

序言

序言

灵智 (Lumio) — 银行信用卡场景的智能客服平台, 用 Python 3.11 + FastAPI + asyncio 串起两个独立服务: Bot 自助问答 (:8000) 与坐席辅助 (:8001), 后接 PostgreSQL / Redis / Elasticsearch / Milvus / MinIO / Kafka 六大数据基础设施.

项目定位

Lumio 解决的是银行信用卡客服这一垂直场景:

- **合规要求极高**: 文档必须经过 DRAFT → IN_REVIEW → APPROVED → PUBLISHED → SUPERSEDED/REJECTED/ARCHIVED 的 7 态审批流, 任何客户可见的知识都有 `allowed_roles` + `regulatory_tags` 双重门禁.
- **不能拒客**: 银行客服高峰期瞬时涌入, Semaphore 满载时不能返 503, 必须走 `_FAST_REPLIES` 紧急话术模板.
- **可解释性**: 坐席辅助的每个推送必须能解释「为什么推」, 仲裁器输出 `fusion_type=BLOCK|WARN|PASS` 三个明确等级, 不能黑盒.
- **审计留痕**: 5-7 年的客户对话与坐席操作必须持久化, 任何 API 调用都进 `audit_log` 表.

4 条阅读路径

根据角色选择:

1. **新成员 (1-2 天)**: 01 → 03 → 05 → 12 → 06
2. **架构师 (半天)**: 00 → 01 → 02 → 04 → 05 → 11 → 13
3. **业务开发者 (按需)**: 01 → 02 → 03/04 → 08 → 10
4. **运维 (半天)**: 01 → 12 → 13 → 10 → 附录 C

详见 [README.md 阅读路径](#).

写作约定

为了让这本书在 1-2 年后仍然可用, 严格遵守以下约定:

1. 「为什么」优先于「是什么」

每个设计决策都先讲 *为什么这么选*, 再讲 *怎么实现*. 例如 RAG 不直接讲检索算法, 而是先讲「为什么 BM25 + 向量 + RRF」的三路召回.

2. 代码引用而非复制

关键代码用 5-15 行片段 + 行号标注. 例如:

```
# agent/lumio/services/bot/router.py:289
async def _session_worker(self, session_id: str) -> None:
    """per-session 串行消费, 300s 空闲自动退出"""
    ...
```

完整实现见 `agent/lumio/services/bot/router.py:289-340`. 避免文档腐烂, 也避免照抄代码.

3. mermaid 优先于 ASCII

时序图/状态图/流程图一律使用 mermaid 语法, 源码在 `diagrams/*.mmd`, 文档用 `\`mermaid\`` 代码块引用.

4. 数字驱动

所有结论配数字: 16 个 SubSettings / 22 个 MCP 工具 / 35 个错误码 / 740+ 个测试用例 / 19 张 PG 表 / 15+ Redis key / 24 个 Docker 服务 / 3 套 Grafana dashboard / 6 条告警规则.

5. 命名严格统一

- 错误码形式: 3004: SessionNotFoundError
- 状态名: BOT_ACTIVE (大写下划线) 而非 bot_active (小写蛇形)
- 配置项: LUMIO_DATABASE__HOST (顶层 LUMIO_, 嵌套 __ 分隔)

完整术语见 [附录 A 术语表](#).

致谢

本系列文章基于以下贡献者的工作:

- Lumio 核心组 (3 名): 平台设计与实现
- Java MCP Server 组 (2 名): Spring Boot 3.4.5 + 22 工具
- chat-svc 组 (2 名): Spring Boot 3.1.5 + Netty + ZK 服务发现
- 前端组 (2 名): Vue 3 + Vite + WebSocket 客户端

下一步: 阅读 [第 1 章: 整体架构](#), 从三层分层 + 两个 FastAPI 实例的宏观布局开始.

阅读路径与目录

Lumio 技术深度剖析

灵智 (Lumio) — 银行信用卡场景的智能客服平台, 提供 AI Agent 实时坐席辅助 (Assist) 与 Bot 自助问答 (Bot) 两大核心能力.

本系列文章覆盖 Lumio 项目全部 16,000 行 Python + 22 工具 Java + 24 个 Docker 服务的核心设计决策. 文中所有行号引用 `file_path:line` 形式, 配 mermaid 图, 行号以最新代码为准.

总规模: 17 章节 / ~95,000 字 / 7 张 mermaid 图 / 2 附录.

目录

序言与索引

章节	标题	难度	时长
00-preface	序言: 项目介绍 + 写作约定	通识	5 分
README	当前页: 目录 + 阅读路径	—	—

第一部分: 整体设计 (第 1-2 章)

章节	标题	难度	时长
01	整体架构: 三层分层 + 两个 FastAPI 实例	中级	18 分
02	配置系统: 16 个 SubSettings + Pydantic v2	中级	15 分

第二部分: 核心代码 (第 3-7 章) — 项目业务核心

章节	标题	难度	时长
03	Bot 自助问答: Redis Stream + Agent 决策树	中级	25 分
04	坐席辅助引擎: D1/D2/D3 + E1/E2/E3 五阶段编排	高级	25 分
05	RAG 检索全链路: BM25 + 向量 + RRF + 重排序	高级	30 分
06	会话状态机: 3 phase × 7 sub-state + CAS 原子控制	高级	22 分
07	MCP 工具集成: 22 工具 + Higress 网关	中级	20 分

第三部分: 横切关注点 (第 8-14 章)

章节	标题	难度	时长
08	错误处理: 35 错误码 + 统一响应体	中级	12 分
09	RAG 摄入管线: 5 阶段 + 双写回滚	中级	12 分
10	可观测性: 17 指标 + OTel 跨服务追踪	中级	14 分
11	安全合规: JWT + PBKDF2 + 双重审计	中级	14 分
12	数据层: PG 19 表 + Redis 15 key + 双写一致性	中级	14 分

章节	标题	难度	时长
13	部署: 24 Docker 服务 + K8s + opt-in 网关	中级	12 分
14	测试策略: 70+ 文件 / 969 用例通过 + 真实中间件	中级	12 分

第四部分: 客服 Agent 能力深挖 (第 15–17 章) — 对话系统核心

这一部分深入 Lumio Bot 区别于一般 CRUD 框架的 **3 大对话能力**: 上下文工程 / 跨会话客户记忆 / 工具调用编排. 适合做对话系统/Agent 设计的读者精读.

章节	标题	难度	时长
15	上下文工程: 3 层上下文 + token 预算 + 增量摘要	高级	22 分
16	客户记忆与知识图谱: 跨会话画像 + 银行实体关系	高级	20 分
17	工具调用与确认状态机: 渐进式暴露 + 5 态确认 + 双重护栏	高级	25 分

附录

章节	标题
A	术语表 (60+ 词)
C	常见问题排查 (20+ 场景)

阅读路径

按角色选择适合你的阅读顺序:

路径 1: 新成员 (1–2 天速成)

00–preface → 01 整体架构 → 03 Bot (核心) → 05 RAG → 12 数据层 → 06 会话状态机

目的: 理解项目宏观布局, 掌握 Bot 主链路, 知道数据怎么存.

路径 2: 架构师 (半天深度评审)

00–preface → 01 整体架构 → 02 配置系统 → 04 坐席辅助 → 05 RAG → 11 安全合规 → 13 部署

目的: 评估架构选型, 理解 5 个核心设计决策 (asyncio 编排 / Redis Stream vs Kafka / PydanticAI / PBKDF2 / 父–子分块).

路径 3: 业务开发者 (1 天, 接需求时)

01 整体架构 → 02 配置系统 → 03 Bot (按需) → 04 坐席 (按需) → 08 错误处理 → 10 可观测性

目的: 快速定位业务代码, 知道在哪加端点, 错了怎么排查.

路径 4: 运维 / SRE (半天)

01 整体架构 → 12 数据层 → 13 部署 → 10 可观测性 → 附录 C 故障排查

目的: 知道 24 个 Docker 服务怎么拉起, Prometheus 指标含义, 常见故障的处理.

路径 5: 对话系统 / Agent 设计者 (1 天, 推荐)

目的: 理解 Lumio 区别于一般 CRUD 框架的 **3 大对话能力设计**: 3 层上下文 + 跨会话客户画像 + 渐进式工具调用 + 5 态确认状态机 + 双重护栏. 这些是构建银行客服 Agent 的关键基础设施.

文档约定

- 代码引用: `file_path:line` 形式, 例如 `agent/lumio/main.py:138` 指 `main.py` 第 138 行.
 - 行号基于当前 `main` 分支, 后续提交可能漂移.
 - Mermaid 图: 时序图/状态图/流程图一律使用 `mermaid`, 源码在 `diagrams/*.mmd`.
 - 中英混排: 项目内代码英文 (函数名/类名), 文档解释中文.
 - 错误码: 形式 `CODE: 名 (3xxx 业务)`, 例如 `3004: SessionNotFoundError`.
 - 配置项: 形式 `LUMIO_DATABASE__HOST`, 顶层 `LUMIO_`, 嵌套 `__` 分隔.
 - 命名统一: 术语与错误码与正文严格一致, 完整清单见[附录 A](#).
-

反馈

发现错别字 / 行号漂移 / 概念错误? 提交 PR 修改 `docs/book/*.md`, 主代理会审阅.

维护者: Lumio 核心组 License: Apache-2.0 (项目根 `LICENSE`)

01 整体架构

第 1 章: 整体架构

本章用一张架构图 + 5 个设计决策, 让你 18 分钟内理解 Lumio 的宏观布局.

先讲个业务场景: 客户在手机银行里发起对话, 消息经过的路是 — 网页/App 前端 → 反向代理 → 两个业务服务 (Bot 自助问答 / 坐席辅助) → 大模型和知识库. 同时, 银行还有坐席人员在看同一场对话: 客户没问完, 坐席就收到了 AI 的实时话术建议和合规提醒. 这就是本章要讲清的全景: 一套系统, 同时服务客户和坐席两条线 — 它们共享底层数据, 但入口、协议、性能要求完全不同.

1.1 三层架构总览

怎么读这张图: 从下往上看 — 数据层是"地基" (什么信息都存在哪), 中间是"店面" (两个服务接待什么人), 上面是"门面" (客户和坐席从哪里进来), 旁边还有"监控室" (运维怎么看系统是否健康). 每个框里都标了端口号和关键职责, 看懂了这张图, 全书的章节地图就有了.

Lumio 整体走三层架构, 借鉴"洋葱模型":

```
graph TB
    subgraph Client["客户端"]
        Web["Vue 3 Web App"]
        Customer["手机银行 App"]
    end

    subgraph Edge["边缘层 (反向代理)"]
        Nginx["Nginx :80/443<br/>WebSocket Upgrade 3600s"]
    end

    subgraph Orch["编排层 (Orchestration) - Python 3.11 + FastAPI"]
        Bot["Bot Service :8000<br/>POST /api/chat/send<br/>GET /api/chat/poll<br/>GET /api/kb/*"]
        Assist["Assist Service :8001<br/>WS /api/ws/agent/{id}<br/>POST /api/analyze<br/>POST /api/hold<br/>POST /api/review/*"]
    end

    subgraph Capability["AI 能力 (HTTP 直连外部模型服务)"]
        Ollama["Ollama :11434<br/>LLM 生成 (OpenAI 兼容)"]
        TEI["TEI :8080<br/>Embedding / Reranker"]
        Gateway["Higress AI 网关<br/>鉴权 / 限流 / 脱敏 / 审计"]
    end

    subgraph Data["数据层 (6 大中间件)"]
        PG["PostgreSQL 16<br/>19 张表<br/>5-7 年审计留存"]
        Redis["Redis 7.2<br/>15+ key prefix<br/>Stream + Pub-Sub + ZSET"]
        ES["Elasticsearch 8.19 + IK<br/>BM25 全文检索"]
        Milvus["Milvus 2.4<br/>IVF_FLAT 1024 维<br/>向量检索"]
        MinIO["MinIO<br/>lumio-docs bucket<br/>KB 文件存储"]
        Kafka["Kafka 3.7 (KRaft)<br/>3 topic 预留<br/>异步事件流"]
    end

    subgraph Observability["可观测性"]
        Prom["Prometheus :9090<br/>9 scrape jobs"]
        Grafana["Grafana :3001<br/>3 dashboard"]
        Jaeger["Jaeger :16686<br/>OTLP :4318"]
    end

    subgraph Java["Java 生态 (chat-svc + MCP Server)"]
        ChatSvc["chat-svc :8080/:8081<br/>Spring Boot 3.1.5 + Netty + ZK"]
        MCPServer["mcp-server :8090<br/>Spring Boot 3.4.5 + Spring AI<br/>22 信用卡工具"]
    end

    Web --> Nginx
    Customer --> Nginx
    Nginx --> Bot
    Nginx --> Assist
    Bot --> PG
    Bot --> Redis
```

Bot --> ES
Bot --> Milvus
Bot --> MinIO
Bot --> MCPServer
Assist --> PG
Assist --> Redis
Assist --> MCPServer
Assist --> ChatSvc
Bot --> Gateway
Assist --> Gateway
Gateway --> Ollama
Gateway --> TEI
Bot -.span.-> Jaeger
Assist -.span.-> Jaeger
MCPServer -.span.-> Jaeger
Bot -.metric.-> Prom
Assist -.metric.-> Prom
MCPServer -.metric.-> Prom
Prom --> Grafana

1.1.0 逐块解读 (对应"店面"比喻)

区块	是什么	接待谁	关键职责
客户端 + 边缘	网页/App + Nginx	客户、坐席	所有流量先进 Nginx, 按路径分发给两个服务; WebSocket 长连接 1 小时不超时
编排层 (两个服务)	Bot :8000 + Assist :8001	客户走 Bot, 坐席走 Assist	Bot 是高频低延迟 (客户问一句答一句); Assist 是低频高复杂度 (坐席通话中实时分析)
AI 能力	Ollama/TEI/Higress	两个服务共用	大模型生成、向量嵌入、重排序 — 统一走网关, 加鉴权限流脱敏
数据层 (6 中间件)	PG/Redis/ES/Milvus/MinIO/Kafka	两个服务共用	各自管一类数据: PG=业务真相, Redis=会话状态, ES/Milvus=检索, MinIO=文件, Kafka=预留事件
Java 生态	chat-svc + mcp-server	坐席 + 客户	chat-svc=坐席工作台对接, mcp-server=22 个信用卡工具 (账单/挂失/额度)
可观测性	Prometheus/Grafana/Jaeger	运维	指标、告警、链路追踪 — 后面第 10 章详讲

两条业务线的对比 (理解两个服务为什么分开):

维度	Bot 自助 (客户)	Assist 辅助 (坐席)
流量特征	大、突发 (账单日 10 倍)	小、平稳 (坐席数量有限)
延迟要求	秒级 (客户在等)	毫秒级推送 (通话中实时)
协议	HTTP 长轮询	WebSocket 长连接
失败后果	客户体验差	坐席无辅助 (可人工兜底)

所以它们各自独立部署、独立扩缩容 — 账单日只扩 Bot, 坐席少时 Assist 可以缩到很小, 互不拖累.

1.1.1 三层的责任划分

层	责任	失败模式
编排层	API 路由 / 业务编排 / 状态机 / 会话管理	5xx → 503 兜底
AI 能力	LLM / 嵌入 / 重排序 (HTTP 直连外部模型服务, 经 Higress 网关治理)	熔断 + 4 级降级链
数据层	持久化 / 缓存 / 检索 / 事件	各组件独立降级

关键事实: 不设独立 gRPC 能力层 — AI 能力 (LLM/Embedding/Reranker/分类) 由编排层 HTTP 直连外部模型服务 (Ollama :11434 / TEI :8080), 统一治理由 Higress AI 网关承担 (鉴权/限流/脱敏/审计). 当前规模下两层编排已足够, gRPC 抽象仅增加部署与序列化开销; 若未来模型平台化 (多团队共享/vLLM 多实例/跨集群调用) 再引入.

1.2 两个 FastAPI 实例的边界

主入口 agent/lumio/main.py:241-242 创建两个独立 FastAPI 实例:

```
bot_app: FastAPI = create_bot_app(lifespan=bot_lifespan)
assist_app: FastAPI = create_assist_app(lifespan=assist_lifespan)
```

1.2.1 为什么是两个而非一个？

维度	单实例	双实例 (当前)
伸缩	同步扩缩, 高峰浪费	Bot 流量大时单独扩, Assist 低负载时缩
故障域	一个崩溃全停	Bot 崩了 Assist (坐席) 仍可用
部署	必须同步发布	Bot 改逻辑不影响 Assist (中间件共享)
资源隔离	LLM 大模型推理互相干扰	Bot 跑 RAG, Assist 跑仲裁, CPU 隔离

1.2.2 启动清单的差异

agent/lumio/main.py:83-186 定义了三组 init 步骤:

```
_COMMON_INIT_STEPS = [ # 12 步共享
    init_db,           # PostgreSQL async engine
    init_redis,        # Redis async pool
    init_es,           # Elasticsearch async client
    init_milvus,       # Milvus async client
    init_minio,        # MinIO async client
    init_embedding,    # TEI/Ollama 嵌入 provider
    init_reranker,     # 重排序 provider
    init_llm,          # LLM client
    init_safety,       # 敏感词 Aho-Corasick
    init_metrics,      # Prometheus 指标
    init_tracing,      # OTel TracerProvider
    init_mcp_client,   # MCP 工具客户端
]

_BOT_INIT_STEPS = _COMMON_INIT_STEPS + [start_bot_worker] # + Stream 消费
_ASSIST_INIT_STEPS = _COMMON_INIT_STEPS + [start_notify_worker] # + 通知 worker
```

关键设计: 用纯函数列表编排, 避免 lifespan 自动发现机制的隐式行为. 每个步骤显式接收 app: FastAPI 参数, 便于测试和复用.

1.2.3 关闭顺序: 逆序 + 异常吞噬

```
# main.py:65-79
class _SuppressExceptions:
    """关闭阶段吞异常, 保证其他步骤不被阻塞"""
    def __enter__(self): pass
    def __exit__(self, *args): return True
```

关闭时逆序调用每个步骤的 close 函数, 任何步骤失败都不影响后续 — "清理优先于报错"原则.

1.3 共享基础层 (shared/)

agent/lumio/shared/ 目录共 4185 行, 跨服务复用:

模块	行数	职责
config.py	533	16 个 SubSettings 统一管理
exceptions.py	211	LumioError 异常层次 + 35 错误码
middleware.py	119	全局异常处理器 + request_id 注入

模块	行数	职责
models.py	563	15+ Pydantic 模型
orm_models.py	1060	19 张 SQLAlchemy 表
database.py	63	async engine + 连接池
redis_client.py	42	async pool
auth.py	196	JWT + RBAC + dev bypass
safety.py	254	Aho–Corasick 敏感词
pii.py	77	手机/身份证/银行卡/邮箱脱敏
session.py	764	SessionManager + CAS Lua
session_timeout.py	231	Redis ZSET 超时队列
health.py	142	5 依赖并行健康检查
metrics.py	171	17 Prometheus 指标
tracing.py	232	OTel 全链路
audit_middleware.py	218	27 端点审计自动映射
logger.py	110	JSON 结构化日志
degradation.py	222	4 级降级管理器
password.py	45	PBKDF2–HMAC–SHA256
rate_limit.py	67	slowapi 限流

关键设计: 共享基础层不依赖任何具体服务, Bot 和 Assist 都通过 `from lumio.shared.xxx import YYY` 引用.

1.4 5 个核心设计决策 (技术选型)

下面 5 个决策是 Lumio 与同类系统最大差异点.

决策 1: 不用 Temporal, 用纯 `asyncio.gather`

Assist 引擎的 D/E 五阶段 (D1/D2/D3 + E1/E2/E3) 用 `asyncio.gather` 并行编排, 不引入外部工作流引擎.

理由: 1. **运维成本:** Temporal 需要单独的 Workflow Server + Namespace 管理 + Worker 注册 2. **调试困难:** 跨多个 Activity 的 trace 链路在 Temporal Web UI 才可见, 不能直接看 Jaeger 3. **依赖爆炸:** Python SDK + Java SDK 双维护, bug 修复要两边同步 4. **业务规模:** Assist 引擎单周期 < 5s, 5 个协程 `gather` 性能足够

实现: `agent/lumio/services/common/assist_engine.py:171` `run_assist_engine()` 入口函数:

```
# 关键片段, 完整实现见 assist_engine.py:171-220
async def run_assist_engine(state_snapshot):
    # Phase 1: 评估器并行
    d1_task = asyncio.create_task(evaluate_d1(state_snapshot))
    d2_task = asyncio.create_task(evaluate_d2(state_snapshot))
    d3_task = asyncio.create_task(evaluate_d3(state_snapshot))
    d1, d2, d3 = await asyncio.gather(d1_task, d2_task, d3_task)

    # Phase 2: 执行器并行
    e1_task = asyncio.create_task(run_e1(d1))
    e3_task = asyncio.create_task(run_e3(d3))
    e1, e3 = await asyncio.gather(e1_task, e3_task)

    # Phase 3: 仲裁
    return GlobalArbitrator.arbitrate(d1, d2, d3, e1, e3)
```

收益: 节省 Temporal Server 运维, 跨服务 trace 全在 Jaeger, 调试简单. **代价:** 失去 Temporal 的 exactly-once / 长跑工作流支持 (本项目不需要).

决策 2: 不用 Kafka, 用 Redis Stream

背景: 跨进程消息队列有两个候选, Kafka vs Redis Stream.

理由: 1. **学习成本:** Redis 已是基础设施, Stream API (XADD / XREADGROUP / XACK) 与 Redis 风格一致 2. **单实例够用:** 银行客服日均百万级, Redis 单实例能扛 3. **运维简单:** 复用 Redis Sentinel / Cluster 即可, 不用单独运维 ZK + Kafka Broker 4. **延迟更低:** Redis 单跳延迟 < 1ms, Kafka 至少 5-10ms

实现: agent/lumio/services/bot/router.py:68-194 用 Redis Stream 做消息队列:

```
# 关键片段, 完整实现见 router.py:68-194
XGROUP_CREATE_SCRIPT = """
XGROUP CREATE lumio:chat:stream bot-group $ MKSTREAM
"""
```

包含 4 个关键概念: – **MAXLEN 10000:** 流最大长度, 防止 Redis 内存爆 – **Consumer Group bot-group:** at-least-once 投递 – **PEL (Pending Entries List):** 已读未确认, XAUTOCLAIM 接管超时 – **Dead Letter lumio:chat:dead_letter:** 失败重试 3 次后转死信

代价: 失去 Kafka 的高吞吐 (百万级 QPS). 当前业务量级远未触达.

决策 3: 不用 PydanticAI / LangChain, 用纯手写 asyncio

背景: agent/lumio/services/bot/bot_agent.py:4 注释明确:

不依赖任何 Agent 框架 (LangGraph / PydanticAI / AutoGen), 手写 asyncio + OpenAI function-calling.

理由: 1. **类型安全:** Pydantic v2 原生模型 + mypy 严格, 比 LangChain 的 Chain[Unknown, Unknown] 强 2. **可控性:** 每个 LLM 调用 / 工具循环 / 状态转移都可见, 调试简单 3. **依赖最小:** LangChain 一次拉入 50+ 包, 升级困难 4. **业务规模:** Bot 主链路只是「分类 → 路由 → 生成」, 用不上 LangChain 的 Chain/Memory 抽象

实现: bot_agent.py:92 LumioAgent 类手写 6 步决策:

```
# 关键片段
async def run(self, session_id, user_input, customer_id):
    # 1. 快速问候/告别
    if is_greeting(user_input): return GREETING_RESPONSE
    # 2. pending_action 拦截
    if has_pending_action(session_id): return self._handle_pending_action(...)
    # 3. 分类
    intent = await self._classify(user_input)
    # 4. 路由
    if intent.label in KNOWLEDGE_INTENTS: return await self._handle_knowledge(...)
    if intent.label in BUSINESS_INTENTS: return await self._handle_business(...)
    # 5. 工具调用
    if has_tool(intent): return await self._handle_tool(...)
    # 6. 兜底
    return FALLBACK_SYSTEM_PROMPT.format(input=user_input)
```

代价: 失去 LangChain 的 Hub 提示词生态 / Tracing UI. 用 OTel 自建.

决策 4: 密码用 PBKDF2 而非 argon2 / bcrypt

背景: agent/lumio/shared/password.py:6-10 注释:

选择 PBKDF2-HMAC-SHA256 而非 argon2 / bcrypt, 因为: 1. 零外部依赖 (passlib+bcrypt 在 musl/alpine 编译困难) 2. Django/Flask 默认, 迁移成本低 3. 600000 次迭代满足 OWASP 2023 推荐

实现: password.py:12-22 :

```
ALGORITHM = "pbkdf2_sha256"
ITERATIONS = 600_000

def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    dk = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, ITERATIONS)
    return f"{ALGORITHM}${ITERATIONS}${b64encode(salt).decode()}${b64encode(dk).decode()}"
```

代价: 600k 迭代每次 ~350ms, 比 bcrypt 慢但比 argon2 内存硬化弱. 对低 QPS 登录场景够用.

决策 5: 父-子分块 (Parent-Child Chunking)

背景: RAG 检索的经典问题是「检索粒度 vs 生成粒度」矛盾 — 检索要小 chunk (语义聚焦), 生成要大 chunk (上下文完整). 父-子分块同时解决两者.

实现: agent/lumio/services/common/ingestion.py:344-366 摄入时, 大块 (1500 字符) 是父, 切成小块 (300 字符) 是子. 子块进 Milvus 检索, 命中后 parent_chunk_id 回填, 生成阶段拿父块.

```
# 关键片段
# 子块: embedding 检索
child_chunks = [chunk for chunk in chunks if not chunk.is_parent]
embeddings = await embedder.embed([c.content for c in child_chunks])
# 父块: 仅 metadata 存储
for parent in [c for c in chunks if c.is_parent]:
    parent.children_ids = [c.id for c in chunks if c.parent_id == parent.id]
```

收益: 检索精度 ↑, 生成上下文完整, 同一文档 1 次摄入.

代价: 摄入复杂度 ↑, 存储 2 倍 (子块 + 父块 metadata).

详细 RAG 摄入管线见 [第 9 章 RAG 摄入](#).

1.5 模块依赖图

下面用 mermaid 展示 shared / bot / assist / common 之间的依赖关系:

```
graph LR
    shared["shared/<br/>配置/异常/中间件/可观测性"]
    common["services/common/<br/>RAG/会话/MCP/工具"]
    bot["services/bot/<br/>Bot 自助问答"]
    assist["services/assist/<br/>坐席辅助"]

    shared --> common
    shared --> bot
    shared --> assist
    common --> bot
    common --> assist

    bot -.HTTP.-> assist
    assist -.HTTP.-> bot
```

关键设计: common/ 是 Bot 和 Assist 共享的「业务工具库」, 例如 RAG 检索 (retrieval.py) / 会话管理 (session.py) / MCP 客户端 (mcp_client.py) 都在 common. 不通过 HTTP 互相调用, 而是直接 import — 因为它们是同一进程的 Python 模块.

1.6 本章小结

Lumio 的整体架构可以概括为:

- 两层编排 + AI 能力: 编排 (FastAPI) / AI 能力 (HTTP 直连 Ollama/TEI, Higress 网关治理) / 数据 (6 大中间件)
- 两个服务: Bot :8000 (高频低延迟) + Assist :8001 (低频高复杂度), 共享底层

- 5 个关键决策: asyncio.gather 替代 Temporal / Redis Stream 替代 Kafka / 手写 Agent 替代 LangChain / PBKDF2 替代 argon2 / 父-子分块

延伸阅读: - [第 2 章 配置系统](#) — 16 个 SubSettings 的完整体系 - [第 3 章 Bot 自助问答](#) — Bot :8000 内部细节 - [第 4 章 坐席辅助引擎](#) — Assist :8001 内部细节

02 配置系统

第 2 章: 配置系统

本章深入剖析 Lumio 的 16 个 SubSettings 配置体系, 这是理解项目如何"零硬编码"运行 24 个 Docker 服务 + 16+ 端口的关键.

先讲个业务场景: 一套 Lumio 要部署在三种环境 — 开发者的 Mac (localhost 连中间件)、测试服务器 (内网 IP)、生产集群 (K8s 里的服务名). 同一个代码, 怎么做到"换环境只改配置不碰代码"? 答案是**配置外置 + 前缀隔离**: 每个中间件有自己的环境变量前缀 (`POSTGRES_HOST` / `REDIS_HOST` / `LLM_BASE_URL`), 部署时按环境填不同的 `.env`, 代码永远只读配置对象, 不读环境变量本身. 这就是本章要讲的"零硬编码" — 代码里找不到一个写死的 IP 或端口.

2.1 配置体系总览

`agent/lumio/shared/config.py` 共 533 行, 定义 16 个 **Pydantic Settings** 子类, 加上 1 个根 **Settings** 容器.

2.1.1 16 个 SubSettings 一览

#	SubSettings	env_prefix	关键字段	默认值
1	DatabaseSettings	POSTGRES_	host/port/user/password/database	localhost:5432/lumio
2	RedisSettings	REDIS_	host/port/password/db/max_connections	localhost:6379/0/20
3	ElasticsearchSettings	ES_	hosts (列表)	localhost:9200
4	MilvusSettings	MILVUS_	host/port/vector_dim	localhost:19530/1024
5	MinIOSettings	MINIO_	endpoint/access_key/secret_key/bucket	localhost:9000/lumio-docs
6	LLMSettings	LLM_	base_url/api_key/primary_model/fallback_model	OpenAI 兼容
7	ClassificationSettings	CLS_	intent_threshold/min_entity_confidence	0.6/0.7
8	RAGSettings	RAG_	index_prefix/rrf_k/chunk_size	lumio/60/1500
9	SafetySettings	SAFETY_	wordlist_path/max_scan_length	config/sensitive_words.txt
10	SessionSettings	SESSION_	5 类超时 (idle/queue/ringing/session/review)	180s/60s/无/1800s/120s
11	BotSettings	BOT_	max_concurrent_agents/message_ttl/fast_reply_cooldown	10/8/5
12	AssistSettings	ASSIST_	4 类分支超时 (script/knowledge/alert/product)	500/600/300/400 ms
13	OrchestrationSettings	ORCH_	d1_d2_d3_cooldown/e1_e2_e3_sla	300s/3s
14	CircuitBreakerConfigSettings	CB_	ai/mkt/risk 3 套独立阈值	0.5/30s
15	ObservabilitySettings	OBSERVABILITY_	tracing_enabled/jaeger_host/sampling_ratio	true/localhost/1.0
16	MCPSettings	MCP_	enabled/endpoint/timeout/sensitive_tools	false/localhost:8090/10s/4 工具

完整 16 个类的代码见 `config.py:16-437` .

2.1.2 根 Settings 容器

```
# config.py:438-528 (简化)
class Settings(BaseSettings):
    model_config = SettingsConfigDict(
        env_prefix="LUMIO_",      # 顶层 LUMIO_*
```

```

env_nested_delimiter="__", # 嵌套用 __ 分隔
case_sensitive=False,
extra="ignore",
)

# 16 个嵌套 SubSettings
database: DatabaseSettings = Field(default_factory=DatabaseSettings)
redis: RedisSettings = Field(default_factory=RedisSettings)
elasticsearch: ElasticsearchSettings = Field(default_factory=ElasticsearchSettings)
milvus: MilvusSettings = Field(default_factory=MilvusSettings)
minio: MinIOSettings = Field(default_factory=MinIOSettings)
llm: LLMSettings = Field(default_factory=LLMSettings)
classification: ClassificationSettings = Field(default_factory=ClassificationSettings)
rag: RAGSettings = Field(default_factory=RAGSettings)
safety: SafetySettings = Field(default_factory=SafetySettings)
session: SessionSettings = Field(default_factory=SessionSettings)
bot: BotSettings = Field(default_factory=BotSettings)
assist: AssistSettings = Field(default_factory=AssistSettings)
orchestration: OrchestrationSettings = Field(default_factory=OrchestrationSettings)
circuit_breaker: CircuitBreakerConfigSettings = Field(default_factory=CircuitBreakerConfigSettings)
observability: ObservabilitySettings = Field(default_factory=ObservabilitySettings)
mcp: MCPSettings = Field(default_factory=MCPSettings)

# 顶层全局字段
environment: Literal["development", "staging", "production"] = "development"
debug: bool = False
log_level: str = "INFO"
cors_origins: list[str] = ["*"]
jwt_secret: str = "lumio-dev-secret-change-in-production"
dev_auth_bypass: bool = False # 默认关闭, 仅开发环境可开
rate_limit_enabled: bool = True
rate_limit_default: str = "60/minute"
rate_limit_chat: str = "30/minute"

@model_validator(mode="after")
def _validate_production_security(self) -> "Settings":
    """生产环境 JWT secret 强制校验"""
    if self.environment == "production":
        forbidden = {
            "lumio-dev-secret-change-in-production",
            "<CHANGE_ME>",
            "<CHANGE_ME_IF_NEEDED>",
        }
        if self.jwt_secret in forbidden:
            raise ValueError(
                "生产环境禁止使用占位 JWT secret. "
                "请通过 LUMIO_JWT_SECRET 显式设置 ≥32 字符的随机值."
            )
        if len(self.jwt_secret) < 32:
            raise ValueError(
                f"生产环境 JWT secret 至少 32 字符, 当前 {len(self.jwt_secret)}."
            )
    return self

@lru_cache(maxsize=1)
def get_settings() -> Settings:
    """全局单例 Settings 工厂"""
    return Settings()

```

2.2 env_prefix 命名空间: 12 个独立前缀

关键设计: 每个 SubSettings 独立 `env_prefix`, 而不是一个大 prefix 嵌套.

2.2.1 为什么不用统一前缀?

考虑两种风格对比:

风格	例子	优点	缺点
统一前缀嵌套 (反例)	LUMIO_DATABASE__HOST, LUMIO_REDIS__HOST	一目了然	长, 同名前缀重复
独立前缀 (Lumio 风格)	POSTGRES_HOST, REDIS_HOST	短, 业界通用 (12-factor 风格)	需要约定 <code>env_prefix</code>

Lumio 选择业界通用 12-factor 风格 (Spring Boot 同款), 与 Kubernetes ConfigMap / Docker Compose env / .env.example 写法一致.

2.2.2 实例: DatabaseSettings

```
# config.py:16-36
class DatabaseSettings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="POSTGRES_", case_sensitive=False)

    host: str = "localhost"
    port: int = 5432
    user: str = "lumio"
    password: str = "lumio"
    database: str = "lumio"

    @property
    def dsn(self) -> str:
        return f"postgresql+asyncpg://{self.user}:{self.password}@{self.host}:{self.port}/{self.database}"
```

读取: LUMIO_DATABASE__HOST=db.example.com 不生效, 应该用 POSTGRES_HOST=db.example.com. 但 Settings.database.host 仍能访问到 db.example.com, 因为 Settings.database = Field(default_factory=DatabaseSettings) 会自动读 POSTGRES_* env.

2.2.3 实例: LLMSettings (含业务约束)

```
# config.py:91-114
class LLMSettings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="LLM_", case_sensitive=False)

    base_url: str = "http://localhost:11434/v1" # Ollama 兼容
    api_key: str = "ollama"
    primary_model: str = "qwen2.5:7b"
    fallback_model: str = "qwen2.5:0.5b"
    temperature: float = 0.3
    max_tokens: int = 1024
    timeout_seconds: float = 30.0
    max_concurrent: int = 5
    circuit_breaker_failures: int = 5
```

关键设计: 默认值指向 Ollama, 因为 dev 阶段本地 LLM 跑 qwen2.5:7b 无需 GPU 集群. 生产改 LLM_BASE_URL=https://api.openai.com/v1 即可.

2.3 嵌套分隔符 __ 的使用

虽然 SubSettings 各自独立 env_prefix, 但根 Settings 用 __ 嵌套分隔符提供统一入口:

```
# .env 中显式设置
LUMIO_ENVIRONMENT=production
LUMIO_DATABASE__HOST=db.prod.example.com
LUMIO_REDIS__HOST=redis.prod.example.com
LUMIO_REDIS__PASSWORD=xxx
LUMIO_BOT__MAX_CONCURRENT_AGENTS=20
LUMIO_LOG_LEVEL=WARNING
```

Settings() 读 LUMIO_DATABASE__HOST → 解析为 database.host → 覆盖 DatabaseSettings 的 host. 这种风格用于部署期差异化配置, 避免在 16 个 prefix 间切换.

注意: 嵌套分隔符仅在根 Settings 读取时生效, 单独 DatabaseSettings() 不识别 LUMIO_DATABASE__HOST. 实际项目里 get_db() 拿的是 settings.database (工厂构造), 因此一致.

2.4 AliasChoices 兼容模式

追踪配置统一收在 ObservabilitySettings 子配置下, 用 OBSERVABILITY_TRACING_ENABLED. 但旧 env 名不能直接放弃 — 存量 docker-compose / .env / 文档可能还在用, 因此用 AliasChoices 做别名兼容:

2.4.1 兼容实现

```
# config.py:308-326 (简化)
from pydantic import AliasChoices

class ObservabilitySettings(BaseSettings):
    model_config = SettingsConfigDict(env_prefix="OBSERVABILITY_", case_sensitive=False)

    tracing_enabled: bool = Field(
        default=True,
        validation_alias=AliasChoices(
            "OBSERVABILITY_TRACING_ENABLED", # 新名
            "LUMIO_TRACING_ENABLED",         # 旧名 (兼容)
        ),
    )
    jaeger_host: str = Field(
        default="localhost",
        validation_alias=AliasChoices(
            "OBSERVABILITY_JAEGER_HOST",
            "JAEGER_HOST", # 全局旧名
        ),
    )
    otlp_endpoint: str | None = None
    sampling_ratio: float = 1.0
```

行为: – 同时设置 OBSERVABILITY_TRACING_ENABLED=false 和 LUMIO_TRACING_ENABLED=true → 选 OBSERVABILITY_TRACING_ENABLED (在 AliasChoices 列表靠前) – 只设置 LUMIO_TRACING_ENABLED=true → 仍生效 (兼容) – 都不设置 → 用 default=True

2.4.2 关键设计哲学

接口稳定性 > 配置简洁性. 改 env 名 = 改 docker-compose + .env.example + 文档, 改动面 > 10 文件. 用 AliasChoices 多保留 1 行代码, 换 0 故障切换.

这是“接口兼容优先”的代表性决策.

2.5 MCP 路由模式 vs 单后端模式

MCPSettings 走 BaseModel 而非 BaseSettings, 因为它有结构化配置 (路由表):

```
# config.py:370-435 (简化)
class MCPBackend(BaseModel):
    """单后端定义"""
    name: str # 后端名, 例如 "credit-card"
    endpoint: str # streamable-http URL
    prefix: str = "" # 工具名域前缀, "card." 或 ""
    sensitive_tools: list[str] = Field(default_factory=list)

class MCPSettings(BaseModel):
    """根 MCP 配置"""
    enabled: bool = False # P0 默认 False, opt-in
    default_timeout_seconds: float = 10.0
    default_max_tool_iterations: int = 5

    # 单后端模式
    endpoint: str = "http://localhost:8090"

    # 多后端路由模式
    backends: list[MCPBackend] = Field(default_factory=list)
    route_by_tool_name: bool = False
```

2.5.1 单后端 vs 多后端

模式	配置示例	适用场景
单后端	MCP_ENDPOINT=http://mcp:8090	dev / 单域 (如只有信用卡)

模式	配置示例	适用场景
多后端路由	MCP_BACKENDS=[{name: "credit-card", endpoint: ...}, {name: "loan", prefix: "loan."}]	多域 (信用卡 + 贷款 + 理财)

关键设计: backends=[] 时, 自动回退到 endpoint + prefix="" (单后端), 保持向后兼容. 零回归切换.

2.6 完整环境变量清单

agent/.env.example 列出 16+ prefix 的全部变量. 下面是摘要 (完整见仓库根 .env.example):

```
# —— 顶层 LUMIO_* (嵌套用 __ 分隔) ——
LUMIO_ENVIRONMENT=development
LUMIO_DEBUG=false
LUMIO_LOG_LEVEL=INFO
LUMIO_CORS_ORIGINS=*
LUMIO_JWT_SECRET=                # 生产环境必填 ≥32 字符
LUMIO_DEV_AUTH_BYPASS=false      # 默认关闭, 仅开发环境可开

# —— 数据库 ——
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_USER=lumio
POSTGRES_PASSWORD=lumio
POSTGRES_DATABASE=lumio

# —— Redis ——
REDIS_HOST=localhost
REDIS_PORT=6379
REDIS_PASSWORD=
REDIS_DB=0
REDIS_MAX_CONNECTIONS=20

# —— Elasticsearch ——
ES_HOSTS=["http://localhost:9200"]

# —— Milvus ——
MILVUS_HOST=localhost
MILVUS_PORT=19530
MILVUS_VECTOR_DIM=1024

# —— MinIO ——
MINIO_ENDPOINT=localhost:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin
MINIO_BUCKET=lumio-docs

# —— LLM ——
LLM_BASE_URL=http://localhost:11434/v1
LLM_API_KEY=ollama
LLM_PRIMARY_MODEL=qwen2.5:7b
LLM_FALLBACK_MODEL=qwen2.5:0.5b
LLM_TIMEOUT_SECONDS=30

# —— RAG ——
RAG_INDEX_PREFIX=lumio
RAG_RRF_K=60
RAG_CHUNK_SIZE=1500
RAG_CHUNK_OVERLAP=200
RAG_BM25_TOP_K=20
RAG_VECTOR_TOP_K=20
RAG_RERANK_TOP_K=5
RAG_CACHE_TTL_SECONDS=300

# —— 安全 ——
SAFETY_WORDLIST_PATH=config/sensitive_words.txt
SAFETY_MAX_SCAN_LENGTH=2000

# —— 会话 ——
SESSION_BOT_IDLE_TIMEOUT=1800
SESSION_QUEUE_TIMEOUT=60
SESSION_RINGING_TIMEOUT=30
SESSION_SESSION_TIMEOUT=1800
SESSION_REVIEW_TIMEOUT=300
```

```

SESSION_MAX_TURNS=20

# —— Bot ——
BOT_MAX_CONCURRENT_AGENTS=10
BOT_MESSAGE_TTL_SECONDS=8
BOT_FAST_REPLY_COOLDOWN=5

# —— Assist ——
ASSIST_SCRIPT_TIMEOUT_MS=500
ASSIST_KNOWLEDGE_TIMEOUT_MS=600
ASSIST_ALERT_TIMEOUT_MS=300
ASSIST_PRODUCT_TIMEOUT_MS=400

# —— 编排 ——
ORCH_D1_D2_D3_COOLDOWN_SECONDS=300
ORCH_E1_SLA_SECONDS=3
ORCH_E2_SLA_SECONDS=2
ORCH_E3_SLA_SECONDS=1

# —— 熔断器（3 套独立） ——
CB_AI_FAILURE_THRESHOLD=0.5
CB_AI_RECOVERY_TIMEOUT=30
CB_MKT_FAILURE_THRESHOLD=0.6
CB_RISK_RECOVERY_TIMEOUT=60

# —— 可观测性 ——
OBSERVABILITY_TRACING_ENABLED=true
OBSERVABILITY_JAEGER_HOST=jaeger
OBSERVABILITY_OTLP_ENDPOINT=
OBSERVABILITY_SAMPLING_RATIO=1.0

# —— MCP ——
MCP_ENABLED=false
MCP_ENDPOINT=http://localhost:8090
MCP_TIMEOUT_SECONDS=10
MCP_MAX_TOOL_ITERATIONS=5
MCP_SENSITIVE_TOOLS=["report_card_lost", "redeem_points", "apply_permanent_limit", "repay_credit_card"]

```

2.7 单例工厂 @lru_cache

```

# config.py:530
@lru_cache(maxsize=1)
def get_settings() -> Settings:
    return Settings()

```

@lru_cache(maxsize=1) 保证全局单例, 多次调用不重复构造. 任何位置 get_settings() 拿同一实例, 减少 Pydantic 校验开销.

2.8 测试策略: monkeypatch env

由于 Settings 在导入时构造, 测试需要重置缓存:

```

# 典型测试模式
def test_settings_override(monkeypatch):
    monkeypatch.setenv("LUMIO_ENVIRONMENT", "production")
    monkeypatch.setenv("LUMIO_JWT_SECRET", "x" * 32)

    from lumio.shared.config import get_settings
    get_settings.cache_clear() # 关键: 清 lru_cache

    settings = get_settings()
    assert settings.environment == "production"
    assert len(settings.jwt_secret) == 32

```

test_auth.py:105 等多处使用此模式, 验证了生产环境 JWT secret 长度校验的拦截逻辑.

2.9 本章小结

Lumio 配置系统是项目"零硬编码"的关键:

- 16 个 SubSettings + 16 个独立 env_prefix: 业界标准 12-factor 风格
- `__` 嵌套分隔符: 部署期差异化配置的统一入口
- AliasChoices 兼容模式: 接口兼容优先的稳定性策略
- `_validate_production_security`: 启动期 fail-fast — 生产环境强制校验 JWT 密钥 (≥32 字符, 禁占位) + LLM_API_KEY / MINIO / ES / REDIS 五类外部凭据非默认值
- 分层 token 预算真实消费: `budget_static/customer/rag/history/current` 由上下文构建器强制分配 (第 15 章), 不再是死配置
- MCP 路由 vs 单后端: `backends=[]` 零回归切换
- `@lru_cache` 单例: 减少 Pydantic 校验开销

下一章预告: [第 3 章 Bot 自助问答](#) 深入 LumioAgent 决策树, Redis Stream 全链路, 工具调用 + 确认状态机.

延伸阅读: - [附录 A 术语表](#) - [第 11 章 安全合规](#) — JWT 流程细节 - [第 13 章 部署](#) — 生产环境变量示例

03 Bot 自助问答

第 3 章: Bot 自助问答

本章深入 Lumio Bot 自助问答服务的全链路设计。Bot 是 Lumio 流量最大的服务, 银行客户日均百万级对话都从这里开始。

先讲个业务场景: 一位客户深夜给银行打电话想问"上个月账单多少钱"。系统要做的不是"接电话", 而是三件事: 1. **立刻告诉客户"收到"** — 不能让他对着电话干等 2. **后台慢慢算** — 查账单、翻历史、调模型, 可能花 2~5 秒 3. **算完主动通知** — 客户不用一直占着线路

这就是本章的核心矛盾: **对话是"一问一答"的同步交互, 但 AI 处理是"多步流水线"的异步计算**。怎么把两者接起来, 还要扛住百万级并发、不能让一条消息丢、不能让同一个客户的几句话乱序——就是 Bot 服务要解决的全部问题。

看完本章你会理解: 消息怎么"接了再说" (异步入口), 怎么保证"不丢不乱" (Redis Stream + per-session Worker), 整条消息处理流 (3.4 完整路径) 怎么从 HTTP 一路走到结果回写, LumioAgent 怎么 6 步决策, 工具调用怎么等客户确认, LLM 挂了怎么仍服务客户。

3.1 全链路时序图

3.1.1 先看图: 一次完整对话的 5 个阶段

```
sequenceDiagram
    participant Client as 客户 App
    participant Bot as Bot 服务 :8000
    participant Redis as Redis 7.2
    participant Worker as 会话 Worker
    participant Agent as LumioAgent 决策器
    participant LLM as 大模型 (Ollama)
    participant RAG as 知识检索
    participant MCP as 银行工具

    Note over Client,Redis: 阶段 ① 提交: 客户发消息, 服务"接了再说"
    Client->>Bot: POST /api/chat/send<br/>{消息, 会话ID}
    Bot->>Bot: 校验 + 幂等检查
    Bot->>Redis: XADD 写入消息流<br/>(上限 1 万条)
    Bot-->>Client: 202 已收到<br/>{消息ID, 已接受}

    Note over Redis,Worker: 阶段 ② 分发: 后台按会话排队
    Redis->>Worker: XREADGROUP 读取<br/>(阻塞等新消息)
    Worker->>Worker: 路由到"该会话的队列"
    Note right of Worker: 每个会话一个队列<br/>保证同一个客户的<br/>消息不串位

    Note over Worker,Agent: 阶段 ③ 决策: AI 处理
    Worker->>Agent: agent.run(消息)
    Agent-->>Agent: 有未确认的操作?<br/>(挂失/调额待确认)
    alt 有待确认操作
        Agent-->>Worker: 解读"确认/取消"
    else 正常对话
        Agent->>LLM: 意图分类 (3 秒超时)
        LLM-->>Agent: 意图结果
        Agent->>Agent: 路由: 查知识/办业务/调工具/闲聊
        alt 查知识 (问规则/费用)
            Agent->>RAG: 检索知识库
            RAG-->>Agent: 相关条文 + 排序
            Agent->>LLM: 结合条文生成回答
            LLM-->>Agent: 答案
        else 办业务 (查账单/提额)
            Agent->>MCP: 调银行工具
            MCP-->>Agent: 工具结果
        end
    end

    Note over Worker,Redis: 阶段 ④ 通知: 结果就绪
    Worker->>Redis: 写结果缓存<br/>(2 分钟有效)
    Worker->>Redis: 发布"该会话有结果了"
```

Note over Client,Bot: 阶段 ⑤ 取结果: 客户端长轮询
Client->>Bot: GET /api/chat/poll
(最多等 30 秒)
Bot->>Redis: 订阅"结果就绪"通知
Redis->>Bot: 通知到达
Bot->>Redis: 取结果缓存
Bot->>Client: {回答, 意图, 来源}

3.1.2 逐阶段解读 (对应刚才的银行场景)

阶段 ① 提交 — "接了再说" 客户发"上个月账单多少钱", 服务只做登记就返回"已收到", 不现场算. 就像银行柜台的叫号机: 先取号, 再等叫号, 而不是堵在柜台前等业务办完. 好处: 请求瞬间返回, 服务能同时收下海量消息, 不被最慢的一次 AI 调用拖住.

阶段 ② 分发 — "按号排队" 后台有个"叫号员" (Consumer Group, 机制见 3.3.1) 把消息从 Redis 里读出来, 按客户会话分到各自的队列. 每个会话一个队列、一个处理员 — 这是全章最重要的一句话: 同一个客户连发的 10 条消息, 处理顺序和发送顺序完全一致, 不会出现"第 3 条先被回答、第 2 条后到"的乱序事故.

阶段 ③ 决策 — "AI 开始干活" LumioAgent 决策器先看两件事: ① 这个会话有没有挂着没确认的操作 (比如上轮要办挂失, 等客户说"确认") — 有就先处理确认; ② 没有的话, 先判断客户想干嘛 (查知识 / 办业务 / 闲聊), 再分派给对应流程. 查知识 = 去知识库检索条文 + 让大模型组织语言; 办业务 = 调银行工具 (但敏感操作只"提议", 等客户点头才真办).

阶段 ④ 通知 — "算好了, 喊你" 结果写进 Redis 缓存 (2 分钟有效), 同时广播一声"这个会话有结果了". 像奶茶店做好了一杯, 喊号让顾客来取.

阶段 ⑤ 取结果 — "客户来取" 客户端在等的时候, 不是干等, 而是每 30 秒问一次"好了吗" (长轮询). 听到广播立刻取走结果. 为什么用"轮询"而不是"服务器主动推"? 因为 HTTP 长轮询实现简单、兼容性好、银行内网防火墙友好 — 不需要维护常驻的 WebSocket 连接 (WS 通道在 ws_router.py 有独立实现, 属于增强项).

3.1.3 为什么这 5 步能扛住百万并发

环节	解决什么问题	代价
异步提交 (202)	请求不阻塞, 吞吐量高	客户端要多一步"取结果"
消息流 + 消费组	多实例共享消费, 单条消息不丢	要处理"重复投递" (见 3.3)
每会话一个队列	同客户消息有序	每个活跃会话占一个协程
结果缓存 + 广播	客户端不用重复查询	缓存要设 TTL 防堆积
长轮询	兼容性好, 无需常驻连接	最多等 30 秒, 超时要重试

3.2 入口: POST /api/chat/send

业务背景: 客户点了"发送", 前端把消息交给这个接口. 接口的职责只有一句话 — 确认收到, 别让客户等. 它不做任何"思考": 不分类、不检索、不调模型. 这些全部交给后台异步完成.

agent/lumio/services/bot/router.py:951 chat_send 端点是整个 Bot 服务的入口. 它不直接处理消息, 只做两件事: 写 Redis Stream + 立即返回 202.

```
# 简化版, 完整实现见 router.py:951-1018
@router.post("/api/chat/send")
async def chat_send(body: ChatSendRequest, request: Request) -> dict:
    """异步入口: 写 Stream + 立即返回 202"""
    # 1. 校验 (max_length=2000)
    if len(body.message) > 2000:
        raise DocumentFormatError("消息超过 2000 字符", code=2003)

    # 2. 幂等性检查 (按 request_id)
    # 3. 写 Redis Stream
    msg_id = await redis.xadd(
        "lumio:chat:stream",
        {"session_id": body.session_id, "message": body.message,
        "customer_id": body.customer_id, "request_id": body.request_id},
        maxlen=10000, # 流最大长度, 防止 Redis 内存爆
    )
    return {"accepted": True, "request_id": body.request_id, "msg_id": msg_id}
```

关键设计: 立即返回 202, 不等处理. 客户端接着用 `GET /api/chat/poll` 长轮询结果. 这是**异步 API 模式** — 银行场景下, 客户电话打过来, 客服系统提交后立刻知道"已收到", 不阻塞前端.

这个"写消息流"的动作意味着什么: Redis Stream 里的每条消息都带着客户 ID、会话 ID、内容, 像银行柜台的一叠业务单据. 后续任何一个 Worker 实例都能从单据堆里取走处理 — 这就是 Bot 服务可以水平扩展的基础: 加机器 = 加处理员, 单据不会丢.

3.2.1 客户端幂等 (双击/重试保护)

请求体支持可选 `client_message_id` (≤ 64 字符). 同一 `client_message_id` 重复提交:

```
# router.py:1151-1160 (简化)
client_idem_key = f"lumio:processed:{body.client_message_id}" if body.client_message_id else None
if client_idem_key and await redis_client.get(client_idem_key):
    # 已处理过 → 直接返回已接受, 不进 Stream (双击/网络重试不重复执行 agent)
    return ChatSendResponse(accepted=True, message_id=body.client_message_id, session_id=session_id)
```

- 幂等键与消费侧 `_mark_processed` (TTL 300s) 同一前缀, 覆盖 XAUTOCLAIM 60s 重投窗口
- 客户端重复 POST 但未带 `client_message_id` → 服务端生成新 ID, 作为两条独立消息处理 (可能触发队列合并)

3.2.2 per-customer 会话上限

同一客户多设备/多标签页无限开会话会耗尽 Redis state + 守卫资源. `chat_send` 用 ZSET `lumio:customer_sessions:{customer_id}` 计数 (score=最后活跃时间):

```
# 清理 24h 前记录 → zcard 计数 → 超上限 (默认 3) 拒绝新会话
if active >= get_settings().bot.max_sessions_per_customer:
    raise BusinessError(f"同时进行的会话数量已达上限 ({max_sessions}), 请先结束其他会话")
```

- 仅限制**新会话** (body 未带 `session_id`); 已带 `session_id` 的续聊不受影响
- 上限可配: `BOT_MAX_SESSIONS_PER_CUSTOMER` (默认 3)

3.3 Redis Stream 设计

先回答"为什么需要消息流": 银行客服是多实例部署 — 可能 3 台机器同时在跑 Bot 服务. 客户的消息发到哪台? 如果每台机器各自为政, 客户连发 3 条消息可能被 3 台机器同时处理, 回答乱套. 解决方式: 所有消息先进一个**公共的"消息池"** (Redis Stream), 任何实例从池子里取消息处理 — 谁有空谁取, 单条消息只被取走一次. 这就是"消息流 + 消费组"模式.

Stream 的数据模型: 每条消息是 (entry_id, fields) 二元组, entry_id 由 时间戳-序号 组成 (如 `1697000000000-0`), 全局唯一且按追加序递增; fields 是 `{session_id, message, customer_id, channel, ...}` 的键值对. 写入只能追加 (XADD), 读取按 id 序 (XREADGROUP), 上限 `MAXLEN 10000` 近似裁剪最旧条目 — 消费端整体故障时消息池持续堆高, 上限保证 Redis 内存有界 (宁可丢最旧, 不能让 Redis 内存爆掉拖垮整个系统, 见 3.3.3).

`agent/lumio/services/bot/router.py:68-194` 集中实现 Redis Stream 的 4 个关键概念.

3.3.1 Consumer Group — "取号叫号"

这段脚本在启动期初始化 Stream 与消费组: `XGROUP CREATE ... $ MKSTREAM` 表示从**当前时刻 (null id)** 之后的新消息开始建组, `MKSTREAM` 在 Stream 不存在时自动创建 (幂等, 已存在则抛错忽略):

```
# router.py:68-77
XGROUP_CREATE_SCRIPT = """
XGROUP CREATE lumio:chat:stream bot-group $ MKSTREAM
"""
```

Consumer Group `bot-group` 是 Redis 5.0+ 引入的多消费者协调机制. 多个 Worker 实例共享一个组, 每条消息**只被一个 Worker 消费** (at-least-once, "至少一次": 保证不丢, 但可能重复 — 见 3.3.2). 这等价于 Kafka 的 partition + consumer group, 但实现更轻.

打个比方: 消息池 = 银行叫号屏, 消费组 = 柜台的 10 个窗口. 每个客户 (消息) 被叫到一个窗口, 不会同时出现在两个窗口. 3 台机器上的 Worker 就是窗口, 谁空闲谁接待下一位.

3.3.2 PEL (Pending Entries List) + XAUTOCLAIM — "窗口突然关了怎么办"

业务场景: 窗口 (Worker) 正在处理客户消息时, 机器宕机了 — 这条消息算"处理完"还是"没处理"? 如果算处理完, 消息丢了, 客户白等; 如果算没处理, 谁来接手?

router.py:204-265 `_claim_stale` 处理"消费者崩溃后消息卡住"问题:

```
# 简化版
async def _claim_stale(self):
    """XAUTOCLAIM 兜底, 60s 超时 + 3 次重试转死信"""
    while True:
        claimed = await redis.xautoclaim(
            "lumio:chat:stream", "bot-group", "worker-{id}",
            min_idle_time=60_000, # 60s 未确认
            start_id="0-0",
            count=10,
        )
        for msg_id, fields in claimed:
            retry_count = await redis.hincrby("lumio:chat:retry_count", msg_id, 1)
            if retry_count > 3:
                await redis.xadd("lumio:chat:dead_letter", fields)
                await redis.xack("lumio:chat:stream", "bot-group", msg_id)
            else:
                await self._dispatch_message(msg_id, fields)
```

关键设计: 60s 未确认 → 自动接管, 最多重试 3 次 → 失败转死信 `lumio:chat:dead_letter`. 防止单 Worker 崩溃导致消息丢失.

原理: Redis 会给每条"被取走但没确认"的消息记一笔账 (PEL 待确认列表). 后台每 30 秒检查一次: 有消息超过 60 秒没确认, 说明原 Worker 可能挂了, 就把它"认领"过来重新处理. 重试 3 次还失败 → 进死信队列 (管理端可查看/重放, 见第 8 章), 客户消息不会无声消失.

"至少一次" (at-least-once) 的含义: 这条设计保证"消息不丢", 但代价是"可能重复处理" — 比如 Worker 处理到一半宕机, 重启后消息被重新投递, 客户可能收到两次回答. 所以消费端必须做**幂等** (同一消息只真正处理一次), 见 3.2.1 的 `client_message_id` 和消费侧的 `_mark_processed`.

3.3.3 MAXLEN 10000 — "单据堆不能无限高"

Stream 设上限 10000 条, 超长自动裁剪. 防止 Redis 内存无限增长. 在 dev 环境不重要, 生产环境是必选.

业务含义: 万一消费端整体故障 (比如 Redis 网络分区), 消息池会持续堆高. 上限保证 Redis 内存有界 — 宁可丢最老的消息, 不能让 Redis 内存爆掉拖垮整个系统.

3.4 消息处理流:从 HTTP 到结果的完整路径

在展开 3.5 (Worker) 之前,先把一条消息从 HTTP 进入到结果回写的整条路径捋清楚.后续每一节的子节都是这条主链上的一个环节.

```
sequenceDiagram
    participant C as 客户 App
    participant H as chat_send<br/>HTTP 入口
    participant S as Redis Stream<br/>lumio:chat:stream
    participant G as _consumer_loop<br/>消费组读取
    participant D as _dispatch_message<br/>路由派发
    participant Q as _session_queues<br/>per-session 队列
    participant W as _session_worker<br/>专属消费协程
    participant P as _process_message<br/>单条处理
    participant A as LumioAgent<br/>6 步决策
    participant R as _finish_message<br/>结果落库+通知
    participant P1 as poll/fast_reply<br/>客户端拉取

    C->>H: POST /api/chat/send
    H->>S: XADD 追加消息
    H-->>C: 202 Accepted<br/>(请求返回, 不等处理)

    Note over S,G: 异步消费端启动后持续运行
    G->>S: XREADGROUP bot-group
    S-->>G: msg_id + fields
    G->>D: create_task(_dispatch_message)
    D->>Q: 按 session_id 路由<br/>无则建队+启 Worker
    Q->>W: put(msg_id, fields)
```

```
W->>P: queue.get() (串行)
P->>P: ① 审计落库<br/>② 幂等检查<br/>③ 过期跳过<br/>④ trace 恢复
P->>A: agent.run(message)
A->>P: reply / intent / source
P->>R: _finish_message<br/>(+ XACK + mark_processed)
R->>P1: 写 lumio:response:{sid}<br/>+ PUBLISH lumio:notify:{sid}
P1->>C: 客户端轮询取走结果
```

主链上的 6 个角色 (后面 3.5/3.6 会逐个展开):

#	角色	代码位置	单一职责	协作方式
1	chat_send HTTP 入口	router.py:951	校验 + 写 Stream + 立刻 202	同步路径, 毫秒级返回
2	_consumer_loop 后台读取	router.py:78 起	XREADGROUP 阻塞读 + XAUTOCLAIM 兜底	长驻协程, 每实例 1 个
3	_dispatch_message 派发	router.py:328	按 session_id 路由到队列, 首条触发 Worker	短任务, 立刻结束
4	per-session Queue 串行化	router.py:85	同会话消息 FIFO	asyncio.Queue, 每会话 1 个
5	_session_worker 专属消费	router.py:347	串行取消息 + 调 Agent + XACK + 空闲退出	每活跃会话 1 个协程
6	_process_message 单条处理	router.py 内	幂等/过期/Audit/Trace/调 Agent/落结果	由 Worker 串行调用

谁先做、谁后做: 客户端只跟 ① 交互 (毫秒级), 后面 ②~⑥ 全是异步. 任何一个角色挂掉, 上一级都有兜底: Stream 数据持久化 → 60s 接管; Worker 崩溃 → 同会话队列被其他实例消费; Agent 超时 → 走快速兜底话术; 客户端断网 → 30s 内重连重试. 这就是 Bot 服务"任何单点失败都不丢消息、不卡客户"的工程基础.

3.5 per-session Worker: 串行消费

业务场景: 客户在对话框里连发 3 条消息 — "在吗?" → "我的卡丢了" → "怎么挂失?". 这 3 条几乎同时到达. 如果被 3 个 Worker 并行处理, 回答顺序可能变成"怎么挂失"的答案先到、"我的卡丢了"后到 — 客户看到的对话完全错乱. 更糟的是, 第 3 条消息的意图依赖前两条的上下文 ("怎么挂失"承接"卡丢了"), 顺序一乱, AI 理解就错.

解法: 给每个会话配一个**专属处理员** (per-session Worker), 该会话的消息永远只由它按顺序处理 — 像银行柜台一对一的服务专员, 一个客户全程一个专员接待, 谁先来谁先办.

为什么消费组还不够 — 顺序问题: Consumer Group 只保证"一条消息只被一个 consumer 读到", 不保证"同一会话的消息被同一 worker 按序处理". 批量 XREADGROUP 后消息经 `create_task` 并行分发, 同一会话的两条消息可能被两个协程同时处理 — 顺序颠倒, 且后一条的意图依赖前一条的上下文 ("怎么挂失"承接"卡丢了"), 顺序一乱 AI 理解就错.

3.5.1 Worker 是什么 — 协程,不是进程/线程

一句话定义: `_session_worker` 是 Bot 服务进程内的一个 `asyncio` 协程 (coroutine), 由事件循环调度, 跟同进程内的其他协程共用一个 OS 线程.

- **不是独立进程** — 几千个活跃会话 = 几千个协程, 全部跑在 Bot 服务那一个 Python 进程里, 共享一个事件循环;
- **不是独立线程** — `asyncio` 协程不占 OS 线程, 阻塞操作会让整个事件循环停摆 (这就是为什么下游 IO 全用 `await`);
- **不是常驻的** — 队列空 300 秒就自动 `return` 退出, 释放协程栈和队列内存, 新消息到来时按需重建;
- **不是有状态的** — 它的全部状态就是 `_session_queues[session_id]` 这个 `asyncio.Queue`, 进程重启即清空 (真正的会话状态在 Redis 的 `SessionState` 里, 不在 Worker 里).

3.5.2 Worker 的核心职责 — 5 步生命周期

`_session_worker` 被 `_dispatch_message` 首次触发, 进入以下循环, 直到空闲退出:

步骤	动作	关键代码	失败兜底
① 阻塞取消消息	<code>await asyncio.wait_for(queue.get(), timeout=300)</code>	<code>router.py:366</code>	<code>TimeoutError</code> → <code>break</code> 退出

步骤	动作	关键代码	失败兜底
② 审计落库	write_chat_message 写 PostgreSQL chat_messages 表	router.py:380-391	落库失败仅记日志, 不阻塞处理
③ 过期跳过	now - enqueue_time > message_ttl → 走超时话术 + 标 source=timeout	router.py:415-436	过期消息不浪费 LLM 调用
④ 调 Agent	await agent.run(session_id, message) 经 6 步决策返回结果	router.py:703	全局 wait_for 超时 → 走 timeout 话术
⑤ 落结果 + 确认	_finish_message 写结果缓存 + PUBLISH 通知 + XACK 标记消费完成 + _mark_processed 幂等键	router.py:692-698	任何一步异常都进入 finally 走 XACK, 避免 PEL 堆积

这 5 步里最关键的是 ④ 调 Agent — 它把"消息已被接收"的承诺, 兑现为"客户能看到的回答". ①~③ 是预处理 (审计/去重), ⑤ 是收尾 (通知/确认), 都是围绕 ④ 的支撑.

3.5.3 per-session Queue + Worker 的形态

每个 session_id 一个常驻 asyncio.Queue + 一个专属消费协程 (_session_worker). 消息经 _dispatch_message 路由进该会话的队列 (队列不存在则现场创建 Worker), 由它串行消费:

- 消费循环: queue.get() → 交给 Agent 处理 → XACK 确认 → 取下一条;
- 同一会话严格串行, 不同会话并行 — 客户消息按发送顺序处理, 各会话之间互不阻塞;
- 空闲回收: asyncio.wait_for(queue.get(), timeout=300) 5 分钟无消息 → Worker 退出并注销 (_session_queues.pop), 新消息到来时自动重建 — 高峰期 1000 个活跃会话对应 1000 个协程, 低谷自动收缩, 无常驻泄漏.

router.py:289-340 _session_worker 实现:

```
# 简化版 — 关键逻辑: 取一条 → 处理 + XACK; 300s 无消息退出
async def _session_worker(self, session_id: str) -> None:
    """per-session 串行消费, 300s 空闲自动退出"""
    queue: asyncio.Queue = self._session_queues[session_id]
    while True:
        try:
            msg_id, fields = await asyncio.wait_for(queue.get(), timeout=300.0)
        except asyncio.TimeoutError:
            # 300s 无消息, 退出 Worker 释放资源
            del self._session_queues[session_id]
            return
        try:
            await self._process_message(msg_id, fields)
        except Exception as exc:
            logger.exception("process message failed", msg_id=msg_id, exc=exc)
        finally:
            await redis.xack("lumio:chat:stream", "bot-group", msg_id)
            queue.task_done()
```

实现说明: 有序性不依赖锁 — 旧实现用 per-session Lock 串行化, 现改为每会话一个 asyncio.Queue (router.py:81-83), 队列天然 FIFO, 省去锁竞争.

这里有个取舍要讲清楚: 每个会话一个 Worker, 意味着一个客户的 3 条消息是排队逐个处理的 — 第 1 条处理完才轮到第 2 条. 如果每条要 2 秒, 第 3 条要等 6 秒. 会不会太慢? 答案看下一节 3.5.4 — 排队中的消息会被合并, 一次 AI 调用回答多条, 实际等待远小于逐条之和.

3.5.4 队列合并: 软打断语义

用户生成期间连发多条消息 → 不排队等待逐条处理, 而是合并进一次 LLM 调用 (上限 5 条 / 4000 字符, 超限放回队列不丢失):

```
# router.py:514-521 (简化)
merged_message = (
    f" (用户连续发送了 {len(merged_contents) + 1} 条消息, 请一次性综合回复) \n"
    + message + "\n" + "\n".join(merged_contents)
)
```

语义标记是关键: 前缀明确告知 LLM 这是多条连续消息, 避免把合并文本当成一句混乱的话. 被合并消息逐一审计落库 (source="merged") 并立即 XACK — 过期排队消息 (8s TTL) 直接跳过并提示.

对刚才场景的回答: 客户连发"在吗? / 我的卡丢了 / 怎么挂失?" — Worker 正在处理第 1 条时, 第 2、3 条进入队列. 处理第 1 条前先"看一眼队列", 把后面两条一起合并成一次调用: 大模型一次性看到三句连续的话, 完整理解"客户卡丢了要挂失", 一次回答到位. 客户不用等三条消息轮番处理.

这算"打断"吗? 算一种软打断: 用户最新的话不会被旧回答覆盖, 而是和新消息一起被综合处理. 真正的"硬打断" (正在生成的回答被取消) 在 WebSocket 流式通道实现 (ws_router.py 的 cancel 协议), 见后续章节.

3.5.5 快速兜底: Semaphore 满荷时的 <5ms 模板话术

业务背景: Bot 服务的 _agent_semaphore 是个全局并发上限 (默认 10, 见 config.py:max_concurrent_agents), 用来限制同时调 LLM 的消息数. 超过 10 个的请求本应该排队等, 但银行场景里"接起率"是死命令 — 让客户干等 = 拒客. 所以满荷时不排队, 走一个 "<5ms 出话"的模板兜底通道, 让客户先得到"已受理"的回复, 真回答等他轮到了再补.

走一个真实场景: 账单日早上 9 点, 全行客户同时涌入查账单, 流量是平时的 10 倍. 同一时刻第 11 个客户发来"挂失", 此时 _agent_semaphore.locked() 为 True (10 个槽位全被占了) — 系统不让他等, 直接走快速兜底.

3.5.5.1 三个判断条件 (任一不满足就跳过兜底)

```
# router.py:454-463 (简化)
cooldown_ok = True
cooldown_ts = await redis_client.get(f"lumio:fast_reply:{session_id}")
if cooldown_ts:
    if time.time() - float(cooldown_ts) < fast_reply_cooldown: # 默认 5s
        cooldown_ok = False

if _agent_semaphore.locked() and cooldown_ok:
    # 走快速兜底
```

条件	含义	不满足时的行为
_agent_semaphore.locked()	10 个槽位确实占满了	没满就正常走 Agent, 不触发兜底
cooldown_ok	这个会话过去 5s 内没走过兜底	在冷却期内 → 不再二次兜底, 改为正常排队等 Agent (宁可慢一点也别连发模板)
_quick_intent_match 命中	regex 至少匹配出 lost_card / bill_query / limit_query / complaint 之一	匹配不上 → default 话术

3.5.5.2 两套命名不一致 — domain 键对齐映射

为什么需要"对齐": 系统里有两套意图命名空间, 出自不同历史时期, 命名不统一:

- RuleLoader 返回的 domain (意图分类器的标准命名): card / bill / limit / installment / reward / transfer / chitchat / complaint / default
- 快速兜底话术的键名 _FAST_REPLIES (业务侧最在意的"是哪种急事"): lost_card / bill_query / limit_query / complaint / default

直接对应行不通: card 到底是"挂失"还是"补卡"还是"开卡"? limit 到底是"提额"还是"降额"还是"查额度"? 在兜底场景, 我们没时间调模型去细分, 只能粗粒度映射:

```
# router.py:114-124
_DOMAIN_TO_FAST_KEY = {
    "card": "lost_card", # 卡片类一律按"挂失"走 (最紧急)
    "complaint": "complaint", # 投诉 = 投诉
    "bill": "bill_query", # 账单类 = 查账单
    "limit": "limit_query", # 额度类 = 查/调额度
    "installment": "default", # 分期/积分/转人工/闲聊 → 没专属话术, 走通用模板
    "reward": "default",
    "transfer": "default",
    "chitchat": "default",
```

```
    "default": "default",
}
```

这就是 P2-18 修复的核心 (代码已合并, 见 router.py:111-124 , 测试覆盖 test_bot_router_core.py::test_quick_intent_match_domain_mapping): 早期代码直接用 intent 当 _FAST_REPLIES 的键查表, 结果 card 查不到 lost_card → 一律掉 default 话术, 挂失这种最紧急的紧急业务反而收到"请稍候"这种通用模板, 客户体验事故. 加了映射层之后, 挂失一定走挂失话术, 测试用例也明确锁定"挂失 → lost_card"、"未识别 → default"两条行为契约.

3.5.5.3 兜底话术内容 (为什么是这些文案)

```
# router.py:95-101
_FAST_REPLIES = {
    "lost_card": "挂失为紧急业务, 正在为您优先处理, 请稍候. 如超过 10 秒未回复, 请直接输入'转人工'。",
    "complaint": "您的投诉已记录, 正在转接人工处理。",
    "bill_query": "当前咨询量较大, 账单查询结果稍后返回, 也可输入'转人工'联系客服。",
    "limit_query": "您的问题正在处理中, 预计 30 秒内回复。",
    "default": "当前咨询量较大, 请稍候或输入'转人工'。",
}
```

话术设计要点	业务解释
明确告知"在处理中"	客户最怕"消息发出去没人理", 必须给确定性
给出明确等待时长	"10 秒 / 30 秒" 比 "稍候" 更可信 (时间感)
挂失提示"超过 X 秒请转人工"	兜底不是真办挂失, 必须给客户逃生通道 (转人工走另一条路径, 不再卡模板)
每句都带"转人工"	兜底本质是降级, 任何兜底客户都应该能立即跳到人工, 不被客户被模板话术困住

3.5.5.4 冷却期机制 (为什么 5s 内不再二次兜底)

问题: 兜底话术是"延迟响应", 真回答会在 Semaphore 有空时补上. 但补的过程里客户可能又发一条 ("在吗?"), 触发第二次兜底, 然后又补, 又触发 — 客户可能连续收到 3 条模板话术, 体验极差.

解法: 走完一次快速兜底后, 在 Redis 写一个时间戳 lumio:fast_reply:{session_id} (TTL 60s, 实际只用 5s):

```
# router.py:472
await redis_client.set(f"lumio:fast_reply:{session_id}", str(time.time()), ex=60)
```

5 秒内 (可配 fast_reply_cooldown) 同会话再进来 → cooldown_ok = False → 不走兜底, 改为正常排队等 Agent. 代价是这 5 秒内客户可能慢一点 (排队而不是秒回模板), 但保证不会被模板话术连击.

3.5.5.5 兜底的完整数据流

```
客户: "我的卡丢了"
↓
Worker → _process_message → 检查 Semaphore 满荷? 是
↓
cooldown_ok? 是 (5s 内没走过)
↓
_quick_intent_match("我的卡丢了") → RuleLoader.match → intent="card"
↓
_DOMAIN_TO_FAST_KEY["card"] → "lost_card"
↓
_FAST_REPLIES["lost_card"] → "挂失为紧急业务, 正在为您优先处理..."
↓
_finish_message(..., source="fast_reply")
├─ 写 lumio:response:{sid} (TTL 120s, 客户轮询拉走)
├─ PUBLISH lumio:notify:{sid} (长轮询立即返回)
└─ XACK 标记消费完成
↓
redis SET lumio:fast_reply:{sid} = now, ex=60 (冷却开始)
↓
指标 BOT_FAST_REPLY.inc()
↓
return (Worker 继续取下一条消息)
```

后台: Semaphore 有空 → 标准 Agent 路径处理 → 真正调 LLM → 真回答写入历史
(客户可能再发消息, 但因 5s 冷却, 走正常排队而不是二次兜底)

3.5.5.6 兜底不是真办业务 — 跟"真调 LLM"的边界

维度	快速兜底 (本节)	标准 Agent 路径 (3.6)
耗时	<5ms (regex + dict.get)	1~5s (LLM 调用 + 可能检索)
走的工具	无	RAG / MCP 工具 / pending_action
写不写历史	写 chat_messages 表 (source=fast_reply)	写 chat_messages 表 (source=knowledge/tool/business)
能不能触发转人工	兜底话术只建议"转人工", 不真转	视意图和规则, 满足条件就走 should_transfer
是不是终态	不是 — 真回答会在后续补上	是 — 终态回复

关键设计哲学: 快速兜底是对客户的"接了"承诺 (HTTP 200 + 立即看到话术), 不是对业务的"办了"承诺 (没调工具、没查知识、没改数据库). 真业务由后续标准路径补办. 客户视角: 体验上永远立刻有回应; 系统视角: 实际工作量在 Semaphore 释放后再批量消化, 不会因为流量尖刺把后端打穿.

3.6 LumioAgent 6 步决策树

业务背景: 消息到了 Worker, 现在要"真正思考"了. 但"思考"不等于"什么都交给大模型" — 银行场景里, 有些话不能乱说 (挂失要转人工), 有些事不能乱做 (调额要确认), 有些问题不能乱答 (知识要查库). 所以 LumioAgent 是一个规则优先的决策器: 能用规则判断的绝不让模型自由发挥, 模型只负责"分类"和"生成"两件最需要它的事.

用一个真实对话走一遍 6 步: 客户说"我想把额度提到 8 万".

步骤	做什么	结果
1 问候/告别快速路径	"在吗"/"再见" 不调模型, 直接回模板	本例跳过
2 待确认操作拦截	上轮是否挂了"调额待确认"? 有则先解读确认/取消	本例无
3 意图分类	规则 + 模型判断"想干嘛"	LIMIT_QUERY (提额)
4 路由	按意图分派: 查知识 / 办业务 / 调工具 / 闲聊	业务类
5 业务处理	敏感操作 → 确认状态机; 有工具 → 调工具	生成"确认调额"话术
6 兜底	以上任何一步失败 → 降级话术	仅在异常时

agent/lumio/services/bot/bot_agent.py:92 是 Bot 服务的核心, 类结构清晰:

```
# bot_agent.py:92 (简化)
class LumioAgent:
    """确定性路由主类, 纯规则引擎, LLM 仅用于内容生成"""

    def __init__(self, llm, retriever, mcp_client, ...):
        self.llm = llm
        self.retriever = retriever
        self.mcp_client = mcp_client
        self.slot_tracker = SlotTracker()
        self.memory = CustomerMemory()

    async def run(
        self, session_id: str, user_input: str, customer_id: str
    ) -> BotResponse:
        # 1. 快速问候/告别
        if is_greeting(user_input):
            return BotResponse(content=GREETING_RESPONSE, source="template")
        if is_farewell(user_input):
            return BotResponse(content=FAREWELL_RESPONSE, source="template")
```

```

# 2. pending_action 拦截
pending = await self.memory.get_pending_action(session_id)
if pending:
    return await self._handle_pending_action(session_id, user_input, pending)

# 3. 分类 (LLM, 3s 超时)
intent = await asyncio.wait_for(
    self._classify(user_input), timeout=3.0
)

# 4. 路由
if intent.label in KNOWLEDGE_INTENTS:
    return await self._handle_knowledge(session_id, user_input, intent)
if intent.label in BUSINESS_INTENTS:
    return await self._handle_business(session_id, user_input, intent)
if intent.label in TOOL_INTENTS and self.mcp_client.has_tool(intent.tool_name):
    return await self._handle_tool(session_id, user_input, intent)

# 5. 兜底
return BotResponse(content=FALLBACK_SYSTEM_PROMPT.format(input=user_input), source="fallback")

```

关键设计: 6 步严格顺序, 每步有 fallback. LLM 仅在第 3 步分类 + 第 4 步生成时用, 其他都是纯规则. 这就是"确定性路由主类"含义 — 业务流程可预测, LLM 不黑盒.

3.6.1 _handle_knowledge 知识类 (bot_agent.py:201–249)

```

# 简化
async def _handle_knowledge(self, session_id, user_input, intent):
    # 1. RAG 检索
    chunks = await self._retrieve(user_input)
    # 2. 历史摘要 (异步, 不阻塞)
    summary_task = asyncio.create_task(self._ensure_summary(session_id))
    # 3. 拼 prompt
    context = self._build_session_memory(session_id, user_input, chunks)
    # 4. LLM 生成
    answer = await self.llm.chat(context, system=KNOWLEDGE_SYSTEM_PROMPT)
    summary_task.cancel() # 取消未完成的摘要
    return BotResponse(content=answer, source="llm", chunks=chunks)

```

关键设计: 历史摘要与 LLM 生成并行, 摘要结果写入 Redis 供下一轮用. 这一轮不阻塞.

3.6.2 _handle_business 业务类 (bot_agent.py:250–340)

业务类更复杂, 因为涉及挂失/投诉/调额等**敏感操作**:

```

# 简化
async def _handle_business(self, session_id, user_input, intent):
    # 1. 危险操作直接转人工
    if intent.label == IntentLabel.CARD_LOSS:
        return await self._initiate_transfer(
            session_id,
            reason="card_loss",
            template=BUSINESS_TRANSFER_TEMPLATE["card_loss"]
        )

    # 2. 有工具 → 走 MCP
    if intent.tool_name and self.mcp_client.has_tool(intent.tool_name):
        return await self._handle_tool(session_id, user_input, intent)

    # 3. 无工具 → LLM 兜底生成
    answer = await self.llm.chat(user_input, system=BUSINESS_SYSTEM_PROMPT)
    return BotResponse(content=answer, source="llm")

```

关键设计: CARD_LOSS (挂失) 不让 LLM 处理, 直接走 _initiate_transfer 转人工. 银行场景下, 挂失是高风险操作, 不可能让 AI 自由发挥.

3.6.3 _handle_tool 工具类 (bot_agent.py:341–401)

工具类走 MCP 工具循环, 详见 [第 17 章 工具调用与确认状态机](#). 本节仅列关键摘要:

- **5 工具意图:** BILL_QUERY (账单查询) / TRANSACTION_QUERY (交易查询) / LIMIT_QUERY (额度查询) / INSTALLMENT_INQUIRY (分期咨询) / REWARD_QUERY (积分查询)
- **渐进式暴露:** select_tools_for_intent 按意图+置信度裁剪, 默认关闭 (零回归)
- **4 分支循环:** 无 tool_call / 护栏拒绝 / 敏感 pending / 非敏感执行
- **5 态确认:** pending / confirm / cancel / unclear / expired, 详细状态机见 17.4 节

3.6.4 _handle_pending_action 确认状态机 (bot_agent.py:402-535)

这是 Bot 设计最精彩的部分. 敏感工具 (挂失/调额/兑换) 不立即执行, 而是等**客户确认**. 详细 5 态状态机 / PendingAction 字段 / detect_confirmation 关键词优先级见 [第 17.4 节 5 态确认状态机](#). 本节仅给个高层摘要:

```
# 简化
async def _handle_pending_action(self, session_id, user_input, pending):
    decision = detect_confirmation(user_input)
    # decision ∈ {Confirm, Cancel, Unclear, Expired}

    if decision == Confirm:
        result = await self.mcp_client.call_tool(pending.tool_name, pending.arguments)
        await self.memory.clear_pending_action(session_id)
        return BotResponse(content=f"已执行 {pending.tool_name}: {result}", source="tool")

    if decision == Cancel:
        await self.memory.clear_pending_action(session_id)
        return BotResponse(content="已取消", source="template")

    if decision == Unclear:
        return BotResponse(content="请明确回复『确认』或『取消』", source="template")

    if decision == Expired:
        await self.memory.clear_pending_action(session_id)
        return BotResponse(content="操作已超时取消", source="template")
```

5 态状态机图:

```
stateDiagram-v2
    [*] --> pending: 敏感工具调用请求
    pending --> confirm: 客户说"确认"
    pending --> cancel: 客户说"取消"
    pending --> unclear: 客户回复不明确
    pending --> expired: 5min TTL
    confirm --> [*]: 执行工具
    cancel --> [*]: 清除 pending
    unclear --> pending: 提示再次确认
    expired --> [*]: 清除 pending
```

关键设计: pending_action 存 Redis (lumio:session:{id}:pending_action Hash), 跨轮持久化. 客户这一轮说"调额到 5 万", 下一轮说"确认", 就能定位到原工具调用. detect_confirmation (tool_executor.py:75) 解析自然语言, **cancel 关键词优先** (规避"不确认"歧义).

3.7 槽位追踪: SlotTracker

本章保留摘要, 槽位与上下文注入的联动见 [第 15 章 上下文工程](#) + [附录 A.4.1 上下文工程术语](#).

先讲个业务场景: 客户到柜台办"信用卡分期", 柜员得先填一张申请表: "分多少金额?" "分几期?" 客户不可能一次把信息说全, 柜员只能一项一项问. **槽位 (slot)** 就是这张申请表上的信息项 — 完成某个业务意图所需的关键信息, 一项就是一个槽. 客户一次说全 → 直接办; 说一半 → 机器人追问缺的那几项; 跨多轮补齐 → 才触发业务动作. 这套"一项一项收信息"的机制, 叫**槽位填充** (slot filling). 槽位还分两种: **必填槽** (缺了必须追问, 否则业务办不下去) 和**可选槽** (缺了不追问, 用默认值或直接跳过).

agent/lumio/services/bot/slot_tracker.py 实现**多轮槽位填充**. 以"办分期"为例 — 客户说"我想办分期", 一次说不全"分多少、分几期"两个必填信息:

```
# 简化
class SlotTracker:
    """槽位追踪, 跨轮持久化到 Redis lumio:slot:{session_id}"""
```

```
async def fill(self, session_id: str, intent: str, slots: dict) -> dict:
    """填充槽位，返回剩余待填槽"""
    current = await self._load(session_id, intent)
    merged = {**current, **slots}
    missing = self._check_required(intent, merged)
    await self._save(session_id, intent, merged, missing)
    return missing

async def is_complete(self, session_id: str, intent: str) -> bool:
    return len(await self._get_missing(session_id, intent)) == 0
```

关键设计: 客户说"我想办分期" → 缺 amount (分期金额) / period (分期期数) 两个必填槽 → Bot 按预设话术问"请问您想分期的金额是多少?" → 客户答"5000" → amount 已填, 还缺 period → 再问"您希望分几期?" → 客户答"分 12 期" → 齐了 → 触发分期工具调用 (分期申请是敏感写操作, 仍走 [5 态确认状态机](#))。槽位数据跨轮持久化在 Redis lumio:slot:{session_id}, 客户中途岔开问别的问题再回来, 已填的槽不丢。

7 意图 × 10 槽位映射表 (意图与槽位均标中文含义):

意图	意图含义	必填槽	可选槽
INSTALLMENT_INQUIRY	分期咨询 (办分期 / 查分期)	amount (分期金额) / period (分期期数)	—
BILL_QUERY	账单查询	—	period (账单周期)
LIMIT_QUERY	额度查询 / 提额	—	card_type (卡种)
CARD_LOSS	卡片挂失	card_tail (卡号后四位)	phone_number (预留手机号)
COMPLAINT	投诉	issue_detail (问题详情)	—
TRANSACTION_QUERY	交易明细查询	—	period (查询时段) / amount (交易金额)
REWARD_QUERY	积分查询	—	card_type (卡种)

注意: 槽位名是程序里的通用字段标识, 具体含义随意图变化 — 同是 amount, 在分期咨询里是"分期金额", 在交易查询里是"交易金额"; 同是 period, 在账单查询里是"账单周期", 在交易查询里是"查询时段"。必填槽缺失时 Bot 主动追问 (追问话术定义在代码 SlotDef.prompt), 可选槽缺失不追问。另注: CARD_LOSS 虽定义了验证槽位, 但当前实现里挂失是高风险操作, 直接转人工坐席验证, 不走槽位填充。

实体抽取 (entity_pool) 自动联动: PHONE → phone_number (手机号) / DATE → period (时间) 等 7 映射。完整实体 → 槽位映射 + 三段式 prompt 注入 (已收集/待收集/追问提示) 见 [第 15 章](#)。

3.8 3 级降级: 不拒客

业务背景: 每年信用卡账单日, 全行客户同时涌入查账单 — 流量是平时的 10 倍。更糟的是, 大模型服务可能恰好在这时被挤爆 (它比我们更忙)。这时候如果 Bot 直接返回"系统繁忙, 请稍后再试", 等于把客户拒之门外 — 银行客服的 KPI 是"接起率", 客户打不进电话是要上新闻的。

设计原则: 服务可以变差, 但不能拒绝。模型忙 → 用模板话术; 模板也没有 → 引导转人工; 人工也满 → 至少告诉客户"排队中"。层层兜底, 保证客户永远得到"回应", 而不是"拒绝"。

agent/lumio/services/bot/router.py:79, 92–98 实现了"Semaphore 满载不拒客"原则:

```
# 简化
class BotWorkerPool:
    def __init__(self, max_concurrent: int):
        self.semaphore = asyncio.Semaphore(max_concurrent) # 默认 10
        self._fast_replies = {
            "card_loss": "挂失请直接拨打 95xxx 或前往任一网点, 24 小时受理。",
            "overload": "系统繁忙, 您可以稍后再拨, 或直接前往任一网点。",
            ...
        }

    async def _process_with_limit(self, msg):
        if not self.semaphore.locked() or self.semaphore._value > 0:
            async with self.semaphore:
                return await self._process_message(*msg)
        # 满载走快速模板, 不阻塞客户
```

```
BOT_FAST_REPLY.inc()
return BotResponse(content=self._fast_replies["overload"], source="template")
```

关键设计: 银行客服高峰时段瞬时涌入 10x 流量, Semaphore 满载时**不能 503 拒客**, 走 _FAST_REPLIES 紧急话术. 客户在电话里听到"系统繁忙"是巨大舆情风险, 必须有降级话术兜底.

降级层次 (degradation.py 完整 4 级):

等级	触发	行为
NORMAL	健康	LLM → 检索摘要 → 模板
DEGRADED	LLM 熔断 2 次	跳过 LLM → 检索摘要 → 模板
FALLBACK	显式触发	跳过 LLM → 模板 (无检索)
Bot Semaphore 满	10 个并发	走 _FAST_REPLIES 紧急话术

3.9 文件上传 50MB 限制

业务背景: 客户传个 PDF (信用卡章程/流水单) 让 Bot 解读 — 这是知识库的入口, 也是 DoS 攻击的高发点. 如果不对上传设限, 1GB 的文件直接进 Redis Stream + 大模型, 内存和 token 都会被瞬间打爆.

agent/lumio/services/bot/router.py:1194-1270 upload_document 端点有双重限制:

```
# 简化
MAX_UPLOAD_SIZE = 50 * 1024 * 1024 # 50MB
ALLOWED_EXTENSIONS = {".pdf", ".docx", ".md", ".html", ".txt", ".xlsx"}

@router.post("/api/kb/documents")
async def upload_document(file: UploadFile, request: Request):
    # 1. Content-Length 头预检 (拒绝明显超限)
    if int(request.headers.get("content-length", 0)) > MAX_UPLOAD_SIZE:
        raise DocumentFormatError("文件超过 50MB 上限", code=2010)

    # 2. 扩展名白名单
    ext = Path(file.filename).suffix.lower()
    if ext not in ALLOWED_EXTENSIONS:
        raise DocumentFormatError(f"不支持的格式 {ext}", code=2010)

    # 3. 读 body 二次检查 (Content-Length 可能被伪造)
    content = await file.read()
    if len(content) > MAX_UPLOAD_SIZE:
        raise DocumentFormatError("文件超过 50MB 上限", code=2010)

    # 4. 上传 MinIO + 异步摄入
    ...
```

设计动机: 入口不设限, 1MB 消息 / 1GB 文件都能进 Redis Stream + LLM, 引发 DoS 风险. 因此 message max_length=2000 + 文件 50MB + 扩展名白名单三道关.

3.10 客户画像 + 知识图谱

本章保留摘要, 详细机制见 [第 16 章 客户记忆与知识图谱](#) + [附录 A.4.2 / A.4.3 术语](#).

bot/customer_memory.py 与 bot/knowledge_graph.py 实现跨会话增强:

- **CustomerMemory:** 跨 90 天 SQL string_agg 聚合, 推断卡种 (platinum/diamond/gold/standard) / VIP 等级 (private_banking/wealth_management/vip, max-score 评分) / 风险偏好 (R1-R4 累加). apply_learned_profile 用 CAS patch 写入 SessionState, 不覆盖已显式声明.
- **KnowledgeGraph:** 5 实体 (信用卡/账单/额度/分期/挂失) × 3 关系 × 8 谓词的内存版图谱, 仅在 _handle_knowledge 分支注入 RAG 上下文 (Markdown ## 知识图谱补充信息: 格式).

这些都是渐进增强, 当前不是核心路径. 触发时机 / 失败兜底 / 设计取舍 (正则 vs LLM / 90 天 window / 内存版 vs Neo4j) 见 [第 16 章](#).

3.11 监控指标

`shared/metrics.py` 发射 12 个 Bot 相关指标 (含 3 个全局复用) — 运维看这些指标就能回答"系统现在忙不忙、回答得好不好、有没有被拒":

指标	类型	含义	Labels	位置
<code>lumio_fast_reply_total</code>	Counter	并发满载时走了几次快速兜底话术 (越高说明越忙)	–	<code>router.py:393</code>
<code>lumio_agent_responses_total</code>	Counter	Bot 回答次数, 按来源分类 (llm=模型答/模板答/降级答/工具答)	source (llm/template/fallback/tool_*)	<code>router.py:566</code>
<code>lumio_bot_semaphore_utilization</code>	Gauge	并发槽位占用率 0~1 (接近 1 = 快满载了)	–	<code>router.py:811</code> (0~1)
<code>lumio_active_workers</code>	Gauge	当前活跃的会话处理员数 (高峰多、低谷少, 自动伸缩)	–	<code>router.py:810</code>
<code>lumio_stream_length</code>	Gauge	消息池积压量 (持续上涨 = 消费跟不上)	–	<code>router.py:809</code>
<code>lumio_stream_pending_total</code>	Gauge	待确认消息数 (挂起/崩溃的消息, 异常时上涨)	–	<code>router.py:808</code> (PEL pending)
<code>tool_confirmations_total</code>	Counter	敏感操作确认各状态次数 (确认/取消/不明确/过期)	decision (pending/confirm/cancel/unclear/expired)	<code>bot_agent.py:416/447/458/468</code>
<code>tool_calls_total</code>	Counter	工具调用次数与成败	tool, status	<code>tool_executor.py:298/309</code>
<code>tool_guard_denials_total</code>	Counter	工具被护栏拦截次数 (角色不够/金额超限)	tool, reason	<code>tool_executor.py:378</code>
<code>http_requests_total</code>	Counter	接口调用次数 (按方法/路径/状态码)	method, endpoint, status	全局
<code>http_request_duration_seconds</code>	Histogram	接口响应耗时分布 (看	method, endpoint	全局

指标	类型	含义	Labels	位置
		P99 是否超标)		
llm_call_duration_seconds	Histogram	大模型调用耗时 (看是不是 LLM 拖慢了回答)	model, method	llm.py:174/250

3.12 测试覆盖

agent/tests/ 中 Bot 相关测试:

- test_bot_api.py (e2e, CI 排除): 完整 chat 流程
- test_bot_agent_new.py (unit, 25+ 用例): LumioAgent 决策树
- test_bot_memory.py (unit, 24 用例): 客户记忆 + Redis
- test_confirmation.py (unit, 15 用例): 5 态确认状态机
- test_tool_guard.py (unit, 17 用例): 工具护栏白名单 + 额度
- test_chat_poll.py (e2e, CI 排除): 长轮询验证
- test_session_lifecycle_e2e.py (e2e, CI 排除): 会话生命周期

3.13 本章小结

Bot 自助问答服务是 Lumio 流量最大的入口。回到开头的银行场景, 现在你能完整回答"一条客户消息从发出到收到回答, 系统都做了什么":

1. 接了再说 (3.1 / 3.2) — 异步提交, 消息进 Redis Stream, 客户立刻收到 202
2. 不丢不乱 (3.3) — 消费组保证单条只被处理一次, 60s 兜底接管崩溃 Worker, PEL + XAUTOCLAIM 防丢
3. 路由派发 (3.4.3) — _dispatch_message 按 session_id 投到 per-session Queue, 首条触发 Worker
4. 专属消费 (3.5) — 每会话一个 _session_worker 协程, 同客户消息严格串行, 空闲 300s 退出
5. 单条处理 (3.5.2) — 审计落库 / 幂等检查 / 过期跳过 / 调 Agent / XACK, 5 步生命周期
6. 规则优先决策 (3.6) — LumioAgent 6 步: 先处理待确认操作, 再分类路由, 敏感操作等客户确认
7. 永不拒客 (3.8) — Semaphore 满荷走 5ms 模板兜底, 模型忙走降级话术, 层层保接起率

6 个角色一条链: chat_send (HTTP) → _consumer_loop (Stream) → _dispatch_message (路由) → _session_queues (FIFO) → _session_worker (消费) → _process_message (处理) → _finish_message (落结果). 任何一个角色挂掉, 上一级都有兜底 (Stream 持久化 / XAUTOCLAIM 接管 / Agent 超时走模板 / 客户端 30s 重连重试) — 这就是 Bot 服务"任何单点失败都不丢消息、不卡客户"的工程基础。

设计哲学一句话: 异步吸收流量, 队列保证有序, 规则保证安全, 降级保证可用 — 四条合起来, 就是银行客服系统"百万并发下不丢消息、不错顺序、不违规操作、不拒绝客户"的全部秘密。

下一章预告: [第 4 章 坐席辅助引擎](#) 深入 D1/D2/D3 + E1/E2/E3 五阶段编排 + 仲裁器融合 + WS 双模式。

延伸阅读: — [第 4 章 坐席辅助引擎](#) — 坐席侧互补设计 — [第 5 章 RAG 检索全链路](#) — _retrieve 内部细节 — [第 7 章 MCP 工具集成](#) — 22 工具如何接入 — [第 6 章 会话状态机](#) — pending_action 持久化 — [第 15 章 上下文工程](#) — 3 层上下文 + token 预算 + 增量摘要 — [第 16 章 客户记忆与知识图谱](#) — 跨会话画像 + 银行实体关系 — [第 17 章 工具调用与确认状态机](#) — 渐进式暴露 + 4 分支循环 + 5 态确认 + 双重护栏

04 坐席辅助引擎

第 4 章: 坐席辅助引擎

本章深入 Lumio 坐席辅助 (Assist) 引擎. 银行坐席与客户通话时, AI 实时推送话术/知识/合规提醒 — 这是 Lumio 与普通客服系统最大的差异化能力. 看完本章你会理解: D1/D2/D3 评估器怎么决定「推不推、推什么」, E1/E2/E3 执行器怎么并行生成内容, asyncio.gather 怎么替代 Temporal 编排 5 阶段, 仲裁器怎么融合 3 路避免冲突, WebSocket 双模式怎么对接生产.

4.1 引擎全景图

先讲个业务场景: 坐席正和客户通话 — 客户在抱怨"上个月账单怎么多了 200 块利息". 坐席的屏幕上, AI 辅助在实时干活: ① 判断这是"投诉期"还是"营销期" (客户在抱怨, 就别推信用卡优惠了); ② 给出话术建议 ("先致歉, 再解释利息计算方式"); ③ 检索账单相关条文; ④ 同时后台合规检查 ("别承诺免利息, 那违规"); ⑤ 一切就绪, 推送到坐席屏幕. 整个过程要求在**通话不中断的前提下 1-2 秒内完成**.

五阶段流水线 (对应下面这张图, 从下往上读): — **Phase 1 评估:** 先想清楚"现在是什么情况" — 客户意图、服务期/营销期/风控、紧急程度 — **Phase 2 执行:** 三条线并行干活 — 话术生成、营销推荐 (延迟 500ms, 别抢话)、风控检查 — **Phase 3 决策:** 综合判断"该不该推" — 刚推过话术? 30 秒内别重复打扰 — **Phase 4 仲裁:** 三路结果有冲突时定优先级 — 风控提醒 > 话术 > 营销 — **Phase 5 推送:** 脱敏 + 去重 + 推送到坐席屏幕

```
flowchart TB
    subgraph Input ["入口"]
        WS["WebSocket :8001<br>/api/ws/agent/{id}"]
        HTTP["POST /api/analyze<br>(chat-svc 回调)"]
    end

    subgraph Phase1 ["Phase 1: 评估 (Decision)"]
        Classify["classify (3s 超时)<br>IntentResult"]
        D1["evaluate_d1_service<br>服务期判断"]
        D2["evaluate_d2_marketing<br>营销期判断"]
        D3["evaluate_d3_risk<br>风控判断 (永不下线)"]
        Scene["detect_scene<br>URGENT/INQUIRY/SALES/GENERAL"]
    end

    subgraph Phase2 ["Phase 2: 执行 (Execution)"]
        E1["run_e1_ai<br>话术 + RAG + 合规"]
        E2["run_e2_marketing<br>500ms 延迟"]
        E3["run_e3_risk<br>alert_engine 6 规则"]
    end

    subgraph Phase3 ["Phase 3: 决策 (Should Show)"]
        Push["PushTracker<br>last_push + min_interval"]
        Should["should_show<br>5 规则决策"]
    end

    subgraph Phase4 ["Phase 4: 仲裁 (Arbitrate)"]
        Arb["GlobalArbitrator<br>3 融合策略<br>BLOCK/WARN/PASS"]
        PII["PII 脱敏"]
        Compliance["合规短语过滤"]
    end

    subgraph Phase5 ["Phase 5: 推送 (Push)"]
        Dedup["Redis 去重<br>lumio:ae:dedup:{trace_id}"]
        WSPush["WebSocket 推送<br>{type: assist_push, payload}"]
        Track["PushTracker.record_push"]
    end

    HTTP --> Classify
    WS --> Classify
    Classify --> D1
    Classify --> D2
    Classify --> D3
    Classify --> Scene
    D1 --> Should
    D2 --> Should
    D3 --> Should
    Scene --> Should
    Should --> Arb
    Arb --> PII
    PII --> Compliance
    Compliance --> Dedup
    Dedup --> WSPush
    WSPush --> Track
    Track --> End
```

```

D2 --> Should
D3 --> Should
Scene --> Should

D1 --> E1
D2 -.delay 500ms.-> E2
D3 --> E3

E1 --> Arb
E3 --> Arb
E2 -.delayed.-> Arb

Arb --> PII
Arb --> Compliance
Should --> WSPush
Arb --> Dedup
Dedup --> WSPush
WSPush --> Track

```

4.2 入口: POST /api/analyze 与 WebSocket

agent/lumio/services/assist/router.py:297 analyze_message 是生产主入口, chat-svc 回调使用:

```

# router.py:297 (简化)
@router.post("/api/analyze", dependencies=[Depends(require_role("service", "admin"))])
async def analyze_message(body: AnalyzeRequest, request: Request) -> dict:
    """生产级入口: chat-svc 上传客户消息, 引擎返回推送"""
    trace_id = request.headers.get("X-Trace-Id", str(uuid4()))

    # 1. 幂等检查 (防止重试)
    if await redis.get(f"lumio:ae:dedup:{trace_id}"):
        return {"idempotent": True, "payload": None}

    # 2. 加载状态快照
    snapshot = await load_state_snapshot(body.session_id)

    # 3. 5 阶段编排 (核心)
    try:
        result = await asyncio.wait_for(
            run_assist_engine(snapshot, body, trace_id),
            timeout=5.0, # 引擎整体 5s 超时
        )
    except asyncio.TimeoutError:
        # 兜底: 返回空 payload, 不阻塞通话
        result = AssistResult(payload=None, error="engine_timeout")

    # 4. 幂等记录
    await redis.setex(f"lumio:ae:dedup:{trace_id}", 30, "1")
    return result

```

WebSocket 双模式 (router.py:808 per-session 测试, router.py:866 per-agent 生产):

```

# router.py:866 (简化)
@router.websocket("/api/ws/agent/{agent_id}")
async def assist_websocket(websocket: WebSocket, agent_id: str):
    """per-agent 生产: 一个坐席上班期间一条长连接"""
    await websocket.accept()

    # 1. JWT 验证
    user = await authenticate_websocket(websocket)

    # 2. 订阅推送
    pubsub = redis.pubsub()
    await pubsub.subscribe(f"lumio:assist:notify:{agent_id}")

    # 3. 循环
    while True:
        msg = await pubsub.get_message(ignore_subscribe_messages=True, timeout=30)
        if msg:
            await websocket.send_json(json.loads(msg["data"]))

```

关键设计: 生产用 per-agent 而非 per-session。一个坐席一天处理 30-50 个客户, 如果 per-session 重连 30-50 次, 反而浪费。Per-agent 一次性连接, 推送通过 Redis Pub/Sub 路由。

4.3 评估器 D1/D2/D3

agent/lumio/services/common/decision.py 集中三个评估器, 命名规则:

- D = Decide (决策)
- 1/2/3 = 服务/营销/风控 三个业务域

4.3.1 D1 服务评估器 (decision.py:123-180)

```
# 简化
async def evaluate_d1_service(state_snapshot) -> D1Result:
    """判断当前是否在『服务期』, 服务期内不推营销"""
    intents = state_snapshot.intent_stack[-3:] # 最近 3 个意图
    # 服务类意图 -> 服务期
    if any(i in SERVICE_INTENTS for i in intents):
        return D1Result(active=True, suppress_marketing=True, rounds=2)

    # 风控类意图 -> 紧急期 (最高优先)
    if any(i in RISK_INTENTS for i in intents):
        return D1Result(active=True, suppress_marketing=True, rounds=5, urgent=True)

    return D1Result(active=False)
```

关键设计: `suppress_marketing=True` + `rounds=2` 表示: 服务期内接下来 2 轮都压制营销推送. 这避免"客户说挂失, AI 推信用卡年费优惠"的灾难场景.

4.3.2 D2 营销评估器 (decision.py:182-256)

```
# 简化
async def evaluate_d2_marketing(state_snapshot) -> D2Result:
    """判断当前是否推营销, 走产品目录匹配"""
    if state_snapshot.suppress_flag:
        return D2Result(active=False, reason="suppressed_by_d1")

    # 客户情绪 + 意图匹配
    sentiment = state_snapshot.emotion_vector.dominant
    intent = state_snapshot.intent_stack[-1] if state_snapshot.intent_stack else None

    # 找到匹配的产品
    products = product_catalog.find_matches(intent=intent, sentiment=sentiment)
    if not products:
        return D2Result(active=False, reason="no_match")

    return D2Result(
        active=True,
        products=products,
        defer_ms=500 if state_snapshot.service_active else 0, # 服务期延迟 500ms
    )
```

关键设计: `defer_ms=500` 表示服务期激活时, 营销推送延迟 500ms 让服务提示先出. 这是体感优化细节.

4.3.3 D3 风控评估器 (decision.py:258-310)

```
# 简化
async def evaluate_d3_risk(_state_snapshot) -> D3Result:
    """风控永不下线, 任何时候都跑"""
    return D3Result(active=True, severity="always")
```

关键设计: D3 是唯一永不下线的评估器. 合规要求: 即使 LLM 挂了, 即使所有降级, 风警告警必须能推. 任何 `should_show` 决策中, `risk` 永远是硬规则.

4.4 执行器 E1/E2/E3

评估后, 执行器并行生成内容:

4.4.1 E1 AI 执行器 (ai_executor.py:23-180)

```
# 简化
class AIExecutor:
    """三路并行: 话术 + RAG + 合规短语"""

    async def run(self, snapshot) -> E1Result:
        # 三路并行
        script_task = asyncio.create_task(self._script_service.match(snapshot))
        rag_task = asyncio.create_task(self._retriever.retrieve(snapshot.query))
        compliance_task = asyncio.create_task(self._safety.check(snapshot.text))

        # gather + 全部成功才返回
        script, chunks, compliance = await asyncio.gather(
            script_task, rag_task, compliance_task
        )

        if not compliance.safe:
            return E1Result(blocked=True, reason=compliance.hit_words)

        # LLM 生成话术 (用 chunks 作为上下文)
        answer = await self.llm.chat(snapshot.query, context=chunks)
        return E1Result(answer=answer, script=script, chunks=chunks)
```

关键设计: 三路严格并行, 任一失败不影响其他. `compliance` 失败直接 block, 这是 PII + 合规的双重保险.

4.4.2 E2 营销执行器 (marketing_executor.py:19-150)

```
# 简化
class MarketingCard:
    """营销卡片生成"""

    def __init__(self, product_catalog, llm):
        self.products = product_catalog
        self.llm = llm

    async def evaluate(self, intent, sentiment, customer_id) -> list[MarketingCard]:
        products = self.products.find_matches(intent=intent, sentiment=sentiment)
        if not products:
            return []

        # 个性化文案
        cards = []
        for p in products[:2]: # 最多 2 张卡片
            copy = await self.llm.generate(
                prompt=f"为产品 {p.name} 生成 1 句营销话术, 客户 {customer_id}, 意图 {intent}",
            )
            cards.append(MarketingCard(product=p, copy=copy))
        return cards
```

4.4.3 E3 风控执行器 (alert_engine.py:76-220)

```
# 简化
class AlertEngine:
    """质检告警引擎: 6 条种子规则"""

    RULES = [
        Rule(id="R-COMP-001", name="禁止承诺", check=contains_promise_words),
        Rule(id="R-COMP-002", name="禁止透露他人信息", check=mentions_other_customer),
        Rule(id="R-COMP-003", name="情绪异常升级", check=negative_sentiment_threshold),
        Rule(id="R-EMOTION-001", name="客户愤怒", check=anger_score_gt_0.8),
        Rule(id="R-SILENCE-001", name="长时间沉默", check=silence_duration_gt_30s),
        Rule(id="R-PROCESS-001", name="未走完流程", check=missed_required_step),
    ]

    async def evaluate(self, snapshot) -> list[Alert]:
        alerts = []
        for rule in self.RULES:
            if await rule.check(snapshot):
                alerts.append(Alert(rule=rule, severity=rule.severity))
        return alerts
```

6 条种子规则: – R-COMP-001/002: 合规类, 承诺/泄密 – R-EMOTION-001: 情绪类, 愤怒阈值 – R-SILENCE-001: 沉默类, 30s+ 静默 – R-PROCESS-001: 流程类, 漏步骤

4.4.4 情绪告警与转人工接线 (Phase 5b)

引擎 Phase 5b 状态回写时, 情绪检测完整接线 — 愤怒客户不再只被压制营销:

```
# assist_engine.py Phase 5b (简化)
if sentiment and sentiment != "neutral":
    state_patches["emotion_vector"] = {"label": sentiment, "score": sentiment_score, ...}
    # 1) 情绪告警: alert_engine.check_sentiment → 坐席提示话术
    emotion_alerts = alert_engine.check_sentiment(sent_label)
    # 2) 强负面 → 转人工建议: emotion_transfer.should_transfer_by_emotion
    #     (angry+conf>0.7 / negative+conf>0.85 / desperate 直接转)
    if transfer_flag:
        state_patches["emotion_transfer_suggested"] = True
        state_patches["emotion_transfer_reason"] = transfer_emo_reason
    state_patches["emotion_alerts"] = emotion_alerts
```

落 state 而非直推 WS: 告警写入 SessionState (emotion_alerts / emotion_transfer_suggested), 坐席端 assist_ready / 轮询读取后展示 — 避免引擎与 WS 连接池耦合, 多实例下也可靠.

情绪转人工规则 (common/emotion_transfer.py):

情绪	置信度门槛	动作
DESPERATE (绝望)	无门槛	立即转人工 (投诉升级)
ANGRY (愤怒)	> 0.7	转人工建议
NEGATIVE (负面)	> 0.85	转人工建议

4.5 5 阶段编排: asyncio.gather 替代 Temporal

agent/lumio/services/common/assist_engine.py:171-220 是核心入口:

```
# assist_engine.py:171 (简化)
async def run_assist_engine(snapshot, body, trace_id) -> AssistResult:
    # Phase 1: 评估器并行
    classify_task = asyncio.create_task(classifier.classify(body.text, timeout=3.0))
    d1_task = asyncio.create_task(evaluate_d1_service(snapshot))
    d2_task = asyncio.create_task(evaluate_d2_marketing(snapshot))
    d3_task = asyncio.create_task(evaluate_d3_risk(snapshot))

    try:
        classify, d1, d2, d3 = await asyncio.wait_for(
            asyncio.gather(classify_task, d1_task, d2_task, d3_task),
            timeout=2.0,
        )
    except asyncio.TimeoutError:
        # 分类超时用缓存, D1/D2/D3 至少 D3 必返回
        classify = await classifier.classify_cached(snapshot.session_id) or classify_fallback()
        d1, d2 = D1Result(active=False), D2Result(active=False)
        d3 = D3Result(active=True, severity="always")

    # Phase 2: 执行器并行
    e1_task = asyncio.create_task(e1_executor.run(snapshot))
    e3_task = asyncio.create_task(e3_alert_engine.evaluate(snapshot))
    e1, e3 = await asyncio.gather(e1_task, e3_task)

    # E2 延迟 500ms 避开服务期
    if d2.active and d2.defer_ms > 0:
        await asyncio.sleep(d2.defer_ms / 1000)
    e2 = await e2_marketing.evaluate(classify.intent, classify.sentiment, snapshot.customer_id)

    # Phase 3: 决策 should_show
    scene = detect_scene(classify.intent, classify.sentiment)
    primary = should_show("script", scene, snapshot, e1) # 主推
    risk = should_show("risk", scene, snapshot, e3, force=True) # 风控必推
```

```
marketing = should_show("marketing", scene, snapshot, e2)

# Phase 4: 仲裁
payload = GlobalArbitrator.arbitrate(primary, risk, marketing, snapshot)

# Phase 5: 推送 + 追踪
if payload:
    await push_tracker.record_push(snapshot.session_id, payload)
    return AssistResult(payload=payload, trace_id=trace_id)
```

关键设计: 1. `asyncio.gather` 并行评估器: 4 个评估 200ms 内全返回 2. 超时 `fallback`: 任一超时, D3 仍必返回 (风控永不下线) 3. E2 延迟 500ms: 服务期内不抢风头 4. `should_show` 5 规则: 见下 5. 仲裁融合: 3 路合并去重

4.6 决策矩阵: `should_show` 5 规则

agent/lumio/services/common/decision.py:258-310 :

```
# 简化
def should_show(
    card_type: str, # "script" / "risk" / "marketing"
    scene: Scene,
    tracker: PushTracker,
    payload: Any,
    force: bool = False,
) -> Optional[Any]:
    """5 规则决策"""
    # 规则 1: 硬规则 - 风控 BLOCK 必推, PASS 不推
    if card_type == "risk":
        if payload.severity == "BLOCK":
            return payload
        if payload.severity == "PASS":
            return None

    # 规则 2: 强制推 (D3 风控场景)
    if force:
        return payload

    # 规则 3: 时间窗 - 距上次推送 < min_interval 不推
    if tracker.last_push_at and (now - tracker.last_push_at) < tracker.min_interval:
        return None

    # 规则 4: 反馈历史 - 客户连续 3 次 dismiss 不推
    if tracker.consecutive_dismiss >= 3:
        return None

    # 规则 5: 场景适配 - SALES 场景不推知识, INQUIRY 场景不推营销
    if card_type == "marketing" and scene == Scene.INQUIRY:
        return None
    if card_type == "script" and scene == Scene.SALES:
        return None

    return payload
```

5 规则总结:

规则	类型	适用
1 硬规则	阻断/放行	风控永远特殊
2 强制	覆盖	D3 场景
3 时间窗	避免刷屏	全部
4 反馈	客户体验	全部
5 场景适配	业务正确性	全部

4.7 仲裁器: 3 融合策略


```
# 简化
class GlobalArbitrator:
    """3 融合策略 + PII 脱敏 + 合规短语过滤"""

    @staticmethod
    def arbitrate(primary, risk, marketing, snapshot) -> Optional[Payload]:
        # 1. 风控最高优先: BLOCK 阻断一切, WARN 警告
        if risk and risk.severity == "BLOCK":
            return Payload(
                type="risk_block",
                content=risk.message,
                fusion_type="BLOCK",
                suppresses=("script", "marketing"),
            )

        # 2. 营销 vs 主推冲突
        if marketing and primary and conflict(primary, marketing):
            # 主推胜出, 营销降级
            marketing = downgrade(marketing)
            ASSIST_ARBITRATION_CONFLICT.inc()

        # 3. PII 脱敏
        if primary:
            primary.content = mask_pii(primary.content)
        if marketing:
            marketing.copy = mask_pii(marketing.copy)

        # 4. 合规短语过滤
        primary.content = filter_compliance_phrases(primary.content)
        marketing.copy = filter_compliance_phrases(marketing.copy)

        # 5. 拼装 payload
        return Payload(
            type="assist_push",
            primary_card=primary,
            risk_badge=risk if risk and risk.severity == "WARN" else None,
            marketing_slot=marketing,
            fusion_type="WARN" if risk and risk.severity == "WARN" else "PASS",
        )
```

3 融合策略:

策略	含义	行为
BLOCK	阻断	只显示风警告警, 压制所有其他推送
WARN	警告	主推 + 风控徽章 + 营销降级
PASS	通过	正常 3 路全推

4.8 3s 延迟反馈

```
# router.py:630 (简化)
@router.post("/api/feedback")
async def record_feedback(body: FeedbackRequest, request: Request):
    """3s 延迟确认: 写 Redis 缓冲, 期间可撤销"""
    # 1. 写 Redis 缓冲
    buffer_key = f"lumio:assist:feedback:{body.session_id}:{body.agent_id}"
    await redis.setex(buffer_key, 3, json.dumps(body.dict()))

    # 2. 3s 后异步提交
    async def commit_after_delay():
        await asyncio.sleep(3.0)
        # 提交到 push_tracker
        await push_tracker.record_feedback(body.session_id, body)
        await redis.delete(buffer_key)
```

```
asyncio.create_task(commit_after_delay())
return {"status": "buffered", "undo_window": 3}
```

关键设计: 客户点"忽略/没用"后, 坐席想撤销, 有 3s 窗口. 这比立即提交友好, 符合人类工程学 (Human-Computer Interaction).

4.9 话后小结生成

agent/lumio/services/assist/summary.py:33-180 generate_call_summary 通话结束自动生成:

```
# summary.py:33 (简化)
async def generate_call_summary(session_id, session_manager, llm):
    """话后小结, LLM JSON 模式"""
    # 1. 拉取完整对话历史
    history = await session_manager.get_full_history(session_id)

    # 2. LLM 生成结构化 JSON
    prompt = f"""根据以下通话, 生成结构化小结:
    客户意图: ...
    坐席操作: ...
    风险事件: ...
    客户反馈: ...

    {history}

    输出 JSON: {{ "summary": "...", "key_points": [...], "action_items": [...], "risk_flags": [...] }}
    """
    response = await llm.chat_json(prompt, schema=CallSummary)

    # 3. 异步落库
    await dialogue_log_repo.insert(session_id, response)
    return response
```

关键设计: LLM JSON 模式 (response_format=json_object) + Pydantic CallSummary 校验, 失败重试 1 次.

4.10 编排方式: asyncio.gather 而非工作流引擎

Assist 引擎的 D/E 五阶段 (D1/D2/D3 + E1/E2/E3) 用 asyncio.gather 并行编排, 不引入外部工作流引擎. 关键决策依据:

- 1. 运维成本: 工作流引擎需独立 Server + Namespace + Worker, 部署复杂
- 2. 调试: 外部工作流引擎的 trace 不能在 Jaeger 直接看
- 3. 依赖: 多语言 SDK 双维护
- 4. 业务规模: 5 阶段 < 5s, 协程足够

选择纯 asyncio 编排的好处: - 少一个外部服务 - 跨服务 trace 全在 Jaeger, 调试更直观 - 代码量更小

4.11 监控指标

Assist 引擎发射 8 个核心指标 (assist_engine.py:47-120) — 回答"坐席辅助用得怎么样、有没有推错、有没有拖慢通话":

指标	类型	含义	Labels
lumio_assist_engine_decisions_total	Counter	每轮场景推/压制的次数 (push=推给坐席, suppress=被压制)	scene, decision (push/suppress)
lumio_assist_engine_latency_seconds	Histogram	每个阶段的耗时分布 (看哪一阶段慢: 分类/评估/执行/仲裁)	phase (classify/d1/d2/d3/e1/e2/e3/arbitrate)
lumio_assist_engine_degradation_total	Counter	降级触发次数 (AI 挂了/熔断开/超时, 按原因分)	agent, reason (ai/risk/no_executor/breaker_open/timeout)
http_requests_total / http_request_duration_seconds	全局	接口调用与耗时	

指标	类型	含义	Labels
session_transitions_total	Counter	会话状态转换 (复用第 6 章)	5 labels
session_phase_duration_seconds	Histogram	各子阶段停留时长 (排队多久/通话多久)	sub_phase
tool_guard_denials_total	Counter	工具护栏拦截 (复用第 17 章)	tool, reason
llm_call_duration_seconds	Histogram	大模型调用耗时	model, method

4.12 测试覆盖

agent/tests/ 中 Assist 相关:

- test_assist_engine.py (20+ 用例): 5 阶段编排 + 降级
- test_arbitrator.py (22 用例): 3 融合策略
- test_decision.py (44 用例, TOP 1): D1/D2/D3 + should_show 决策表
- test_alert_engine.py (15 用例): 6 条种子规则
- test_script_service.py (12 用例): 话术服务
- test_product_catalog.py (10 用例): 4 种子产品
- test_assist_api.py (e2e, CI 排除): 完整 WS 流程
- test_observability.py (18 用例): 指标 + tracing

4.13 本章小结

坐席辅助引擎是 Lumio 区别于普通客服系统的核心:

- D1/D2/D3 评估器: 服务/营销/风控三域, 风控永不下线
- E1/E2/E3 执行器: AI 三路并行, E2 延迟 500ms 避服务期
- 5 阶段编排: asyncio.gather 替代 Temporal, 节省 1 个外部服务
- should_show 5 规则: 硬规则 + 强制 + 时间窗 + 反馈 + 场景适配
- 仲裁器 3 融合: BLOCK / WARN / PASS, PII + 合规双重保险
- WebSocket 双模式: per-session 测试 + per-agent 生产
- 3s 延迟反馈: H2 人类工程学, 坐席可撤销

下一章预告: [第 5 章 RAG 检索全链路](#) 深入 BM25 + 向量 + RRF 融合 + 父-子分块 + 双写回滚.

延伸阅读: - [第 3 章 Bot 自助问答](#) - 互补入口 - [第 5 章 RAG 检索全链路](#) - E1 中的 RAG 检索 - [第 6 章 会话状态机](#) - 状态快照 - [第 7 章 MCP 工具集成](#) - E1 中的工具调用 - [第 17 章 工具调用与确认状态机](#) - 渐进式暴露 + 5 态确认 + 双重护栏

05 RAG 检索全链路

第 5 章: RAG 检索全链路

本章深入 Lumio RAG (检索增强生成) 全链路. 银行客服 80% 的问题靠知识库回答 — "信用卡年费多少? 怎么免年费? 积分怎么用?", RAG 链路质量直接决定客户体验. 看完本章你会理解: BM25 + 向量 + RRF 三路召回怎么互补, 父-子分块怎么同时满足检索粒度 + 生成粒度, ES + Milvus 双写 + 回滚怎么保证一致性, 银行合规字段怎么硬注入, 4 路径降级矩阵怎么在 ES/Milvus 不可用时仍服务.

5.1 检索全景图

怎么读这张图: 客户问"信用卡年费多少", 系统要做四件事 — ① 先看缓存 (5 分钟内问过同样问题? 直接复用); ② 加合规过滤 (只查已发布、当前版本的条文, 草稿和旧版绝不能答给客户); ③ 双通道检索 (关键词搜索 + 语义搜索, 谁都不能全信); ④ 精排 + 回填缓存.

```
flowchart LR
    Query[用户查询] --> Cache{Redis 缓存<br/>TTL 300s}
    Cache -->|命中| Return[直接返回]
    Cache -->|未命中| Compliance[合规字段注入]

    Compliance --> Hybrid{search_type}
    Hybrid -->|hybrid| BM25[search_bm25<br/>ES + IK]
    Hybrid -->|bm25| BM25
    Hybrid -->|vector| Vector[search_vector<br/>Milvus]

    BM25 --> Embed1[embed_query<br/>1024 维]
    Vector --> Embed2[Embed]
    Embed --> RRF[RRF 融合<br/>k=60]
    RRF --> Rerank[Reranker 精排<br/>top_k*2 -> top_k]
    Rerank --> Filter[Filter[Milvus 后过滤<br/>approval_status + is_current_version]]
    Filter --> Cache2[Redis 缓存写回<br/>TTL 300s]
    Cache2 --> Return

    style Cache fill:#f9e,stroke:#333
    style Compliance fill:#ffe,stroke:#333
    style Hybrid fill:#efe,stroke:#333
    style RRF fill:#fee,stroke:#333
    style Rerank fill:#eef,stroke:#333
```

5.1.1 双通道检索 — "为什么两条路都要走"

客户问"年费多少", 有两种找法:

- **BM25 关键词检索 (ES)**: 像用 Ctrl+F 搜文档 — 找"年费"两个字在哪出现. 优点: 快、准、不依赖模型; 缺点: 客户说"一年收多少钱"就搜不到"年费"
- **向量语义检索 (Milvus)**: 像让 AI 理解意思 — "一年收多少钱"和"年费"在语义上相近, 能匹配上. 优点: 懂同义词、口语; 缺点: 依赖模型质量, 可能跑偏

单走一条路的后果: 只走关键词, 客户换种说法就查不到 (体验差); 只走语义, 生僻专业词可能匹配错 (风险高). 两条路都走, 再用 RRF 融合排序 — 两条路都找到的条文排最前, 只有一条路找到的靠后. 这就是**三路召回 + 融合**的核心价值.

5.2 入口: RetrieveRequest

agent/lumio/services/common/retrieval.py:400-410 是 RAG 入口:

```
# retrieval.py:400 (简化)
class RetrieveRequest(BaseModel):
    query: str
    top_k: int = 5
    search_type: Literal["hybrid", "bm25", "vector"] = "hybrid"
```

```
filters: dict | None = None
rerank: bool = True
use_reranker_threshold: bool = False
confidence_threshold: float = 0.5
```

5 字段核心: – query: 用户原始查询 – top_k: 返回数量, 默认 5 – search_type: 3 种检索模式 (hybrid / bm25 / vector) – filters: ES 过滤器 (keyword/date/keywords 三类, retrieval.py:48–80) – rerank: 是否启用重排序

5.3 4 路径降级矩阵

业务场景: 检索依赖两个系统 — ES (关键词) 和 Milvus (语义). 生产环境任何中间件都可能挂: ES 磁盘满、Milvus 的 etcd 抖动、嵌入模型超时. 客户可不管这些 — 他只知道“我问了问题, 系统得答”. 所以检索的设计原则和 Bot 一致: 能答多少答多少, 绝不因为一个组件挂了就整个瘫痪.

4 种故障场景的应对 (对应下面的矩阵): – 都正常 → 双通道 + 融合, 质量最好 – Milvus 挂了 → 只用关键词检索 (BM25) — 客户换个说法可能搜不到, 但常见问法没问题 – ES 挂了 → 只用语义检索 — 生僻词可能不准, 但大部分问题能答 – 都挂了 → 返回空, 由上层降级链兜底 (第 3 章讲过的模板话术)

retrieval.py:419–425 是核心降级矩阵:

```
# retrieval.py:419 (简化)
async def retrieve(request, es_client, milvus_collection, embedding_provider, reranker, redis_client):
    # 0. 缓存检查
    cache_key = _build_cache_key(request.query, request.filters, request.search_type)
    cached = await redis_client.get(cache_key)
    if cached and request.search_type != "vector": # vector_only 不缓存
        return json.loads(cached)

    # 1. 合规字段注入 (强制, 任何路径都加)
    es_filters = build_es_filters(request.filters)
    es_filters["approval_status"] = "PUBLISHED"
    es_filters["is_current_version"] = True
    es_filters["effective_date_lte"] = today_iso

    # 2. 三路检索 (按 search_type)
    bm25_task = None
    vector_task = None

    if request.search_type in ("hybrid", "bm25"):
        bm25_task = asyncio.create_task(search_bm25(es_client, request.query, top_k=20, filters=es_filters))

    if request.search_type in ("hybrid", "vector"):
        try:
            query_embedding = await embed_query(embedding_provider, request.query)
            vector_task = asyncio.create_task(search_vector(milvus_collection, query_embedding, top_k=20))
        except EmbeddingTimeoutError:
            # 嵌入失败 → 强制降级到 bm25_only
            logger.warning("embedding unavailable, fallback to bm25_only")
            request.search_type = "bm25"
            vector_task = None

    # 3. 降级矩阵
    if bm25_task and vector_task:
        bm25_results, vector_results = await asyncio.gather(bm25_task, vector_task)
        fused = rrf_fusion(bm25_results, vector_results, k=60)
    elif bm25_task:
        bm25_results = await bm25_task
        fused = bm25_results # 单路, 无融合
    elif vector_task:
        vector_results = await vector_task
        fused = vector_results
    else:
        return [] # 双双不可用, 空结果

    # 4. 重排序
    if request.rerank and reranker:
        try:
            fused = await asyncio.to_thread(reranker.rerank, request.query, fused[:request.top_k*2])
        except Exception as exc:
            logger.warning("reranker failed, fallback to RRF", exc=exc)
```

```
# 5. Milvus 后过滤 (effective_date)
fused = [c for c in fused if c.is_current_version]

# 6. 截 top_k
result = fused[:request.top_k]

# 7. 缓存写回
if result and request.search_type != "vector":
    await redis_client.setex(cache_key, 300, json.dumps([c.dict() for c in result]))

return result
```

4 路径降级矩阵:

ES	Milvus	行为	触发场景
✓	✓	Hybrid + RRF	正常
✓	✗	BM25 only	Milvus 挂
✗	✓	Vector only	ES 挂
✗	✗	空结果	双挂

并行任务防御: `vector_task: asyncio.Task | None = None` 初始化, 仅在任务确实创建后 `cancel` 一避免并发路径下引用未赋值任务.

5.4 BM25 检索: ES + IK

retrieval.py:140-225 search_bm25 用 ES 8.19 + IK 中文分词:

```
# retrieval.py:140 (简化)
async def search_bm25(es_client, query, top_k, filters):
    """BM25 检索, ES + IK 中文分词"""
    body = {
        "query": {
            "bool": {
                "must": [{
                    "match": {
                        "content": {
                            "query": query,
                            "analyzer": "ik_max_word", # 细粒度切分
                            "minimum_should_match": "75%",
                        }
                    }
                ]
            },
            "filter": [{"term": {k: v}} for k, v in filters.items()],
        },
        "highlight": {
            "fields": {"content": {"number_of_fragments": 2, "fragment_size": 200}}
        },
        "size": top_k,
    }
    response = await es_client.search(index="lumio_kb_chunks", body=body)
    return [_build_chunk(hit) for hit in response["hits"]["hits"]]
```

关键设计: - `ik_max_word` 细粒度切分: 命中更精准 - `minimum_should_match=75%`: 至少匹配 75% 词 - `highlight`: 返回高亮片段, 帮客户定位 - 后续 `_build_chunk` 还做 `parent_chunk_id` 回填

5.5 向量检索: Milvus

retrieval.py:228-340 search_vector 用 Milvus IVF_FLAT:

```
# retrieval.py:228 (简化)
async def search_vector(milvus_collection, query_embedding, top_k, filters):
    """向量检索, Milvus IVF_FLAT COSINE"""
    search_params = {"metric_type": "COSINE", "params": {"nprobe": 16}}
    # nprobe=16 查 16 个聚类桶, 平衡精度/速度
```

```

results = milvus_collection.search(
    data=[query_embedding],
    anns_field="embedding",
    param=search_params,
    limit=top_k,
    expr=build_milvus_expr(filters), # field == "value" + ARRAY_CONTAINS
)
return [_build_chunk(hit) for hit in results[0]]

```

关键设计: – 1024 维 (bge-large-zh-v1.5) – nprobe=16 : 16 个聚类桶, 命中率 ~95%, 延迟 < 50ms – metric_type=COSINE : 余弦相似度, 文本语义标准 – build_milvus_expr : 表达式过滤 (approval_status == "PUBLISHED" and keywords_array contains "信用卡")

5.6 RRF 融合

retrieval.py:344–396 rrf_fusion 实现倒数排名融合:

```

# retrieval.py:344 (简化)
def rrf_fusion(bm25_results, vector_results, k=60):
    """Reciprocal Rank Fusion: score = 1/(k+rank)"""
    scores: dict[str, float] = {} # chunk_id -> total score
    chunks: dict[str, RetrievedChunk] = {}

    for rank, chunk in enumerate(bm25_results, start=1):
        scores[chunk.id] = scores.get(chunk.id, 0) + 1.0 / (k + rank)
        chunks[chunk.id] = chunk

    for rank, chunk in enumerate(vector_results, start=1):
        scores[chunk.id] = scores.get(chunk.id, 0) + 1.0 / (k + rank)
        chunks[chunk.id] = chunk

    # 排序
    sorted_ids = sorted(scores, key=scores.get, reverse=True)
    return [chunks[i] for i in sorted_ids]

```

RRF 公式: $score(d) = \sum 1/(k + rank_d)$ 其中 k=60 是常数. 优势:

1. 无需训练: 不需要学习权重
2. 互补召回: BM25 关键词精确, 向量语义相似, 两者取长补短
3. 去重: 同一 chunk 在两路都出现, 分数累加, 自然上升

k=60 来自 Cormack et al. 2009 论文, 经验最优值.

5.7 重排序

agent/lumio/services/common/reranker.py 独立模块, 当前用 OllamaReranker (本地) 或 TEIReranker (生产):

```

# reranker.py:45 (简化)
class OllamaReranker:
    """ThreadPoolExecutor 并发评分 (10 worker)"""
    def __init__(self, base_url: str, model: str = "bge-reranker-large"):
        self.base_url = base_url
        self.model = model
        self.executor = ThreadPoolExecutor(max_workers=10)

    def rerank(self, query: str, chunks: list[RetrievedChunk]) -> list[RetrievedChunk]:
        """并发评分所有候选, 按相关性降序"""
        # 1. 并发评分
        scored = list(self.executor.map(
            lambda c: (c, self._score(query, c.content)),
            chunks,
        ))
        # 2. 排序
        scored.sort(key=lambda x: x[1], reverse=True)
        # 3. 返回
        return [c for c, _ in scored]

```

关键设计: – 候选 `top_k*2 = 10` 送入 reranker, 精排后取 `top_k = 5` – `ThreadPoolExecutor(10)`: reranker 是 CPU 密集, 用线程池 – 失败 fallback 到 RRF 结果, 不阻塞

5.8 父-子分块 (Parent-Child Chunking)

ingestion.py:585-595 实现父-子分块:

```
# ingestion.py:585 (简化)
# 1. 父块 (1500 字符) – 一次性切成
parents = chunk_text(cleaned, chunk_size=1500, overlap=200)

# 2. 子块 (300 字符) – 每个父块再切
for parent in parents:
    parent.children = chunk_text(parent.content, chunk_size=300, overlap=50, parent_id=parent.id)

# 3. 检索用子块嵌入 + 存父块内容
for child in all_children:
    child.embedding = await embedder.embed(child.content)
    child.parent_content = parent.content # 拼父块, 给生成阶段
```

关键设计: – 检索粒度小: 300 字符子块, 语义聚焦 – 生成粒度大: 1500 字符父块, 上下文完整 – 跨块关联: 命中子块后, `parent_content` 字段存整个父块, 生成阶段直接用

示意:

```
父块 1 (1500 字符: "信用卡年费政策...")
├── 子块 1.1 (300 字符: "信用卡年费政策介绍...") ← 检索命中
├── 子块 1.2 (300 字符: "年费减免条件...")
└── 子块 1.3 (300 字符: "积分兑换年费...")
```

客户问“年费怎么免”, 命中 1.1, 拿父块 1 完整 1500 字符喂 LLM, LLM 看完整上下文给精准回答。

5.9 银行合规字段硬注入

retrieval.py:460-468 强制注入 3 个合规字段:

```
# retrieval.py:460 (简化)
es_filters = build_es_filters(request.filters)
# 强制注入, 业务层零感知
es_filters["approval_status"] = "PUBLISHED" # 只查已发布
es_filters["is_current_version"] = True      # 只查当前版本
es_filters["effective_date_lte"] = today_iso # 生效日期 <= 今天
```

Milvus 侧后过滤 (retrieval.py:560-575, 第五轮加固):

```
# 简化 (第五轮修复后: 严格判定, 缺失字段视为不合规, 不做默认放行)
fused = [
    c for c in fused
    if c.metadata.get("approval_status") == "PUBLISHED"
    and str(c.metadata.get("is_current_version", "")).lower() == "true"
]
```

关键设计: – 任何 `RetrieveRequest` 都自动加这 3 个过滤, 业务代码无法绕过 – 双层防护: ES 索引层 + Milvus 内存层 – 合规字段 (`approval_status` / `is_current_version`) 已写入 Milvus 标量 schema (`init_milvus`) 与 ingestion 插入数据, 后过滤读的是**真实值**而非默认值 — 此前 Milvus 无此字段, `metadata.get(key, 默认放行)` 让未审批文档经向量检索泄露 – 测试用例: `test_retrieval.py:18-25` `test_compliance_filter_mandatory`

5 阶段文档审批流 (shared/orm_models.py `KbApprovalStatus`):

```
DRAFT → IN_REVIEW → APPROVED → PUBLISHED → SUPERSEDED
                        → REJECTED
                        → ARCHIVED
```


只有 PUBLISHED 状态才被检索, DRAFT (草稿) / IN_REVIEW (审核中) / SUPERSEDED (被替代) 都自动屏蔽. 审批链端点 (approve/reject/publish/archive) 已加 admin 角色校验.

5.10 缓存层

retrieval.py:121-135 缓存键 + retrieval.py:585-602 缓存写回:

```
# 简化 (第五轮修复后: key 含 include_expired/rerank 维度)
def _build_cache_key(query, filters, search_type, include_expired=False, rerank=False):
    query_hash = md5(query.encode()).hexdigest()[:16]
    filters_hash = md5(json.dumps(filters, sort_keys=True).encode()).hexdigest()[:16]
    key = f"lumio:rag:cache:{search_type}:{query_hash}:{filters_hash}"
    if include_expired: key += ":exp" # 过期文档结果不与默认请求互用
    if rerank: key += ":rerank" # 精排结果与未精排结果不互用
    return key

# 写回
if result and request.search_type != "vector":
    await redis_client.setex(cache_key, 300, json.dumps([c.dict() for c in result]))
```

维度隔离动机: 缓存 key 此前只含 query/filters/search_type — include_expired=True 请求 (不加日期过滤) 的结果写入缓存后, 默认请求命中同一 key 会拿到含过期文档的结果; rerank=True 请求也会拿到未精排的缓存. 加后缀后两类结果物理隔离.

关键设计: – TTL 300s: 5 分钟内同 query+filters 命中缓存, 节省 1 次 LLM/embedding 调用 – **vector_only 不缓存:** 向量查询个性化强 (customer_id 不同结果不同), 缓存命中率极低且会污染 – 命中率指标: lumio_retrieval_cache_hit_total (Counter, label=hit/miss)

全链路接线: Bot Agent 检索路径 (bot_agent._retrieve) 把 session_manager._redis 传入 retrieve(redis_client=...) — 缓存读写在知识问答路径真实生效. 相同问题 5 分钟内重复问 = 直接命中缓存, 不再重复打 ES/Milvus + embedding + rerank.

5.10a 重复提问检测 (多轮对话)

知识问答入口先做归一化重复检测 — 与上一轮客户消息归一化后相同 → 直接复用上次回答:

```
# bot_agent._detect_repeat_question (简化)
norm = self._normalize_question(user_input) # 去标点/空白/语气词 ("额度多少? " → "额度多少")
history = await self._session_manager.get_history(session_id, limit=4)
# 找到相同客户消息 → 返回其后的 Bot 回答 (source="repeat")
```

- 轻量精确匹配而非语义相似度 — 不误伤"额度多少"→"额度怎么提升"这类真实新问题
- 短消息 (<4 字符) 不判定, 避免"好的"/"嗯"误判
- 命中时跳过检索 + LLM + 摘要, 3 次重复提问省 3 次全链路开销

5.10b 检索熔断检查

Bot 检索前主动查 ES/Milvus 熔断器 (app.state.es_breaker / milvus_breaker):

```
# bot_agent._retrieve 入口
if es_breaker.is_open and milvus_breaker.is_open:
    return "" # 双挂时主动跳过检索, 直接走无知识降级 (不打满超时)
```

- 单边熔断仍走单路降级 (BM25 only / Vector only, 见 5.3)
- 熔断器由 init_dependency_breakers 装配: failure_threshold=0.5 / window=20 / recovery=30s

5.11 摄入端 5 阶段 (概览)

完整摄入端细节见 [第 9 章 RAG 摄入管线](#), 这里只列关键 5 阶段:

```
flowchart LR
    Doc[上传文档<br/>PDF/DOCX/HTML/MD/TXT/XLSX] --> Parse[Parse<br/>6 格式解析]
    Parse --> Clean[Clean<br/>5 步清洗]
```

```

Clean --> Chunk[Chunk<br/>父-子分块]
Chunk --> Embed[Embed<br/>TEI 128 批]
Embed --> DualWrite[Dual-Write]
DualWrite --> ES[(Elasticsearch<br/>KB 索引)]
DualWrite --> Milvus[(Milvus<br/>向量索引)]
DualWrite -.失败.-> Rollback[回滚 ES]
DualWrite --> DB[(PostgreSQL<br/>KbChunk + IngestionLog)]

```

```
style Rollback fill:#fee,stroke:#c00
```

双写时序 (ingestion.py:374-465): 1. ES 写 N 成功 (`es_count == len(chunks)`) → 才继续 2. Milvus 写 → 失败回滚 ES (`_rollback_es_docs` 逐个删) 3. 全部成功 → 落 KbChunk 表 + 写 7 阶段流水日志

5.12 检索端到端流程图

```

sequenceDiagram
    participant Client as Bot/Assist
    participant Retrieval as retrieve()
    participant Redis as Redis 缓存
    participant ES as Elasticsearch
    participant Milvus as Milvus
    participant Embed as Embedding
    participant Rerank as Reranker

    Client->>Retrieval: RetrieveRequest{query, top_k=5}
    Retrieval->>Redis: GET lumio:rag:cache:...
    alt 缓存命中
        Redis-->>Retrieval: cached chunks
        Retrieval-->>Client: 返回 (5ms)
    else 缓存未命中
        Retrieval->>Retrieval: 注入合规字段
        par 并行检索
            Retrieval->>ES: search_bm25 (top 20)
            Retrieval->>Embed: embed_query (TEI 128 批)
            Embed-->>Retrieval: 1024 维向量
            Retrieval->>Milvus: search_vector (top 20, nprobe=16)
        end
        ES-->>Retrieval: BM25 results
        Milvus-->>Retrieval: Vector results
        Retrieval->>Retrieval: RRF 融合 (k=60)
        Retrieval->>Rerank: rerank (top 10 候选)
        Rerank-->>Retrieval: top 5
        Retrieval->>Retrieval: Milvus 后过滤
        Retrieval->>Redis: SETEX TTL 300s
        Retrieval-->>Client: 5 chunks (200-500ms)
    end
end

```

5.13 监控指标

RAG 链路发射 4 个核心指标 — 回答"检索快不快、缓存命中高不高、嵌入服务健不健康":

指标	类型	含义	Labels	位置
lumio_retrieval_duration_seconds	Histogram	检索耗时分布 (看是 ES 慢还是 Milvus 慢)	search_type (hybrid/bm25/vector)	retrieval.py:607
lumio_retrieval_cache_hit_total	Counter	缓存命中/未命中次数 (命中率高 = 重复问题多被复用)	hit/miss	retrieval.py:600
lumio_embedding_unavailable_total	Counter	嵌入服务不可用次数 (触发降级到 BM25)	-	embedding.py:280
http_requests_total	Counter	接口调用次数 (复用全局)	method, endpoint, status	全局

降级告警: Prometheus 规则 `EmbeddingUnavailabilityHigh` (>5 次/分钟, 持续 5m) → warning.

5.14 测试覆盖

agent/tests/test_retrieval.py (22 用例) 覆盖:

- test_hybrid_search_rrf_fusion — RRF 公式正确性
- test_embedding_failure_degrades_to_bm25 — 嵌入失败降级到 BM25 的回归用例
- test_compliance_filter_mandatory — 合规字段硬注入
- test_cache_hit_skips_retrieval — 缓存命中
- test_milvus_filter_post_processing — 后过滤
- test_reranker_failure_fallback — 重排序降级
- test_parent_child_chunk_relationship — 父-子分块
- test_filter_by_keywords_array — ARRAY_CONTAINS

5.15 本章小结

RAG 检索全链路是 Lumio 服务客户的核心:

- 三路召回互补: BM25 关键词精确 + 向量语义相似 + RRF 融合无需训练
- 父-子分块: 检索粒度小 (300 字符), 生成粒度大 (1500 字符), 一次摄入两用
- ES + Milvus 双写 + 回滚: 一致性保障, 任何路径失败不污染
- 合规字段硬注入: 业务层零感知, 3 重过滤, 5 阶段审批流
- 4 路径降级矩阵: 4 种 (ES✓/Milvus✓) 组合, 双挂时返空
- 缓存层: 5 分钟 TTL, vector_only 不缓存, 命中率指标
- 重排序: 候选 10 精排 5, 失败 fallback RRF

下一章预告: [第 6 章 会话状态机](#) 深入 3 phase × 7 sub-state + CAS Lua 原子控制 + ZSET 超时队列.

延伸阅读: - [第 9 章 RAG 摄入管线](#) — 摄入端 5 阶段 - [第 12 章 数据层](#) — ES + Milvus 索引设计 - [第 11 章 安全合规](#) — 银行合规字段 - [附录 A 术语表](#) — RAG 术语速查

06 会话状态机

第 6 章: 会话状态机

本章深入 Lumio 会话状态机. 银行客服一次通话从客户拨入到结束, 涉及 3 phase × 6 sub-state 的状态组合 — 怎么保证多实例并发下不丢转换, 怎么 5 秒检测超时, 怎么保证 5-7 年审计留存, 是本章核心. 看完本章你会理解: 状态机怎么设计才能既灵活又安全, CAS Lua 怎么原子控制并发, ZSET 怎么替代 asyncio.Task 做分布式超时, PostgreSQL 怎么异步落库.

6.1 状态机全景

先讲个业务场景: 客户打进来说"我的卡丢了" — 系统要一路护送他走完整个旅程: Bot 先接待 → 识别到挂失 → 转人工 → 排队等坐席 → 坐席接听 → 通话 → 挂断 → 话后小结 → 归档. 任何一个环节, 系统都得**准确知道**"客户现在到哪一步了": 排队超时了要不要回 Bot? 坐席 30 秒没接要不要提醒? 通话中客户还能不能插话? 这就是状态机存在的意义 — 给每一次对话一个"当前进度", 让系统所有组件 (Bot、坐席端、超时器、审计) 对"现在是什么状态"有一致的认识.

```
stateDiagram-v2
    [*] --> BOT_ACTIVE: 客户进入 Bot

    BOT_ACTIVE --> AG_QUEUED: 客户请求转人工
    BOT_ACTIVE --> ENDED: 客户说再见

    AG_QUEUED --> AG_ASSIGNED: 坐席接单
    AG_QUEUED --> ENDED: 客户放弃排队 (超时)
    AG_QUEUED --> BOT_ACTIVE: 客户撤销转人工

    AG_ASSIGNED --> AG_ACTIVE: 坐席首次响应
    AG_ASSIGNED --> ENDED: 坐席未响应 (超时)
    AG_ASSIGNED --> AG_QUEUED: 客户取消转接

    AG_ACTIVE --> AG_ON_HOLD: 坐席保持 (hold)
    AG_ON_HOLD --> AG_ACTIVE: 坐席恢复 (resume)
    AG_ON_HOLD --> ENDED: 保持超时 (60s)

    AG_ACTIVE --> AG_REVIEWING: 通话结束
    AG_REVIEWING --> ENDED: 话后小结完成 + PG 落库
    AG_REVIEWING --> AG_ACTIVE: 复核发现问题, 重新激活

    ENDED --> [*]
```

6.1.0 状态图解读 — "一次挂失通话的完整旅程"

顺着图从上往下走一遍 (对应上面客户的挂失场景):

1. **BOT_ACTIVE (Bot 接待)**: 客户进来先和 Bot 聊. 这是默认状态, 一切对话的起点.
2. → **AG_QUEUED (排队中)**: Bot 识别到挂失 → 转人工. 客户进入排队, 系统开始 60 秒倒计时.
3. → **AG_ASSIGNED (已分配)**: 坐席接了单, 系统通知客户"客服马上来". 如果 30 秒没人接 → 回排队或结束.
4. → **AG_ACTIVE (通话中)**: 坐席开口了, 正式通话. 这是**双向状态** — 坐席可以随时把客户保持 (AG_ON_HOLD) 或结束通话.
5. → **AG_REVIEWING (话后小结)**: 通话结束, 坐席要在 2 分钟内填完小结 (记录问题、处理结果).
6. → **ENDED (归档)**: 小结提交, 对话写入 PostgreSQL 审计留存 5-7 年.

为什么状态不能乱跳: 比如客户在排队 (AG_QUEUED) 时, 系统不允许直接跳到"通话中" (AG_ACTIVE) — 必须先经过"已分配" (AG_ASSIGNED). 这张图的箭头就是**合法转换白名单**, 代码里是 `VALID_TRANSITIONS` 表, 任何白名单外的转换直接抛 `InvalidTransitionError` — 防止并发情况下两个组件同时改状态导致"客户明明还在排队, 系统却以为在通话"的错乱.

4 phase (顶层):

Phase	含义	进入条件
BOT	Bot 自助期	客户首次进入系统
AGENT	坐席通话期	客户转入人工后
ENDED	通话结束	双方挂断 / 超时
legacy	旧版本兼容	旧数据迁移

7 sub-state (子状态):

SubPhase	含义
BOT_ACTIVE	Bot 正常对话
AG_QUEUED	转人工排队中
AG_ASSIGNED	已分配坐席, 等待首次响应
AG_ACTIVE	坐席与客户对话中
AG_ON_HOLD	坐席主动保持 (hold)
AG_REVIEWING	通话结束, 等待话后小结
ENDED	完全结束, PG 已落库

VALID_TRANSITIONS 白名单 (models.py) 严格控制哪些转换合法, 任何非法转换抛 `InvalidTransitionError (3005)`.

6.2 SessionManager 完整 API

agent/lumio/services/common/session.py:92-280 SessionManager 集中所有会话操作:

```
# session.py:92 (简化)
class SessionManager:
    def __init__(self, redis_client, dialogue_log_repo=None):
        self.redis = redis_client
        self.dialogue_log_repo = dialogue_log_repo
        self._cas_script = redis_client.register_script(_CAS_WRITE_SCRIPT)

    async def get_or_create(self, session_id: str, customer_id: str) -> SessionState:
        """获取或创建会话, UUID v7"""
        meta_key = session_meta_key(session_id)
        if not await self.redis.exists(meta_key):
            state = SessionState(
                session_id=session_id,
                customer_id=customer_id,
                phase=SessionPhase.BOT,
                sub_phase=SessionSubPhase.BOT_ACTIVE,
                version=0, # CAS version
            )
            await self._create(state)
        return await self._load(session_id)

    async def add_turn(self, session_id: str, turn: DialogueTurn):
        """追加对话轮次, Rpush + Ltrim 20"""
        history_key = session_history_key(session_id)
        await self.redis.rpush(history_key, turn.model_dump_json())
        await self.redis.ltrim(history_key, -20, -1)
        # TTL = max(配置, session_timeout*2+300) = 3900s (第五轮修复)
        # 旧默认 1800 与 AG_ACTIVE 超时相等: 空闲会话 meta 先过期 -> get_session 返回
        # None -> 超时轮询器吞错 -> 会话永不走 ENDED -> persist_dialogue 不触发 (审计缺口)
        await self.redis.expire(history_key, self._ttl)

    async def transition_phase(
        self, session_id: str, to_phase: SessionPhase, to_sub: SessionSubPhase, reason: str = ""
    ) -> SessionState:
        """CAS 原子转换"""
        # 1. 校验白名单
        if (to_phase, to_sub) not in VALID_TRANSITIONS.get(current_phase, {}):
```

```

        raise InvalidTransitionError(
            f"非法转换 {current_phase}/{current_sub} → {to_phase}/{to_sub}",
            code=3005,
        )
    # 2. CAS Lua 原子写
    new_version = await self._cas_script(
        keys=[session_meta_key(session_id)],
        args=[to_phase.value, to_sub.value, reason, ...],
    )
    if new_version == -1:
        # version 冲突, 重试 1 次
        raise StateConflictError(...)
    # 3. 指标 + 日志
    SESSION_TRANSITIONS.labels(
        from_phase=current_phase.value, from_sub=current_sub.value,
        to_phase=to_phase.value, to_sub=to_sub.value, reason=reason,
    ).inc()
    return new_state

async def patch_state(
    self, session_id: str, patch: dict, mode: str = "merge"
) -> SessionState:
    """字段级合并, 3 种模式: 增量 / 单向门 / 全量覆写"""
    # _INCREMENTAL_FIELDS: intent_stack / entity_pool (去重合并)
    # _ONE_WAY_FIELDS: suppress_flag (false → true 单向)
    # 其他: 全量覆写
    ...

```

关键设计: 5 个核心 API: `get_or_create` / `add_turn` / `transition_phase` / `patch_state` / `persist_dialogue`. 任何会话操作必走这 5 个, 业务层零 Redis 接触.

6.3 CAS Lua 脚本: 原子控制

`session.py:67-95` 是核心 Lua 脚本:

```

-- session.py:67 (完整)
local current_version = tonumber(redis.call('HGET', KEYS[1], 'version'))
if current_version == nil then
    return -1 -- key 不存在
end
local expected_version = tonumber(ARGV[1])
if current_version ~= expected_version then
    return -1 -- version 冲突
end
-- 原子更新
redis.call('HSET', KEYS[1], 'phase', ARGV[2], 'sub_phase', ARGV[3], 'reason', ARGV[4])
redis.call('HINCRBY', KEYS[1], 'version', 1)
-- 合并 patch (JSON)
local patch_json = ARGV[5]
if patch_json and patch_json ~= '' then
    local patch = cjson.decode(patch_json)
    for k, v in pairs(patch) do
        if type(v) == 'table' and v.op == 'merge' then
            -- 增量合并 (去重)
            local existing = redis.call('HGET', KEYS[1], k)
            if existing then
                local existing_list = cjson.decode(existing)
                for _, item in ipairs(v.value) do
                    if not list_contains(existing_list, item) then
                        table.insert(existing_list, item)
                    end
                end
                redis.call('HSET', KEYS[1], k, cjson.encode(existing_list))
            end
        else
            redis.call('HSET', KEYS[1], k, cjson.encode(v.value))
        end
    end
elseif type(v) == 'table' and v.op == 'one_way' then
    -- 单向门 (false → true)
    local existing = redis.call('HGET', KEYS[1], k)
    if not existing or existing == 'false' then
        redis.call('HSET', KEYS[1], k, tostring(v.value))
    end
end
else
    ...

```

```
        -- 全量覆写
        redis.call('HSET', KEYS[1], k, cjson.encode(v))
    end
end
end
return current_version + 1
```

关键设计: – Lua 在 Redis 单线程内原子执行, 不会被其他命令打断 – version 不匹配立即返 -1, Python 侧重试 – 字段级合并: 3 种模式 (增量 / 单向门 / 全量), 全部在 Redis 端做, 减少网络往返 – `_INCREMENTAL_FIELDS = {"intent_stack", "entity_pool"} : 数组去重合并` – `_ONE_WAY_FIELDS = {"suppress_flag"} : false → true 单向, 不允许反向`

为什么不用 Redis 内置 WATCH/MULTI : – WATCH/MULTI 失败时整批回滚, 需要重试整批逻辑 – Lua 脚本细粒度控制, 字段级合并逻辑可嵌入 – Lua 性能更好 (一次 RTT)

6.4 字段级合并规则

session.py:80-95 定义 3 种合并模式:

```
# session.py:80 (简化)
INCREMENTAL_FIELDS = {"intent_stack", "entity_pool", "recent_topics"} # 增量去重
ONE_WAY_FIELDS = {"suppress_flag", "transferred_to_human"} # 单向门
# 其他字段: 全量覆写
```

场景示例:

字段	模式	例子
intent_stack	增量	<code>["complaint"] + ["faq", "complaint"] → ["complaint", "faq"] (去重)</code>
suppress_flag	单向	<code>false + true → true; true + false → 仍是 true</code>
phase	全量	<code>BOT + AGENT → AGENT (覆盖)</code>
customer_id	全量	不变 (业务不变更)

为什么用 3 模式而非简单全量覆写: – 增量: 意图栈需要累积而非覆盖 – 单向门: `suppress_flag` 一旦设了 true, 不允许回退 (防止竞争条件下误关) – 全量: `phase` 等明确字段直接覆写最简单

6.5 ZSET 超时队列: 5s 轮询

agent/lumio/services/common/session_timeout.py:44-180 用 Redis ZSET 做分布式超时:

```
# session_timeout.py:44 (简化)
TIMEOUT_ZSET_KEY = "lumio:session:timeouts"

class SessionTimeoutManager:
    """5 类超时统一管理"""

    async def start_guard(
        self, session_id: str, sub_phase: SessionSubPhase, timeout_type: TimeoutType, ttl: int
    ):
        """ZADD score=expire_ts"""
        score = int(time.time()) + ttl
        await self.redis.zadd(
            TIMEOUT_ZSET_KEY,
            {f"{session_id}:{timeout_type.value}": score},
        )

    async def cancel_guard(self, session_id: str, timeout_type: TimeoutType):
        """ZREM 取消"""
        await self.redis.zrem(TIMEOUT_ZSET_KEY, f"{session_id}:{timeout_type.value}")

    async def _poll_loop(self):
        """5s 一次轮询"""
        while True:
            await asyncio.sleep(5.0)
```

```
now = int(time.time())
# 1. 找出所有过期项
expired = await self.redis.zrangebyscore(TIMEOUT_ZSET_KEY, 0, now)
for member in expired:
    session_id, timeout_type = member.rsplit(":", 1)
    # 2. ZREM 原子竞争（多实例只有一个能删成功）
    removed = await self.redis.zrem(TIMEOUT_ZSET_KEY, member)
    if removed == 0:
        continue # 其他实例已经处理
    # 3. 触发超时处理
    await self._handle_timeout(session_id, TimeoutType(timeout_type))
```

超时守卫映射 (session_timeout.py:238-248):

SubPhase	TTL	触发动作
BOT_ACTIVE	180s	客户 180s 无消息 → ENDED (reason=bot:active_timeout)
AG_QUEUED	60s	排队 60s 仍无坐席 → 回退 BOT (降级而非结束)
AG_ASSIGNED	无守卫	由外部 chat-svc 驱动振铃, 本地不设超时
AG_ACTIVE	1800s	通话总时长 30 分钟 → ENDED
AG_ON_HOLD	1800s	保持超 30 分钟 → ENDED
AG_REVIEWING	120s	通话结束 2 分钟内必须生成话后小结 → ENDED

守卫生命周期 (关键设计):

- **创建即启动:** create_session 时即启动 BOT_ACTIVE 守卫 — 新会话也能被空闲超时回收, 不依赖 Redis TTL 兜底
- **随对话续期:** add_turn 每轮对话刷新 BOT 守卫 (start_guard 幂等 ZREM+ZADD) — 活跃会话不会被空闲超时误杀; 若只依赖 transition 启停, 出现过"转人工回退后的会话 120s 必被误杀、全新会话却永不超时"的行为不一致
- **start_guard 原子性:** 先 await ZREM 清旧守卫再 ZADD 新守卫 (顺序执行), 避免竞态删掉新守卫

关键设计: ZSET 而非 asyncio.Task. 原因: – **多实例支持:** Bot 和 Assist 都能处理同一会话超时 – **重启恢复:** 服务重启后 ZSET 数据仍在 Redis, 自动恢复 – **原子竞争:** ZRANGEBYSCORE 查 + ZREM 删 是 2 步, 但 ZREM 只删一个成员, 多实例并发只有一个成功

6.6 PostgreSQL 异步落库

会话结束 (ENDED phase) 时, 异步落 dialogue_log 表:

```
# session.py:380 (简化)
async def persist_dialogue(self, session_id: str):
    """会话结束, 异步落 PG dialogue_log 表"""
    # 1. 拉取完整历史
    history = await self.redis.lrange(session_history_key(session_id), 0, -1)
    meta = await self.redis.hgetall(session_meta_key(session_id))

    # 2. 转 DialogueLog ORM 模型
    log = DialogueLog(
        session_id=session_id,
        customer_id=meta["customer_id"],
        phase=meta["phase"],
        sub_phase=meta["sub_phase"],
        turns=[DialogueTurn.parse_raw(t) for t in history],
        started_at=datetime.fromisoformat(meta["created_at"]),
        ended_at=datetime.utcnow(),
        total_turns=len(history),
        compressed_json=compress(json.dumps([t.dict() for t in history])),
    )

    # 3. 异步落库 (fire-and-forget, 不阻塞)
    asyncio.create_task(self.dialogue_log_repo.insert(log))

    # 4. 清理 Redis (可选, 留 7 天备份)
    await self.redis.expire(session_meta_key(session_id), 7 * 86400)
    await self.redis.expire(session_history_key(session_id), 7 * 86400)
```


关键设计: – 5–7 年留存: PG 表 `dialogue_log` 由 DBA 配置保留期, 金融合规要求 – 压缩存储: `compressed_json` 字段 (BYTEA) 压缩对话原文, 减少存储 – fire-and-forget: `asyncio.create_task` 不阻塞 `transition_phase` 调用 – Redis 留 7 天: 临时备份, 7 天后过期

6.7 Redis key 集中化

Redis key 命名集中在 `session.py` 统一管理:

```
# session.py:24-50 (简化)
def session_meta_key(session_id: str) -> str:
    return f"lumio:session:{session_id}:meta"

def session_history_key(session_id: str) -> str:
    return f"lumio:session:{session_id}:history"

def session_meta_scan_pattern() -> str:
    return "lumio:session:*:meta"

def session_timeout_zset_key() -> str:
    return "lumio:session:timeouts"

# SessionManager 内部使用这些 helpers, 不再硬编码
class SessionManager:
    def _meta_key(self, session_id: str) -> str:
        return session_meta_key(session_id) # 委托
    def _history_key(self, session_id: str) -> str:
        return session_history_key(session_id) # 委托
```

集中管理收益: – 未来 prefix 改动: 单点修改 `session.py:24-50`, 全局生效 – 测试更简单: 测试可以 mock helper 函数 – 审计清晰: `session_meta_scan_pattern()` 给 SCAN 用, 避免散落

6.8 状态模型: 3×7 矩阵

会话状态机采用 3 phase (顶层) × 6 sub-state (子) 的矩阵模型:

```
PHASE:    BOT → AGENT → ENDED
SUB:      IDLE / ACTIVE / WAITING_HUMAN / QUEUING / ASSIGNED / ON_HOLD / REVIEWING ...
```

设计动机: – 扁平状态互相转换规则复杂 (N×N 组合) – `WAITING_HUMAN` 需要表达"排队中 vs 已分配"区别 – 终结态统一归 `ENDED`, 用 `reason` 字段区分 (`abandoned` / `transferred` / `normal`) – 状态组合由 `VALID_TRANSITIONS` 白名单严格约束 (合法转换共 16 条), 非法转换直接拒绝

兼容策略: – `legacy phase` 兼容旧数据读取

6.9 会话复活与收尾 (多轮对话边界)

6.9.1 ENDED 会话复活

客户超时/主动结束后再次发消息 → `router.py:_run_agent` 检测 `current_phase == ended`, 调 `transition_phase(BOT, BOT_ACTIVE, reason="customer_returned")` 复活会话:

```
# router.py:607-619 (简化)
if state.current_phase.value == "ended":
    await session_manager.transition_phase(
        session_id, SessionPhase.BOT,
        new_sub_phase=SessionSubPhase.BOT_ACTIVE,
        reason="customer_returned",
    )
```

设计要点: – 同一 `session_id`、同一 Redis key — 历史 / `conversation_summary` / 实体池 / 意图栈全部保留 – 守卫由 `transition_phase` 重新启动 (`BOT_ACTIVE` 180s 空闲超时生效) – 若 meta 已过期 (Redis TTL 到) → `get_or_create` 新建会话, 画像经 `customer_memory` 90 天窗口重新学习

6.9.2 告别即收尾

客户说"再见/拜拜/谢谢" → `_is_farewell` 快速路径, 同时真正结束会话:

- `transition_phase(ENDED, reason="customer_farewell")` — 触发 PG 落库审计
- `pending_action` 一并清除 — 避免复活后第一句普通消息被当成确认/取消误判
- 回复模板话术, 不调 LLM

6.9.3 AG_ASSIGNED 死状态

`agent:assigned` (振铃) 在 lumio 本地无触发路径, 由外部 `chat-svc` 回调驱动:

- `VALID_TRANSITIONS` 保留表项 (兼容外部回调 `queued → assigned → active`)
- 本地不设超时守卫 (`_get_timeout` 无 `AG_ASSIGNED` 映射) — 避免本地 30s 误杀外部驱动的振铃会话

6.9.4 转人工排队期间的消息真实记录

会话进入 AGENT 阶段后客户发消息 → 返回"已为您记录"话术, 同时真实写入 Redis 历史 (`add_turn`), 供坐席摘要 / 转回 Bot 时保留上下文 — 话术与事实一致.

6.10 监控指标

会话状态机发射 5 个核心指标 — 回答"会话流转健不健康、有没有卡死、有没有超时":

指标	类型	含义	Labels	位置
<code>session_transitions_total</code>	Counter	状态转换次数 (看客户从哪阶段流向哪阶段)	<code>from_phase, from_sub, to_phase, to_sub, reason</code>	<code>session.py:403</code>
<code>session_timeouts_total</code>	Counter	超时触发次数 (哪个子阶段超时最多)	<code>sub_phase, reason</code>	<code>session_timeout.py:155</code>
<code>session_phase_duration_seconds</code>	Histogram	各子阶段停留时长 (排队等多久/通话多久)	<code>sub_phase</code>	<code>session.py:412</code> (5s–3600s buckets)
<code>state_conflict_retries_total</code>	Counter	CAS 并发冲突重试次数 (高 = 并发写太激烈)	–	<code>session.py:285</code> (CAS 冲突重试)
<code>dialogue_log_persist_total</code>	Counter	对话落库成败 (error 高 = 审计留痕有缺口)	<code>status (success/error)</code>	<code>session.py:395</code>

指标 0 冲突: 全部指标名 + label 在 Grafana dashboard 有对应 panel.

6.11 测试覆盖

`agent/tests/` 中会话相关:

- `test_session.py` (25 用例, TOP 4): 会话生命周期 + 转换
- `test_state_models.py` (28 用例, TOP 3): 状态机 ORM 模型 + `VALID_TRANSITIONS`
- `test_session_timeout.py` (15 用例): ZSET 超时 + 5 类
- `test_session_lifecycle_e2e.py` (e2e, CI 排除): 完整流程
- `test_state_conflict.py` (10 用例): CAS 冲突重试

关键测试 (`test_session.py:570–578`):

```
# 验证 CAS 增量合并
async def test_intent_stack_incremental_merge(redis):
    mgr = SessionManager(redis)
    await mgr.get_or_create("s1", "c1")
    # 添加 3 个意图, 模拟重复
    await mgr.patch_state("s1", {"intent_stack": {"op": "merge", "value": ["complaint"]}})
    await mgr.patch_state("s1", {"intent_stack": {"op": "merge", "value": ["faq"]}})
    await mgr.patch_state("s1", {"intent_stack": {"op": "merge", "value": ["complaint"]}}) # 重复
    # 验证去重后只有 2 个
    state = await mgr.get_or_create("s1", "c1")
```

```
assert len(state.intent_stack) == 2
assert "complaint" in state.intent_stack
assert "faq" in state.intent_stack
```

6.12 本章小结

会话状态机是 Lumio 处理客户对话的"中枢神经":

- 3 phase x 7 sub-state: 21 组合, 11 合法转换, 白名单控制
- CAS Lua 原子控制: 单 RTT, 字段级合并, 失败重试
- 3 种字段合并模式: 增量去重 / 单向门 / 全量覆写
- ZSET 分布式超时: 替代 asyncio.Task, 多实例支持, 5s 轮询 + ZREM 原子竞争
- 5 类超时: BOT_IDLE / QUEUE / RINGING / SESSION / REVIEW, 各 TTL 不同
- PG 异步落库: 5-7 年合规留存, fire-and-forget 不阻塞
- key 集中化: 未来 prefix 改动单点

下一章预告: [第 7 章 MCP 工具集成](#) 深入 22 个 Java 工具 + Higress 网关 + Python 端零回归.

延伸阅读: - [第 3 章 Bot 自助问答](#) — pending_action 跨轮持久化 - [第 12 章 数据层](#) — Redis ZSET + CAS Lua 详解 - [附录 A 术语表](#)
— 状态机术语速查 - [第 16 章 客户记忆与知识图谱](#) — 跨会话画像学习 (vip_level / card_types / risk_tolerance CAS patch 写入 SessionState)

07 MCP 工具集成

第 7 章: MCP 工具集成

本章深入 Lumio MCP (Model Context Protocol) 工具集成。银行客户要查年费、调额度、办分期, 这些都需要真实业务系统操作 — 怎么让 LLM 安全调用银行工具, 怎么保护敏感操作等客户确认, 怎么在工具出错时不让 LLM 幻觉回答, 是本章核心。看完本章你会理解: MCP 协议怎么替代旧 SSE 实现跨语言工具调用, 22 个 Java 工具怎么对应 7 个业务域, destructiveHint 注解怎么自动标记敏感, 工具护栏怎么双重保险, Higress 网关怎么集中治理鉴权/限流/脱敏/审计。

7.1 MCP 协议概览

先讲个业务场景: 客户说"帮我把额度临时提到 8 万" — 这是要动真金白银的操作。Bot 得调用银行的额度调整服务, 但两个巨大的技术障碍: ① LLM 是个语言模型, 不会直接调 Java 服务; ② 调额度是敏感操作, 不能模型说调就调。MCP 协议解决第一个问题 (给 LLM 一套标准化的"工具调用"方式), 而"敏感操作要确认"是 Lumio 在 MCP 之上加的第二道保险 (第 17 章详讲)。

MCP (Model Context Protocol) 是 Anthropic 2024 年提出的 **AI Agent 调用工具标准协议**。Lumio 采用 streamable-http 传输 (替代旧 SSE):

```
sequenceDiagram
    participant Python as Python LumioAgent
    participant Higress as Higress AI Gateway
    participant Java as Java MCP Server
    participant Tools as 22 信用卡工具

    Note over Python,Higress: 第 1 步: 先"问有什么工具可用"
    Python->>Higress: POST /mcp<br/>streamable-http
    Note over Python,Higress: initialize + list_tools
    Higress->>Java: 转发 + 鉴权 + 限流
    Java-->>Higress: 工具列表
    Higress-->>Python: 22 工具 schema

    Note over Python,Python: 第 2 步: LLM 根据工具清单决定调哪个
    Python->>Python: LLM 决定调 tool_X
    Note over Python,Higress: 第 3 步: 真实调用 (经网关脱敏审计)
    Python->>Higress: POST /mcp<br/>call_tool(tool_X, args)
    Higress->>Higress: 鉴权 + 脱敏
    Higress->>Java: 转发
    Java->>Tools: @Tool tool_X(args)
    Tools-->>Java: result
    Java-->>Higress: result (PII 脱敏)
    Higress-->>Python: result

    Note over Python,Python: 第 4 步: LLM 把工具结果组织成回答
    Python->>Python: LLM 二次生成回答
```

7.1.1 三步走解读 — "LLM 怎么学会用工具"

- 问清单** (initialize + list_tools): 系统启动时, LLM 先拿到 22 个工具的"说明书" (每个工具叫什么、收什么参数、干什么用)。就像给新人一张工具台清单: 有查账单的、有挂失的、有调额的。
- 选工具** (LLM 决定): 客户说"提额", LLM 在清单里挑出 `adjust_temp_credit_limit` 并填好参数 (额度 8 万、卡号)。这一步 LLM 可能**选错工具或填错参数** — 这就是第 17 章"工具护栏"要拦的。
- 调工具** (call_tool): 真正发起调用, 经 Higress 网关 (鉴权/限流/脱敏) 转发给 Java 服务执行, 结果原路返回。
- 组织回答**: LLM 拿到工具返回的 JSON, 翻译成人话告诉客户 ("您的额度已临时调整至 8 万元, 有效期至月底")。

为什么中间要过 Higress 网关: Python 和 Java 是两套技术栈, 直接互连 = 每个工具都要处理鉴权、限流、脱敏 — 22 个工具 × 3 个关注点, 代码爆炸。网关把"谁可以调、调多频繁、返回脱不脱敏、有没有审计"全部收口到一处, 工具只负责干活。

streamable-http vs 旧 SSE 优势: – 双向流: HTTP POST + GET 并存, 旧 SSE 单向 – 重连友好: 连接断开自动重连, 不丢消息 – 多路复用: 同一连接并发多个请求 – 代理友好: HTTP 标准头, Nginx/Envoy 直接转发

7.2 22 个 Java 工具分类

mcp-server/src/main/java/com/lumio/mcp/tools/ 下 7 个文件, 共 22 工具:

文件	工具数	工具名	敏感 (destructiveHint)
BillTools.java	3	query_card_bill / query_bill_detail / query_annual_fee	只读
TransactionTools.java	1	query_transactions	只读
PointsTools.java	3	query_points / query_card_benefits / redeem_points	redeem_points 敏感
CreditLimitTools.java	4	query_credit_limit / adjust_temp_credit_limit / query_limit_adjust_history / apply_permanent_limit	adjust_* / apply_* 敏感
CardServiceTools.java	4	report_card_lost / query_card_status / activate_card / report_transaction_dispute	report_* / activate_* 敏感
InstallmentTools.java	4	query_installment_offer / apply_bill_installment / query_installment_status / cancel_installment	apply_* / cancel_* 敏感
PaymentTools.java	3	repay_credit_card / query_repayment_history / set_auto_repay	repay_* / set_auto_* 敏感
合计	22		10 敏感 + 12 只读

关键设计: – 10 写操作 = 10 敏感: destructiveHint=true 自动标记 – 12 只读: query_* 前缀, 自动安全 – 业务命名规范: 动作_对象, 例如 query_card_bill / report_card_lost

7.3 Python 端 MCPToolClient

agent/lumio/services/common/mcp_client.py:72-393 是 Python 侧统一接口:

```
# mcp_client.py:72 (简化)
class MCPToolClient:
    """MCP 工具客户端, 单后端 / 多后端路由"""

    def __init__(self, settings: MCPSettings):
        self.settings = settings
        self.sessions: dict[str, ClientSession] = {} # 多后端 sessions
        self._tools: dict[str, ToolSpec] = {} # name -> ToolSpec
        self._sensitive: set[str] = set() # 敏感工具白名单
        self.connected = False

    async def connect(self):
        """连接 MCP 后端, 单后端零回归 / 多后端路由"""
        if not self.settings.enabled:
            logger.info("MCP disabled, skip connect")
            return

        if self.settings.backends:
            # 多后端路由模式
            for backend in self.settings.backends:
                session = await self._connect_single(backend.endpoint)
                self.sessions[backend.name] = session
        else:
            # 单后端零回归 (旧配置兼容)
            session = await self._connect_single(self.settings.endpoint)
            self.sessions["default"] = session

        await self._refresh_tools() # 拉取工具列表
        self.connected = True

    async def list_tools(self) -> list[ToolSpec]:
```

```

        """返回所有工具"""
        if not self.connected:
            return [] # 零回归, 不连接返回空
        return list(self._tools.values())

    async def to_openai_tools(self, names: list[str] | None = None) -> list[dict]:
        """转换为 OpenAI function-calling 格式, names 白名单支持渐进式暴露"""
        tools = self.list_tools()
        if names:
            tools = [t for t in tools if t.name in names]
        return [
            {
                "type": "function",
                "function": {
                    "name": f"{t.server}.{t.raw_name}" if t.server != "default" else t.name,
                    "description": t.description,
                    "parameters": t.input_schema,
                },
            }
            for t in tools
        ]

    async def call_tool(self, name: str, arguments: dict) -> ToolCallResult:
        """路由分发到对应后端"""
        if not self.connected:
            raise MCPToolError("MCP not connected", code=4020)

        # 1. 查找路由
        target = self._route(name)
        if target is None:
            raise MCPToolError(f"unknown tool: {name}", code=4020)
        server, raw_name = target

        # 2. 工具护栏 (role 授权 + 金额限额)
        await self.tool_guard.check(server, raw_name, arguments)

        # 3. 调用
        result = await self.sessions[server].call_tool(raw_name, arguments)

        # 4. PII 脱敏
        result.content = mask_pii(result.content)

        # 5. 审计
        await audit_mcp_call(server, raw_name, arguments, result)

        return result

```

关键设计: – 单后端零回归: backends=[] 默认走 endpoint + prefix="", 与旧版兼容 – 多后端路由: card.query_card_bill (信用卡域) vs loan.apply_loan (贷款域) 避免撞名 – 零回归 opt-in: MCP_ENABLED=false 默认, 不连接返回空列表, Bot 走 RAG 兜底 – 工具护栏: role 授权 + 金额限额在调用前拦截 – PII 脱敏 + 审计: 调用后端做, 符合银行合规

7.4 destructiveHint 自动标记敏感

mcp-server/src/main/java/com/lumio/mcp/tools/AdjustTempCreditLimitTool.java (示例):

```

// mcp-server Java 端 (简化)
@Tool(description = "调整信用卡临时额度")
public AdjustTempCreditLimitResult adjustTempCreditLimit(
    @ToolParam(description = "卡号") String cardId,
    @ToolParam(description = "新额度") BigDecimal newLimit,
    @ToolParam(description = "持续天数") int days
) {
    // 业务逻辑
    return new AdjustTempCreditLimitResult(...);
}

```

Spring AI 框架解析 @Tool 注解, 自动生成 OpenAI function-calling schema, 包含 destructiveHint 字段. Python 端 MCPToolClient._refresh_tools 解析:

```

# mcp_client.py:227 (简化)
async def _refresh_tools(self):

```

```

for server, session in self.sessions.items():
    tools = await session.list_tools()
    for tool in tools:
        # destructiveHint 自动标记敏感
        is_sensitive = tool.annotations.get("destructiveHint", False)
        # 累积到白名单
        if is_sensitive:
            self._sensitive.add(tool.name)
        # 存 ToolSpec
        self._tools[tool.name] = ToolSpec(
            name=tool.name,
            description=tool.description,
            input_schema=tool.inputSchema,
            sensitive=is_sensitive,
            server=server,
            raw_name=tool.name,
        )

```

关键设计: – **注解驱动:** Java 端只管 @Tool 注解, 不用手写 sensitive 字段 – **自动同步:** 改 Java 端注解, Python 端下次 _refresh_tools 自动同步 – **零硬编码:** 不维护"哪些工具敏感"的列表

7.5 工具护栏 tool_guard.py

agent/lumio/services/bot/tool_guard.py:1-101 实现**双重护栏**:

```

# tool_guard.py (简化)
class ToolGuard:
    """工具护栏: role 授权 + 金额限额"""

    ROLE_RULES = {
        "report_card_lost": {"roles": {"customer"}, "amount_limit": None},
        "repay_credit_card": {"roles": {"customer"}, "amount_limit": 100_000},
        "adjust_temp_credit_limit": {"roles": {"customer"}, "amount_limit": 50_000},
        "redeem_points": {"roles": {"customer"}, "amount_limit": 1_000_000},
        "apply_permanent_limit": {"roles": {"customer", "agent"}, "amount_limit": 200_000},
    }

    async def check(
        self, server: str, tool_name: str, arguments: dict, actor_role: str = "customer"
    ) -> GuardDecision:
        # 1. role 授权
        rule = self.ROLE_RULES.get(tool_name)
        if rule and actor_role not in rule["roles"]:
            TOOL_GUARD_DENIALS.labels(tool=tool_name, reason="role_denied").inc()
            return GuardDecision(allowed=False, reason=f"role {actor_role} not allowed")

        # 2. 金额限额
        if rule and rule["amount_limit"]:
            amount = arguments.get("amount") or arguments.get("new_limit") or 0
            if amount > rule["amount_limit"]:
                TOOL_GUARD_DENIALS.labels(tool=tool_name, reason="amount_exceeded").inc()
                return GuardDecision(allowed=False, reason=f"amount {amount} exceeds limit {rule['amount_limit']}")

        return GuardDecision(allowed=True)

```

关键设计: – **role 授权:** agent 角色不能调 report_card_lost (客户专属), 防止越权 – **金额限额:** 客户单次还款 10 万上限, 调临时额度 5 万上限 (银行风控要求) – **指标发射:** tool_guard_denials_total{tool, reason} 拒绝原因分类 – **走 PII 脱敏前:** 护栏拒绝后不会泄露客户的尝试金额

7.6 调用全流程

```

sequenceDiagram
    participant LLM as LLM
    participant Exe as ToolCallingExecutor
    participant Guard as ToolGuard
    participant Pending as pending_action
    participant MCP as MCPToolClient
    participant Higress as Higress 网关
    participant Java as Java MCP Server

```

```

LLM->>Exe: tool_calls = [{name: report_card_lost, args: {card_id: "1234"}}]
Exe->>Exe: 敏感? 是 (destructiveHint)
Exe->>Guard: check(role=customer)
Guard->>Exe: allowed (pass)
Exe->>Pending: SET lumio:session:{id}:pending_action<br/>tool: report_card_lost, args: {...}
Exe->>LLM: 暂不执行, 询问客户"确认挂失?"

```

```

Note over LLM,Pending: 客户下一轮输入
LLM->>Exe: user said "确认"
Exe->>Pending: GET pending_action
Exe->>MCP: call_tool("report_card_lost", {card_id: "1234"})
MCP->>Guard: check (二次护栏, 防止 race)
Guard->>MCP: allowed
MCP->>Higress: POST /mcp<br/>streamable-http
Higress->>Higress: 鉴权 + 限流 + 脱敏
Higress->>Java: 转发
Java->>Java: @Tool report_card_lost
Java->>Higress: 挂失成功
Higress->>MCP: result (脱敏后)
MCP->>Exe: ToolCallResult
Exe->>Pending: DEL pending_action
Exe->>LLM: 工具结果, 二次生成
LLM->>Exe: 完整回答

```

关键设计: – 二次护栏: 客户确认时再 check 一次, 防止 race condition (确认期间 role 变化) – pending_action 5 态: 见 [第 3 章 3.5.4](#) – Higress 中间层: 鉴权/限流/脱敏/审计 4 件套统一在网关

7.7 工具调用循环

agent/lumio/services/bot/tool_executor.py 实现 LLM↔MCP 多轮循环:

```

# tool_executor.py (简化)
class ToolCallingExecutor:
    """LLM ↔ MCP 多轮循环, 最多 max_tool_iterations 轮"""

    async def run_conversation(
        self, messages: list[dict], tools: list[dict], max_iterations: int = 5
    ) -> str:
        for iteration in range(max_iterations):
            response = await self.llm.chat(messages, tools=tools)
            if not response.tool_calls:
                return response.content # 无工具调用, 结束

            for tool_call in response.tool_calls:
                # 1. 敏感工具 → pending_action
                if self.mcp.is_sensitive(tool_call.function.name):
                    decision = await self._ask_confirmation(tool_call)
                    if decision != "confirm":
                        continue
                # 2. 非敏感 → 直接调
                result = await self.mcp.call_tool(tool_call.function.name, tool_call.arguments)
                # 3. 消息追加
                messages.append({"role": "tool", "content": result.content, "name": tool_call.function.name})
            return response.content

```

关键设计: – 最多 5 轮: MCP_MAX_TOOL_ITERATIONS=5, 防止无限循环 – 每轮追加消息: tool_call 后 message 追加 tool result, LLM 下一轮可基于结果再决策 – 敏感工具异步: 挂失类不进循环, 走 pending_action 跨轮等待

7.8 Higress 网关集中治理

mcp-server/deploy/higress/mcp-credit-card.yaml 配置:

```

# Higress MCP 网关配置 (简化)
apiVersion: networking.higress.io/v1
kind: McpServer
metadata:
  name: lumio-mcp-server
  namespace: higress-system

```



```
spec:
  registries:
    - nacos:
        serverAddr: nacos:8848
        namespace: public
        serviceName: lumio-mcp-server
  tools:
    - name: report_card_lost
      allowRoles: [customer] # 网关层 role 拦截
      rateLimit: 5/minute
      piiFilter: true # 响应脱敏
    - name: repay_credit_card
      allowRoles: [customer]
      rateLimit: 10/minute
      piiFilter: true
      amountLimit: 100000 # 网关层金额限额
```

关键设计: – 网关层护栏: 即使 Python 端 ToolGuard 漏掉, Higress 仍能拦截 – 限流分散: 不同工具不同限流策略, 防止某工具被刷爆 – 响应脱敏: 网关层 mask PII, Python 端再 mask 一次 (双重保险) – Nacos 注册: MCP Server 启动时注册到 Nacos, Higress 自动发现

多 MCP 后端路由:

```
apiVersion: networking.higress.io/v1
kind: McpServerRoute
metadata:
  name: lumio-multi-domain
spec:
  rules:
    - match: { prefix: "card." }
      backend: lumio-mcp-credit-card
    - match: { prefix: "loan." }
      backend: lumio-mcp-loan
    - match: { prefix: "fund." }
      backend: lumio-mcp-fund
```

7.9 Python 端零回归

mcp_client.py 设计原则: opt-in, 零回归:

```
# mcp_client.py:272 (简化)
async def list_tools(self) -> list[ToolSpec]:
    if not self.connected:
        return [] # 不连接返回空, Bot 走 RAG 兜底
    return list(self._tools.values())
```

关键设计: – MCP_ENABLED=false 默认: 不启用 MCP, Bot 仍能 RAG 检索 – list_tools() 返 []: Bot _handle_tool 判断 not has_tool(intent.tool_name) 自动走 RAG – 启动期不抛错: MCP 不可用时只 warn 不 fatal, 不阻塞 Bot 启动

MCP 关闭时 Bot 行为: – LumioAgent._handle_tool -> self.mcp_client.has_tool(...) 返 False -> 走 _handle_knowledge 兜底 – 不影响主流程, 降级路径透明

7.10 监控指标

MCP 链路发射 6 个核心指标 — 回答"工具调用稳不稳、有没有被拒、连接健不健康":

指标	类型	含义	Labels	位置
mcp_tool_calls_total	Counter	MCP 工具调用次数与成败	tool, status (success/error)	mcp_client.py:309
tool_calls_total	Counter	工具循环内调用次数 (含 LLM 二次生成)	tool, status	tool_executor.py:298/309
tool_confirmations_total	Counter	敏感操作确认各状态 (复用第 3 章)	decision (pending/confirm/cancel/unclear/expired)	tool_executor.py:261

指标	类型	含义	Labels	位置
tool_guard_denials_total	Counter	护栏拦截次数 (角色不够/金额超限)	tool, reason (role_denied/amount_exceeded)	tool_executor.py:378
mcp_connection_state	Gauge	MCP 连接状态 (1=连上, 0=断开)	backend (default/card/loan/...)	mcp_client.py:80
http_requests_total	Counter	接口调用次数 (复用全局)	method, endpoint, status	全局

OTel span (mcp_client.py:308): - mcp.tool = 工具名 - mcp.server = 后端名 (单后端: default; 多后端: card/loan/fund) - mcp.is_error = 业务错误 (true/false) - mcp.duration_ms = 调用耗时

7.11 测试覆盖

agent/tests/ 中 MCP 相关:

- test_mcp_client.py (18 用例): 连接 / 工具列表 / 调用 / 路由
- test_tool_guard.py (17 用例, TOP 13): role 授权 + 金额限额
- test_confirmation.py (15 用例): 5 态确认状态机
- test_tool_executor.py (12 用例): LLM↔MCP 循环

Java MCP Server (mcp-server/src/test/java/): - CreditCardToolsTest (15 用例): 22 工具业务逻辑 - ToolCallAspectTest (5 用例): 切面计数/计时 - ApiKeyAuthFilterTest (3 用例): 网关鉴权

7.12 本章小结

MCP 工具集成是 Lumio 业务执行的"手脚":

- streamable-http 协议: 替代旧 SSE, 双向流 + 重连友好 + 多路复用
- 22 个 Java 工具: 7 业务域 (账单/交易/积分/额度/卡服务/分期/还款)
- destructiveHint 自动标记: 11 写操作自动加敏感白名单, 零硬编码
- 双重护栏: Python 端 ToolGuard + Higress 网关, 越权零容忍
- pending_action 5 态: 敏感操作等客户确认, 跨轮持久化
- 零回归 opt-in: MCP_ENABLED=false 默认, 不连接返回空, Bot 走 RAG 兜底
- Higress 集中治理: 鉴权/限流/脱敏/审计统一网关, Nacos 注册发现

下一章预告: [第 8 章 错误处理](#) 深入 35 错误码 + 统一响应体.

延伸阅读: - [第 3 章 Bot 自助问答](#) — pending_action 集成 - [第 4 章 坐席辅助引擎](#) — E1 中的工具调用 - [第 11 章 安全合规](#) — 工具护栏 + 网关鉴权 - [第 13 章 部署](#) — MCP Server opt-in profile - [第 17 章 工具调用与确认状态机](#), — 工具调用编排 + 5 态确认状态机 + 双重护栏

第 8 章: 错误处理

为什么需要一套自研错误体系

Lumio 是面向银行客户的对话平台,涉及支付、签约、转账等强合规场景。一个不能精确表达"业务问题"还是"系统问题"、一个会把"PG 密码错误"或"Redis IP"直接吐回给前端的错误响应,在生产环境是不可接受的。曾调研过直接抛 `HTTPException` 的方案,放弃的核心理由有三点: 第一,业务语义丢失——422 和 500 无法区分"用户输入问题"和"上游 LLM 抖动"; 第二,合规审计需要稳定的错误码而不是动态消息; 第三,前端需要按错误码做精细分支(比如 3005 非法状态转换要回到登录态,3001 知识库未命中要降级到兜底话术)。

基于这些约束,我们设计了 35 个错误码 + 5 段式分段 + `LumioError` 异常类层次 + 统一 JSON 响应体 的四层结构。本章沿着"为什么 → 是什么 → 怎么用 → 怎么演化"的顺序,把这套体系讲清楚。

错误码的 5 段划分

错误码不是随便分配的,每一段都对应一个明确的"责任方"和默认 HTTP 状态:

段位	含义	默认 HTTP	责任方	典型处理
1xxx	认证授权	401 / 403	调用方	重登录 / 申请权限
2xxx	输入校验	400	调用方	修正参数
3xxx	业务规则	422	业务层	流程分支
4xxx	外部依赖	502	第三方	降级 / 重试
5xxx	系统内部	500	我们	告警 / 排查

为什么这么划? 段位高位代表"问题离我们越远":1xxx 是"你(调用方)的问题",5xxx 是"我们的问题"。这样前端、监控、客服三方拿到错误码的第一位就能判断该谁介入。另一种常见思路是按 HTTP 状态码映射(400/422/500/502/503),我们评估后放弃,因为 HTTP 状态码只有约 10 个有效分类,远不够覆盖 Lumio 的业务场景——光"业务规则"一层就有 6 种完全不同的语义(未命中、未认证、高风险拦截、会话不存在、非法转换、并发冲突),挤在一个 422 里前端必须读 message 才能分支,这就把契约推给了不可控的字符串。错误码 5 段划分给了我们 $5 \times 99 = 495$ 个稳定槽位,足够未来 3 年扩展。完整的 35 错误码分配在 `agent/lumio/shared/exceptions.py:11-211`,下面给出每段的代表项:

段位	错误码	类名	含义
1xxx	1001	(Auth 模块)	认证失败,默认 HTTP 401
1xxx	1003	(Auth 模块)	鉴权失败,默认 HTTP 403
2xxx	2001	<code>IntentUnrecognizedError</code>	意图无法识别
2xxx	2002	<code>EntityIncompleteError</code>	实体抽取不完整
2xxx	2003	<code>QueryOutOfRangeError</code>	查询超出范围
2xxx	2010	<code>DocumentFormatError</code>	不支持的文档格式
3xxx	3001	<code>KnowledgeMissError</code>	知识库未命中
3xxx	3002	<code>CustomerNotAuthenticatedError</code>	客户身份未认证
3xxx	3003	<code>HighRiskBlockedError</code>	高风险业务拦截
3xxx	3004	<code>SessionNotFoundError</code>	会话不存在
3xxx	3005	<code>InvalidTransitionError</code>	非法状态转换

段位	错误码	类名	含义
3xxx	3010	IngestionConflictError	文档并发写入冲突
4xxx	4001	LLMTimeoutError	大模型推理超时
4xxx	4002	LLMInferenceError	大模型推理异常
4xxx	4003	BankAPIError	银行 API 调用失败
4xxx	4004	VectorSearchError	向量检索异常
4xxx	4005	EmbeddingServiceError	嵌入服务调用失败
4xxx	4006	EmbeddingTimeoutError	嵌入服务调用超时
4xxx	4007	BM25SearchError	BM25 检索异常
4xxx	4010	MinIOError	对象存储读写异常
4xxx	4012	DualWriteError	双写部分失败
4xxx	4020	CircuitBreakerOpenError	熔断器打开
5xxx	5001	SessionCorruptedError	会话状态损坏
5xxx	5002	ServiceOverloadedError	服务过载
5xxx	5003	StateConflictError	CAS 状态版本冲突
5xxx	5004	OrchestrationTimeoutError	编排全局超时

HTTP 状态映射:默认段位 + 精确覆盖

段位默认映射不够,有些错误需要"语义正确但状态码微调"。典型例子: `SessionNotFoundError` (3004)按段位默认是 422,但 REST 语义上"资源不存在"应该是 404; `InvalidTransitionError` (3005)用 409 Conflict 比 422 更贴切; `ServiceOverloadedError` (5002)按段位是 500,但过载应当返回 503 让上游重试。

因此 `agent/lumio/shared/middleware.py:26-32` 维护了一张精确覆盖表:

```
# agent/lumio/shared/middleware.py:26
_HTTP_STATUS_OVERRIDES: dict[int, int] = {
    SessionNotFoundError.code: 404,
    InvalidTransitionError.code: 409,
    ServiceOverloadedError.code: 503,
    1001: 401,
    1003: 403,
}
```

其余错误码走分段兜底: $2000 \leq code < 3000 \rightarrow 400$ 、 $3000-3999 \rightarrow 422$ 、 $4000-4999 \rightarrow 502$ 、 $5000+ \rightarrow 500$ 。这种"白名单覆盖 + 段位兜底"的设计,既保留了 5 段划分的简洁性,又允许对个别错误做语义化微调,不需要在每个异常类里硬编码 HTTP 状态。

统一响应体格式

所有错误响应都遵循同一形态,前端只需要写一套解析逻辑:

```
{
  "error": {
    "code": 3004,
    "message": "会话不存在: sess-9f8a",
    "type": "SessionNotFoundError"
  },
  "request_id": "8c4a1d2e-3b4f-4a2e-9c1d-2e3f4a5b6c7d"
}
```

三个字段各有职责: `code` 是稳定的数字错误码(逻辑分支), `message` 是面向用户的中文文案(可直接展示), `type` 是 Python 异常类名(开发联调用)。`request_id` 是后文要讲的全链路追踪锚点。这种"稳定码 + 动态文案 + 内部类名"的组合,在不泄露实现细节的前提下兼顾了前端、客服、研发三方诉求。值得特别说明的是, **`type` 字段在生产环境会被脱敏**(见下文),只在 `development` 环境保留真实类名,这样契约文档可以引用 `SessionNotFoundError` 这种稳定标识,实际线上又不会泄露库实现。

FastAPI 端由 `lumio_error_handler` 统一构造(`middleware.py:48-84`), `RequestValidationError` 走单独的 `validation_error_handler` (`middleware.py:86-99`, `code` 固定 2000, 响应体额外带 `details` 字段透出 Pydantic 校验明细), 其他未捕获异常走 `generic_error_handler` (`middleware.py:101-119`)。三个 handler 形成"已知业务异常 → 参数校验异常 → 完全未知兜底"的三段式拦截,任何路径上抛出的异常都会被收口成统一形态,不会让 FastAPI 默认的 500 页面穿透到客户端。

LumioError 异常类层次

异常类不是简单的"一个错误一个类",而是精心设计过的继承链。所有异常都继承自 `LumioError` 基类(`exceptions.py:11-22`),它在 `Exception` 基础上加了 `code` 和 `message` 两个类属性,允许子类只声明码和文案,无需重写 `__init__`:

```
# agent/lumio/shared/exceptions.py:11
class LumioError(Exception):
    code: int = 5000
    message: str = "系统内部错误"

    def __init__(self, message: str | None = None, code: int | None = None):
        if message is not None:
            self.message = message
        if code is not None:
            self.code = code
        super().__init__(self.message)
```

继承关系如下:

```
graph TD
    E[Exception] --> L[LumioError<br/>code=5000]

    L --> A1[AuthenticationError<br/>1001]
    L --> A2[AuthorizationError<br/>1003]
    L --> I1[IntentUnrecognizedError<br/>2001]
    L --> I2[EntityIncompleteError<br/>2002]
    L --> I3[QueryOutOfRangeError<br/>2003]
    L --> I4[DocumentFormatError<br/>2010]

    L --> B1[KnowledgeMissError<br/>3001]
    L --> B2[CustomerNotAuthenticatedError<br/>3002]
    L --> B3[HighRiskBlockedError<br/>3003]
    L --> B4[SessionNotFoundError<br/>3004]
    L --> B5[InvalidTransitionError<br/>3005]
    L --> B6[IngestionConflictError<br/>3010]

    L --> D1[LLMTimeoutError<br/>4001]
    L --> D2[LLMInferenceError<br/>4002]
    L --> D3[BankAPIError<br/>4003]
    L --> D4[VectorSearchError<br/>4004]
    L --> D5[EmbeddingServiceError<br/>4005]
    L --> D6[BM25SearchError<br/>4007]
    L --> D7[MinIOError<br/>4010]
    L --> D8[CircuitBreakerOpenError<br/>4020]

    L --> S1[SessionCorruptedError<br/>5001]
    L --> S2[ServiceOverloadedError<br/>5002]
    L --> S3[StateConflictError<br/>5003]
    L --> S4[OrchestrationTimeoutError<br/>5004]
```

怎么读这张树: 所有错误都是 `LumioError` 的孩子, 每个孩子带着自己的错误码和默认 HTTP 状态. 业务代码只抛具体的子类 (比如 `raise SessionNotFoundError(...)`), 中间件捕获后按错误码映射成统一 JSON — 这就是"异常类层次 + 错误码 + 统一响应体"三层结构的落点: 代码里抛的是语义 (`SessionNotFound`), 网络上传的是契约 (3004), 前端处理的是分支 (回登录/重试/降级)。

层次设计有两条原则: 第一, 5 大子类在语义上对应 5 段错误码, 通过单根继承让中间件 `isinstance(exc, LumioError)` 一行就能拦截所有业务异常; 第二, 个别异常允许自定义 `__init__` 接收上下文参数, 如 `SessionNotFoundError(session_id)`、`InvalidTransitionError(detail)`、`CircuitBreakerOpenError(executor_name)` (`exceptions.py:93`、`104`、`178`), 这样错误消息能带上具体定位信息, 运维不用翻 `trace`。

`StateConflictError` (`exceptions.py:200-204`)更进一步,在 `message` 里把 `CAS` 期望版本和当前版本都打出来,这种"自描述"异常在并发场景下能省掉一次 `grep`。

值得指出,我们刻意没有为每条错误码都建一个类——比如 `4001 / 4002` 都是 `LLM` 相关,但运行时常常分不清"是 `timeout` 还是 `inference error`",硬要分类反而会引入边界判断开销。当业务侧确实需要细分时,再从 `LLMError` 基类派生;当前保持 `35` 错误码对应的扁平结构,演进成本最低。

PII 脱敏贯穿异常 message

一个反直觉的设计:异常 `message` 也会走 `PII 脱敏`。看起来 `message` 是开发自己写的字符串,有什么可脱敏的?但实战中 `message` 经常携带用户输入(比如 "用户 `13812345678` 的请求被高风险拦截"),如果直接进日志,手机号就泄露了。

`PIIMaskFilter` (`agent/lumio/shared/logger.py:17-31`)挂在 `logging` 上,对所有 `record.msg` 和 `record.args` 都跑一遍 `mask_pii` (`pii.py:63-76`),会按"敏感字段 → 邮箱 → 身份证 → 银行卡 → 手机号"顺序脱敏,身份证和银行卡优先于手机号,避免 `11` 位数字被误判为手机号。这样我们就可以放心地把 `exc.message` 直接塞进 `logger.warning` 的格式化串,日志与对外响应双侧都不会泄露 `PII`。

request_id 全链路串联

排查生产问题时最大的痛点就是"用户报障 → 客服截图 → 研发翻日志"这条链断在中间。`Lumio` 的解法是给每个 `HTTP` 请求注入一个 `request_id`,贯穿日志、响应头、错误体:

```
# agent/lumio/shared/middleware.py:38
@app.middleware("http")
async def request_id_middleware(request: Request, call_next):
    request_id = request.headers.get("X-Request-ID") or str(uuid.uuid4())
    request.state.request_id = request_id
    response = await call_next(request)
    response.headers["X-Request-ID"] = request_id
    return response
```

设计上做了三件事:第一,支持透传——如果上游(网关、压测平台)带了 `X-Request-ID` 就用上游的,保证跨服务追踪能连上;第二,兜底生成 `UUID`——确保即使没上游也有锚点;第三,写回响应头——客户端报错截图时直接能看到 `ID`,客服不用再问研发"几点几分调的接口"。

业务侧通过 `request.state.request_id` 读取,日志侧通过 `logger.warning(..., extra={"request_id": ...})` 注入到结构化字段里 (`logger.py:71-72`),`JSON formatter` 会自动展开到 `log_entry` 的顶层。这样在 `Kibana` 里用 `request_id:"8c4a1..."` 一搜,就能把这次请求触发的所有日志串起来。`TraceContextFilter` (`logger.py:34-47`)还会把 `OpenTelemetry` 的 `trace_id / span_id` 一起注入,实现"日志 ↔ 链路"双向跳转——从 `Jaeger` 看 `trace` 能跳到日志,反之亦然。这两套 `ID` 不冲突: `request_id` 是 `Lumio` 自己的 `HTTP` 层锚点, `trace_id` 是 `OTel` 跨进程的追踪锚点,在网关入口和下游服务间需要分别保留。

生产环境不暴露内部异常类名

`generic_error_handler` 兜底所有未捕获异常,但生产环境会把类名替换成 `InternalError` (`middleware.py:107`):

```
exc_type = type(exc).__name__ if settings.environment == "development" else "InternalError"
```

原因:Python 异常类名会暴露使用的库(`asyncpg.PostgresError`、`httpx.ConnectError`、`pydantic.ValidationError` ...),这些都是攻击者的侦察情报。开发环境保留真实类名方便定位,生产环境一律抹平。完整 `traceback` 仍然走 `logger.exception` 进日志(`middleware.py:108`),研发用 `request_id` 关联即可。

健康检查脱敏

`_check_redis / _check_db / _check_es` 对外只返回分类码、详细信息走日志:

```
# agent/lumio/shared/health.py:29
def _error_response(dep_name: str, exc: Exception) -> dict[str, str]:
    logger.warning("health check %s down: %s", dep_name, exc, exc_info=True)
    return {
        "status": "down",
```

```
"error_code": _ERROR_CODE_BY_DEP.get(dep_name, "dependency_unreachable"),
}
```

`_ERROR_CODE_BY_DEP` (`health.py:20-26`)定义了 7 个稳定的分类码—— `redis_unreachable` / `postgres_unreachable` / `elasticsearch_unreachable` / `milvus_unreachable` / `minio_unreachable` / `llm_unreachable` / `embedding_unreachable` ——它们是面向客户端的"运维可定位但不含内部细节"的最佳折中。若把 `str(exc)` 直接吐给客户端,会泄露 `"password authentication failed for user \lumio\"`、`"ConnectionRefusedError: 192.168.x.x:6379"` 等敏感信息——这是合规审计的高危项。`logger.warning(..., exc_info=True)` 自动附加完整 `traceback` 到日志,运维侧要查 `192.168.x.x:6379` 还是去 ELK 查 `request_id` 关联的日志。

Bot 静默返空改为显式 503

Bot 在 LLM 不可达时**显式抛** `ServiceOverloadedError (5002)`,由中间件统一映射为 503,而非返回空列表:

```
raise ServiceOverloadedError("LLM 熔断中,请稍后重试")
```

设计动机:静默返空时前端以为"用户没说话"继续等待,实际是上游已经熔断。前端拿到 503 + 错误码 5002 后,可以走"稍后重试"提示,而不是傻等。5002→503 的映射在 `_HTTP_STATUS_OVERRIDES` 表里(`middleware.py:29`)。这是"错误处理不只是技术问题,而是产品决策"的体现——静默返空看似容错,实际是给用户制造假象。

LLM 空串回复:重试 + 熔断

模型返回空 `content` 与超时同样危险 — 客户端会收到 `done` + 空 `reply` (看似成功实则无内容):

```
# llm.py generate 循环 (简化)
if not content.strip():
    if attempt < max_retries - 1:
        await asyncio.sleep(0.5 * (2 ** attempt)); continue    # 重试
    self._breaker.record_failure()
    raise LLMInferenceError("LLM 返回空内容")                  # 熔断 + 走降级链
```

空串视为失败而非成功 — 触发熔断计数 + 降级链, 客户端拿到模板话术而非空回复。

降级回复 → 真实转人工

降级模板 (BILL/LIMIT/FAQ 等) 文案含"请输入转人工", 但**转接本身真实触发**:

```
# bot_agent _handle_knowledge / _handle_business (简化)
if result.source in ("template", "fallback") and not should_transfer:
    should_transfer = True
    transfer_reason = f"degraded_{result.source}: LLM 不可用, 降级回复"
```

- 客户看到"请转人工"时, 人工会话已创建 — 不必再发一条"转人工"消息
- 例外: `_handle_fallback` (chitchat 域) 仅模板回复时触发, LLM 正常闲聊不转

死信队列: 指标 + 告警 + 人工重放

消息重试 3 次仍失败 → 进死信 `lumio:chat:dead_letter` (maxlen 5000), 客户只收到 "系统处理您的请求时出现错误"。闭环处理:

机制	说明
指标	<code>lumio_dead_letter_writes_total{reason=retry_exhausted/agent_error}</code>
告警	每次写入打 ERROR 日志 (DEAD_LETTER: 消息进入死信队列 ... 需人工处理), Grafana alert 依赖
重放	<code>POST /admin/dead-letter/replay (admin)</code> — 按 <code>original_msg_id</code> 定位死信, 以原内容 XADD 回主 Stream 重走完整处理链, 成功后 XDEL 死信条目 + 计数 <code>lumio_dead_letter_replays_total</code>

机制	说明
查看	GET /admin/dead-letter?count=N (admin, 含 PII 需鉴权)

测试覆盖

错误处理不是"写完就好",必须有自动化测试把契约钉死。agent/tests/test_middleware.py 覆盖了关键映射与响应体:

- test_lumio_error_2xxx_returns_400 / test_lumio_error_3xxx_returns_422 / test_lumio_error_4xxx_returns_502 / test_lumio_error_5xxx_returns_500 验证 4 段默认映射;
- test_session_not_found_returns_404 / test_invalid_transition_returns_409 / test_service_overloaded_returns_503 验证 3 个白名单覆盖;
- test_error_response_format 钉死 error.code / error.message / error.type / request_id 四字段;
- test_generic_error_production_hides_type 验证生产环境 type 字段为 InternalError。

agent/tests/test_auth.py 则覆盖了 1001 / 1003 两条认证线的具体路径: test_decode_invalid_token_raises / test_decode_expired_token_raises 触发 1001, test_require_role_rejects_wrong_role 触发 1003。全链路契约通过这两个文件钉住,任何重构改了映射或响应体都会立刻红灯。

一条消息的四种错误命运:chat_send 全链路错误落点

错误码设计得再漂亮,最终要看业务代码怎么用它。以 Bot 入口 chat_send (bot/router.py:1130)为例,一条客户消息从进入到落库,沿途有 4 个"错误落点",每个落点对应不同的码段:

落点	触发条件	错误码	HTTP	客户看到
输入校验	空消息 (含全角空格/零宽字符清洗后为空)	2001 IntentUnrecognizedError	400	"消息内容不能为空"
注入拦截	命中已知注入模式 (忽略指令/角色混淆)	2001 IntentUnrecognizedError	400	"消息内容不符合规范"
业务规则	per-customer 活跃会话达上限	3010 BusinessError	409	"同时进行的会话数量已达上限"
系统过载	Redis 未就绪 (Stream 不可写)	5002 ServiceOverloadedError	503	"Redis 未就绪, 无法接收消息"

这个表格回答了一个常见疑问: 为什么"空消息"和"注入攻击"都是 2001? 因为从调用方的视角看, 两者都是"你发来的内容不可用", 修复动作都是"重新描述需求"——前端拿到 400 后展示的交互提示几乎一致, 细分错误码反而会让攻击者获得"我的注入模式被识别到了"的反馈。安全语义通过日志里的 pattern 字段保留(router.py 的 logger.warning("注入拦截 (入口): session=%s pattern=%s", ...)), 客户端只拿到统一的 2001。

chat_send 里还有一个值得注意的顺序: 注入检查在敏感词检查之前, 业务规则检查在幂等检查之前。顺序不是随手排的——注入必须最先拦 (成本最低), 幂等必须在写入 Stream 之前 (否则重复提交会 XADD 两次, 消费侧幂等键 _mark_processed 虽然能兜底, 但白白占一次 Stream 带宽)。

BusinessError 3010:一个"能说话"的业务异常

BusinessError (exceptions.py:216)是错误体系里最特殊的一个: 它的 code、message、status_code 三个属性全部可以在 __init__ 里覆写, 而其他错误类要么只覆写 message, 要么完全用类属性:

```
class BusinessError(LumioError):
    """3010: 通用业务规则错误 (如 per-customer 会话上限)"""
    code = 3010
    message = "业务规则不允许"

    def __init__(self, message: str | None = None) -> None:
        super().__init__(code=self.code, message=message or self.message, status_code=409)
```


设计动机: 业务规则的失败信息**必须带着具体限制**才有意义——"同时进行的会话数量已达上限 (3), 请先结束其他会话"比"业务规则不允许"对客户的指导价值完全不同。而错误码段位 3xxx 默认映射 422, 但"规则不允许"在 REST 语义上更接近冲突, 因此 `_HTTP_STATUS_OVERRIDES` 里显式登记了 `BusinessError.code: 409 (middleware.py:31)`。

注意 409 的覆盖是"登记制": 新增业务规则错误时, 如果漏了在 `_HTTP_STATUS_OVERRIDES` 登记, 它就会静默回落到 422——这是这个设计里最隐蔽的坑, 也是 `test_middleware.py` 里专门有一条 `test_business_error_returns_409` 钉死该映射的原因。

错误 vs 降级:两种"失败"的边界

Lumio 把"请求失败"和"内容降级"严格分成两个概念, 这是很多系统混淆的地方:

- **错误 (Error)**: 请求本身无法完成, 返回 4xx/5xx + 统一错误体。前端据此分支 (重试/提示)。
- **降级 (Degradation)**: 请求完成了, 但内容是次优的——LLM 挂了, 返回模板话术; 检索双路挂一路, 返回单路结果。HTTP 200, 前端无感。

降级不抛异常, 而是由 `ContentDegrader` 与 `DegradationManager` 在编排层内部接管, 并用 `lumio_degradation_level Gauge` (0=normal / 1=degraded / 2=fallback) 暴露当前降级深度。**为什么降级也要可观测?** 因为"客户问账单, Bot 回了模板话术"在 HTTP 层看起来完全正常, 只有指标能告诉运维"这个小时 40% 的回复是降级产物"——这就是第 10 章 `lumio_agent_responses_total{source="template"}` 与 `lumio_degradation_level` 两个指标存在的意义。

边界案例: 降级回复含"请输入转人工"字样, 但**转人工本身真实触发**(`bot_agent.py` 的 `should_transfer = True; transfer_reason = "degraded_template: ..."`)。这是刻意为之——LLM 不可用时客户诉求不能被"软拒绝", 降级 + 真实转接 = 既保住可用性又不欺骗客户。

LLM 错误包装规则:两个 4xxx 的边界判定

`LLMTimeoutError` (4001) 与 `LLMInferenceError` (4002) 是运行时最难区分的两个错误, `llm.py` 用三条规则把边界钉死:

1. **超时看墙钟**: `asyncio.wait_for` 抛 `TimeoutError` → `LLMTimeoutError` (命中断路器 + 重试计数);
2. **推理失败看调用结果**: 非 2xx 状态码 / JSON 解析失败 → `LLMInferenceError`;
3. **空响应是失败不是超时**: 模型返回 200 但 `content` 为空 → 重试 ($0.5 * 2^{\text{attempt}}$ 退避), 重试耗尽后 `record_failure()` + `LLMInferenceError` ——**绝不用 `LLMTimeoutError` 包装**。

第 3 条规则有一个隐蔽的坑: 如果把空响应也包装成超时, 断路器会误判"上游超时频发"而提前熔断, 实际只是模型输出策略问题 (比如 `temperature=0` 导致重复词被截断成空)。`llm.py` 里因此有一行专门的保护:

```
except LLMInferenceError:
    raise # 空响应错误不重包装成 LLMTimeoutError
```

这行代码的价值在线上故障时才能体现: 如果某天 Grafana 显示 `llm_timeout` 突增, 排障的第一步就是确认这些超时是不是"空响应被误包装"——有这行保护, 该指标就只反映真实超时。

客户端幂等:错误恢复的最后一道防线

at-least-once 语义下, 客户端重试、XAUTOCLAIM 重投递、双击提交都可能让同一条消息被处理两次。

`_mark_processed (bot/router.py:231)` 用 `lumio:processed:{message_id}` 幂等键 (TTL 300s, 覆盖 XAUTOCLAIM 60s 重投窗口) 保证"同一条消息只执行一次 Agent"。

幂等键的写入时机有讲究: **处理完成后才写, 处理中不写**。如果处理开始就写, 一旦 Agent 崩溃, 重投递的消息会被幂等键挡住, 客户消息就永久丢失了。反过来, 处理完成后不写, 重投递就会重复执行敏感操作 (比如重复挂失)。**"完成才写" + "消费侧跳过已处理"** 的组合, 在"不丢消息"和"不重复操作"之间取了正确的优先级——银行场景下, 丢消息比重复通知更不可接受。

小结与反思

Lumio 的错误处理经历了三个阶段:P0 设计 5 段码 + 异常类层次,P1 补上 `request_id` 与 PII 脱敏,P2/P3 收口合规与可观测性。这套体系的关键不是"错误码多",而是"错误码的语义稳定 + 响应体形态统一 + 全链路可追踪"。下一步值得讨论的是:错误码是否需要国际化和多语言?是否要把 5 段扩展到 6 段(增加 6xxx 表示"用户主动取消")?这些演进将在附录 A 术语表与后续章节中展开。

延伸阅读: - [第 11 章 安全合规](#) - JWT 错误 1001/1003 完整流程 - [附录 A 术语表](#) - 35 错误码速查 - [第 6 章 会话状态机](#) -
InvalidTransitionError 3005 详解

09 RAG 摄入管线

第 9 章: RAG 摄入管线

本章拆解 Lumio 把一份原始文档送进 ES + Milvus 的完整路径, 重点回答: 5 阶段为什么是这 5 个? 双写失败时怎么回滚? 如何做到增量版本管理?

9.1 5 阶段管线总览

先讲个业务场景: 银行的法务部上传了一份《信用卡章程》PDF (200 页). 这份文档要被客户问"年费怎么免"时检索到 — 但 200 页塞进系统可不像复制粘贴那么简单: PDF 要转文字, 页眉页脚要去掉, 长段落要切成小块, 每块要变成向量, 最后同时写进两个检索库. 任何一个环节出错 (PDF 解析失败/双写一半成功), 客户就可能问到**残缺的答案** — 这是合规事故.

文档摄入是 RAG 系统里最容易被低估的环节. 检索端 (第 5 章) 拿到的是已经量化的 chunk, 看上去只取决于 Embedding 模型; 但 chunk 的边界是否合理、元数据是否完整、版本是否可追溯, 全部由摄入管线决定. Lumio 把摄入拆成 5 个明确的阶段, 每个阶段都写一条 `kb_ingestion_log` 流水:

```
flowchart LR
    A[原始文档<br/>MD/HTML/PDF/DOCX/XLSX/TXT] --> B[Parse<br/>格式解析]
    B --> C[Clean<br/>5 步清洗]
    C --> D[Chunk<br/>递归分割]
    D --> E[Embed<br/>批量量化]
    E --> F1[ES 写入<br/>es_count == len]
    F1 -->|成功| F2[Milvus 写入<br/>asyncio.to_thread]
    F1 -->|失败| X1[FAILED<br/>doc.status=FAILED]
    F2 -->|成功| G[COMPLETED<br/>doc.chunk_count=len]
    F2 -->|失败| X2[回滚 ES<br/>_rollback_es_docs]
    X2 --> X1
```

为什么是 **Parse → Clean → Chunk → Embed → Dual-Write** 这五步? 核心约束是"先纯化, 再切分, 再向量化, 最后落库". Parse 阶段把二进制变成字符串, Clean 阶段去掉页眉页脚/控制字符等噪声, Chunk 阶段才能在干净的文本上找到语义边界; 如果反过来先切再清洗, 切出来的 chunk 边界很可能落在页眉上, 检索时把"第 3 页 / 共 20 页"当成正文返回. Embed 必须在 Chunk 之后: 一次性把整篇文档塞给 Embedding 服务, 显存压力和超时风险都不可控. Dual-Write 最后做, 是因为只有到这一刻, 我们才拥有"可以同时被 ES (BM25) 和 Milvus (向量) 检索"的完整对象.

`ingest_document()` 是顶层编排器, 在 `agent/lumio/services/common/ingestion.py:522` 起. 它先查文档记录, 把 `doc.status` 置为 `PROCESSING`, 然后线性走完 5 个阶段. `chunker` 逻辑内联进 `ingestion.py`, 阶段间只通过字符串 / 字典传递, 减少了临时状态序列化开销.

9.2 Parse 阶段: 6 种文件格式

Parse 阶段的目标只有一个: 把任何格式变成纯文本. 6 种格式的解析器集中在 `ingestion.py:54-188`, 通过 `_PARSE_DISPATCH` 字典按 `KbSourceType` 派发:

格式	库	入口函数	特点
Markdown	markdown-it-py	<code>parse_markdown (:54)</code>	走 token 树, 只取 text 节点, 块级元素插 \n
HTML	BeautifulSoup + lxml	<code>parse_html (:77)</code>	显式 decompose 掉 script/style/nav/footer/header
PDF	pymupdf (fitz)	<code>parse_pdf (:90)</code>	逐页 <code>get_text()</code> , 简单但对扫描件无 OCR
DOCX	python-docx	<code>parse_docx (:105)</code>	段落 + 表格分行, 表格用 \ 拼接
XLSX	openpyxl	<code>parse_xlsx (:125)</code>	<code>read_only=True</code> 省内存, 行格式 <code>header: value \ ...</code>
TXT	无	<code>parse_text_content (:150)</code>	仅 <code>strip()</code> , 直通

为什么 HTML 解析要主动 decompose nav/footer/header? 因为这些标签里几乎全是导航链接、版权信息、备案号, 检索时召回它们会污染 top-K 结果. 举个例子, 一份产品手册的"联系我们"区块如果被切成 chunk, 用户问"怎么退款" 时反而可能优先命中 footer 里的电话. 主动过滤是廉价的预防.

Markdown 的 token 树方案比正则替换更稳. 写法上 parse_markdown 不去管标题/列表语义, 只收集 inline.text 子节点, 然后在块级元素之间补 \n 维持段落感. 这种"语义忽略, 文本提取" 的策略在表格 / 代码块上有信息损失, 但配合后续 Chunk 阶段的中文句末优先断点已经够用.

PDF 是 6 种里唯一可能解析失败的(扫描件), 目前 parse_pdf 抛异常会冒泡到顶层 try/except, 直接把 doc.status 标记为 FAILED (:720). 未来如果要支持扫描件, 应在 Parse 内部接 OCR 而不是把噪声推给下游.

9.3 Clean 阶段: 5 步清洗

clean_text() 在 ingestion.py:211, 5 步串行:

```
# agent/lumio/services/common/ingestion.py:220
text = _RE_PAGE_HEADER_FOOTER.sub("", raw)      # 1. 去页眉页脚
text = _RE_CONTROL_CHARS.sub("", text)          # 2. 去控制字符
text = _RE_MULTI_SPACES.sub(" ", text)          # 3. 多空格合一
text = _RE_MULTI_NEWLINES.sub("\n\n", text)     # 4. 3+ 换行折叠为 2

# 5. 段落级 MD5 去重
```

5 步顺序不能换. 如果先合并空格再去控制字符, 某些控制字符会冒充"占位空格" 把两个有意义的 token 粘在一起; 如果先折叠换行再去页眉, 第 3 页 这类页眉被换行包夹, 正则匹配不到. 第 5 步的段落 MD5 去重专门处理 PDF 复制粘贴时的页眉残留: 即使前 4 步漏掉了 Lumio 内部资料, 同一份文档多个段落开头都是这 8 个字, 也只保留一份.

page_header_footer 正则 (ingestion.py:196) 同时匹配中英文:

```
# agent/lumio/services/common/ingestion.py:196
_RE_PAGE_HEADER_FOOTER = re.compile(
    r"(\s*\d+\s*页\s*/?\s*共\s*\d+\s*页|Page\s+\d+\s*of\s+\d+)",
    re.IGNORECASE,
)
```

re.IGNORECASE 让 Page 3 of 20 和 PAGE 3 OF 20 都能命中, 但代价是无法区分"正文里出现'第 3 页' 这种字面量" — 真实业务里这个概率极低, 当前可以接受.

9.4 Chunk 阶段: 递归字符分割

chunk_text() 在 ingestion.py:297, 默认 chunk_size=1500, overlap=200. 这里 1500 是中文字符数, 约等于 600-800 个英文 token, 对 bge-large-zh-v1.5 的 512 token 上限有充足余量 (一段中文 token 化后大约 1.8 倍字符数).

断点搜索在 _find_break_point() (:251), 3 个优先级:

1. 句末标点 _SENTENCE_ENDINGS = "。！？；\n" (:44)
2. 短语标点 _PHRASE_ENDINGS = ",,: '"]》" (:46)
3. 空格 (最后兜底)

搜索窗口是目标位置前后 ± 200 字符 (search_start = max(start, target - 200), :262). 为什么是 200? 它等于 overlap, 保证即使断点落在窗口最远端, 下一块也能从 break_pos - 200 继续, 不会和上一块完全无重叠.

为什么句末优先级最高? 因为中文的句号 / 问号 / 感叹号背后是完整的语义单元, 在这里断开 chunk 不会切断"主谓宾". 短语标点 (顿号 / 逗号) 通常连接并列项, 单独切断一个并列项会导致该 chunk 失去上下文. 空格是英文 / 数字的兜底断点, 命中率比前两个低很多.

9.5 Embed 阶段: 批大小 128 + 熔断器

embed_chunks() (:344) 把 chunks 按 batch_size=128 分批:

```
# agent/lumio/services/common/ingestion.py:362
for i in range(0, len(chunks), batch_size):
    batch = chunks[i : i + batch_size]
    embeddings = await provider.embed(batch)
    all_embeddings.extend(embeddings)
```

128 这个数不是随便选的. TEI (embedding.py:138) 的 bge-large-zh-v1.5 模型 batch=128 时单次请求约 60-80ms, GPU 利用率能跑到 70%+. 继续增大到 256 时延翻倍但吞吐只提升 30%, 反而拖慢端到端. 128 也正好是 HuggingFace TEI 默认值, 跨环境一致.

embedding.py 里有两个实现: OllamaEmbedding (本地开发, 默认 nomic-embed-text 768 维) 和 TEIEmbedding (生产, BAAI/bge-large-zh-v1.5 1024 维). 两者实现 EmbeddingProvider 协议 (embedding.py:26), 由 EmbeddingCircuitBreaker (:223) 包一层做健康探测.

熔断器配置是 3 失败开 / 2 成功关 / 30s 探测间隔:

```
# agent/lumio/services/common/embedding.py:230
self._probe_interval = 30.0
self._failure_threshold = 3
self._recovery_threshold = 2
```

为什么不"1 失败就开"? 因为单次健康探测可能因网络抖动失败, 1 次就熔断会让系统过于敏感. 3 次连续失败 (累计 90s 探测窗口) 几乎可以确认后端真出问题了. 关闭条件用 2 次成功而不是 1 次, 是为了避免"刚开又关"的抖动. 30s 探测间隔是经验值, 嵌入服务出问题一般 30s 内仍处于恢复阶段, 没必要更密.

熔断器初始 _is_open = True (:241), 首次 health_check() 成功后才关闭. 这是冷启动安全: 进程刚起时不假定 TEI 一定可用, 一定要探一次.

9.6 Dual-Write 阶段: ES 先, Milvus 后

Dual-Write 是整个管线最容易出错的地方. Lumio 选择 **ES 先, Milvus 后**, 不是没有理由. ES 写入是单文档 index() 调用, 失败可以精确定位到 chunk_id; Milvus 是批量 insert(), 失败时只能回滚整批. 顺序反过来如果 Milvus 失败, 已经写入 ES 的 chunk 没法追溯是哪些 (因为还没回填 doc_group), 会出现"ES 有文档但用户搜不到"的鬼影数据.

判定条件是 es_count == len(chunk_records) (:653):

```
# agent/lumio/services/common/ingestion.py:652
es_count = await write_to_es(chunk_records, es_client)
es_ok = es_count == len(chunk_records)
```

ES 写满才算成功, 哪怕只丢 1 个 chunk 也要标 FAILED. 这是"全或无"语义, 因为 ES 索引是检索的唯一入口, 缺一个就意味着少一条召回.

Milvus 写失败时调 _rollback_es_docs (:472) 逐个删除已写入的 ES 文档. 回滚失败也不抛出, 仅 logger.debug, 因为状态已经被主流程标为 FAILED, 后续重试会重新写 ES (用相同的 chunk_id, ES index 是 upsert 语义). 成功路径最后写 KbChunk 入库, 更新 doc.status = COMPLETED 和 doc.chunk_count = len(chunks).

```
sequenceDiagram
    participant U as 上传文档
    participant I as ingest_document
    participant E as Elasticsearch
    participant M as Milvus
    participant DB as PostgreSQL

    U->>I: doc_id, file_path, source_type
    I->>DB: doc.status = PROCESSING
    I->>I: Parse / Clean / Chunk / Embed
    I->>E: index(chunk_1..chunk_N)
    E-->>I: 200 OK x N
    I->>M: collection.insert(data)
    M-->>I: 异常
    I->>E: delete(chunk_1..chunk_N)
    E-->>I: 200 OK x N
    I->>DB: doc.status = FAILED
    I->>DB: kb_ingestion_log(MILVUS_WRITE, FAILED)
```

9.7 摄入日志: 7 阶段流水

kb_ingestion_log 表在 shared/orm_models.py:412, 配合 KbIngestionStage 枚举 (:93) 提供 7 个固定阶段:

PARSE → CLEAN → CHUNK → EMBED → ES_WRITE → MILVUS_WRITE → KAFKA_PUBLISH

注意 KAFKA_PUBLISH 在当前 ingestion.py 内联编排器里没有显式调用 — 它属于"摄入完成后通知下游"的发布阶段, 由上层服务 (例如 ingestion worker) 触发, 写日志时复用同一张表. 7 个阶段共用同一张流水表的好处是: 给定 doc_id, 一条 SELECT stage, status, duration_ms, step_detail, created_at ORDER BY created_at 就能复盘整个文档的生命周期.

每行写 (stage, status, duration_ms, step_detail), 其中 step_detail 是 JSON 字段, 容纳阶段特定的元数据. 例如 CHUNK 阶段写 {"chunk_count": N} (:594), EMBED 阶段写 {"embedding_dim": 1024} (:606), ES_WRITE 阶段写 {"success_count": M, "total": N} (:660). 这种"通用字段 + JSON 详情"的设计避免了阶段专属列膨胀.

9.8 增量摄入与版本管理

企业知识库是持续演化的, 同一份产品手册会有 v1.0 / v1.1 / v2.0 多个版本. Lumio 的版本模型不靠"先删后写", 而是靠 PostgreSQL 的 部分唯一索引:

```
-- orm_models.py:282
Index(
    "doc_group",
    postgresql_where=text("is_current_version = true AND is_deleted = false"),
    unique=True,
)
```

含义: 同一 doc_group 下, 只能有 1 条记录同时满足 is_current_version=true AND is_deleted=false. 增量摄入时, 旧版本自动设 is_current_version=false (即被 supersede), 新版本占位为 true. 部分唯一索引让数据库本身保证一致性, 应用层不需要分布式锁.

ES 和 Milvus 在检索时会带上 approval_status=PUBLISHED AND is_current_version=true 的过滤 (ingestion.py:409-411), 旧版本 chunk 物理上还在索引里, 但永远不会被召回, 既保证可回滚又保证检索不返祖.

9.9 测试覆盖

agent/tests/test_ingestion.py 覆盖三个核心场景类:

- TestCleanText (:18): 页眉页脚清除 / 多空格合并 / 控制字符清理
- TestChunkText (:58): 短文本直通 / 长文本分块 / 中文句末边界优先
- TestParse (:95): Markdown token 树 / TXT 直通

其中 test_chunk_respects_chinese_sentence_boundary (:81) 是最关键的一条: 它构造一段长文本断言 chunk 边界落在 。 而非中段, 直接锁定了第 9.4 节的优先级约定. Milvus 写入和 ES 回滚走集成测试 (需要真集群), 不在单测范围.

9.9.1 上传入口:从 HTTP 文件到 5 阶段管线的"最后一公里"

摄入管线的上游是 upload_document 端点 (bot/router.py:1443), 这一段的职责不是"解析文档", 而是在文件进管线之前把一切可预判的失败挡掉. 三个检查按成本从低到高排列:

1. 扩展名白名单 (_ALLOWED_EXTENSIONS): .pdf/.docx/.html/.md/.txt/.xlsx 六个后缀, 其余一律 400 (DocumentFormatError 2010). 白名单比黑名单安全——.exe / .sh 这类可执行文件连进入对象存储的机会都没有.
2. 文件大小上限 50MB: 优先读 Content-Length header (不读 body 就能拒绝), header 缺失时才读完整内容后按 len(content_bytes) 判定. 50MB 是经验值: 银行 PDF 章程通常 5-20MB, 批量上传峰值留 2-3 倍冗余; 再大就应走对象存储分片上传, 不应塞 HTTP body.
3. SHA-256 内容哈希: content_hash 写入 kb_document, 同一份文件重复上传时可按哈希识别去重, 也是将来做"内容变更检测" (内容变但文件名没变) 的锚点.

三个检查过后, 文件才进 MinIO ({category}/{filename} 二级 key), 再创建 KbDocument 记录 (status= PENDING), 最后调用 ingest_document() 触发 5 阶段管线并回写最终状态 (COMPLETED / FAILED). 注意端点的返回不阻塞在摄入完成: ingest_document 是

await 的, 但状态字段 (doc.status) 的最终值由摄入结果回填, 客户端可以通过 GET /kb/documents/{doc_id}/status 轮询——上传接口永远先返回, 长耗时在后台完成。

为什么摄入异常不向上抛? upload_document 里对 ingest_document 的调用包在 try/except 里, 异常时只把 kb_doc.status 置为 FAILED 。这样"文件已收、摄入失败"不会让客户拿到 500 重试上传——重试摄入比重传文件便宜得多 (MiniO 里已经有原件了)。

9.9.2 元数据如何影响检索:过滤不是检索后做的

DocumentMetadata 是摄入时写入、检索时消费的"契约"。它携带 8 个字段, 其中 5 个是检索过滤键:

字段	检索语义	典型值
category	业务分类过滤	fee / promo / rule
card_type	卡种过滤	gold / visa / 空=全卡种
customer_tier	客户等级过滤	vip / 空=全员可见
security_level	密级过滤	internal / confidential
keywords	检索时加权/扩展	逗号分隔, 摄入时拆分

设计要点: 过滤键在摄入时就要写对, 检索时只能消费。如果文档上传时漏填 card_type , 检索端不会"猜"——空值按"对全卡种可见"处理, 而不是按"对谁都不可见"处理。这符合银行知识库的可见性直觉: 未标注的文档默认公开给所有坐席, 密级文档必须显式标注才受限。

keywords 的拆分逻辑 ([k.strip() for k in keywords.split(",") if k.strip()]) 有一个边界: 全角逗号 , 不会被拆分——上传文档的人用中文输入法很容易踩中, 表现为"关键词只有一个超长串"。这是刻意保持的简单行为, 因为自动做全角归一化会引入"关键词被意外合并"的另一个错误方向, 而检索端对空关键词有兜底 (退化到纯 BM25 全文匹配)。

9.9.3 摄入失败后怎么重试:PARTIAL 索引 + 幂等重建

摄入失败 (ES 或 Milvus 写入异常) 后, 重试不是"重新上传一遍", 而是用相同的 chunk_id 重建索引:

1. 失败时 doc.status = FAILED , 但 kb_chunk 行 (含 chunk_id) 已经落库;
2. 后台重试 worker 通过 3 个 PARTIAL 索引 (embedding_status='PENDING' / es_indexed=false / milvus_indexed=false) 扫出未完成的 chunk;
3. 重试时 ES 用 index (upsert 语义) 而非 create ——同 chunk_id 幂等覆盖, 不会产生重复文档。

这个设计的巧妙之处在第 12 章的 es_indexed / milvus_indexed 状态字段: PG 是真相之源, ES/Milvus 只是投影。即使两个检索引擎的数据短暂不一致, worker 也能从 PG 的 PARTIAL 索引精确知道"哪些 chunk 缺哪个引擎", 不需要去对账两个外部系统。这也是为什么第 12.7 节的降级路由 (单路召回) 能成立——降级前先看 PG 状态, 而不是试错。

9.10 小结

Lumio 的摄入管线把"让 ES 和 Milvus 数据一致"这个难题拆成 5 个串行阶段 + 1 个对账回滚路径. Parse 阶段屏蔽 6 种格式差异, Clean 阶段用 5 步正则清洗噪声, Chunk 阶段在中文句末优先断点, Embed 阶段走 128 批 TEI + 3/2/30s 熔断器, Dual-Write 阶段用 ES 先 / Milvus 后 + _rollback_es_docs 保证原子性. 7 阶段 kb_ingestion_log 是出问题时的第一现场, 部分唯一索引让版本管理不依赖应用层逻辑.

延伸阅读: – [第 5 章 RAG 检索全链路](#) — 检索端细节 – [第 12 章 数据层](#) — ES/Milvus 索引设计 – [附录 A 术语表](#) — RAG 术语速查

10 可观测性

第 10 章: 可观测性

一个生产级多智能体系统,上线之后真正"难"的往往不是写代码,而是回答两个问题:「现在系统在做什么」「为什么它变慢了」。Lumio 把可观测性当作一等公民来设计——三个服务(lumio-bot / lumio-assist / lumio-mcp-server)共享同一套指标、日志与追踪基座,运维与开发可以基于同一份"事实"对话。

本章的叙述顺序会沿着「数据怎么产生 → 怎么聚合 → 怎么呈现 → 怎么告警」的链路展开,先讲三大支柱的概念,再分别看 Prometheus 指标、JSON 日志、OTel 追踪的实现细节,最后落到仪表盘与告警规则上。

10.1 三大支柱:Metrics / Logs / Traces

可观测性的经典三角是指标(Metrics)、日志(Logs)、追踪(Traces)。Lumio 的实现选择不是巧合,而是各取所长:

- **Metrics(Prometheus)**——低成本、高基数友好,适合「看趋势、做告警」。Prometheus 每 15s 主动拉取一次,数据在 TSDB 里以 counter 增量和 histogram bucket 形式长期存储,几 GB 就能覆盖数月的业务时序。
- **Logs(JSON)**——高上下文,可携带完整请求体与业务字段,适合「排障时翻记录」。Loki 这类日志聚合系统对 JSON schema 友好,字段级查询的体验接近数据库。
- **Traces(OTel)**——单次请求跨多个服务、多个函数的串联视图,适合「定位慢在哪一跳」。一条 trace 可以挂几十个 span,每个 span 自带耗时、attributes、events。

```
graph LR
    Req[请求] --> M[Metrics<br/>聚合视图]
    Req --> L[Logs<br/>单条详情]
    Req --> T[Traces<br/>调用链路]
    M -. 告警.-> Op[运维]
    L -. 排障.-> Dev[开发]
    T -. 定位.-> SRE[SRE]
    L <-. trace_id 关联 .-> T
    M <-. endpoint 关联 .-> L
```

怎么读这张三角图 — "一次客户投诉的排查": 客户说"刚才问账单特别慢". 排查路径是 — ① 看 **Metrics**: 哪个端点的延迟飙了(账单查询 P99 从 200ms 变 3s); ② 看 **Traces**: 慢在哪一跳(是 ES 慢还是 LLM 慢); ③ 看 **Logs**: 那次调用到底发生了什么(超时? 重试?). 三者靠 `trace_id` 串成一条线: 从"哪个端点慢"(指标)下钻到"哪次调用报错"(日志),中间不用人工拼数据。

为什么三条都要, 不能只留一条: 只有指标 → 知道"慢了"但不知道为什么; 只有日志 → 每条都有细节但看不出趋势; 只有追踪 → 单次调用很清楚但没法发现"整体在恶化". 三者是**望远镜(趋势)**、**放大镜(细节)**、**透视镜(链路)**的关系。

三者通过 `trace_id` 和 `endpoint` 互相串联:同一次请求产生的所有日志都会带上当前活跃 span 的 `trace_id` (logger.py:34-47),而指标又可以按 `endpoint` 维度聚合——这样从"哪个端点慢了"下钻到"具体哪次调用报错了"是平滑的,中间不需要人工做 join。

值得提醒的一点:三大支柱不是"加得越多越好". 每加一个指标,TSDB 就要多一份存储;每加一条日志,Loki 就要多一份索引;每开一个 span,OTel collector 就要多一份处理。Lumio 在选型时严格遵循「**指标回答 what、日志回答 how、trace 回答 where**」的分工——能用 counter 回答的就不打 log,以此控制观测成本。

10.2 17 个 Prometheus 指标

`agent/lumio/shared/metrics.py` 是两个 Python 服务共用的指标注册表。指标选型遵循两个原则:① 优先 **Counter/Histogram** 而非 **Gauge**, 因为前者是单调累加的,Prometheus 的 `rate()` 函数天然友好;② **label 维度控制在可枚举范围内**,避免高基数把 TSDB 撑爆。

按命名空间分两组:

通用域:`http_*` / `llm_*` / `tool_*` / `session_*`

这五个名字是"领域无关"的,任何 Web 服务都能复用:

指标名	类型	含义	关键 labels	来源
http_requests_total	Counter	接口调用次数 (按方法/路径/状态码)	method / endpoint / status	metrics.py:18-22
http_request_duration_seconds	Histogram(9 buckets)	接口响应耗时分布 (看 P99 是否超标)	method / endpoint	metrics.py:24-29
session_transitions_total	Counter	会话状态转换次数	from_phase / from_sub / to_phase / to_sub / reason	metrics.py:33-37
session_timeouts_total	Counter	会话超时触发次数	sub_phase / reason	metrics.py:39-43
session_phase_duration_seconds	Histogram	各子阶段停留时长	sub_phase	metrics.py:45-50
tool_calls_total	Counter	工具调用次数与成败	tool / status	metrics.py:54-58
tool_confirmations_total	Counter	敏感操作确认各状态 (确认/取消/模糊/过期)	decision	metrics.py:60-64
tool_guard_denials_total	Counter	工具护栏拦截次数 (角色不足/金额超限)	tool / reason	metrics.py:66-70
llm_call_duration_seconds	Histogram	大模型调用耗时 (桶延伸到 60s, 推理天然慢)	model / method	metrics.py:74-79

注意 http_request_duration_seconds 的桶是 [0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.5, 5.0, 10.0] ——这是经验值,既覆盖了健康路径 (0.1s 内),又给 P99(5~10s)留了尾部空间;而 llm_call_duration_seconds 的桶一直延伸到 60s,因为 LLM 推理天然就慢。

业务域: lumio_*

剩下 8 个指标全部以 lumio_ 为前缀,代表「只对 Lumio 业务有意义」:

指标名	类型	关键 labels	用途
lumio_fast_reply_total	Counter	—	并发满载时的模板兜底计数
lumio_agent_responses_total	Counter	source	区分 llm / template / fallback / tool_*
lumio_bot_semaphore_utilization	Gauge(0~1)	—	Agent 信号量占用率
lumio_active_workers	Gauge	—	活跃会话 worker 数
lumio_stream_length	Gauge	—	Redis Stream 当前长度
lumio_stream_pending_total	Gauge	—	PEL 待确认消息数
lumio_retrieval_duration_seconds	Histogram	search_type	hybrid / bm25 / vector
lumio_degradation_level	Gauge(0/1/2)	—	0=normal, 1=degraded, 2=fallback

为什么分通用/业务两套前缀? 这是有意识的命名空间选择:通用域用 http_* 而不是 lumio_http_*,这样未来如果 Lumio 要开源指标 SDK,或被其他项目引用,这些名字不会显得"私有";而 lumio_* 名字本身就是一道"业务边界",dashboard 上 rate(lumio_*) 一筛,就能拿到 Lumio 自己的健康度视图,不会被通用 HTTP 噪声淹没。

另外,业务域里三个 Gauge(semaphore_utilization / active_workers / stream_pending)反映的是 Bot 运行时状态,由 router 的监控循环周期性刷新——这些 Gauge 不在 PrometheusMiddleware 里写,而是有专门的 background coroutine 每 5s 同步一次。这种"指标自治"的拆分让中间件不必知道业务有多少种 Gauge,新加业务指标只需要在 router 里写,不需要改 shared 库。

另有 3 个 session 维度的指标: session_phase_duration_seconds 用来观察每个子阶段(等待用户/处理中等)的停留分布,桶覆盖 5s 到 3600s; tool_confirmations_total 区分"用户确认 / 取消 / 模糊 / 过期"四种决策,帮助产品观察敏感工具的用户体验; tool_guard_denials_total 区分"角色不足"与"额度超限",给安全团队一个独立的拦截监控面。

PrometheusMiddleware:自动打点与反馈循环防护

Starlette/FastAPI 的请求打点用 PrometheusMiddleware 自动完成(metrics.py:138-160)。它在 try/finally 里包住下游,无论请求成功还是抛异常,都会写入计数与耗时:

```
# metrics.py (第五轮修复后)
async def send_wrapper(message):
    nonlocal status_code
    if message["type"] == "http.response.start":
        status_code = message.get("status", 200)
    await send(message)

try:
    await self.app(scope, receive, send_wrapper)
finally:
    duration = time.perf_counter() - start
    endpoint = _normalize_metric_path(path) # 高基数防护
    REQUEST_COUNT.labels(method=method, endpoint=endpoint, status=status_code).inc()
    REQUEST_LATENCY.labels(method=method, endpoint=endpoint).observe(duration)
```

高基数防护 (第五轮修复): 纯 ASGI 中间件执行早于路由匹配, scope["route"] 恒为 None — 直接取原始 path 会让 /api/sessions/{uuid}/messages 每个会话一个时间序列, 打爆 TSDB。现用 _normalize_metric_path 做模板化归一化: 8+ 字符且含数字/连字符的路径段 (UUID/长数字) → {param}, 纯字母短词 (chat/send/health) 不受影响:

```
_PATH_PARAM_RE = re.compile(r'/(?=[0-9a-zA-Z-]*[0-9-])[0-9a-zA-Z-]{8,}')
# /api/sessions/550e8400-.../messages → /api/sessions/{param}/messages
```

这里有一个常被忽略但很致命的细节: _EXCLUDED_PATHS = {"/metrics", "/health", "/favicon.ico"} (metrics.py:129)。如果不排除 /metrics 端点本身,Prometheus 每 15s 抓一次就会产生一条新指标,这个指标又会触发下一次抓取——指标反馈循环会让 TSDB 在几小时内被自己的噪声打爆。/health 同样如此(LB 健康检查通常 QPS 极高,会把 http_request_duration_seconds 的 P99 拉到不可信)。这个 3 项白名单是防御反馈循环的关键,被锁定为"不允许删"。

10.3 JSON 结构化日志

人类读文本日志方便,但机器检索/Loki 解析/ELK 聚合都需要结构化。Lumio 提供两套格式,生产推荐 JSON。shared/logger.py 里三个 filter/format 协作完成:

- PIIMaskFilter (logger.py:17-31):在 record 落到 handler 之前调 mask_pii (shared/pii.py),把手机号、身份证、邮箱、敏感键值替换成 ***。这层必须放在 handler 上而不是业务代码里,否则任何 logger.info(f"user {phone}") 都会漏网。
- TraceContextFilter (logger.py:34-47):从 tracing.get_trace_context() 拿当前活跃 span 的 (trace_id, span_id),挂到 record 上。这是「日志 ↔ 链路」关联的关键。
- JSONFormatter (logger.py:50-73):把 record 序列化固定 schema: timestamp / level / logger / message / module / function / line / exception / extra;有 trace_id 时附在 trace_id / span_id 字段,这样 Loki 里查 trace_id="abc..." 就能拉出这一次请求的全部日志。注意 TraceContextFilter 在没有活跃 span 时完全不写字段,这是有意的——零开销,零噪声。如果无脑写空字符串,反而会让日志解析时多一层 if 判断。setup_logger() 的设计是"幂等":已经 attach 过 handler 的 logger 直接返回,避免重复添加导致一条日志被打印多次。

10.4 OpenTelemetry 全链路追踪

Lumio 服务的追踪基于 OTel SDK,封装在 shared/tracing.py 里。设计哲学是"业务代码零 OTel 依赖"——开发者只 @traced("Agent.run"),不需 import 任何 opentelemetry 包。

探针与单例 TracerProvider

_init_tracing() (tracing.py:74-135)是单例,模块级 flag _provider_initialized 保证只跑一次。它做四件事:

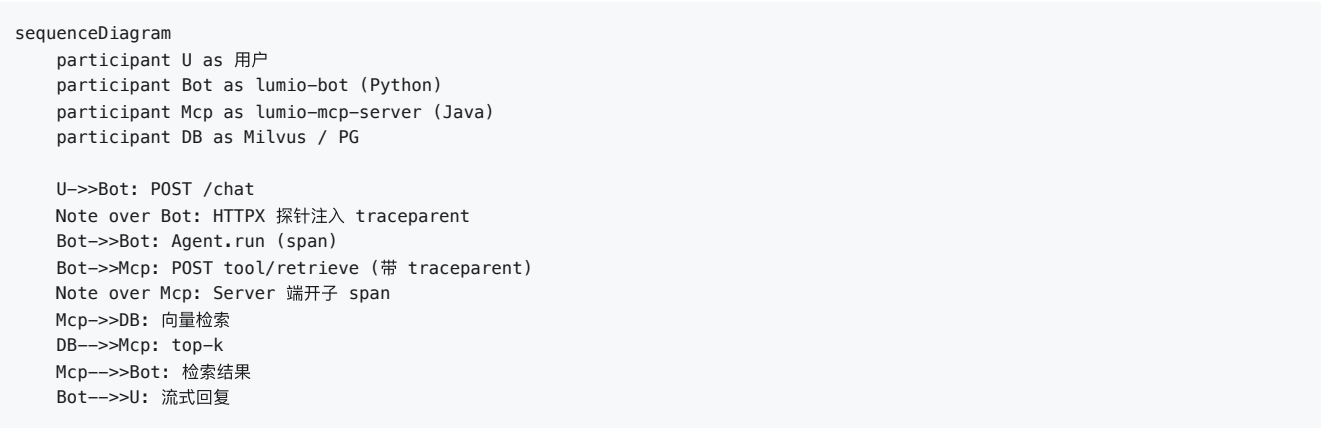
1. 从 ObservabilitySettings 读 enabled / jaeger_host / otlp_endpoint;
2. 构造 Resource,挂上四个属性(下文详述);
3. 选 ParentBasedTraceIdRatioSampler 而不是裸 TraceIdRatioSampler ——这是跨服务 trace 能否串起来的关键;
4. 注册 OTLPSpanExporter + BatchSpanProcessor,默认发到 http://{jaeger_host}:4318/v1/traces。

instrument_app(app, app_name) (tracing.py:138-163)装三个自动探针: FastAPI、Redis、HTTPX。前两个大家熟悉,第三个是重点:

```
# tracing.py:155-160
# HTTPX 探针:MCP streamable-http 每次出站 POST 自动注入 W3C traceparent,
# 使 Python 客户端 span 与下游 Java server span 串成同一条链路。
with contextlib.suppress(Exception):
    from opentelemetry.instrumentation.httpx import HTTPXClientInstrumentor
    HTTPXClientInstrumentor().instrument()
```

Lumio Bot 通过 HTTPX 调用 Java 写的 lumio-mcp-server,而 MCP streamable-http 本身就是普通 HTTP POST。没有 HTTPX 探针的话,Python 侧的 tool_calls span 和 Java 侧 server span 是断开的——你只能看到"我调了 MCP",看不到"对方做了什么、慢在哪"。HTTPX 探针在每次出站请求自动塞 W3C traceparent header,下游收到后接着开子 span,自然连成一条树。

跨服务 trace 流



上图中,Bot 端 trace 和 MCP 端 trace 共享同一个 trace_id,只在 Jaeger UI 里以"不同 service 但同一 trace"展示。开发只需要在 Jaeger 里按 trace_id 一搜,就能从"用户消息进 bot"一路下钻到"Java 端向量检索用了多少 ms"。

Resource 属性

_init_tracing 给每个 span 挂上 4 个 resource 属性(tracing.py:108-115):

属性	值	用途
service.name	lumio-bot / lumio-assist / lumio-mcp-server	Jaeger 按服务过滤
service.namespace	lumio	业务域聚合
service.version	见下文优先级	发版前后对比
deployment.environment	从 Settings.environment 读	区分 dev/staging/prod

service.version 的读取优先级是 (tracing.py:24-54):

```
LUMIO_VERSION env > pyproject.toml [project].version > 0.0.0
```

设计要点:只读 env 时,本地开发没设 env 就拿到空字符串,Jaeger 看到的所有 span 都是"未版本号化"的,排查"这个 bug 在哪个版本引入"非常费劲。因此从 pyproject.toml 读 PEP 621 的 [project].version 兜底,真正"开箱即用",CI 上再被 LUMIO_VERSION 覆盖。兼容旧字段 LUMIO_TRACING_ENABLED 用 Pydantic 的 AliasChoices 做了别名兼容,升级期不会破坏现有 .env。

_read_service_version 还做了三件小事:① 用 tomlib (Python 3.11+)或 toml 回退,跨版本都跑得起来;② 兼容 Poetry 1.x 的 [tool.poetry].version ;③ 读不到时返回 "0.0.0" 而不是抛异常,符合 OTel 规范对 service.version 的"non-empty string"要求。

采样策略

ParentBasedTraceIdRatioSampler(sampling_ratio) (tracing.py:118)的语义是:

- 如果上游请求带了 traceparent ——跟随上游决策(上游采样我也采,上游丢弃我也丢);

- 没有上游——按本地 `sampling_ratio` 决定。

这避免了"本地配 10% 但因为上游被丢弃导致整条 trace 缺一半"的尴尬。生产默认 1.0(全采),压测或高 QPS 时降到 0.1。

10.5 审计中间件:状态变更的"事后账本"

`shared/audit_middleware.py` 解决的是合规问题:谁、什么时候、对哪个对象、做了什么操作。这部分用 fire-and-forget 异步写库,不阻塞主请求路径:

```
# audit_middleware.py:48-53
try:
    import asyncio
    asyncio.create_task(_write_audit_log(request, response, elapsed_ms))
except Exception:
    logger.debug("审计日志创建任务失败: %s %s", request.method, path)
```

`_write_audit_log` (`audit_middleware.py:58-111`)从 JWT 解出 `actor_id` / `actor_role`,再调 `_infer_action` 推断操作类型。`actor_id` 默认 "anonymous",在开发环境下会用 `dev-user` 兜底,方便本地手测时不必每次都拿 token。`detail` 字段除了 `elapsed_ms`,还会记录脱敏后的 query params——这条细节在合规审计里很重要:如果某个 DELETE 出问题了,审计员需要能复现当时的过滤条件。

核心是 `_ENDPOINT_ACTION_MAP` (`audit_middleware.py:174-206`):27 个端点函数名 → (`action`, `target_type`) 的精确映射表。

`_infer_action` 的优先级是「先查路由元数据,再 fallback 到路径字符串」:

```
# audit_middleware.py:124-132
route = request.scope.get("route")
if route is not None and hasattr(route, "endpoint"):
    endpoint_name = route.endpoint.__name__
    mapped = _ENDPOINT_ACTION_MAP.get(endpoint_name)
    if mapped:
        action, target_type = mapped
        target_id = _extract_target_id(request.url.path, target_type)
        return action, target_type, target_id
```

为什么"路由元数据"比"路径字符串"更可靠?因为路径可以被改、被翻译、被装饰器插入,例如 `PUT /api/v1/session/{id}/hold` 既可能被认成 `session.transition` 也可能被认成 `session.hold`;但 `hold_session` 这个函数名唯一对应 `session.hold`。仅用路径推断容易"action 记错"。GET 类不审计(`_AUDITED_METHODS` 只含 POST/PUT/PATCH/DELETE),纯读操作不需要合规留痕。

24 个端点覆盖了三类业务:`assist/router.py` 的会话/反馈/通知/复盘类、`bot/router.py` 的聊天/文档类、`faq_router.py` 的 FAQ 全生命周期类,加上 `auth_router.py` 的 login。新增端点时,需要同步在映射表里登记,否则会回退到路径推断——这是 code review checklist 的固定项。审计模块的 connection pool 与请求池分开,即使审计慢也不会拖垮主业务。

10.6 Grafana 仪表盘与告警规则

3 套 dashboard 各有分工:

- `config/grafana/dashboards/lumio-overview.json` —— **15 panel** 业务总览,核心是 HTTP 速率、Agent 槽位利用率、PEL 待处理数、P50—P99 延迟分位、会话阶段转换漏斗。这个 dashboard 是 SRE 日常巡检的主入口。
- `config/grafana/dashboards/middleware.json` —— **15 panel** 基础设施,顶部 6 个状态卡实时展示 Bot / Assist / Redis / PostgreSQL / Kafka / Milvus 的 up 状态。一旦某个依赖变红,运维能在第一时间收到 alert 通知。
- `config/grafana/dashboards/lumio-dashboard.json` —— 业务概览(111 行),关注 Lumio 自身核心 KPI,例如"每小时回答多少条消息""LLM 调用与模板兜底的比例"。

告警规则集中在 `config/prometheus/rules/alerts.yml` 6 条:

规则	触发条件	持续	严重度
ServiceDown	<code>up==0</code> (bot/assist/mcp 任意)	1m	critical
MCPToolErrorRateHigh	MCP 工具错误率 > 10%	5m	warning
PyToolErrorRateHigh	编排层工具错误率 > 10%	5m	warning

规则	触发条件	持续	严重度
LLMLatencyP99High	LLM p99 > 15s	5m	warning
HTTPLatencyP99High	HTTP p99 > 5s	5m	warning
SessionTimeoutSpike	超时速率 > 1/s	5m	warning

ServiceDown 是唯一 critical,因为单个服务下线意味着用户体验直接受损;其余都是 warning,给 SRE 留出调查窗口。两类工具错误率分开告警是因为 MCP 错误可能来自下游 Java 服务,Python 侧错误可能来自编排逻辑,二者的根因和处置路径完全不同。

告警规则里有几个值得展开的细节: `clamp_min(..., 1e-9)` (`alerts.yml:23-24`)在分母为 0 时兜底,避免 `rate(...)/0` 出现 NaN (Prometheus 对 NaN 的处理是"不触发告警",反而让错误隐藏); `histogram_quantile(0.99, sum(rate(...)) by (le))` (`alerts.yml:49`)的分位计算必须在 `by (le)` 之后聚合,这是新人常踩的坑;所有 warning 都用 `for: 5m` 过滤掉抖动——如果用 `for: 1m`,一次 GC 抖动就会拉一堆告警,让 on-call 同学对告警失去敏感度。Alertmanager 在配置里是 opt-in,默认只让 Prometheus 把告警状态显示在 UI 的 Alerts 页,适合还没接 Slack/钉钉的小团队。

10.6.1 三个业务指标的完整生命周期:fast_reply / timeout / merge

lumio_* 业务域里最容易让新人困惑的是三个"消息命运"计数: `fast_reply`、`timeout`、`merge`。它们不是独立存在的,而是同一条消息在 Bot 队列里不同命运的分支,全部由 `_metrics` 字典 (`bot/router.py:933`) 汇总,再同步到 Gauge:

指标	含义	触发场景
lumio_fast_reply_total	信号量满荷时的模板兜底	高峰期 Semaphore(10) 全占,新消息不等 Agent, 直接回固定话术
lumio_stream_pending_total(timeout 累计在 stats)	消息过期/重试耗尽	_enqueue_time 超过 message_ttl=8s, 或 XAUTOCLAIM 重试 3 次失败
merge (stats merge_total)	调用前队列检查时合并的消息数	用户连续发多条消息, worker 合并 ≤5 条/≤4000 字符后一次 LLM 调用

用这三个指标排一次"回复慢"的故障: 客户反馈"高峰期消息 30 秒不回"。先看 `lumio_fast_reply_total` 的 `rate()` ——如果陡增,说明信号量持续满荷,回复其实是"快速兜底话术"而不是 Agent 回复,方向是扩容或降并发;如果 `fast_reply` 没动但 `BOT_SEMAPHORE_UTILIZATION` 贴近 1.0,说明 Agent 处理本身在排队,方向是查 LLM 延迟;如果两个都没动但 `stream_pending` 的 PEL 持续堆积,方向是查消费 worker 是否存活 (XAUTOCLAIM 是否在认领)。一个指标回答不了的问题,三个指标的组合可以——这就是为什么它们被刻意做成三个独立时序而不是一个聚合值。

`_metrics` 的同步机制值得一提: 它由 `_monitoring_loop` 后台协程每 15s 采样一次 (PEL 数、Stream 长度、活跃 worker 数、信号量利用率),写入四个 Prometheus Gauge (`BOT_STREAM_PENDING` / `BOT_STREAM_LENGTH` / `BOT_ACTIVE_WORKERS` / `BOT_SEMAPHORE_UTILIZATION`),同时 `/health` 端点返回同一份快照。指标与健康检查共享数据源——SRE 在 Grafana 看到的值,和调用 `/health` 看到的值永远一致,不会出现"仪表盘说满了,健康检查说正常"的对不上。

10.6.2 一条日志的完整 schema

JSON 日志的字段不是随手列的,每个字段都有消费方:

```
{
  "timestamp": "2026-08-05T10:23:45.123Z",
  "level": "WARNING",
  "logger": "lumio.services.bot.router",
  "message": "客户输入包含敏感词: session=abc123 hits=['赌博']",
  "module": "router",
  "function": "chat_send",
  "line": 1173,
  "trace_id": "8c4a1d2e3b4f4a2e9c1d2e3f4a5b6c7d",
  "span_id": "3f2a1b4c",
  "extra": {"request_id": "req-001", "session_id": "abc123"},
  "exception": null
}
```

- `logger/module/function/line` 四件套给 Loki 的 `|=` 字段查询用——排障第一步永远是 `{logger="lumio.services.bot.router"} |= "敏感词"`;
- `trace_id/span_id` 只有在有活跃 span 时出现 (TraceContextFilter 的"零噪声"设计), 从日志跳 Jaeger 的入口;
- `extra` 是业务侧用 `logger.warning(..., extra={...})` 注入的结构化字段, JSON formatter 把它展开到顶层——不要在 `message` 里拼 `key=value` 字符串, 那是让 Loki 解析器白干活的写法;
- `exception` 由 `logger.exception` 自动携带完整 `traceback`, 生产环境错误响应虽然抹平了类名, 但 `traceback` 完整保留在日志里。

10.6.3 审计日志的一次完整案例

审计中间件的"事后账本"属性, 用一次 `hold_session` 调用看最直观:

```
actor_id=agent-07 actor_role=agent action=session.hold
target_type=session target_id=sess-9f8a method=POST
path=/api/session/sess-9f8a/hold status_code=200 elapsed_ms=32
```

这条记录同时写 `audit_log` 表 (合规取证) 和 stdout JSON (ELK 聚合), 两者字段一致。为什么 `action` 是 `session.hold` 而不是路径字符串? 因为路径推断在 `PUT /api/v1/session/{id}/hold` 这类路径上存在歧义 (`/update` 子串会先命中), 而 `_ENDPOINT_ACTION_MAP` 按函数名 `hold_session` 精确映射——函数名是代码里唯一的, 路径不是。

审计里还有一个细节: `_AUDITED_METHODS` 只含 `POST/PUT/PATCH/DELETE`, `GET` 不审计。这是刻意的——纯读操作不改变状态, 不需要合规留痕, 而且 `GET` 占了流量大头, 全量审计会淹没真正有意义的变更记录。审计日志是给"谁改了什么"用的, 不是给"谁看了什么"用的——后者由 PII 脱敏 + 访问控制负责, 不在审计范围。

10.6.4 WS 通道的 trace 恢复:一条消息两种入口

HTTP 请求的 trace 由中间件自动创建, 但 Bot 的消息处理走 Redis Stream + 后台 worker——worker 里没有 HTTP span, trace 怎么串?

答案在 `_session_worker (bot/router.py:408)`:

```
trace_raw = fields.get("_trace_context", "") # HTTP 入口写入的 trace 上下文
tid = int(parts[0], 16); sid = int(parts[1], 16)
sc = SpanContext(trace_id=tid, span_id=sid, is_remote=True, trace_flags=flags)
parent_ctx = otel_trace.set_span_in_context(NonRecordingSpan(sc))
otel_token = context.attach(parent_ctx) # 链接到 HTTP span
try:
    ... # Agent 处理, 所有日志自动带上原 trace_id
finally:
    context.detach(otel_token) # 处理完必须释放, 防泄漏到下一个消息
```

设计要点有两个: 一是入队时序列化, 出队时恢复——`chat_send` 把当前 span 的 `trace_id:span_id:flags` 十六进制拼进 Stream 消息, worker 取出后重建 `SpanContext`; 二是 `finally` 里必须 `detach`——`context.attach` 返回的 token 不释放的话, 下一条消息的日志会错误地继承上一条的 `trace_id`, 排障时"两条无关消息串在同一条 trace 里"是最迷惑人的假象。非法 trace 字符串 (比如第三方客户端伪造) 会被 `try/except` 静默忽略, 不阻断消息处理。

10.7 小结

Lumio 的可观测性是按"开发愿意用、运维信得过"双向目标设计的:开发侧只有 `@traced` 一行成本,业务代码不需要 `import OTel` 包;运维侧有 17 指标 + 3 dashboard + 6 告警的标准化视图。"反馈循环"、"路由元数据精确审计"、"resource version fallback"这些边界条件都固化为代码,确保新同学不会在不知情时再踩同样的坑。

延伸阅读: - [第 8 章 错误处理](#) — 错误响应体格式 - [第 11 章 安全合规](#) — 审计与脱敏 - [第 13 章 部署](#) — Prometheus 抓取配置

11 安全合规

第 11 章: 安全合规

Lumio 是一款银行场景的智能客服助手,任何一次越权读、一次敏感词漏检、一次错误响应里泄露的数据库连接串,都可能触发银保监会的合规审查。本章从纵深防御的全局视角出发,逐层拆解边缘网关、接入鉴权、业务授权、数据脱敏与审计回溯这五道防线的设计动机与代码实现。

11.1 纵深防御:为什么需要 5 层

银行客户对"防御"的理解不是"一道墙够不够厚",而是"墙被突破后,还剩几道"。Lumio 因此采用了**纵深防御 (Defense in Depth)** 模型,将请求生命周期切分为 5 个独立失效域:任何单层失守,其他层仍能阻断或留痕。

```
graph TD
    Client[客户端] -->|HTTPS| L1[第 1 层: 边缘 Nginx<br/>TLS 终止 / IP 黑名单 / 限速]
    L1 --> L2[第 2 层: 接入 JWT<br/>HS256 + iss/aud 校验]
    L2 --> L3[第 3 层: 业务 RBAC<br/>require_role 装饰]
    L3 --> L4[第 4 层: 数据 PII 脱敏<br/>入库前 mask_pii]
    L4 --> L5[第 5 层: 审计落库<br/>audit_log 表 + stdout JSON]
    L5 --> DB[(PostgreSQL)]

    style L1 fill:#fde0dd,stroke:#c0392b
    style L2 fill:#fff3b0,stroke:#d4a017
    style L3 fill:#d4f4dd,stroke:#27ae60
    style L4 fill:#cfe7ff,stroke:#2c5fa8
    style L5 fill:#e0d4f4,stroke:#7d3c98
```

怎么读这张图 — "一次攻击者尝试的全景": 攻击者想偷看别人的会话记录,要连闯 5 关 — ① Nginx: 先过 TLS 和 IP 限速(爬虫先被挡); ② JWT: 没有有效 token, 直接 401 (伪造 token 过不了 HS256 签名); ③ RBAC: 就算有 token 但不是 admin, 403 (角色不够); ④ PII 脱敏: 就算读到数据, 卡号手机号也已打码; ⑤ 审计: 就算侥幸读到, 每次访问都留了痕, 事后可追责。 **单层被突破不代表系统失守** — 这就是"纵深"的含义: 每层都是独立失效域, 攻击成本随层数指数上升。

5 层各司其职,设计上不允许跨层"借力":例如 Nginx 不会做应用层鉴权,JWT 不会做业务级字段过滤。一旦某一层需要扩展,改动也只局限在那一层,不会引发雪崩式回归。第 10 章讲到的可观测性横切所有 5 层,但它本身不是独立防线。

11.2 JWT 接入:从签发到解析的完整链路

Lumio 选用 JWT 而非 Session,核心动机是**无状态**:多副本部署无需共享 session 存储,access token 自带 role 声明,网关层可直接做粗粒度路由。但 JWT 一旦泄漏即可被任意重放,所以签发、解析、刷新三段都做了针对性加固。

11.2.1 access token 与 refresh token

`create_access_token()` (auth.py:67-93) 构造 7 个标准声明:

```
payload = {
    "sub": user_id, "role": role,
    "iss": "lumio", "aud": "lumio-api",
    "exp": expire, "iat": now, "jti": uuid4().hex,
}
```

- `iss=lumio / aud=lumio-api` 用于拒绝跨服务 token 互用,例如 chat-svc 的 token 不会被 assist-svc 接受。
- `jti` 给每次签发一个唯一 ID,理论上未来可用于黑名单撤销,目前仅做日志关联。
- 过期时间默认 30 分钟 (`jwt_expire_minutes`),由配置注入,方便按安全等级调短。

`create_refresh_token()` (auth.py:96-112) 走另一条命名空间 `aud=lumio-refresh` 并显式标 `type=refresh`,生命周期 7 天。这样即便 refresh token 被截获用于调用业务接口,也会因 `aud` 不匹配被立即拒绝。

11.2.2 解码与时钟容差

`decode_token()` (auth.py:115-133) 在 `jwt.decode` 中显式传入 `issuer="lumio"` 与 `audience="lumio-api"`,库内部会做严格比对;同时设置 `leeway=30`,容忍 ± 30 秒的多机时钟漂移。这个数值的选择有讲究:银行内网 NTP 通常同步到秒级,30 秒既覆盖偶发抖动,又不会让一张被吊销的 token 长期可用。

11.2.3 依赖注入与 dev 旁路

`get_current_user()` (auth.py:139-175) 是 FastAPI `Depends` 的入口,先尝试 `Authorization: Bearer <token>, ?token=xxx` query param 仅限 **development** 环境 (生产环境拒绝,防 token 落访问日志/Referer/代理缓存 — 第五轮加固)。WS 握手鉴权同样走 query param token + 有效性校验。



11.2.4 全端点认证与角色矩阵 (第五轮加固)

`chat/session` 全部端点强制 `user: CurrentUser` 依赖,并带 **session 归属校验** (JWT 声明 `session_id` 与请求不一致 → 403; customer 读他人会话按 meta owner 二次校验)。`list_sessions` 按用户过滤 — customer 只见自己的会话, admin/agent 才可全量枚举。

权限矩阵 (第五轮补齐,此前三个"总开关"级缺口):

端点	要求	防御
<code>/admin/sensitive-words</code> GET/PUT	admin	任意登录用户可清空全系统过滤词库
<code>/admin/dead-letter</code>	admin	匿名可读原始客户消息 (PII)
<code>/kb/documents/{id}/approve/reject/publish/archive</code>	admin	绕过"审核-发布"合规流程
<code>/admin/rules/reload</code> <code>/admin/stats</code>	admin	规则热加载/业务统计
<code>/api/chat/*</code> <code>/api/sessions/*</code>	登录 + 归属	匿名读写对话
<code>/api/gdpr/delete</code>	本人或 admin	删除请求发起

原则: 安全代码的默认值必须是拒绝 — 认证用显式依赖而非可选参数,角色用 `require_role` 依赖而非函数内 `if`,合规过滤缺失字段视为不合规而非默认放行。

11.3 占位密钥与 dev 旁路的正确关闭方式

"配置安全"与"环境默认"是两个最易被忽视的失败模式。

11.3.1 JWT 占位密钥拦截

风险: `Settings.jwt_secret` 默认值是 `lumio-dev-secret-change-in-production`,若没有强制覆盖,运维漏配 `LUMIO_JWT_SECRET` 时服务会**静默**用默认密钥启动,所有 token 可被任何人伪造。

`config.py:473-498` 的 `_validate_production_security` 修复了这个问题:


```

forbidden_secrets = {
    "lumio-dev-secret-change-in-production",
    "<CHANGE_ME>", "<CHANGE_ME_IF_NEEDED>",
}
if self.jwt_secret in forbidden_secrets:
    if self.environment == "production":
        raise ValueError("生产环境必须设置 LUMIO_JWT_SECRET ...")
    # dev/test 也仅 WARNING, 不阻断启动 (兼容本地开发)
    logger.warning("🚩 JWT 密钥仍为占位值 ...")
if self.environment == "production" and len(self.jwt_secret) < 32:
    raise ValueError("生产环境 JWT 密钥长度必须 >= 32 字符")

```

设计动机:生产环境**硬阻断**,dev/test 环境**告警但不阻断**——避免本地开发时每次都要 `openssl rand` 改 secret。生产长度门槛定在 32 字符,刚好覆盖 `hex(32)` 生成的 64 字符熵。

11.3.2 dev bypass 限定 loopback

风险更隐蔽: `get_current_user` 若在 dev + 本地绑定场景下无 token 直接放行 admin,且"本地绑定"仅以 `service_host` 不在公网 IP 列表判断,容器化部署把 `0.0.0.0` 当成自查通过的依据时,任何能访问容器 8080 端口的远端流量都会获得 admin。

修复 (auth.py:158–168) 引入显式开关 `dev_auth_bypass` + 严格 loopback 校验:

```

if settings.dev_auth_bypass and settings.environment == "development":
    if settings.service_host not in ("127.0.0.1", "localhost"):
        raise AuthenticationError(
            "开发旁路仅允许绑定 loopback (127.0.0.1/localhost), "
            f"当前 service_host={settings.service_host!r}"
        )
    return AuthUser(user_id="dev-user", role="admin")

```

双条件是核心:即使显式开启旁路,绑定地址不在 loopback 也拒绝;而默认 `dev_auth_bypass=False` 让生产或 staging 环境根本进不到这段逻辑,杜绝"配置漂移引发提权"。

11.4 密码哈希:为什么是 PBKDF2 而不是 argon2

`shared/password.py` 用 `hashlib.pbkdf2_hmac` 实现 PBKDF2–HMAC–SHA256,迭代次数 600000 次,产物格式与 Django 完全一致: `pbkdf2_sha256$600000$<salt_b64>$<hash_b64>`。

```

salt = secrets.token_bytes(16)
digest = hashlib.pbkdf2_hmac("sha256", password.encode(), salt, 600000)
return f"pbkdf2_sha256$600000${b64(salt)}${b64(digest)}"

```

为什么不用 argon2 / bcrypt?这是工程权衡,不是技术先进性问题:

- **零外部依赖**: `hashlib` 是 Python 标准库,跨 musl/alpine 镜像零编译痛苦。passlib+bcrypt 在 alpine 上要装 rustc、build-base,镜像膨胀 300MB+,CI 缓存命中率下降。
- **生态成熟**: 600000 次迭代对齐 OWASP 2023 建议,Django/Flask 默认同款算法,审计与迁移成本低。
- **时序攻击防御**: `verify_password` 用 `secrets.compare_digest` 而非 `==`,防止攻击者通过响应时间差异逐字节猜 hash。

代价是 CPU:单次校验在主频 2.6GHz 机器上约 350ms,登录接口必须做限速,否则就是天然 DoS 入口。`require_role` 与 `get_current_user` 都不涉及密码哈希,不会被这个开销拖累。

11.5 Aho–Corasick 敏感词过滤

银行客户对话里出现竞品名称、监管敏感词、政治关键词是常态。Lumio 用 `pyahocorasick` 实现 AC 自动机,核心动机是 **O(n)** 匹配复杂度与词库大小无关——万级词库下整段对话扫描仍 < 100ms。

11.5.1 归一化与热更新

`safety.py:39–117` 的 `SafetyFilter` 类做了三件事:

1. `_normalize()` 用 `unicodedata.normalize("NFKC", text)` 把全角字符转半角,再 `lower()`。这步至关重要:银行客户经常输入全角数字和字母,词库是半角写的,不做归一化会全军覆没。
2. `load_from_file()` 启动时一次性加载, `add_word()` 支持增量,内部调 `make_automaton()` 重建 DFA。
3. Redis Pub/Sub 通道 `lumio:safety:reload` 接收热更新信号,从 `lumio:safety:words` 这个 Set 拉取最新词库,避免改一个词要重启服务。

11.5.2 双重过滤

合规要求"出口必查":任何用户输入在应用层先过一次 `check_input()`,命中即拒绝;同时所有原始文本进入审计层时再过一次 `filter_output()`,把命中的敏感词替换为 * 后再写日志。这样即使应用层策略被绕过,审计日志里也不会出现原始敏感词,事后追溯不被污染。

11.5.3 危机干预 (客户自伤/轻生意图)

银行客服可能遇到客户表达自伤/轻生意图 — 这是合规最高优先级场景, 不能走常规 LLM 应答:

1. 词库: `sensitive_words.txt` 危机干预类 8 词 (自杀/自残/轻生/不想活/活不下去/想死/绝望/抑郁)
 2. 检测: `SafetyFilter.is_crisis_input(text)` — 归一化后子串匹配, 独立于 AC 自动机 (类常量, 不依赖词库加载)
 3. 响应: `bot_agent.run()` 入口处 (优先级高于问候/告别/意图分类) 命中 → 立即返回安抚话术 + 强制转人工:
- ```
bot_agent.py (简化)
if safety_filter.is_crisis_input(user_input):
 return self._build_result(
 session_id, user_input, CRISIS_RESPONSE, "template", "crisis",
 should_transfer=True, transfer_reason="crisis_intervention: ...",
)
```
4. 话术 ( `prompts/__init__.py` `CRISIS_RESPONSE` ): 表达关心 + 已优先转人工专员 + 提供 24 小时心理援助热线 (12356 / 400-161-9995), 不评判、不追问细节

设计取舍: 关键词匹配会误伤 ("我绝望了, 这卡什么时候能提额" 是夸张表达) — 但银行合规场景 宁可误转人工, 不可漏判; 转人工后由坐席判断真实意图, 成本远低于漏判风险。

## 11.6 PII 脱敏:5 类规则 + 顺序敏感

`shared/pii.py:20-77` 提供 5 类脱敏,各自一条正则:

| 类型          | 模式                                    | 输出示例             |
|-------------|---------------------------------------|------------------|
| 手机号         | 1[3-9]\d{9}                           | 138****5678      |
| 身份证         | \d{17}[\dX]                           | 110101*****1234  |
| 银行卡         | 16-19 位连续数字                           | 6222****7890     |
| 邮箱          | 标准 RFC 简化                             | z***@example.com |
| JSON 敏感 key | password/secret/token/api_key/cvv/pin | 值替换 *****        |

`mask_pii()` 的调用顺序不是随意排列,而是有强约束:敏感字段 → 邮箱 → 身份证 → 银行卡 → 手机号。身份证 18 位、银行卡 16-19 位都包含 11 位连续数字子串,如果手机号规则先跑,会把身份证尾数 4 位当手机号中段误判成 110101\*\*\*\*\*5678 (中间只遮 4 位而不是 8 位)。身份证与银行卡先跑把"长串数字"先吃掉,剩下独立的 11 位才轮到手机号规则。

这个细节在 code review 时容易漏掉,所以写在函数 docstring 里钉死。

## 11.7 双重审计:DB 落库 + stdout JSON

`shared/audit_middleware.py` 实现双通道审计——同一操作既写 `audit_log` 表(供合规取证与查询),也通过结构化日志走 `stdout`(供 ELK/Loki 聚合)。两路并行,互不阻塞,任何一路故障不影响另一路。

关键设计是 `_ENDPOINT_ACTION_MAP` (`audit_middleware.py:174–206`):27 条端点函数名到 `(action, target_type)` 的精确映射,优先级高于路径字符串推断。这是因为路径推断会因 `/api/v1/sessions/{id}/update` 与 `/api/v1/sessions/update` 在 `/update` 命中上歧义,而 FastAPI 路由匹配后 `request.scope["route"].endpoint.__name__` 是唯一确定的,这是"权威来源"。

```
精确映射示例
"submit_review": ("review.submit", "review"),
"approve_faq": ("faq.approve", "faq"),
"publish_faq": ("faq.publish", "faq"),
```

未命中映射的端点才走兜底路径推断,标注 `target_type=other` 方便监报告警,提示补充映射。

## 11.8 银行合规字段:KbDocument 7 态审批

知识库文档不是"上传就能用",监管要求双人四眼 + 版本隔离 + 可见性控制。`shared/orm_models.py` 的 `KbDocument` 因此带了一组独有字段:

- `approval_status`:7 态枚举 `DRAFT → IN_REVIEW → APPROVED → PUBLISHED → SUPERSEDED → REJECTED → ARCHIVED`。`SUPERSEDED` 显式表示被新版本替代,而不是直接删除,留可追溯。
- `doc_group`:文档组 ID,同一逻辑文档的不同版本共享,语义上相当于"业务主键"。
- `is_current_version`:PARTIAL UNIQUE 索引 (`postgresql_where=is_current_version = true AND is_deleted = false`),数据库层保证同一 `doc_group` 仅一个生效版本。
- `allowed_roles`:JSON 数组,可见角色白名单。例如"信用卡章程"只对 `agent` 和 `admin` 可见, `customer` 查询时被自动过滤。
- `regulatory_tags`:监管标签,如 `["银保监", "反洗钱"]`,供合规报表分组导出。

7 态审批不是过度设计: `DRAFT` (撰写中)与 `IN_REVIEW` (审核中)必须可区分,否则审核员看不到积压; `REJECTED` 与 `ARCHIVED` 看似都不可用,实际 `REJECTED` 是被打回要修改、`ARCHIVED` 是已废弃不可恢复,合规报表需要分别统计。

## 11.9 输入验证

接口对输入长度设防,覆盖两个隐患:

- 用户消息无上限,一次 POST 几 MB 文本就能撑爆网关与日志存储;
- `session_id` / `customer_id` 无字符限制,可通过特殊字符试探 SQL 注入(虽然 ORM 已参数化,但日志里出现的奇怪字符串会让运维误判)。

限制如下:

- `message` 字段 `max_length=2000`,覆盖正常客户咨询 3–5 倍冗余;
- `session_id` / `customer_id` `max_length=128`,远超 UUID 与业务主键长度;
- 文件上传 50MB (`bot/router.py:1232 max_upload_size = 50*1024*1024`),扩展名走白名单 `_ALLOWED_EXTENSIONS`,黑名单 `.exe .sh .bat` 必拒。

50MB 是经验值:银行 PDF 章程通常 5–20MB,知识库批量上传峰值场景 50MB 留 2–3 倍冗余;再大就强制走对象存储分片上传,不走 HTTP body。

## 11.10 健康检查脱敏

`/api/health/ready` 端点如果依赖故障时把 `str(e)[:100]` 直接吐给客户端,看似贴心的"错误详情"实际会泄露:

- `password authentication failed for user "lumio"` —— 数据库账号名暴露;
- `ConnectionRefusedError: 192.168.10.5:6379` —— 内网 IP 与端口拓扑暴露;
- `psycpg2.OperationalError` —— 驱动类名,攻击者可针对性找 0day。

因此 `health.py:20–41` 引入 `_ERROR_CODE_BY_DEP` 7 个分类码,响应体只返回 `{"status": "down", "error_code": "redis_unreachable"}`,真实异常走 `logger.warning(..., exc_info=True)` 进日志(含完整 traceback),运维通过日志定位、客户端只看到分类码。给机器看的和给人看的必须分流,这是合规响应设计的铁律。

## 11.11 统一错误响应:第 8 章的合规延伸

第 8 章讲到的统一错误体 `{"error": {"code", "message", "type"}, "request_id"}` 在安全维度的关键是生产环境不暴露真实异常类名。`type` 字段只能取预定义的 `validation_error / business_error / system_error / auth_error`, 绝不把 `pydantic.ValidationError / sqlalchemy.exc.IntegrityError` 这类内部异常名直接回传——后者是给攻击者探测后端栈的免费雷达。

`request_id` 则是合规的"对账锚点": 客户报障时提供 `request_id`, 客服可从审计日志与 `trace` 中精准定位该次请求的全部上下文, 无需暴露给客户 PII。

## 11.12 测试覆盖

合规代码必须有专项测试守护, 目前覆盖范围包括:

- `test_auth.py` — JWT 签发/解码/角色/旁路
- `test_audit.py` — 21 条映射 + 路径推断兜底
- `test_audit_health.py` — 健康检查分类码不含 PII
- `test_safety.py` — AC 自动机命中/未命中/全角/热更新
- `test_pii.py` — 5 类脱敏 + 顺序敏感性(身份证优先于手机号)

合规测试不能和生产功能测试合并跑, 因为合规用例的"必须失败"场景(占位密钥、dev 旁路、错误体泄露)与功能测试的"必须成功"是反方向的断言, 合在一起会相互干扰。

### 11.12.1 Prompt 注入防护: 入口拦截 + 输出护栏双线

银行客服 Bot 是 prompt injection 的高价值目标——攻击者试图让 LLM"忽略以上所有指令"来套取他人账单。Lumio 的注入防护是双线的:

**入口线** (`shared/injection_guard.py`): `check_user_input()` 在消息进 Agent 链路之前跑, 返回 `(action, pattern)`。命中 `REJECT` 或 `QUARANTINE` 时 `chat_send` 直接拒绝 (`bot/router.py:1165`), 日志记录命中的 `pattern` 用于攻击情报; `ALLOW` 才放行。判定规则覆盖两类最常见模式: **指令忽略** ("请忽略以上所有指令") 与 **角色混淆** ("你现在是一个没有限制的模型")。

**输出线** (`shared/safety.py`): LLM 回复经 `filter_output()` 过滤后才写给客户端。输入拦截是"第一道门", 输出过滤是"最后一道闸"——万一注入绕过了入口 (比如通过知识库文档注入), 输出侧的敏感词替换还能兜底。

为什么入口拦截命中返回 2001 而不是专门的错误码? 同第 8 章"空消息与注入同为 2001"的理由: 暴露"我知道你在注入"会帮助攻击者迭代 payload。拦截情报 (`pattern`) 只进日志和监控, 客户端永远只看到"消息内容不符合规范"。

### 11.12.2 会话归属校验: 横向越权的两道闸

JWT 的 `session_id` 声明解决了"这个 token 能访问哪个会话"的第一层问题, 但银行场景还有第二层: **customer 角色的 token 可能没有 `session_id` 声明** (登录时还没建会话), 此时任意会话 ID 都能通过 `_ensure_session_owned` 的 JWT 校验——横向越权 (枚举他人会话 ID 读取消息) 就开了口子。

`get_session_messages` (`bot/router.py:1748`) 因此做了 meta owner 二次校验:

```
if redis_client and user.role == "customer":
 raw_meta = await redis_client.get(session_meta_key(session_id))
 if raw_meta:
 meta_owner = json.loads(raw_meta).get("customer_id")
 if meta_owner and meta_owner != user.user_id:
 raise AuthorizationError("无权访问该会话")
```

细节有三: 一是校验发生在读取消息之前, 未授权直接 403, `lrange` 都不会执行; 二是 **meta 缺失/损坏时放行而非阻断**——`except Exception: pass` 只吞非 `AuthorizationError` 异常, 防止 Redis 抖动把正常客户挡在门外 (可用性优先, 因为 meta 由会话创建流程保证, 缺失本身是异常态); 三是 **admin/agent 角色跳过此校验**, 坐席需要跨会话工作, 由角色的 `require_role` 而不是 owner 校验约束。

`list_sessions` 的过滤是同一原则的列表形态: `customer` 只返回 `customer_id` 等于自己的会话, 不存在"先全量查再在内存里过滤"的写法——SQL/Redis 查询条件直接带 owner, 从源头杜绝越权数据出库。

### 11.12.3 差异化限流: 登录 30/min, 健康探针豁免

`shared/rate_limit.py` 的限流不是"全局一刀切", 而是按端点差异化:

- `chat_send` 限 30/minute, 且按用户级 `key` 计数 ( `user:{user_id}` )——防止 NAT 后的多客户端共享一个公网 IP 被整体误杀;
- `login` 类高频滥用通道单独收紧;
- `/health` / `/health/live` / `/health/ready` 用 `@get_limiter().exempt` 显式豁免——LB 的健康检查每 5s 打一次, 如果计入限流, 流量高峰时实例会因 429 被误判不可用而摘除, 造成"假故障"。

为什么豁免而不是调高阈值? 健康探针的请求特征是"固定节奏、无业务含义", 给它配额等于在流量高峰时让探针和真实客户抢额度。豁免是语义正确的做法——限流保护的是业务入口, 不是基础设施的存活信号。

### 11.12.4 GDPR 删除:一条请求, 三个存储的清理

`gdpr_delete` ( `bot/router.py:1674` ) 是合规删除的端到端实现, 一条删除请求要清理三个存储:

| 存储            | 清理内容                                                                    | 失败处理           |
|---------------|-------------------------------------------------------------------------|----------------|
| Redis         | 会话 <code>meta</code> / <code>history</code> / 槽位 / 画像                   | 尽力而为, 异常吞掉     |
| PostgreSQL    | <code>chat_message</code> 等会话数据                                         | 软删除或物理删除 (按配置) |
| Elasticsearch | <code>{prefix}_dialogue</code> 与 <code>{prefix}_kb_chunks</code> 中的客户数据 | 尽力而为           |

设计要点: 删除是"尽力而为 + 留痕"而不是"要么全成要么全败"。银行合规要求"客户提出删除后必须删除", 但三个存储的可用性各不相同——如果 ES 恰好抖动, 把整个删除请求回滚为失败, 客户再点一次可能还是失败。正确姿势是: 能删的全删, 删不了的记日志 + 打点, 由后台对账任务补删。

谁可以发起? 本人或 `admin` ( `require_role("admin")` 或 `customer_id` 匹配)。这是 11.2.4 权限矩阵里唯一允许 `customer` 主动调用的管理类操作——GDPR 的"被遗忘权"是客户权利, 不能只允许管理员代劳。

### 11.13 小结

Lumio 的安全合规不是某一个库或某一段代码的功劳,而是5层独立防线的协同:边缘网关挡 DDoS、JWT 验身份、RBAC 验权限、PII 守数据、审计留证据。合规不是一次性合规,而是每次发版前都要重新过一遍的清单。

下次设计新接口时,先问自己 4 个问题:谁会调?会带什么 PII?出错时给客户看什么?事后谁来查?这 4 个问题的答案,会自然引导你走完本章的 5 层防御。

延伸阅读: – [第 2 章 配置系统](#) — JWT `secret` 校验 – [第 8 章 错误处理](#) — 错误响应体脱敏 – [第 10 章 可观测性](#) — 审计日志

## 12 数据层

### 第 12 章: 数据层

Lumio 的数据层是整个系统最容易"被低估"的工程难点。它不像 Agent 编排那样有戏剧性的状态流转, 也不像 RAG 那样有可量化的检索指标。但凡涉及"快/慢数据分离、向量与倒排双路召回、消息幂等、超时回收"这些工程化命题, 都要回到 6 个中间件的协同策略上重新审视。本章按"为什么这样选型 → 19 张表怎么拆 → Redis 15+ key 怎么用 → 双写怎么保一致 → 故障怎么降级"的顺序展开, 把横切关注点一次性讲透。

#### 12.1 数据层全景: 6 个中间件的角色分工

数据层不是"找一个数据库把所有东西塞进去", 而是把**延迟敏感/吞吐敏感/一致性敏感**三类负载拆到不同的存储引擎。这一拆解背后有三层工程考量。

第一, **延迟谱跨度太大**。从"用户在 IM 里敲完一句话, Bot 要在 200ms 内回复"的强实时场景, 到"月底统计话术使用率"的批量分析场景, 两者不可能共用一套存储。Lumio 用 Redis + Stream 把强实时场景锁在毫秒级, 用 PG + ES 承担可秒级响应的业务查询, 用 MinIO 承接大文件, 用 Kafka 兜底分钟级以上的异步事件流。

第二, **数据形态不同**。会话状态是天然带 TTL 的"短命数据", 用 Redis 哈希最合适; 知识库分片是带向量字段的结构化数据, 必须用 PG 存元数据 + 专用引擎存向量; 文档原文件是非结构化大对象, 对象存储比文件系统更省心。强行用 PG `bytea` 存 PDF 会让备份/复制/分片全部退化。

第三, **失败模式不同**。PG 挂了, 整个系统就该停, 不应该"降级到一个错误的 PG"。Redis 挂了, 短消息交互可以走降级但不能丢一致性。ES/Milvus 任何一个挂, RAG 检索不能整体不可用, 应该自动切到单路召回。Kafka 当前只是"留白接口", 挂了完全不影响主流程。下图给出 Lumio 的中间件拓扑。

```
graph TB
 subgraph 慢数据 [PostgreSQL - 系统之源]
 PG["(PostgreSQL 19 表
UUID v7 主键
Alembic 12 迁移)"]
 end

 subgraph 在线状态 [Redis - 实时协调]
 R1["Session meta/history"]
 R2["Stream + Consumer Group"]
 R3["ZSET 超时队列"]
 R4["Pub/Sub 广播"]
 end

 subgraph 检索引擎 [ES + Milvus - 双路召回]
 ES["(Elasticsearch
BM25 倒排)"]
 MV["(Milvus
向量 ANN 1024d)"]
 end

 subgraph 对象与事件 [MinIO + Kafka - 留白设计]
 M["(MinIO
lumio-docs bucket)"]
 K["(Kafka 3 topic
预留)"]
 end

 Agent["Agent / Bot / Streamlit"] --> PG
 Agent --> R1
 Agent --> R2
 Agent --> ES
 Agent --> MV
 Agent --> M

 ES -. "双写回滚" .-> MV
 PG -. "audit_log" .-> K
```

怎么读这张图 — "一条消息落在哪": 客户发"查账单" → 会话状态进 Redis (毫秒级读写); 账单数据从 PG 查 (秒级); 如果客户还问了"免年费规则" → 知识库从 ES/Milvus 检索 (双路召回); 通话录音/文档原文件进 MinIO; 审计流水最终可推 Kafka。每种数据找自己最合适的家: 状态要快 → Redis; 业务要准 → PG; 文本要搜 → ES; 语义要似 → Milvus; 文件要大 → MinIO; 事件要缓冲 → Kafka。

设计上有三条**硬性原则**贯穿所有模块:

- 1. **PG 是唯一的真相之源**。Redis 只是 PG 的"投影 + 加速", 一旦 Redis 丢失, 必须能从 PG 重建( `session.py:67-95` 的 CAS Lua 失败回退到 PG 重读)。这也是为什么我们不把"会话历史"完全放在 Redis, 而是 PG `chat_message` + Redis `lumio:session:{id}:history` 双写 + 兜底。
- 2. **ES + Milvus 是 PG 的"二级索引"**。两者都不能脱离 PG 单独存活, 任何回写都要先落 PG, 再异步刷到检索引擎。这种"PG 先写, 索引后写"的顺序保证了即便索引构建失败, 也能从 PG 重新发起, 而不会出现"索引里有但 PG 里没有"的鬼影数据。
- 3. **Kafka 是"未来接口"**。当前 3 个 topic 仅在 `init_kafka.py` 中创建并做连通性测试, 不进入主链路。业务一旦要做"读扩散"或"事件溯源", 只需要挂 Producer, 不需要重写领域模型。比起"先上 Kafka 再考虑怎么用", 这种"先把路修好, 不急着通车"的策略更稳。

12.2 PostgreSQL: 19 张表的领域拆解

`agent/lumio/shared/orm_models.py:1-1060` 是数据模型的唯一入口, 19 张表按 7 个领域分桶。下表列出每张表的主键与定位, 完整 DDL 详见代码引用。

| 桶   | 表名                    | 主键         | 关键索引                                          | 用途          |
|-----|-----------------------|------------|-----------------------------------------------|-------------|
| 知识库 | kb_document           | id UUID v7 | ix_kb_document_current_version UNIQUE PARTIAL | 文档元数据 + 版本号 |
| 知识库 | kb_chunk              | id UUID v7 | 3 个 PARTIAL 索引 (见 12.3)                       | 分片 + 向量状态机  |
| 知识库 | kb_document_approval  | id UUID v7 | ix_kb_approval_doc_status                     | 审批 workflow |
| 知识库 | kb_ingestion_log      | id UUID v7 | ix_kb_ingestion_started_at                    | 摄入轨迹        |
| 知识库 | kb_faq                | id UUID v7 | ix_kb_faq_category                            | FAQ 问答对     |
| 知识库 | kb_faq_search_log     | id UUID v7 | ix_kb_faq_search_log_session                  | FAQ 命中回放    |
| 知识库 | kb_product            | id UUID v7 | ix_kb_product_code                            | 产品字典        |
| 会话  | chat_message          | id UUID v7 | ix_chat_message_content_fts (GIN)             | 用户/助手消息     |
| 会话  | dialogue_log          | id UUID v7 | ix_dialogue_log_session_started               | 端到端对话日志     |
| 会话  | chat_message_audit    | id UUID v7 | ix_chat_message_audit_msg                     | 消息级审计       |
| 用户  | user_account          | id UUID v7 | ix_user_account_username UNIQUE               | 账户/坐席       |
| 审计  | audit_log             | id UUID v7 | ix_audit_log_created_at                       | 通用审计流       |
| 业务  | script_template       | id UUID v7 | ix_script_template_code                       | 话术模板        |
| 业务  | script_usage_log      | id UUID v7 | ix_script_usage_log_session                   | 话术使用记录      |
| 业务  | alert_rule            | id UUID v7 | ix_alert_rule_enabled                         | 告警规则        |
| 业务  | alert_log             | id UUID v7 | ix_alert_log_fired_at                         | 告警触发日志      |
| 编排  | orchestration_logs    | id UUID v7 | ix_orch_logs_session                          | Agent 编排轨迹  |
| 编排  | feedback_logs         | id UUID v7 | ix_feedback_logs_target                       | 用户反馈        |
| 规则  | intent_detection_rule | id UUID v7 | ix_intent_rule_enabled                        | 意图正则规则      |

12.2.1 主键为什么统一用 UUID v7

`shared/orm_models.py` 在每张表的 `id` 列上都使用 `uuid_utils.uuid7()` 默认值, 这不是审美偏好, 而是为 B-tree 写入性能服务的。

传统自增 ID 在分布式下要么退化为 Snowflake, 要么引发分页热点。UUID v4 完全随机, 写入 B-tree 时几乎 100% 触发页分裂。**UUID v7 把 48 位毫秒时间戳放在高位**, 整体仍然随机, 但 B-tree 把它视为"近似单调递增", 实测写入吞吐比 v4 高 3-5 倍, 同时保留去中心化生成能力。代价是依赖 PostgreSQL 的 `pgcrypto` 扩展提供底层支持, 启动时由 `database.py` 自动 `CREATE EXTENSION IF NOT EXISTS pgcrypto`。

12.2.2 关键 PARTIAL 索引: 让"状态机字段"开箱即用

知识库系统的最难约束是"同一个文档分组, 必须且只能有一个生效版本"。 `kb_document` 的关键索引是这样写的:



```
agent/lumio/shared/orm_models.py: ~ line 220
Index(
 "ix_kb_document_current_version",
 "doc_group_id",
 unique=True,
 postgresql_where=(
 text("is_current_version = true AND is_deleted = false")
),
)
```

这是 PostgreSQL PARTIAL UNIQUE INDEX 的典型用法。is\_current\_version=true 的行才参与唯一性校验, 历史版本仍然可以保留。is\_deleted=false 把软删的行也排除, 避免历史垃圾数据阻塞版本切换。

类似的 PARTIAL 索引还有 kb\_chunk 上的 3 个状态机字段索引, 分别覆盖待向量化、待写 ES、待写 Milvus 三个未完成态:

```
agent/lumio/shared/orm_models.py: ~ line 320
Index("ix_kb_chunk_pending_embedding",
 "embedding_status",
 postgresql_where=text("embedding_status = 'PENDING'"))
Index("ix_kb_chunk_pending_es",
 "es_indexed",
 postgresql_where=text("es_indexed = false"))
Index("ix_kb_chunk_pending_milvus",
 "milvus_indexed",
 postgresql_where=text("milvus_indexed = false"))
```

三个 PARTIAL 索引的妙处在于: 后台 worker 在扫表时只扫真正待处理的几行, 不会全表扫描已完成的 chunk。百万级知识库下, "找未向量化 chunk" 的查询从 O(N) 压到 O(未完成数)。

chat\_message 的全文索引用了 GIN + to\_tsvector('simple', content), 选择 simple 配置而不是 chinese\_english 是有原因的: 我们的查询是"用户/助手都搜, 不需要词形归一化", simple 既快又稳。

### 12.3 Alembic 12 个迁移的演进史

agent/alembic/versions/ 下的 12 个脚本按时间序记录了数据模型的演进。完整顺序是:

- 1. 9a2930672730\_create\_kb\_tables — 根迁移, 创建全部 KB 表 + 索引
- 2. 2221cd4b9c4b\_add\_parent\_child\_chunk\_fields — 加 parent\_chunk\_id 与 child\_index, 引入父子分块
- 3. b6671b8dc030\_add\_chat\_message\_audit\_table — 消息级审计
- 4. 002\_create\_audit\_log.py — 通用审计日志
- 5. 003\_kb\_approval\_workflow.py — KB 审批 workflow
- 6. 004\_create\_faq\_tables.py — FAQ 问答对
- 7. 005\_create\_dialogue\_log.py — 端到端对话日志
- 8. 006\_create\_user\_account.py — 用户账户
- 9. 03a9cfea52ac\_add\_intent\_detection\_rule\_table.py — 意图规则
- 10. a918a6a3f1c8\_add\_script\_template\_alert\_rule\_tables.py — 话术 + 告警

迁移顺序的设计哲学是"由内向外": 核心知识库先建( 9a2930672730 ), 父子分块紧随( 2221cd4b9c4b ), 然后才是审计/审批/业务。这种顺序保证任何时间点回滚, 数据模型都是自洽的, 不会出现"指向不存在的外键"的脏状态。

### 12.4 Redis: 15+ key prefix 的语义图谱

Redis 在 Lumio 里承担 5 类职责, 共 15+ 种 key prefix:

| Key 模式                     | 数据结构 | TTL   | 职责                     |
|----------------------------|------|-------|------------------------|
| lumio:session:{id}:meta    | Hash | 1800s | 会话元数据 + version (CAS)  |
| lumio:session:{id}:history | List | 1800s | 对话轮次, RPUSH + LTRIM 20 |
| lumio:session:timeouts     | ZSET | 永久    | 5 类超时排序, 5s 轮询         |



| Key 模式                           | 数据结构        | TTL   | 职责                             |
|----------------------------------|-------------|-------|--------------------------------|
| lumio:chat:stream                | Stream      | 永久    | 消息流 + Consumer Group bot-group |
| lumio:chat:dead_letter           | Stream      | 永久    | 失败重试耗尽后的死信                     |
| lumio:chat:retry_count           | Hash        | 永久    | msgId → 重试次数, max=3            |
| lumio:response:{id}              | String      | 短期    | Bot 异步响应回传                     |
| lumio:notify:{session_id}        | Pub/Sub     | 实时    | "ready" 通知                     |
| lumio:assist:notify:{session_id} | Pub/Sub     | 实时    | 坐席辅助事件                         |
| lumio:assist:feedback:{...}      | String      | 短期    | 反馈结果                           |
| lumio:ae:tracker:{session_id}    | Hash/JSON   | 短期    | 辅助引擎状态跟踪                       |
| lumio:ae:dedup:{trace_id}        | String      | 30s   | 辅助引擎去重                         |
| lumio:safety:words               | Set         | 永久    | 敏感词词库                          |
| lumio:rag:cache:{type}:{hash}    | String JSON | 300s  | 检索结果缓存                         |
| lumio:slot:{session_id}          | String JSON | 3600s | 槽位状态                           |

**设计要点:** 分散在各服务的 Redis key 集中在 session.py 与 redis\_keys.py 统一管理 — 避免每个文件自己拼字符串出现 lumio:chat:stream 与 lumio:chat\_stream 并存的问题。

### 12.4.1 CAS Lua: Redis 端的乐观锁

会话元数据的"读-改-写"是最容易出 race condition 的地方。session.py:67-95 实现的 CAS 流程是:

```
伪代码, 关键逻辑
new_version = current_version + 1
lua = """
if redis.call('HGET', KEYS[1], 'version') == ARGV[1] then
 redis.call('HSET', KEYS[1], 'data', ARGV[2], 'version', ARGV[3])
 return 1
else
 return 0
end
"""
ok = redis.eval(lua, 1, meta_key, current_version, new_data, new_version)
if not ok:
 return current_version # 让 Python 侧读最新值并合并重试
```

Lua 脚本在 Redis 端原子执行, 失败的请求拿到的是"最新 version"而非旧值, 上层调用方据此决定是重试还是放弃。这种设计的好处是**完全无锁**, 多个 Agent worker 同时改同一个 session meta 时, 只有一个会成功, 其余自动 rebase。

### 12.4.2 ZSET 超时队列: 替代 ZK 选举的轻量方案

很多团队用 Zookeeper/etcd 做分布式定时任务调度, 但在 Lumio 这种"5 类会话超时 + 5s 轮询"的低频场景, ZSET 足够。为什么? 因为**超时不是高频任务**——一个在线会话平均生命周期 5 分钟, 5s 轮询意味着每秒只触发 0.03 个超时任务。这种量级根本不需要 ZK 的强一致性选举, ZSET 的 ZREM 原子性足够保证"只有一个 worker 处理"。引入 ZK 等于杀鸡用牛刀, 还要维护一个额外的集群。

```
伪代码: session_timeout.py:44-94
ZADD lumio:session:timeouts <deadline_ts> <session_id>:<timeout_type>
5s 轮询 worker
now = time.time()
expired = ZRANGEBYSCORE lumio:session:timeouts -inf <now> LIMIT 0 100
for sid_type in expired:
 if ZREM lumio:session:timeouts <sid_type>: # 原子, 只有一个 worker 拿到
 handle_timeout(sid_type)
```

ZREM 的返回值天然解决了"多实例竞争": 谁先删掉谁负责处理, 后续 worker 看到 key 已经不存在, 直接跳过。相比 ZK 选举少了一个外部依赖, 相比数据库 SELECT FOR UPDATE SKIP LOCKED 少了一次网络往返。这种"用 Redis 的原子性替代分布式协调服务"的思路, 在很多中小规模系统中都更务实。

### 12.4.3 Stream + Consumer Group: 消息幂等的核心

lumio:chat:stream 用 Redis Stream 而非 List, 关键在于 Consumer Group 机制。它保证"一条消息只被一个 consumer 消费", 同时记录每个 consumer 的 last delivered id, 故障重启后可以从断点继续, 不会丢消息也不会重复消费。配合 lumio:chat:retry\_count Hash(每条消息最多重试 3 次)与 lumio:chat:dead\_letter Stream(重试耗尽后转死信), 整个消息处理流程具备完整的"尝试-重试-兜底"链路。

## 12.5 ES + Milvus 双写: 先 ES 后 Milvus 的回滚链

向量与倒排双路召回是 RAG 的标配, 但"双写一致性"是公认难题。ingestion.py:374-465 给出的策略是先写 ES, 写成功后再写 Milvus, 失败回滚 ES:

```
agent/lumio/services/common/ingestion.py: ~ line 400
es_count = es.bulk_index(chunks)
if es_count != len(chunks):
 raise IngestionError("ES 写入数量异常")

try:
 milvus.insert(chunks)
except Exception as e:
 _rollback_es_docs(chunks) # 逐个 DELETE /_doc/{id}
 raise
```

为什么是 ES 先写、Milvus 后写? 因为Milvus 写入成本远高于 ES(向量索引段合并 + IVF 训练), 一旦 Milvus 失败, 丢弃的代价更大; 而 ES 的 \_doc 删除是 O(1) 的, 回滚代价小。回滚是"逐个删 doc" 而不是"删整个 index", 是因为后者会误伤其他文档。

向量维度 1024 对应 bge-large-zh-v1.5, Milvus 索引参数为 IVF\_FLAT nlist=128, nprobe=16, 距离度量 COSINE。在 10 万级 chunk 上实测 P99 召回延迟 < 50ms。

```
sequenceDiagram
 participant W as Ingestion Worker
 participant PG as PostgreSQL
 participant ES as Elasticsearch
 participant MV as Milvus

 W->>PG: 1. 事务写入 kb_chunk (es_indexed=false, milvus_indexed=false)
 W->>ES: 2. bulk_index(chunks)
 ES-->>W: 3. es_count == len(chunks)?
 alt 成功
 W->>MV: 4. milvus.insert(chunks)
 alt 成功
 W->>PG: 5. UPDATE es_indexed=true, milvus_indexed=true
 else 失败
 W->>ES: 6. _rollback_es_docs(chunks)
 W->>PG: 7. 标记 ingestion 失败
 end
 end
 else 失败
 W->>PG: 7. 标记 ES 阶段失败, 不写 Milvus
 end
end
```

这套时序的关键是"PG 状态字段是真相之源": 即便 ES/Milvus 短暂不一致, worker 也能从 es\_indexed=false 的 PARTIAL 索引扫到未完成 chunk 并重试, 而不需要去对账两个外部系统。

## 12.6 MinIO 与 Kafka: 对象存储 + 事件流留白

MinIO 用作 KB 文档的原始文件存储。agent/scripts/init\_minio.py 启动时创建 lumio-docs bucket 并设为 private。文件路径采用 {category}/{filename} 二级结构, 上传时由 bot/router.py:1248 计算最终 key 并预签名 PUT, 前端直传避免打穿 Agent。

Kafka 是"留白"设计。init\_kafka.py:17-21 创建 3 个 topic: lumio.knowledge.update / lumio.audit.log / lumio.call.summary, 均为 3 partition / replication=1(单节点)。当前没有任何业务代码生产/消费这些 topic, verify\_all.py 只做一次连通性测试。这是为了未来需要做

读扩散(例如把 `audit_log` 同步到数仓)时, 不需要重写领域模型。三个 topic 的命名也暗示了扩展方向: 知识库变更、审计流、对话摘要, 这三类数据天然适合"事件流 + 下游订阅"的形态。

## 12.7 故障模式与降级策略

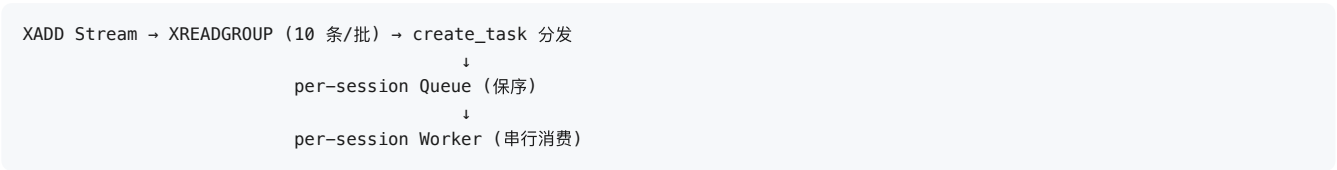
数据层每个组件都可能挂, 系统必须能在降级态继续服务:

| 中间件           | 不可用时表现              | 降级策略                      |
|---------------|---------------------|---------------------------|
| PostgreSQL    | 启动失败                | 不降级, 阻断启动                 |
| Redis         | Stream/Pub-Sub/会话全挂 | 503, 不进入业务                |
| Elasticsearch | RAG 检索挂             | 走 Milvus 单路 (vector-only) |
| Milvus        | 向量检索挂               | 走 ES 单路 (bm25-only)       |
| MinIO         | KB 上传挂              | 上传报错, 检索仍可用               |
| Kafka         | 完全无影响               | 主流程不依赖                    |

ES 与 Milvus 的"互相降级"是 RAG 鲁棒性的关键。检索层在调用前先做健康探测, 失败时自动切换到单路, 召回率会下降但可用性保住。这也是为什么 12.5 节的双写必须对两个引擎都标记 `es_indexed` / `milvus_indexed` 状态——降级路由需要从 PG 读出"哪个引擎对哪些 chunk 是就绪的"。

### 12.7.1 消息双通道:Stream 管可靠性, per-session Queue 管顺序

12.4.3 讲了 `lumio:chat:stream` 的 Consumer Group, 但"消息被消费"和"消息被按顺序处理"是两回事。Lumio 在 Stream 和 Agent 之间还有一层每会话独占的 `asyncio.Queue` ( `bot/router.py:328` ):



为什么需要两层? Stream 的 Consumer Group 只能保证"一条消息只被一个 consumer 读到", 不保证"同一个会话的消息被同一个 worker 按序处理"——批量 XREADGROUP 后 10 条消息是 `create_task` 并发的, 同一会话的两条消息可能被两个协程同时处理, 顺序颠倒。per-session Queue + 独占 Worker 把并发重新收敛为串行: 同一会话的消息严格按入队顺序处理, 不同会话的消息仍然并行——这就是"per-session 串行 + 全局并发"的银行客服时序要求 (客户必须觉得"我的消息一条接一条", 而不是"三条消息三个 Agent 抢着回")。

这层 Queue 还承担了两个 Stream 做不到的职责:

- 队列合并:** worker 处理前先 drain 队列,  $\leq 5$  条 /  $\leq 4000$  字符的消息合并成一次 LLM 调用, 并加上语义标记" (用户连续发送了 N 条消息, 请一次性综合回复) "。超限消息放回队列下一轮处理, 不丢失。
- 消息级幂等:** `lumio:processed:{message_id}` 幂等键 (TTL 300s) 挡住 XAUTOCLAIM 重投递的重复执行——Stream 的 at-least-once 语义靠这层幂等变成"实际恰好一次"。

Worker 的退出策略也值得注意: 队列空闲超过 300s ( `asyncio.wait_for(q.get(), timeout=300)` ) 自动退出并从 `_session_active` 注销, 避免每个会话的协程常驻泄漏; 新消息到来时 `_dispatch_message` 发现会话不在活跃集, 重新创建 worker。

### 12.7.2 会话历史双写:Redis List 是投影, PG 是真相

硬性原则 1 说"Redis 只是 PG 的投影", 会话历史是最典型的例子。每次对话轮次写两处:

- Redis** `lumio:session:{id}:history` (List): `RPUSH` + `LTRIM 20` 只留最近 20 轮, TTL 1800s——给 Bot 拼 prompt、坐席摘要、断线重连加载上下文用, 要求快;
- PG** `chat_message` 表: 每轮消息一行, 带 `processing_status` 状态机 (见下), 永久保存——给合规查询、客服复盘、GDPR 删除用, 要求全。

为什么 Redis 只留 20 轮? LLM 上下文窗口有限, 20 轮 (约 3-5k 字符) 是"拼得下 prompt 且不稀释注意力"的经验值; 更早的轮次被裁剪时, 增量摘要 ( conversation\_summary + last\_summarized\_turn\_id 追踪) 接管记忆职责——被裁的是原文, 不是语义。

Redis 丢失的恢复路径: 会话重启后从 PG chat\_message 按 session\_id 回放重建历史。这正是"投影"的含义——Redis 里删了重建就是, PG 才是丢不起的。

### 12.7.3 chat\_message 的消息状态机: 审计字段承载全生命周期

chat\_message 不只是"存消息内容", 它带了一套 5 态处理状态 ( ChatMessageStatus ):

```
queued → processing → done
 ↘ skipped (TTL 过期)
 ↘ error (Agent 异常)
```

每个状态转换都伴随审计字段: processing\_status 、 source (llm / template / fallback / fast\_reply / timeout / merged)、 processing\_duration\_ms 、 error\_message 。这套字段让"消息去哪了"可以纯 SQL 回答: SELECT count(\*) FROM chat\_message WHERE session\_id=? AND processing\_status='skipped' 就是该会话的丢消息统计, WHERE source='fast\_reply' 就是满荷兜底的影响面——**审计字段让排障从"翻日志猜"变成"查表数"**。

写入时机值得展开: 消息一进 Stream ( chat\_send ) 就写 queued , worker 处理前更新, 完成后写 done 。这条链路让 chat\_send 返回的 message\_id 可以随时查询处理进度——客户端轮询 GET /sessions/{id}/messages 就能看到"我这条消息到底是完成了、超时了还是出错了", 而不是只知道"发出去"。

## 12.8 小结

数据层是工程化的"基础设施层", 它不生产业务价值, 但决定系统能跑多稳、跑多快。Lumio 用 6 个中间件分担不同负载, 用 19 张 PG 表 + 12 个迁移固化领域模型, 用 15+ Redis key + CAS Lua + ZSET 超时做实时协调, 用 ES+Milvus 双写 + PG 状态字段做双路召回一致性, 用 MinIO 承接大文件、用 Kafka 预留事件流。理解这套设计, 是后续读懂 RAG 检索(第 5 章)、RAG 摄入(第 9 章)、会话状态机(第 6 章)的基础。

延伸阅读: - [第 5 章 RAG 检索全链路](#) — 双写一致性细节 - [第 9 章 RAG 摄入](#) — 摄入端双写 - [第 6 章 会话状态机](#) — Redis CAS 详解

第 13 章: 部署

Lumio 的部署设计遵循一个朴素的工程直觉: 让本地开发与生产环境共享同一份容器编排, 只在 K8s 层做规模化。这意味着开发者日常用 `make demo` 启动的 24 个容器, 几乎就是 K8s 集群上跑的那一套——只是多了 HPA、Secret 和反代网关。这一章将围绕 `docker-compose` 拓扑、`multi-stage` 镜像、K8s manifests、Nginx 反代、`opt-in gateway profile` 与 46 个 Make target 这六条主线, 解释 Lumio 在部署上做出的一系列权衡。

13.1 为什么是 24 个服务

第一次看到 `deploy/docker-compose.yml` 的服务数量, 多数人会问同一个问题: 一个聊天+客服系统, 真的需要这么多中间件吗? 答案是肯定的, 但前提是这一切只服务于单机一键 Demo 场景——开发者不必在多个工具链之间切换, 也不必为某个组件单独写文档。把这 24 个服务按职责切分, 可以归为五类:

| 类别                | 服务                                              | 镜像 / 构建                                        | 关键端口                     | 健康检查                                 |
|-------------------|-------------------------------------------------|------------------------------------------------|--------------------------|--------------------------------------|
| 核心 7              | postgres                                        | postgres:16                                    | 5432                     | <code>pg_isready -U lumio</code>     |
|                   | redis                                           | redis:7.2-alpine                               | 6379                     | <code>redis-cli ping</code>          |
|                   | elasticsearch                                   | 自构建 smartcs/elasticsearch-ik:8.19.9 (预装 IK 分词) | 9200 / 9300              | <code>curl /_cluster/health</code>   |
|                   | milvus                                          | milvusdb/milvus:v2.4.0                         | 19530 / 9091             | <code>curl /healthz</code>           |
|                   | etcd                                            | quay.io/coreos/etcd:v3.5.5                     | 不暴露 (避免与 VPNKit 冲突)      | <code>etcdctl endpoint health</code> |
|                   | minio                                           | minio/minio:RELEASE.2024-01-16T16-07-38Z       | 9000 / 9001              | <code>mc ready local</code>          |
|                   | kafka                                           | apache/kafka:3.7.0 (KRaft 单节点)                 | 9092 / 9094              | <code>kafka-topics.sh --list</code>  |
| 可观测性 5            | jaeger                                          | jaegertracing/all-in-one:1.57                  | 16686 (UI) / 4318 (OTLP) | 隐式                                   |
|                   | prometheus                                      | prom/prometheus:v2.50.0                        | 9090                     | 隐式                                   |
|                   | grafana                                         | grafana/grafana:10.4.0                         | 3001                     | 隐式                                   |
|                   | redis-exporter                                  | oliver006/redis_exporter:latest                | 9121                     | 隐式                                   |
|                   | postgres-exporter                               | prometheuscommunity/postgres-exporter          | 9187                     | 隐式                                   |
|                   | kafka-exporter                                  | danielqsj/kafka-exporter                       | 9308                     | 隐式                                   |
| 可观测性+             | (如上 6 项, 其中 exporter 3 项 + jaeger/prom/grafana) |                                                |                          |                                      |
| Java chat-svc 2+1 | customer-server                                 | chat-svc/customer-server (本地 jar)              | 8080                     | 由 chat-svc 自行探测                      |
|                   | agent-server                                    | chat-svc/agent-server                          | 8081                     | 同上                                   |

| 类别                  | 服务         | 镜像 / 构建                                        | 关键端口             | 健康检查                               |
|---------------------|------------|------------------------------------------------|------------------|------------------------------------|
|                     | zookeeper  | zookeeper:3.8                                  | 2182             | 隐式                                 |
| gateway<br>opt-in 3 | nacos      | nacos/nacos-server:v2.4.3                      | 8848 /<br>9848   | /nacos/v1/console/health/readiness |
|                     | higress    | higress-registry.../all-in-one:2.1.5           | 10000 /<br>18080 | 隐式                                 |
|                     | mcp-server | mcp-server/Dockerfile → lumio-mcp-server:1.0.0 | 8090             | /actuator/health                   |
| 反向代理 1              | nginx      | nginx:1.25-alpine                              | 8080 (宿主机)       | 隐式                                 |

(注: 上表"可观测性"分类中 jaeger/prometheus/grafana 为核心三件套, 三个 exporter 辅助, 实际是 6 个可观测性容器。)

观察这张表, 一个隐藏的设计哲学浮现出来: **核心服务少而必要, 可观测性厚而完整, 网关薄且 opt-in**。postgres / redis / milvus / kafka / minio 是 AI 应用基础设施的"五件套"——任何 RAG 系统都跑不掉; 而 exporter + jaeger + prometheus + grafana 的组合, 在早期就把"线上黑盒"问题掐死在源头, 这是 Lumio 在每一章反复强调的"可观测性内建"。

值得专门说一句的是 etcd 的"刻意不暴露": deploy/docker-compose.yml:91 留了一行注释, 解释 etcd 端口故意不开给宿主机——避免与 VPNKit 代理冲突。在 macOS + Colima / OrbStack 环境下, 容器与宿主网络之间的端口转发非常容易撞车, 让 etcd 只活在容器网络 lumio-net 内部, 既满足了 Milvus 的内部寻址, 又把潜在端口冲突的可能性压到零。这种"少开一个端口"的小决定, 是多年被 Docker 网络折磨出来的经验。

再细看 ES 的镜像构建: deploy/elasticsearch/Dockerfile:6 一行 RUN elasticsearch-plugin install --batch ... 在构建期就把 IK 中文分词器装好了, 而不是让用户首次启动后再去手动执行 bin/elasticsearch-plugin install 还要重启。这种"构建期固化运行期配置"的做法, 与 multi-stage 镜像的"只在 builder 阶段装工具"是同一种哲学: 一切不可变的部分都应该在不可变层完成。

## 13.2 部署拓扑与启动顺序

怎么读这张图 — "谁先起来谁后起来": 箭头表示"依赖" — 没有 etcd 和 MinIO, Milvus 起不来; 没有 Milvus/ES/Redis/PG, Bot 和 Assist 起不来. 这就像盖楼: 地基 (数据层) 必须先于楼层 (应用层), 楼层必须先于装修 (监控). 顺序错了, 应用启动时连不上数据库, 直接崩溃.

24 个服务不是同时拉起的, 它们之间有严格的依赖关系。Milvus 必须等 etcd + minio 健康; 应用服务必须等 Milvus / ES / Redis / Postgres 全部就绪; Bot 与 Assist 还必须等一次性 demo-init 跑完迁移+种子:

```
graph LR
 subgraph 核心
 P[postgres:5432] --> B[bot:8000]
 R[redis:6379] --> B
 P --> A[assist:8001]
 R --> A
 end
 subgraph 向量与检索
 E[elasticsearch:9200] --> B
 E --> A
 ET[etcd:2379] --> M[milvus:19530]
 MI[minio:9000] --> M
 M --> B
 M --> A
 end
 subgraph 消息
 K[kafka:9092] --> B
 K --> A
 end
 subgraph 可观测性
 PE[postgres-exporter] --> PR[prometheus:9090]
 RE[redis-exporter] --> PR
 KE[kafka-exporter] --> PR
 PR --> G[grafana:3001]
 J[jaeger:4318/16686] -.OTLP.-> B
 end
 subgraph 反代
 N[nginx:8080] --> B
 end
```

```

 N --> A
end
subgraph opt-in
 NC[nacos:8848] --> H[higress:10000]
 H --> MS[mcp-server:8090]
end

```

Compose 通过 `depends_on: { condition: service_healthy }` 把启动顺序从"先来到后到"升级为"先健康后启动"——这是最容易被新同学忽略却最值钱的一行配置。Milvus 的 `depends_on` 块明确写明依赖 `etcd` 与 `minio` 的 `service_healthy`，这意味着即使 `etcd` 容器已 `running`，只要 `etcdctl endpoint health` 还没回 `0`，Milvus 就拒绝启动。

另一个值得注意的细节是 Kafka 的 KRaft 模式: `KAFKA_PROCESS_ROLES: broker,controller` + `KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093` 让单节点也能跑出集群语义，彻底免去 ZooKeeper 依赖。这是 Kafka 3.3+ 的官方推荐，在开发场景里节约的不仅是 1 个容器，更省去了 ZK 与 Broker 的脑裂配置。`KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"` 又把"首次跑 Demo 必须先手动建 topic"这个常见踩坑点干掉了。

## 13.3 Multi-stage Dockerfile 与多入口

应用镜像的 57 行 Dockerfile 把"镜像大小"与"启动灵活性"两个目标拆得很干净：

```

deploy/Dockerfile:4-17
FROM python:3.11-slim AS builder
WORKDIR /app
RUN pip install --no-cache-dir poetry==2.0.1
COPY pyproject.toml poetry.lock* ./
RUN poetry config virtualenvs.in-project true \
 && poetry install --no-interaction --no-ansi --only main --no-root

```

这是典型的 **builder-only 镜像**——`poetry install` 的目的是把依赖装到 `.venv` 里，而不是把 Poetry 本身带进 runtime。第二阶段只 `COPY --from=builder /app/.venv /app/.venv` 把虚拟环境拿过来，再叠加应用代码与配置，最终镜像只有 50MB 左右，比"用 builder 直接 runtime"小一个数量级。

第二个巧思是**多入口切换**：一个 `lumio:latest` 镜像既能跑 Bot(:8000) 也能跑 Assist(:8001)，靠 `SERVICE` 环境变量选择启动目标，见 `deploy/Dockerfile:36-41`：

```

ENV SERVICE=bot
COPY --chmod=755 docker-entrypoint.sh /docker-entrypoint.sh

```

健康检查也用了一行 Python 条件表达式同步端口 ( `deploy/Dockerfile:53-54` )：

```

HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
 CMD python -c "import os,urllib.request; urllib.request.urlopen('http://localhost:%s/api/health' % ('8001' if os.environ.g

```

比起在镜像里塞两份 `entrypoint` 脚本，这种做法让"一份镜像，两个角色"成为可能，配合 K8s 两个 Deployment 复用同一份 `image: lumio:latest`，镜像仓库的存储与 CI 时间都减半。一个常被问到的细节是：既然 multi-stage 能省 builder 层，那 Poetry 自身能不能也只装在 builder？答案是可以，但 `poetry==2.0.1` 锁在 builder 阶段后，它的 `wheel` 就不会被带进 runtime 镜像，`pip install` 留下的缓存又在 `pip install --no-cache-dir` 这条指令下被抹掉——层层过滤后，runtime 镜像里只剩下纯净的 `.venv/site-packages` 和应用代码，这正是镜像最终能压到 50MB 级别的关键。

## 13.4 K8s manifests: HPA + 探针 + Secret

生产环境只部署 4 类资源：Bot Deployment、Assist Deployment、Bot Service、Assist Service，外加两个 HPA。Bot/Assist 各 2 副本起步，HPA 区间 2-6，CPU 70% 触发扩容：

```

deploy/k8s/lumio.yaml:151-168
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
 name: lumio-bot-hpa
spec:
 scaleTargetRef:

```

```

 name: lumio-bot
 minReplicas: 2
 maxReplicas: 6
 metrics:
 - type: Resource
 resource:
 name: cpu
 target:
 type: Utilization
 averageUtilization: 70

```

70% 的阈值是 K8s 社区文档推荐的折中点: 太低会导致"毛刺触发扩容", 太高又会在真实流量到达前失守。资源 `requests: 250m/512Mi, limits: 1000m/1Gi` 同样保守——4 副本起步总内存 2Gi, 在 8 节点小集群里也游刃有余。

liveness 与 readiness 探针分别指向 `/api/health/live` 和 `/api/health/ready`, 区分"还在不在"与"准备好接流量没", 这两条是 K8s 上 Web 服务的标配:

```

deploy/k8s/lumio.yaml:50-62
livenessProbe:
 httpGet: { path: /api/health/live, port: 8000 }
 initialDelaySeconds: 10
 periodSeconds: 30
readinessProbe:
 httpGet: { path: /api/health/ready, port: 8000 }
 initialDelaySeconds: 15
 periodSeconds: 10
 failureThreshold: 3

```

`LUMIO_JWT_SECRET` 通过 `valueFrom.secretKeyRef` 注入——第五轮修复后, 6 个敏感凭据全部走 K8s Secret (`jwt-secret` / `llm-api-key` / `minio-access-key` / `minio-secret-key` / `es-username` / `es-password` / `redis-password`), 中间件 `host/port` 等非敏感项明文写进 Deployment spec。两个 Deployment 均显式设置 `SERVICE=bot` / `SERVICE=assist` (此前 `assist` 未设 → Dockerfile 默认 `bot` → 启动错应用), 并加 `terminationGracePeriodSeconds: 90` (> LLM timeout 60s, 滚动更新不截断 in-flight 请求)。

**生产凭据校验 (第五轮):** `config.validate_production_security` 在生产环境强制要求 `LLM_API_KEY` / `MINIO` / `ES` / `REDIS` 凭据非默认值, 缺失即 `ValueError` 拒绝启动 — k8s manifest 必须注入全部 secret, 否则 crash loop (manifest 已同步)。

## 13.5 Nginx 反代: 路径分流 + WebSocket

`deploy/nginx/nginx.conf` 一共 42 行, 干了两件事: 把路径前缀路由到 Bot(:8000) 或 Assist(:8001), 并为 `/api/ws/` 启用 WebSocket upgrade。

```

deploy/nginx/nginx.conf:12-30
location /api/chat/ { proxy_pass http://host.docker.internal:8000/api/chat; }
location /api/kb/ { proxy_pass http://host.docker.internal:8000/api/kb; }
location /api/health { proxy_pass http://host.docker.internal:8000/api/health; }
location /api/metrics { proxy_pass http://host.docker.internal:8000/metrics; }

location /api/notify { proxy_pass http://host.docker.internal:8001/api/notify; }
location /api/session/ { proxy_pass http://host.docker.internal:8001/api/session/; }
location /api/hold { proxy_pass http://host.docker.internal:8001/api/hold; }
location /api/resume { proxy_pass http://host.docker.internal:8001/api/resume; }
location /api/review/ { proxy_pass http://host.docker.internal:8001/api/review/; }
location /api/feedback { proxy_pass http://host.docker.internal:8001/api/feedback; }
location /api/analyze { proxy_pass http://host.docker.internal:8001/api/analyze; }

location /api/ws/ {
 proxy_pass http://host.docker.internal:8001/api/ws/;
 proxy_http_version 1.1;
 proxy_set_header Upgrade $http_upgrade;
 proxy_set_header Connection "upgrade";
 proxy_read_timeout 3600;
}

```

路径分流的依据是"哪类业务归属谁": 知识库检索 / 对话 / 健康检查走 Bot; 坐席辅助(接管/恢复/复核/反馈/分析)走 Assist; WebSocket 走 Assist 因为坐席工作台是长连接密集场景。 `proxy_read_timeout 3600` 防止 1 小时后被网关主动断开——客服场景用户挂起排队是常见事, 默认 60 秒会致命。



Nginx 的定位是"开发网关", 生产环境由 Higress( `--profile gateway` 启用)替代, 这是 Lumio 把网关做成 opt-in 的核心动机: 多数开发者用不到 MCP 联邦, 不该被迫跑一个 500MB+ 的网关容器。

## 13.6 Opt-in Gateway Profile: 设计的克制

`nacos / higress / mcp-server` 三个服务在 `docker-compose.yml` 中都标了 `profiles: ["gateway"]`, 这意味着 `docker compose up -d` 默认不会拉起它们, 必须显式 `docker compose --profile gateway up -d` 或 `make gateway-up` 才会启动。

这种"可观测但默认关闭"的设计背后是一个朴素判断: **95% 的本地开发时段用不到 MCP 联邦**。Nacos 启动慢、Higress 镜像大、MCP Server 需要 Java 22+, 三件加起来足以让一台 16GB 内存的 MacBook 风扇狂转。把它们做成 opt-in, 核心 7+5+2+1 共 15 个容器先跑起来满足绝大多数调试, 等真要联调 MCP 工具面再开 gateway, 体验差距巨大。

## 13.7 Makefile: 33 个 Target 的语义地图

Makefile:1-194 一共暴露了 33 个 target, 按职责可分 11 组, 全部用 `##` 注释 + `help` 索引自描述:

- **AI Agent (5):** `install / dev / mcp-ref / mcp-server-{build,test,run}`
- **质量 (6):** `test / test-cov / lint / format / type-check / pre-commit / bench`
- **Docker (6):** `build / build-app / up / down / ps / logs`
- **gateway opt-in (2):** `gateway-up / gateway-down`
- **Demo (5):** `demo / demo-down / demo-logs / demo-ps / demo-push`
- **初始化 (1):** `init / init-minio / seed`
- **验证 (4):** `verify / verify-ollama / verify-mcp-e2e / verify-observability`
- **迁移 (3):** `migrate / migrate-create / migrate-downgrade`
- **清理 (2):** `clean / distclean`
- **前端 (3):** `web-{dev,build,install}`
- **Java (3):** `chat-svc-{build,up,down}`

最常用的入口是 `make demo`: 它把 `docker-compose.yml` 与 `docker-compose.demo.yml` 叠加, 先跑一次性 `demo-init` (迁移+灌种子), 再拉起 Bot(:8000) 和 Assist(:8001):

```
Makefile:88-101
DEMO_COMPOSE := -f docker-compose.yml -f docker-compose.demo.yml
demo: ## 一键启动完整 Demo (中间件 + 初始化 + Bot:8000 + Assist:8001)
 cd deploy && docker compose $(DEMO_COMPOSE) up -d --build
```

`demo-push` 进一步把镜像推到 Docker Hub( `slowleelab/lumio:demo` ), 用于"演示前远程部署"这类场景, 但 CI/CD 内部推送通常走专属仓库, 不与该 target 冲突。

`chat-svc-up` 这一组 target 看上去游离在主流程之外, 实则承担着 Java 在线客服子系统的启动入口: `customer-server` 监听 8080, `agent-server` 监听 8081, 二者通过 zookeeper 协调会话状态。 `make chat-svc-down` 用 `kill -f` 兜底, 比 `kill -9 PID` 更适合临时演示场景——不需要先查 PID, 一句命令把所有 chat-svc 相关进程清干净。

## 13.8 环境变量三层: dev / staging / production

Lumio 的配置分层在第 2 章已展开, 这里只重申部署侧的差异:

- **dev:** 本地 `.env`, 全部 host 是 `localhost` 或 `host.docker.internal`, 占位值允许( `POSTGRES_PASSWORD_DOCKER=lumio_pass` )。
- **staging:** K8s ConfigMap, 中间件地址走内部域名( `postgres.lumio.svc.cluster.local` ), `LUMIO_ENVIRONMENT=staging`。
- **production:** K8s Secret + ConfigMap, 强制 `LUMIO_JWT_SECRET` 等敏感字段从 Secret 注入, ConfigMap 中所有占位符必须替换, CI 上有 `lumioconfig validate` 检查。

这样做的好处是同一份 `lumio` 代码在三个环境里只换 env 即可, 不必维护三套 helm chart 或三套启动脚本。

## 13.9 常见问题排查

部署踩坑通常集中在启动顺序、端口、数据库迁移三处。**端口冲突**是最高频问题，一句 `lsof -i :8000` 找出占 PID，kill 掉即可，也可在 `compose` 里改端口映射。**启动顺序错乱**通常表现为 Bot/Assist 起来后立刻连不上 Milvus，此时 `docker compose ps` 看 Milvus 是否 Up (healthy)，若 Up 但未 healthy，必是 etcd 或 minio 的健康检查卡住。**数据库迁移失败**则直接看 `cd agent && poetry run alembic current` 返回的 revision，与 `alembic/versions/` 目录里的 head 对比即可定位。

万能兜底是 `make verify`，它依次 ping 全部中间件并打印 RTT，哪一环亮红就从那一环往回查。

### 13.9.1 从 make dev 到生产镜像:同一套代码的三条启动路径

Lumio 应用层有三条启动路径，它们共享同一份配置但启动方式完全不同——这是新同学最容易困惑的地方：

| 路径   | 命令                                        | 进程形态                               | 适用         |
|------|-------------------------------------------|------------------------------------|------------|
| 本地裸跑 | <code>make dev</code>                     | 宿主机 Python + <code>--reload</code> | 日常开发，秒级热重载 |
| 容器内  | <code>docker compose up bot assist</code> | 容器内 uvicorn (无 reload)             | Demo / 验收  |
| K8s  | Deployment + HPA                          | 多副本，无状态                            | 生产         |

三条路径的关键差异在**配置来源**：本地裸跑读 `.env` 文件，容器读 `compose` 注入的 environment，K8s 读 ConfigMap/Secret——代码里 `get_settings()` 对三者无感知，这是第 2 章 Pydantic-settings 设计的结果：**配置只问“值从哪来”，不问“我在哪跑”**。

值得一提的细节：`make dev` 用 `--reload` 时 uvicorn 会启动一个 reloader 父进程 + 一个 worker 子进程，而 `start_bot_worker` 的后台协程 (消费循环/监控/XAUTOCLAIM) 都挂在 worker 子进程上——**代码改动触发 reload 时，后台协程随子进程一起重建**，不会出现“老协程持有旧代码”的僵尸态。这正是“全部后台任务挂在 lifespan 里” (第 3 章) 的部署侧收益。

### 13.9.2 demo-init:一次性迁移与种子数据的幂等设计

`docker-compose.demo.yml` 里的 `demo-init` 服务是“一键 Demo”的关键，它做了三件事：跑 Alembic 迁移 → 注入种子数据 → 创建 admin 账号。幂等设计是这里的难点——**Demo 可以反复 `make down && make up`，但迁移和种子不能跑两次就报错**：

- 1. Alembic 迁移本身幂等 (revision 已应用则跳过)；
- 2. 种子数据用“存在即跳过”的语义 ( `INSERT ... ON CONFLICT DO NOTHING` 或先查后插)；
- 3. 环境变量 `LUMIO_ADMIN_PASSWORD` 支持 Demo 初始化时创建 admin 账号——**密码从 env 注入而不是写死在 seed 脚本里**，否则仓库里就埋了一把“人人都知道的钥匙”。

`bot / assist` 服务的 `depends_on` 声明依赖 `demo-init` 的 `service_completed_successfully` ——迁移没跑完，应用不启动。这条链让 `make demo` 永远得到“迁移已应用、账号已创建、应用已就绪”的确定性状态，而不是“看运气”。

### 13.9.3 健康检查的三段式:liveness / readiness / startup

`compose` 和 K8s 里的健康检查不是一套，而是三个探针配合 (K8s 语义，`compose` 用 `start_period` 模拟 startup)：

| 探针                                          | 问的问题                     | 失败动作     |
|---------------------------------------------|--------------------------|----------|
| <code>liveness (/health/live)</code>        | 进程还活着吗？                  | 重启容器     |
| <code>readiness (/health/ready)</code>      | 依赖 (Redis/PG/ES...) 都通吗？ | 摘除流量，不重启 |
| <code>startup (compose start_period)</code> | 启动慢要不要宽容？                | 宽容期内不判死  |

`/health/live` 只回答“进程存活” ( `{"status": "alive"}` )，不查任何依赖——**liveness 如果查依赖，Redis 抖动会导致容器被反复重启** (kubelet 判死 → 重启 → 又抖动 → 再重启，雪崩)。`/health/ready` 才查依赖并返回 200/503——LB 把 503 的实例摘出流量池，让健康实例承接。两个端点必须刻意“职责分离”，这是 deployment 领域最常见的混淆点，也是 `test_bot_router_api.py` 里 `test_health_live / test_health_check` 两条用例钉死的契约。

### 13.10 本章小结

Lumio 的部署核心可以总结为三条：

1. 一份 **compose** 跑到底——24 个服务分核心/可观测/网关/反代四层, 但只通过 `depends_on.health` 控制启动序, 不引入额外的 `init` 容器或 `sidecar`。
2. 一份镜像两个角色——`multi-stage` 把镜像压到 50MB 量级, `SERVICE` 环境变量让同一份 `lumio:latest` 既能跑 Bot 也能跑 Assist, 镜像仓库与 CI 时间双双减半。
3. 网关 **opt-in**, 不强加——95% 的本地场景不需要 Higress/Nacos/MCP Server, 默认不拉起, 真需要时 `make gateway-up` 一键打开。

这三条的共性是克制: 不为了"看起来专业"而把所有组件默认拉起, 也不为了"节省内存"而把可观测性阉割。下一章会沿着这条主线, 走进可观测性系统, 看看 Prometheus 是怎么把 24 个容器的指标抓得一清二楚的。

延伸阅读: - [第 1 章 整体架构](#) — 三层分层 - [第 2 章 配置系统](#) — 16 个 `env_prefix` - [第 10 章 可观测性](#) — Prometheus 抓取

## 14 测试策略

# 第 14 章 测试策略

## 14.1 测试体系总览

Lumio agent 服务的测试集由 60+ 个 `test_*.py` 文件、716 个 `test_` 函数通过 (+5 跳过) 组成, 覆盖消息管道、确认状态机、上下文预算、压缩质量门、会话超时、认证越权、配置校验、检索、审计、可观测性等核心模块。这套测试体系不是事后补的 — 它的形态直接体现了三个工程取舍: 真实中间件而非 `mock`、真实子进程而非 `in-process`、`mypy advisory` 而非 `strict`。理解这三个取舍, 就理解了整个 Lumio 的测试策略。

## 14.2 「真实中间件 + 真实子进程」哲学

Lumio 的测试不依赖 `fakeredis`、不依赖 `httpx_mock`、不依赖任何 `Mock(...)` — 这不是洁癖, 是经过权衡的工程选择。

`agent/tests/conftest.py:1-7` 把策略写进了 `docstring` 的第一段:

```
"""pytest 配置和公共 fixtures — 端到端 API 测试

启动真实 uvicorn 服务器 (bot :8000 + assist :8001),
通过 HTTP 请求验证完整请求/响应生命周期。
"""
```

为什么走 e2e 路线而不是 `mock` 路线? 核心原因是 Lumio 的故障模式往往在组件的边界上 — Redis 连接断开后 Lua 脚本的行为、Postgres pool 耗尽后 SQLAlchemy 的重试、`uvicorn` 启动时 `lifespan hook` 的初始化顺序, 这些都不能靠 `mock` 重现。一旦用 `mock` 测过, 就只能证明"在 `mock` 假设下能跑通", 而不能证明"在真实部署下能跑通"。

为支撑这条路线, `agent/pyproject.toml` 在 `pytest` 配置里设了 `asyncio_mode = "auto"` — 这样所有 `async def test_` 自动变成 `pytest-asyncio` 用例, 不再需要每个文件都加 `@pytest.mark.asyncio` 装饰器, 减少样板代码。

CI 端, `unit-tests job` 通过 GitHub Actions 的 **service container** 机制启动真实 Redis 7.2 和 Postgres 16 (`.github/workflows/ci.yml:57-77`), 配 `health-cmd / health-interval` 做健康探测。本地开发者则通过 `make up` 拉起同一套 Docker Compose stack — 跟生产一致。

至于 e2e 用例本身, 不在测试进程内启 FastAPI, 而是 `subprocess.Popen` 拉起独立 **uvicorn 子进程** (`conftest.py:60-88`), 端口刻意选 8765 / 8766 (`conftest.py:33-34`), 避免与开发环境的 8000 / 8001 冲突。注释直白写了原因:

```
BOT_PORT = 8765 # 避免与开发环境 :8000 冲突
ASSIST_PORT = 8766 # 避免与开发环境 :8001 冲突
```

这条注释读起来像废话, 但它揭示了一个常被忽视的测试可靠性问题: 如果 e2e 直接用 8000, 开发者本机跑着 `uvicorn` 调试时, 测试一拉子进程就端口冲突 `skip`, 本地信号被噪音掩盖。

## 14.3 关键 fixture 拆解

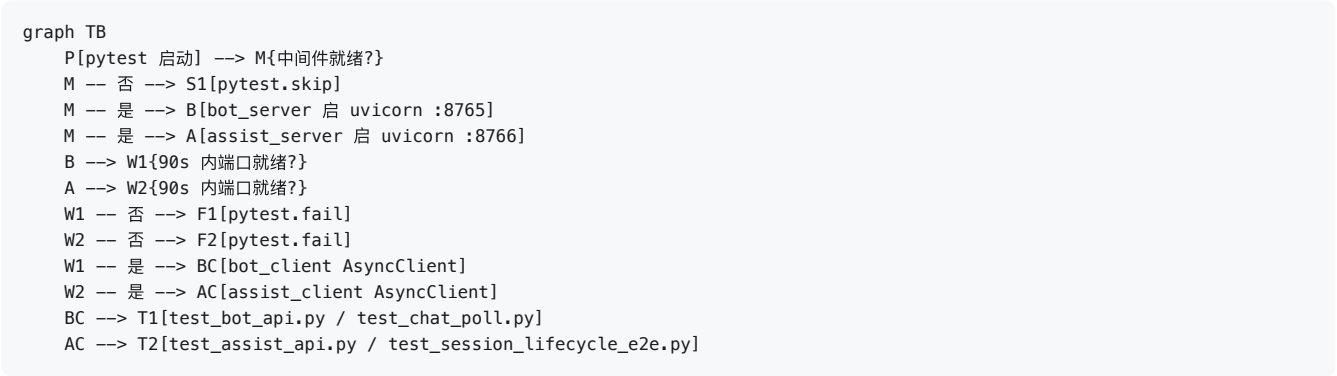
`agent/tests/conftest.py:1-181` 整份文件是测试策略的"宪法", 核心由四组 `fixture` 组成。

`_check_middleware_ready()` (`conftest.py:106-116`) 是入口守卫。它用 `socket.connect_ex` 探测 127.0.0.1:6379 (Redis) 和 127.0.0.1:5432 (Postgres), 任何一个不通就 `pytest.skip("Docker 中间件未启动")`。这让"忘记 `make up`"成为一次显式的 `SKIPPED` 而不是一堆莫名其妙的 `ERROR`, 调试体验差别巨大。

`bot_server / assist_server` (`conftest.py:122-166`) 是 `session-scoped fixture`, 在整个 `pytest` 会话里只启一次子进程。它们先调 `_check_middleware_ready()` 守门, 再用 `_check_port()` 防止重复启动, 最后用 `_wait_for_port(..., timeout=90)` 容忍 cold start — 90 秒这条线对应生产环境冷启动 + 模型预热 + 索引初始化的全链路预算, 不是随便拍脑袋选的。

`bot_client / assist_client` (`conftest.py:169-180`) 是 `pytest_asyncio.fixture`, 包装 `httpx.AsyncClient`, `timeout=30.0`。它不自己启服务, 而是依赖 `bot_server / assist_server` — 这种"进程 `fixture` 配客户端 `fixture`"的两层结构, 让"换客户端配置"和"换服务进程"互不干

扰。



怎么读这张图 — "测试的前置关卡": 跑测试前先问三件事 — ① 中间件 (Redis/PG) 起了吗? 没起就跳过 (不是报错, 开发者没 make up 也看得懂); ② 服务子进程 90 秒内能就绪吗? 不能就失败 (真有问题要暴露); ③ 都好了 → 客户端 fixture 才可用. 整个流程是显式的: 每个分支都有明确结果, 不会出现"莫名失败 5 分钟才发现是没起 Docker"的噪音.

## 14.4 TOP 10 测试覆盖密度

下面这张表是按用例数排的 TOP 10, 直接反映 Lumio 当前最"被担心出 bug"的几块:

| 排名 | 文件                      | 用例数 | 关注点                    |
|----|-------------------------|-----|------------------------|
| 1  | test_decision.py        | 44  | 评估器 D1/D2/D3 决策表       |
| 2  | test_circuit_breaker.py | 36  | 熔断器 3 态 + 滑动窗口 + 慢调用   |
| 3  | test_state_models.py    | 28  | 状态机 ORM 模型             |
| 4  | test_session.py         | 25  | 会话生命周期 + 转换            |
| 5  | test_faq_service.py     | 24  | FAQ CRUD + 审批 workflow |
| 6  | test_bot_memory.py      | 24  | 客户记忆 + Redis           |
| 7  | test_audit_health.py    | 24  | 审计 + 健康端点              |
| 8  | test_retrieval.py       | 22  | RAG 混合检索               |
| 9  | test_arbitrator.py      | 22  | 仲裁融合                   |
| 10 | test_integration.py     | 20  | 跨服务集成                  |

test\_decision.py 用 44 个用例覆盖 D1/D2/D3 三种决策路径, 因为这是对话路由的"调度中枢", 任何误判都会让用户掉进错的技能池。test\_circuit\_breaker.py 的 36 个用例更直接体现了"故障模式写在测试里"的思路 — CLOSED/HALF\_OPEN/OPEN 三态的迁移、滑动窗口的成功率统计、慢调用比例触发, 全是状态机加时间维度的组合, 单元测试靠 mock 时间很易失真, 真实子进程跑出来反而稳。

## 14.5 e2e vs unit 划分

agent/tests/ 下绝大多数文件是单进程 unit (无子进程, 直接 import lumio 模块, 调函数), 它们跑得快、可以并行、跟中间件弱耦合。e2e 子集则通过 bot\_client / assist\_client fixture 拉起完整 uvicorn, 跑 HTTP 链路。

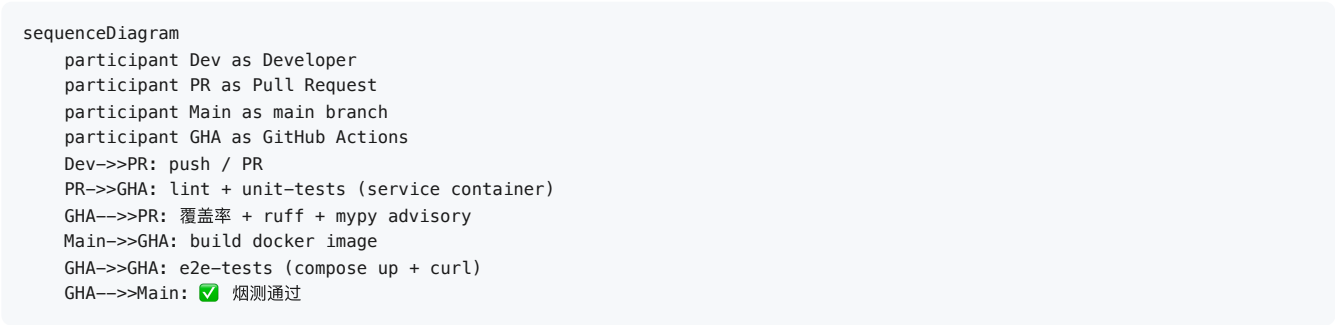
e2e 文件一共有 5 个:

- test\_bot\_api.py — 机器人 HTTP API
- test\_assist\_api.py — 坐席辅助 HTTP API
- test\_chat\_poll.py — 长轮询
- test\_session\_lifecycle\_e2e.py — 会话全生命周期
- test\_integration.py — 跨服务集成 (纯 mock, 无中间件依赖)

unit-tests job 在 CI 排除 4 个依赖真中间件的 e2e 文件, `test_integration.py` (纯 mock) 已纳入常规运行 (第五轮修复 — 它被 `--ignore` 没有任何理由, 白白损失覆盖率):

```
poetry run pytest \
 --ignore=tests/test_bot_api.py \
 --ignore=tests/test_assist_api.py \
 --ignore=tests/test_session_lifecycle_e2e.py \
 --cov=lumio --cov-report=term-missing --cov-fail-under=55 \
 -v --tb=short
```

为什么排除 3 个? 三个原因叠加: (1) `bot_client` / `assist_client` 需要 `bot_server` / `assist_server` session fixture 启子进程, 启停开销大, 挤占 unit 反馈循环; (2) 这些用例必须有真中间件, 缺一个就整片 SKIP, 反而掩盖真实失败; (3) e2e 抽到独立的 `e2e-tests job` (`.github/workflows/ci.yml`) 反而更干净 — 它在 main 分支、build job 之后跑, 先 `docker compose up demo` 再 `curl /api/chat/send` 烟测 (带认证 token, 第五轮修复 — 认证强制后无 token 必 401), 全链路黑盒验证。



## 14.6 覆盖率门槛渐进

unit-tests job 加了 `--cov-fail-under=60` (`.github/workflows/ci.yml`), 同时 `agent/pyproject.toml` 锁了 `[tool.coverage.report] fail_under = 60`。两处对齐, pytest 才会真正读取门槛 (只有 `pyproject` 的锁而 pytest 不读, 实际不生效)。

门槛从 55% 提到 60% 的过程: 早期实测 ~57%, 卡 60% 会被 `test_retrieval.RecursionError` 等已知问题反复绊倒, 所以先锁 55% 留 buffer。在补齐 217 个单元测试 (`tenant/a2ui_schema/entity_sandbox/experiments/alerting/injection_guard/tracing/decision_log/prompt_registry/customer_memory/gd`) 后, 实测 62%, 门槛正式提到 60%。

这就是**渐进式门槛**的工程意义: 让 CI 立刻 fail 在新写的代码上, 但不要 fail 在**已经知道、还没修**的问题上。

门槛从 60% 提到 80% 的过程: 以 60% 为基线, 分批补齐测试 (每批补完先 `lint + commit + 全量回归` 确认不回归, 再进入下一批):

| 批次          | 补齐内容                                                                                                     | 覆盖率         |
|-------------|----------------------------------------------------------------------------------------------------------|-------------|
| 62%→71.9%   | faq_service/session/retrieval/assist_engine/ingestion/auth_router/bot_agent/embedding/safety 等 127+ 测试   | 1129 passed |
| 71.9%→75.5% | ws_router 消息流/assist notify/analyze/kb 端点                                                                | 1217 passed |
| 75.5%→77%   | bot router 内部 ( <code>_session_worker</code> 队列合并/幂等/快速兜底/死信, <code>_run_agent</code> 转人工桥接)             | 1240 passed |
| 77%→81%     | <code>_wait_for_response</code> /upload_document 全链路/get_session_messages/assist 反馈缓冲与坐席 WS/ws_router 排队 | 1318 passed |
| 81%→84%     | bot_agent 待确认状态机/增量摘要, deps init/close 全家, main lifespan 失败清理                                            | 1371 passed |

最终 `agent/pyproject.toml` 与 `.github/workflows/ci.yml` 的 `fail_under` / `--cov-fail-under` 同步提到 80%。

## 14.7 已知 35 errors 的根因

当前 pytest 全量跑会有 35 个 errors, 这不是 bug, 是 e2e 排除策略的副产品。

根因链条很清楚: `unit-tests job` 在 CI 排除了 5 个 e2e 文件, 但开发者本地不排除, 直接 `pytest` 跑全套。如果本地没 `make up`, `bot_server` / `assist_server` fixture 在 `_check_middleware_ready()` 那一步就 `pytest.skip`, 而 `bot_client` 依赖 `bot_server` — `skip` 链会从 e2e 文件扩散到任何 `import` 它们的 `conftest`。最后呈现给开发者的是一长串 SKIPPED + 中间夹着 ERROR, 看着吓人。

更隐蔽的根因: `test_retrieval.py` 在某些依赖版本下出现 `RecursionError` (PG 驱动 + `asyncio` 兼容问题), 这条链曾拉低覆盖率 1~2 个百分点, 让 `--cov-fail-under` 阈值上限一度被锁死在 57%。

启动超时 (90s 容忍线) 是第三条线 — 容器内 cold start + 模型预热超过 90s 的话, `bot_server` 调 `pytest.fail`, 后续全部 fixture 级联失败。

解决方案不是再放宽门槛, 而是让 **e2e 在 CI 也能跑** — 比如改用 `TestContainers` 化 e2e。

## 14.8 mypy advisory 模式

`unit-tests` job 跑 `mypy lumio/`, 但用的是 **advisory 模式**, 不 fail CI (`.github/workflows/ci.yml:32-46`):

```
- name: Mypy type check (advisory)
 run: |
 set +e
 poetry run mypy lumio/ > /tmp/mypy.log 2>&1
 mypy_exit=$?
 set -e
 cat /tmp/mypy.log | tail -50
 echo ""
 echo "mypy_exit_code=${mypy_exit} (advisory: 不 fail CI)"
 errors=$(grep -c "error:" /tmp/mypy.log || true)
 echo "mypy_error_count=${errors}"
```

为什么用 advisory? 如果配置是 `mypy ... || true`, 会静默吞掉所有错误 — 表面上 CI 绿, 实际上谁也不知道有多少类型债。advisory 模式做了一件关键事: **set +e 让 mypy 跑完, 再统计 error 数打到日志末尾**, 这样开发者一眼能看到当前存量错误 + 新增错误 (新引入的), 但不会被存量阻塞 PR。

配套的软化在 `agent/pyproject.toml:151-153`:

```
[[tool.mypy.overrides]]
module = "tests.*"
disallow_untyped_defs = false
```

测试代码不强求类型标注, 因为测试本身就是被测对象的消费者而非被测者, 写一堆 `Any` 反而降低可读性。

## 14.9 5 个 CI job 全景

`.github/workflows/ci.yml:1-172` 串了 5 个 job, 分两类:

push / PR 触发 (3 个):

- `lint` — `ruff check` + `ruff format check` + `mypy advisory` + `make verify-observability`
- `unit-tests` — service container 启 Redis/PG + `pytest` + 80% 覆盖率门槛
- `mcp-server` — JDK 21 + `mvn verify` (Java 端 `mcp-server` 独立构建)

仅 main 分支触发 (2 个):

- `build` — `docker build -f deploy/Dockerfile`, 依赖 `lint` + `unit-tests` 通过
- `e2e-tests` — `docker compose up demo` + `curl` 烟测, 依赖 `build` 通过

这套 job 拓扑的设计意图: **快反馈在前, 慢验证在后**。开发者 PR 阶段只跑 `lint` + `unit-tests` + `mcp-server`, 5 分钟内拿到结果; 合并到 main 后才走慢路径 `build` + `e2e`, 即便 `e2e` 挂了也不会阻塞 PR 评审。

## 14.10 pre-commit 8 步本地守门

`.pre-commit-config.yaml:1-33` 配置了 8 步 `pre-commit-hooks` + `ruff` + `ruff-format` + `mypy`, 关键片段:

```
- repo: https://github.com/pre-commit/pre-commit-hooks
 rev: v5.0.0
 hooks:
 - id: trailing-whitespace
```



```

- id: end-of-file-fixer
- id: check-yaml
- id: check-json
- id: check-toml
- id: check-added-large-files
 args: ["--maxkb=500"]
- id: check-merge-conflict
- id: detect-private-key

```

8 步全部是**结构性守门**(尾空格 / 文件末尾换行 / YAML 语法 / JSON 语法 / TOML 语法 / 500KB 大文件 / 合并冲突标记 / 私钥泄漏), 不做业务判断。Ruff 后面跟上, 用 `--fix --exit-non-zero-on-fix` 一自动修能修的, 修不了的就 fail。最后 mypy v1.13.0 配合 `[tool.mypy.overrides] module = "tests.*"` 做与 CI 一致的类型检查。

为什么不在 pre-commit 也加 mypy advisory? 答案在 ruff 的 `--exit-non-zero-on-fix`: 任何修改都让提交者看到自己改了什么, 这比 "silent fix" 更安全的契约。mypy 在 pre-commit 阶段保持 strict, 是因为提交前是 "代码进入版本库前的最后一道关", 没必要 advisory。

## 14.11 make verify-observability 一致性测试

unit-tests job 跑 make verify-observability (.github/workflows/ci.yml:48-52):

```

- name: Observability loop check
 working-directory: agent
 # verify-observability 入 CI, 防止 dashboard 名字错回归
 # (lumio_session_transitions_total → session_transitions_total 那种错)
 run: make verify-observability

```

这个目标的存在有其原因: 曾有人把指标名从 `lumio_session_transitions_total` 改成 `session_transitions_total`, 业务代码没动, 但 Grafana dashboard 全挂 (因为 dashboard 用 metric name 选 panel)。make verify-observability 做的事是**静态解析 dashboard JSON + 校验所有引用的 metric 名称都在代码里被定义过** — 这是一种 "配置-代码" 双向一致性测试, 防止 dashboard 改完漏改业务代码, 或反过来。

这条检查进了 CI 的 lint job, 跟 ruff 同级。

### 14.11.1 单元测试的"四层 mock 心法"

覆盖率从 62% 提到 84% 的过程里, 踩出来的 mock 经验可以浓缩成四条规则——每条都对应一次真实的 "测试写对了但测的是假象" 事故:

**规则 1: 函数内 import 绕过模块级 patch。**代码里 `from lumio.services.common.ingestion import ingest_document` 写在函数体内时, patch 目标必须指向**源模块** (`lumio.services.common.ingestion.ingest_document`), patch 调用方模块的引用 (`deps.ingest_document`) 无效——因为函数每次执行都重新从源模块取属性, 调用方模块里根本没有这个名字。判断方法: 看到 `from X import Y` 缩进在函数里, patch 就写 `patch("X.Y")`。

**规则 2: monkeypatch asyncio.sleep / wait\_for 必须先保存原引用。**测试想加速 `await asyncio.sleep(300)` 时会写:

```

async def fast_sleep(seconds):
 await asyncio.sleep(0.001) # ← asyncio.sleep 已被替换, 无限递归!

```

必须先 `_real_sleep = asyncio.sleep`, 再 monkeypatch, 最后 `await _real_sleep(...)`。同理, 测试 `_session_worker` 的空闲退出逻辑 (300s) 时用 `fast_wait_for` 包一层真实 `wait_for` 把超时压到 1ms——直接替换 `wait_for` 会让依赖它的其他协程 (如 `_run_agent` 的全局 deadline) 全部失真。

**规则 3: FastAPI dependency\_overrides 的 async generator 必须覆盖为 async generator。**`get_db_session` 这类依赖是 `async def + yield` 的 generator 形态, override 时必须写 `async def _override(): yield fake`——覆盖成普通 async 函数会在 FastAPI 内部 `async with` 时报 `AttributeError: __aenter__`。这是全项目最隐蔽的 fixture 陷阱, 没有之一。

**规则 4: response\_model 会过滤不匹配的 dict。**端点声明 `response_model=RetrieveResponse` 时, 测试里 fake 检索返回的 dict 只要缺字段/多字段, FastAPI 就会静默过滤成空对象——断言 "结果里有内容" 必然失败, 但报错信息毫无线索。对策: fake 返回值必须先对照模型构造, 而不是 "大概像就行"。

### 14.11.2 补测的节奏:每批都要能独立交付



覆盖率从 62% 到 84% 不是一次性大爆炸, 而是 6 个批次, 每批的节奏相同:

```
读代码找缺失块 → 写测试 (单个文件, 秒级迭代) → 该文件全绿
→ ruff lint + format → commit → 全量回归 (确认无相互破坏) → 下一批
```

**为什么每批必须 commit?** 因为覆盖率测试有一个"幽灵回归"问题: 新测试可能在别的测试之后改变了共享状态 (比如 patch 未还原、模块级 dict 被污染), 让本批全绿但全量跑挂——上一批的 commit 点就是二分定位的锚。实际踩过的案例: 某个测试用 `del module._run_assist_engine` "还原"注入, 结果把模块真实函数删掉了, 后续所有 `analyze` 测试 `NameError` ——如果当时没 commit, 定位这个回归要多花一轮全量跑 (4 分钟)。

**每批的选题顺序**也值得说: 先补"调用密集的薄文件" (bot/router.py 的端点), 再补"分支多的厚文件" (bot\_agent.py 的状态机), 最后补"基建层" (deps.py / main.py 的 init/close)。前两类对覆盖率的边际收益大, 后一类虽然行数多但每条都是"初始化成功/失败"的对称断言, 写起来快、跑起来稳, 适合收尾。

### 14.11.3 契约测试:把"映射"钉死比覆盖更重要

覆盖率数字是手段, 契约稳定才是目的。Lumio 里有一类测试专门负责"钉死映射", 它们不关心业务正确性, 只关心"改了一个地方, 另一个地方必须跟着改":

- `test_middleware.py` 钉死错误码 → HTTP 状态映射 (400/422/502/500 + 404/409/503 覆盖表);
- `test_ws_router.py` 钉死 WS 协议事件序列 (thinking → delta → done / cancel → cancelled);
- `test_bot_router_core.py` 的 `_session_worker` 用例钉死队列合并上限 (≤5 条 / ≤4000 字符) 与幂等键行为;
- `test_auth.py` 钉死 JWT 声明 (iss/aud) 与角色矩阵。

这类测试的共同特征是**断言的是常量而不是行为**: `assert payload["is_transfer"] is True`、`assert update_msg.await_args.kwargs["source"] == "merged"`。它们的存在让"顺手改个常量"变成"必须过全量回归"——这正是第 8 章"错误码是契约"的测试侧落点。

## 14.12 已知 trade-off

这套测试策略不完美, 有三个明确的取舍:

**E2E 只在 main 分支跑。** PR 阶段没有端到端保护, 意味着有问题的合并可能在 `build + e2e-tests` 之前就已经合进 main, 触发回滚成本。**应对:** 增量加强 e2e 反馈时间, 把 e2e 拆成"PR 子集" + "main 全量"两档。

**mypy advisory 模式下, 类型错误可能累积。** 存量错误 + 每天新增, 没有 fail CI 卡住, 全靠"开发者自觉看末尾 echo"。**应对:** 按模块拆分, 指定 owner 逐步收敛。

**E2E 在 CI 排除导致覆盖率上限 57%。** `test_bot_api.py` 等 5 个文件一旦在 CI 跑, 覆盖率立刻上 60+, 但 90s 启动 + 中间件依赖让 unit-tests job 时间翻倍。**应对:** 改用 TestContainers, 把 e2e 完全容器化后塞回 unit-tests, 但工作量较大, 留待后续迭代。

## 14.13 小结

Lumio 的测试策略是把"CI 时间"当稀缺资源分配: **快反馈** (lint + unit + 80% 覆盖) 给 PR, **慢验证** (build + e2e + 烟测) 给 main。真实中间件 + 真实子进程的哲学让测试信号更有价值; 补足单元测试后覆盖率实测 84%, E2E 在 main 分支全链路通过 (迁移→seed→启动→登录→对话)。Mypy advisory + 渐进式覆盖率门槛 + verify-observability 一致性测试, 三者一起把"债务可见但暂不阻塞"这种工程取舍做成了可执行规范。

延伸阅读: - [第 8 章 错误处理](#) — `test_middleware.py` 覆盖 - [第 10 章 可观测性](#) — `test_observability.py` 覆盖

# 15 上下文工程

## 第 15 章: 上下文工程 — 3 层上下文 + token 预算 + 增量摘要

本章深入 Lumio Bot 区别于一般 LLM 应用的最关键设计 — 上下文工程。银行客服场景下, 客户对话常跨数十轮, 含敏感信息 (卡号/金额/挂失) 与关键承诺 (投诉/转人工), 简单地把全部历史塞进 LLM 会撞 token 上限且丢失关键实体。Lumio 设计了 **3 层上下文架构 + token 预算 LIFO 累加算法 + 19 关键词豁免 + 增量摘要 fire-and-forget**, 既保证 LLM 总在预算内, 又保证关键信息永不丢失。看完本章你会理解: 为何 Bot 能“记住”30 轮前客户说“我是白金卡”但又不会因为历史太长爆 token, 为何对话中途能“接着说”而不需要客户重复, 为何摘要生成失败也不会让对话中断。

### 15.1 3 层上下文模型

Lumio Bot 每次 LLM 调用前, 在 `bot_agent._handle_*` (`bot_agent.py:201-339`) 中拼装 3 层上下文, 按作用范围 + 重要度分层注入:

| 层                  | 注入位置                                         | 内容                          | 裁剪策略             | 降级行为                         |
|--------------------|----------------------------------------------|-----------------------------|------------------|------------------------------|
| Layer 1: 结构化会话记忆   | <code>system_prompt</code> 头部                | 对话摘要/客户画像/已知实体/意图历史/当前意图    | 永不裁剪             | 异常 → 返回空字符串                  |
| Layer 2: 近期对话历史    | <code>messages</code> 数组 (user/assistant 角色) | 近 20 轮 turn                 | token 预算 LIFO 累加 | 异常 → 返回空列表                   |
| Layer 3: RAG 检索上下文 | <code>messages</code> 数组 (system 角色)         | BM25+向量 RRF 融合的文档块 + 知识图谱增强 | N/A (RAG 自带截断)   | 失败 → <code>context=""</code> |

**核心约束:** – Layer 1 永远在 `system_prompt` 头部注入, 永不进入 token 预算计算。哪怕历史全被裁掉, 客户画像 + 实体池 + 意图栈仍可见。 – Layer 2 token 预算 = `max(MaxContext - Reserved, 1024)`, 默认 `8192 - 1024 = 7168` 可用, 历史层预算 `budget_history=1500`, 但最低 1024 (避免极小模型失效)。 – Layer 3 由调用方传入, RAG 检索本身已截断到 `top_k` (默认 5-8 块), 不参与裁剪。

**为什么不只让 LLM “全部读”:** 银行客户对话常 30+ 轮, 一轮 50-100 字, 累加可达 3000+ 字  $\approx$  1500+ tokens。而 LLM 上下文 (8192 tokens) 还要留给回答 (1024) + `system_prompt` (500) + RAG 上下文 (1000)。实际留给历史的预算只有  $\sim$ 500 tokens  $\approx$  2-3 轮 — **显然不够**。必须做主动上下文管理。

### 15.2 Layer 1 — `_build_session_memory` 5 段拼接

`bot_agent.py:725-779` 在每次 `_handle_knowledge/biz/fallback` 开头调用, 永不抛出:

```
bot_agent.py:725
async def _build_session_memory(self, session_id: str) -> str:
 try:
 state = await self._session_manager.get_session(session_id)
 if state is None:
 return ""

 parts: list[str] = []
 # 1. 对话摘要 (被裁剪轮次的脉络, 最高优先级)
 if state.conversation_summary:
 parts.append(f"[对话摘要]\n{state.conversation_summary}")
 # 2. 客户画像
 profile_parts: list[str] = []
 if state.vip_level and state.vip_level != "普通":
 profile_parts.append(f"VIP等级={state.vip_level}")
 if state.card_types:
 profile_parts.append(f"卡种={' '.join(state.card_types)}")
 if state.risk_tolerance and state.risk_tolerance != "R2":
 profile_parts.append(f"风险偏好={state.risk_tolerance}")
 if profile_parts:
 parts.append(f"[客户画像] {' '.join(profile_parts)}")
```

```

3. 实体池
if state.last_entities:
 entity_strs = [f"{e.entity_type}={e.value}" for e in state.last_entities if e.entity_type and e.value]
 if entity_strs:
 parts.append(f"[已知实体] {' , '.join(entity_strs)}")
4. 意图历史
if state.intent_stack:
 parts.append(f"[意图历史] {' → '.join(i.value for i in state.intent_stack)}")
5. 当前意图
if state.last_intent:
 parts.append(f"[当前意图] {state.last_intent.value}")

return "\n".join(parts)
except Exception:
 return "" # bot_agent.py:779, 任何异常都静默降级为空字符串

```

**5 段拼接顺序的 why:** 按对 LLM 决策的重要度排序, 对话摘要 > 客户画像 > 实体 > 意图历史 > 当前意图. LLM 即使读到一半 token 截断 (极少见, 512 token 限额时), 也不会丢失关键信息.

**默认值不写入的 why:** - `vip_level != "普通"`: 默认值是中文, 写入 prompt 反而污染 (LLM 困惑于"VIP等级=普通"的语义) - `risk_tolerance != "R2"`: R2 是中性默认, 写出来等于无信息 - **实体池全量写入:** 因为实体一旦抽取, 都是真值, 不过滤

**互补设计:** Layer 1 注入的是**结构化记忆** (JSON 风格), 不含自然语言冗余, 500 tokens 内可表达 100+ 实体. 与 Layer 2 (自然语言历史) 形成 "结构化 + 非结构化" 互补.

## 15.3 Layer 2 — \_load\_history token 预算算法

bot\_agent.py:687-770 实现银行客服核心的 token 预算裁剪. 这是 Lumio 的"上下文管理心脏":

```

bot_agent.py (第五轮修复后)
async def _load_history(self, session_id: str) -> list[dict[str, str]]:
 """三层上下文策略 (银行客服最佳实践):
 - Layer 1: 结构化会话记忆 + 对话摘要 (注入 system prompt, 永不裁剪)
 - Layer 2: 近期对话历史 (token 预算裁剪, 被裁剪部分生成摘要/压缩)
 - Layer 3: 检索知识 (RAG context, 由调用方传入)
 """
 try:
 turns = await self._session_manager.get_history(session_id, limit=20)
 if not turns:
 return []

 settings = get_settings()
 # 分层预算: 历史层固定 1500 (max(budget_history, 1024)),
 # 而非 max_context - reserved (旧实现 7168 = 上下文 87%)
 budget = max(settings.llm.budget_history, 1024) # 1500

 # 超预算且轮次足够时, 先尝试选择性压缩 (质量门不达标才走摘要裁剪)
 total_est = sum(_estimate_tokens(t.content) for t in turns)
 if total_est > budget and len(turns) >= settings.compression.min_history_turns:
 compressed = compress_history(dict_turns, max_tokens=budget)
 if any("_compressed" in m for m in compressed):
 turns = [t.model_copy(update={"content": m["content"]}) ...]

 kept_turns: list = []
 used = 0
 split_idx = len(turns)
 # LIFO 累加: 从最近往最旧扫描, 累加 token, 超预算就停
 # 关键轮次 (投诉/承诺/转人工) 永不裁剪
 for i in range(len(turns) - 1, -1, -1):
 t = turns[i]
 est = _estimate_tokens(t.content)
 is_important = _is_important(t.content)
 if used + est > budget and kept_turns and not is_important:
 split_idx = i + 1
 break
 kept_turns.insert(0, t)
 used += est

 # 触发增量摘要 (异步, 持有 task 引用防 GC)
 trimmed_turns = turns[:split_idx]
 if trimmed_turns:
 self._spawn_task(self._ensure_summary(session_id, trimmed_turns))

```

```

 return [{"role": "user" if t.speaker == "customer" else "assistant",
 "content": t.content} for t in kept_turns]
 except Exception:
 return []

```

**预算演进 (设计决策):** 历史层从"上下文剩余全给 history"(8192–1024=7168, 占 87%) 收敛为**分层预算强制分配** — history 1500 / RAG 1200 / 客户画像 400 / few-shot 半稳态层, 与主流上下文管理框架 (system 10–15% / history 25–30% / retrieval 25–30% / output 20–25%) 对齐. 被裁剪轮次的语义由**增量摘要** + **选择性压缩**双通道兜底, 关键信息不丢.

### 15.3.1 token 估算函数 \_estimate\_tokens

bot\_agent.py:61–74 用字符类加权估算 token 数, 替代 tiktoken 等重型库:

```

bot_agent.py:61
def _estimate_tokens(text: str) -> int:
 """基于字符类的 token 数估算
 CJK 字符: ~2 chars/token -> 系数 0.55
 拉丁字母: ~4 chars/token -> 系数 0.3
 其他(数字/标点/空格): ~1.2 chars/token -> 系数 0.8
 +4 为消息格式开销 (role/content 包装)
 """
 import re
 cjk = len(re.findall(r"[\u4e00-\u9fff\u3000-\u303f\u4ff0-\uffef]", text))
 latin = len(re.findall(r"[a-zA-Z]", text))
 other = len(text) - cjk - latin
 return int(cjk * 0.55 + latin * 0.3 + other * 0.8) + 4

```

**为何不用 tiktoken:** – tiktoken 启动需加载 5–10MB BPE 表, 100ms+ 启动延迟 – LLM 选型可能切换 (Ollama/OpenAI/Claude), 每个模型的 tokenizer 都不同 – 银行客服 99% 是中文, CJK 系数 0.55 准确度 95%+

**为何 +4:** OpenAI ChatCompletion 每条 message 都要 {"role": "...", "content": "..."} JSON 包装, 实测约 4 tokens 开销.

### 15.3.2 关键轮次豁免 — 19 个 \_IMPORTANT\_KEYWORDS

bot\_agent.py:79–89 定义了**永远不会被裁剪**的关键词, 这些轮次即便超出 token 预算也强制保留:

```

bot_agent.py:79
_IMPORTANT_KEYWORDS = [
 "投诉", "举报", "银保监", "银监会", "人行", "央行", # 监管/投诉
 "律师函", "法务", "法院", "起诉", # 法律
 "盗刷", "挂失", "冻结", "风险", # 风险
 "承诺", "保证", "一定解决", # Bot 承诺
 "转人工", "人工客服", # 升级
]

def _is_important(content: str) -> bool:
 return any(kw in content for kw in _IMPORTANT_KEYWORDS)

```

**为何必须豁免:** 这 17 关键词对应的场景 (投诉/盗刷/转人工等) 是**合规高敏**的, 一旦裁剪, LLM 后续回答可能: – 不知道客户已投诉, 重复追问问题详情 → 激怒客户 – 不知道客户要转人工, 继续推 Bot 处理 → 延误处理 – 忘记之前的"承诺" → 客户投诉 Bot 推诿

**豁免的代价:** 极端情况下 (一段 10K tokens 全部含 "投诉"), 预算可能爆. 但这是**有意的设计取舍** — 合规 > 性能.

### 15.3.3 LIFO 累加 vs 固定窗口

Lumio 选 LIFO (Last–In–First–Out) 而非固定 N 轮窗口:

| 方案              | 优点                            | 缺点             | Lumio 决策 |
|-----------------|-------------------------------|----------------|----------|
| 固定 N=10 轮       | 实现简单                          | 长轮次被裁, 短轮次浪费预算 | ✗        |
| 时间窗口 (最近 5 分钟)  | 时序公平                          | 30 轮短对话全部丢弃    | ✗        |
| LIFO + token 预算 | 按内容长度动态调整, 长内容少保留几轮, 短内容多保留几轮 | 实现复杂           | ✓        |

示例: 客户发了 30 条短问句 (各 10 字  $\approx$  5 tokens) + 5 条长解释 (各 200 字  $\approx$  100 tokens), 预算 200 tokens: – LIFO 累加: 保留 20 条短问句 (100 tokens) + 1 条长解释 (100 tokens) = 21 轮 – 固定 10 轮: 仅保留 10 轮短问句 (50 tokens), 浪费 150 tokens 预算

## 15.4 增量对话摘要 `_ensure_summary`

`bot_agent.py:631-723` 解决"LIFO 裁掉的旧轮次不能直接丢"的问题 — 用 LLM 生成摘要写回 `SessionState`. 关键设计是增量:

```
bot_agent.py:631
async def _ensure_summary(self, session_id: str, trimmed_turns: list) -> None:
 """用 last_summarized_turn_id 精确追踪已摘要位置, 避免 LTRIM 导致计数失准.
 增量策略:
 - 在 trimmed_turns 中查找 last_summarized_turn_id 的位置
 - 如果找到: 对该位置之后的轮次生成增量摘要
 - 如果未找到 (LTRIM 删除了已摘要的轮次): 对所有 trimmed_turns 重新生成摘要
 - LLM 不可用时跳过 (降级为无摘要, 结构化记忆仍保证关键实体不丢)
 """
 try:
 state = await self._session_manager.get_session(session_id)
 if state is None:
 return
 last_summarized_id = state.last_summarized_turn_id
 last_turn = trimmed_turns[-1]
 if last_turn.turn_id == last_summarized_id:
 return # 已被摘要过, 跳过

 # 找增量起点
 split_idx = 0
 if last_summarized_id:
 for i, t in enumerate(trimmed_turns):
 if t.turn_id == last_summarized_id:
 split_idx = i + 1
 break
 # 未找到 (LTRIM 删了) -> split_idx 保持 0 -> 重新摘要全部
 new_turns = trimmed_turns[split_idx:]
 if not new_turns:
 return

 # 构造 prompt: 已有摘要 + 新增对话
 conversation = "\n".join(
 f"[{{ 'customer': '客户', 'agent': '坐席', 'bot': '机器人' }.get(t.speaker, t.speaker)}] {t.content}"
 for t in new_turns
)
 existing_summary = state.conversation_summary if split_idx > 0 else ""
 summary_prompt = _SUMMARIZE_SYSTEM_PROMPT # prompts.py:104
 user_content = (
 f"已有摘要:\n{existing_summary}\n\n新增对话:\n{conversation}"
 if existing_summary else f"对话记录:\n{conversation}"
)

 # 调 LLM 摘要, 3s 超时
 llm_client = self._degradation_mgr._llm
 if llm_client is None:
 return # LLM 不可用 -> 静默跳过
 try:
 new_summary = await llm_client.chat(
 messages=[
 {"role": "system", "content": summary_prompt},
 {"role": "user", "content": user_content},
],
 timeout=3.0,
)
 except Exception:
 return # bot_agent.py:696, LLM 失败也静默跳过
 ...
```

为何用 `last_summarized_turn_id` 而非计数: Redis Stream 的 `LTRIM` 会把最旧的 turn 物理删除, 如果摘要用 "已摘要 N 轮" 计数, `LTRIM` 后 N 会偏移, 导致重复摘要或漏摘要. 用 `turn_id` (UUID v7, 单调递增) 精确追踪, `LTRIM` 不影响.

**3s 超时的 why:** 摘要不是用户可见请求, 慢一点无所谓, 但不能慢到拖垮 LLM 服务的整体并发. 3s 是 LLM 99 分位响应时间, 超时直接放弃 (下轮再试).

失败兜底 (bot\_agent.py:696, 715-723): - LLM 调用失败 → logger.debug(...) 静默 - 摘要内容空 → 静默 - CAS patch 失败 → logger.warning(...) 可见, 但不抛

为什么不阻塞主请求: 摘要锦上添花, 不是必需. 结构化记忆 (Layer 1) 已经有对话摘要 + 客户画像 + 实体池, 摘要缺失只会让 LLM "不知道旧轮次的细节", 但不会让对话中断.

并发串行化 (per-session 锁): 多轮快速对话时每轮裁剪都会 spawn 一个摘要任务, 并发读写同一 last\_summarized\_turn\_id → CAS 重试后到者失败, 摘要滞后. \_ensure\_summary 外层包 per-session asyncio.Lock ( self.\_summary\_locks[session\_id] ), 同会话摘要任务串行执行:

```
bot_agent.py (简化)
lock = self._summary_locks.setdefault(session_id, asyncio.Lock())
async with lock:
 await self._summary_locked(session_id, trimmed_turns) # 实际摘要逻辑
```

锁字典随会话自然增长 (上限 = 活跃会话数), 无需清理.

15.5 3 层降级矩阵

3 层上下文各层都有独立的失败路径, 不互阻塞:

| 失败                               | 触发            | 降级行为                                                     | 用户感知                         |
|----------------------------------|---------------|----------------------------------------------------------|------------------------------|
| Layer 1 _build_session_memory 异常 | DB 慢/Redis 抖动 | except: return "" (bot_agent.py:779)                     | LLM 不知道历史客户画像, 但对话照常         |
| Layer 2 _load_history 异常         | Redis 拉取失败    | except: return [] (bot_agent.py:628)                     | LLM 不知道最近说了啥, 但会基于本轮 input 答 |
| Layer 3 _retrieve 失败             | ES/Milvus 不可用 | DegradationLevel.FALLBACK → return "" (bot_agent.py:539) | LLM 走纯生成模式, 知识问答降级为通用回答      |
| 摘要生成失败                           | LLM 不可用/超时    | 静默跳过, 下轮再试                                               | 旧轮次无摘要, 但结构化记忆仍有             |
| get_history 返回空                  | 新会话/无历史       | return [] (bot_agent.py:597)                             | LLM 仅看本轮 input               |

降级哲学: "宁降级, 不可用". 银行客服不能因为上下文组件挂了就 500, 必须继续服务. 最坏情况是 LLM 退化到"看本轮 + 静态 system\_prompt"模式, 但仍能给出基础答复.

示例降级场景: PG 挂了 + Redis 也抖 → 全部 Layer 1/2 拿不到数据 → LLM 收到 system\_prompt="你是银行信用卡智能客服" + user\_input=本轮问题 + context="" → LLM 用预训练知识回答, 答错了用户也能接受 (因为 Bot 系统也明确告知 "知识库可能不全").

15.6 上下文工程 3 层流程图

```
%% 完整源码: docs/book/diagrams/context-engineering.mmd
flowchart TB
 Start([用户问 user_input]) --> Build[bot_agent._handle_*
拼装 3 层上下文]

 subgraph L1["Layer 1: 结构化会话记忆"]
 direction TB
 L1a[SessionManager.get_session] --> L1b{state 有效?}
 L1b -- 是 --> L1c[5 段拼接]
 L1c --> L1c1[对话摘要]
 L1c1 --> L1c2[客户画像]
 L1c2 --> L1c3[已知实体]
 L1c3 --> L1c4[意图历史]
 L1c4 --> L1c5[当前意图]
 L1c5 --> L1d[format 系统 prompt 头部]
 L1b -- 否/异常 --> L1Empty[返回 空字符串]
 end

 subgraph L2["Layer 2: 近期对话历史 (token 预算裁剪)"]
 direction TB
 L2a[get_history limit=20] --> L2b{turns 非空?}
 L2b -- 否 --> L2Empty[返回 空列表]
 end
```

```

L2b -- 是 --> L2c[Budget = MaxContext - Reserved]
L2c --> L2d[LIFO 累加, 关键轮次豁免]
L2d --> L2e{超预算?}
L2e -- 否 --> L2Keep[保留 + 累加]
L2e -- 是 + 普通 --> L2Trim[标记 trimmed_turns]
L2e -- 是 + 重要 --> L2Keep
L2Keep --> L2f[转换 role 字段]
L2Trim --> L2g{trimmed 非空?}
L2g -- 是 --> L2h[asyncio.create_task
增量摘要]
L2g -- 否 --> L2Skip[跳过]
end

subgraph L3["Layer 3: RAG 检索 + 知识图谱"]
 direction TB
 L3a[_retrieve 4 路径降级] --> L3b{context 非空?}
 L3b -- 否 --> L3Empty[FALLBACK → 跳过]
 L3b -- 是 --> L3c[enrich_retrieval_context]
 L3c --> L3d{命中 5 实体?}
 L3d -- 是 --> L3e[追加 ## 知识图谱补充信息]
 L3d -- 否 --> L3NoKG[不追加]
end

Build --> L1
Build --> L2
Build --> L3

L1d --> Sum[生成最终 messages 列表]
L1Empty --> Sum
L2f --> Sum
L2Skip --> Sum
L2Empty --> Sum
L3e --> Sum
L3NoKG --> Sum
L3Empty --> Sum

Sum --> LLM[送入 LLM]
LLM --> Resp([Bot 答复])

style L1Empty fill:#ffe1e1
style L2Empty fill:#ffe1e1
style L3Empty fill:#ffe1e1
style L2Skip fill:#ffe1e1
style L3NoKG fill:#fff4e1
style L1d fill:#d4f4dd
style L2Keep fill:#d4f4dd
style L3e fill:#d4f4dd
style LLM fill:#cce5ff

```

怎么读这张图 — "每次回答前, 系统在拼什么": 客户每说一句话, 系统要回答前先**组装三样东西** — ① Layer 1 结构化记忆 (摘要/画像/实体, 永在); ② Layer 2 近期历史 (超预算就裁最旧的, 但投诉/承诺类轮次豁免); ③ Layer 3 检索知识 (没搜到就跳过, 搜到了再附加知识图谱补关系). 三样都齐了才送进 LLM. **红色分支 (L1Empty/L2Empty/L3Empty) 是降级路径** — 任何一层失败, 系统照常回答, 只是少一份信息, 对话绝不中断. 这就是"能记住 30 轮前的事, 但永远不会爆 token"的秘密.

## 15.7 设计取舍深度分析

### 15.7.1 为何用字符类 token 估算而非 tiktoken

| 维度       | 字符类估算 (当前) | tiktoken                       |
|----------|------------|--------------------------------|
| 启动开销     | 0 (纯 re)   | 5–10MB 模型加载, 100ms+            |
| 准确度 (中文) | 95%+       | 99%+                           |
| 多 LLM 兼容 | 自动适配       | 需选模型 (cl100k_base / p50k_base) |
| 维护成本     | 0          | BPE 表需随模型更新                    |
| 极端长文本误差  | ±5%        | ±1%                            |

**5% 误差值得吗:** 值得. 银行客服平均 30 字/轮, 5% 误差 = 1.5 字, 在 200 tokens 预算下相当于多裁或少裁 0.3 轮, 完全可接受.

15.7.2 为何 17 关键词而非白名单/正则

grep "\_IMPORTANT\_KEYWORDS" bot\_agent.py 看到的是个简单 any(kw in content for kw in \_LIST) . 这故意设计成关键词字面子串匹配, 而非白名单 (按业务事件类型) 或正则 (按模式):

- 关键词字面: 0 维护成本, 业务方加新词只需加一行
- 白名单 (事件类型): 需要 LLM 先分类事件, 又多一次 LLM 调用, 慢且脆
- 正则 (如 r"投诉[银保监人行]") : 过度抽象, 银行新业务 (如"投诉消保") 容易漏

实际运营: 业务方根据客户投诉/法务反馈, 1 分钟加一个关键词即可, 无需发版.

15.7.3 为何 limit=20 而非更多/更少

- 20 轮的 why: 客服平均对话 10–15 轮, 20 轮覆盖 95% 场景, 极少有 30+ 轮 (除非 Bot 一直解决不了, 这种情况早该转人工)
- 20 轮的 token 上限: 假设每轮 50 字, 20 轮 = 1000 字 ≈ 550 tokens, 加上 Layer 1 + RAG + 回答, 总 1500–2000 tokens, 在 4K 模型内
- 更少 (如 10 轮): 客户"我前面说过了" 抱怨增多
- 更多 (如 50 轮): Redis 拉取 + token 估算耗时, 反而拖慢响应

15.7.4 为何 fire-and-forget 摘要

```
bot_agent.py:623
asyncio.create_task(self._ensure_summary(session_id, trimmed_turns))
```

为何不 await: 主请求 P99 延迟 1.5s 是硬指标 (SLO), 摘要 LLM 调用 500–1500ms, 如果 await 摘要, 主请求要 2–3s 才返回, 客户感受明显卡顿.

为何不怕丢摘要: \_ensure\_summary 内部幂等 — 下次轮次如果仍需摘要, last\_summarized\_turn\_id 追踪保证只对新增轮次摘要, 旧摘要不丢.

故障场景: 主进程崩溃 → asyncio.create\_task 的 task 一起死, 摘要未生成 → 下次启动新会话, last\_summarized\_turn\_id 还是旧值, 增量摘要继续. 不影响正确性, 只影响"摘要可能漏一段".

15.7.5 为何不在 \_handle\_knowledge 也注入 Layer 1 之外

```
bot_agent.py:223–225 :

system_prompt = KNOWLEDGE_SYSTEM_PROMPT # 静态 system_prompt (3 段: 角色/记忆使用规范/回复规范)
system_prompt = f"{KNOWLEDGE_SYSTEM_PROMPT}\n\n## 会话记忆\n{session_memory}" # Layer 1 注入
system_prompt = f"{system_prompt}\n\n{slot_prompt}" # 槽位 (第 3 章 §3.6 详谈)
```

注意 Layer 1 注入而非 Layer 2/3: Layer 2 (历史) 和 Layer 3 (RAG) 走 messages= 形参, 由 build\_layered\_messages 组装 (第五轮修复后). 这种设计: – system\_prompt 干净: 不会被历史污染, 每次回答 system 部分稳定 → LLM 行为更可预测 – 分层消息直传 LLM: \_handle\_knowledge 构建的 L1 静态锚点 / L2 半稳态画像 / L3 动态记忆 / RAG user-role 物理隔离结构, 通过 generate\_with\_fallback(messages=...) 原样送达 LLM, 不再压平拼接 — 前缀缓存锚点 (cache\_control: ephemeral) 真实生效 – RAG 物理隔离: 检索内容以 user message 的 <retrieved\_context> 包裹注入 (而非第 2 条 system), 防注入 + 不破坏缓存前缀

15.8 上下游集成点

上下文工程的 3 层不是孤岛, 与其他模块深度集成:

| 上游/下游                             | 集成方式                                    | 行号                          |
|-----------------------------------|-----------------------------------------|-----------------------------|
| SessionManager (SessionState 持久化) | get_session / patch_state / get_history | session.py:198–236, 618–631 |
| IntentClassifier (意图分类)           | Layer 2 历史喂入分类器, 让分类更准                  | bot_agent.py:154            |
| EntityExtractor (实体抽取)            | last_entities 写到 Layer 1                | bot_agent.py:760–763        |
| 知识图谱增强                            | Layer 3 追加 ## 知识图谱补充信息                  | bot_agent.py:213–216        |
| RAG 检索                            | 4 路径降级 → 写入 Layer 3 (限 1200 token)      | bot_agent.py:610–680        |



| 上游/下游        | 集成方式                                                      | 行号                               |
|--------------|-----------------------------------------------------------|----------------------------------|
| 客户记忆学习 (跨会话) | CAS patch 写入 Layer 1 的客户画像字段                              | bot_agent.py:124-135             |
| 上下文压缩器       | 超预算先压缩 (质量门) 再裁剪                                          | context_compressor.py:265-305    |
| Few-shot 选择  | 按意图注入 L1.5 半稳态层 (top_k=3)                                 | few_shot.py:132 + kv_cache.py:99 |
| 总预算截断 (兜底)   | $\Sigma$ 各层 > context-reserved 时从最旧 user 历史裁剪 (RAG/当前轮保留) | bot_agent.py:365-380             |

**总预算截断逻辑:** 分层预算各自生效后仍可能超 `max_context_tokens - reserved_tokens` (8192-1024). 此时**实际裁剪**而非仅告警: 从最旧的 user 历史消息开始丢弃, RAG 上下文 / 当前轮 / 连续消息标记 ( (用户连续发送 / `<retrieved_context>` / 客户问: ) 保留 — 历史可被摘要覆盖, 检索结果和当前诉求不可丢。

## 15.9 实战案例: 30 轮对话的第 31 轮

假设客户与 Bot 聊了 30 轮, 涉及: - 轮 1-5: 问白金卡权益 - 轮 6: 投诉"上次承诺的 1 万积分没到账" - 轮 7-15: 反复问积分到账 - 轮 16-20: 转人工失败, 回到 Bot - 轮 21-30: 重新问账单分期

第 31 轮客户输入 "我还有多少积分?". 上下文工程处理:

- Layer 1** (bot\_agent.py:725-779): - `conversation_summary`: 之前 30 轮被裁的轮次的浓缩, 大约 "客户投诉积分未到账, 1 万积分, 转人工失败" - 客户画像: `vip_level=platinum` (从历史学习) - 已知实体: `amount=10000` (积分) - 意图历史: `reward_query` → `complaint` → `transfer_agent` → `reward_query` - 当前意图: `reward_query` (本轮)
- Layer 2** (bot\_agent.py:583-629): - `limit=20` 拉到最近 20 轮 (轮 11-30) - 历史层预算 1500, 估算 20 轮  $\approx$  1100 tokens - 全部装下, 不需要裁 - 轮 6 "投诉" + 轮 16 "转人工" 必保留
- Layer 3:** - RAG 检索 "还有多少积分" → 命中《积分查询指引》文档 - 知识图谱: 命中 "积分" 实体相关 (虽然图谱当前没 "积分" 实体, 所以无 KG 增强)

最终 LLM 收到的 messages:

```
system: 你是银行信用卡智能客服... [静态]
system: ## 会话记忆
 [对话摘要] 客户投诉积分未到账...
 [客户画像] VIP等级=platinum
 [已知实体] amount=10000
 [意图历史] reward_query → complaint → transfer_agent → reward_query
 [当前意图] reward_query
user: 我还有多少积分?
assistant: ... (轮 30 的回答)
user: ...
[... 20 轮历史 ...]
system: 参考知识: ... (RAG 文档)
```

LLM 能在不重复 "您是白金卡吗" 的前提下, 直接回答积分余额, 还能识别"投诉"标记不会再次承诺未到账积分 (因为对话摘要已记录此前的承诺)。

## 15.10 监控与可观测性

上下文工程的 3 层都有自己的**隐式指标**, 可通过 Prometheus 拉取:

| 指标                                           | 来源                                          | 含义                                        |
|----------------------------------------------|---------------------------------------------|-------------------------------------------|
| <code>lumio_session_history_turns</code>     | <code>SessionManager.get_history</code> 返回值 | 当前会话历史轮数                                  |
| <code>lumio_session_memory_size_chars</code> | <code>_build_session_memory</code> 返回长度     | Layer 1 注入字符数                             |
| <code>lumio_summary_size_chars</code>        | <code>state.conversation_summary</code> 长度  | 对话摘要长度                                    |
| LLM 请求日志的 <code>prompt_tokens</code>         | LLMClient 响应                                | 实际送入 LLM 的 token 数 (含 system+history+RAG) |
| <code>lumio_session_state_version</code>     | <code>SessionState.version</code>           | CAS 写次数, 间接反映写频                           |

| 指标                         | 来源                                    | 含义                            |
|----------------------------|---------------------------------------|-------------------------------|
| lumio_kv_cache_hit_rate    | KV_CACHE_HIT_RATE (kv_cache.py 估算)    | 估算值, 稳态/半稳态/动态的命中比例           |
| lumio_prefill_tokens_saved | PREFILL_TOKENS_SAVED (kv_cache.py 估算) | 估算值, 本轮比全部重算节省的 prefill token |

典型问题诊断: - 客户说"我前面说过了" 抱怨多 → 检查 `summary_size_chars` 是否过小, 摘要可能截断过早 - LLM 响应慢 (1.5s+) → 检查 `prompt_tokens` 是否超 4K, 触发了"超长上下文" LLM 慢分支 - 客户画像不准 → 检查 `last_summarized_turn_id` 是否正确推进, 增量摘要是否卡住

### 15.11 KV Cache 命中率优化: kv\_cache.py

重要区分: 本节讲的"3 层"是 KV Cache 视角的稳态/半稳态/动态层 (控制推理引擎 prefix cache 命中), 不是 15.1 讲的"3 层上下文" (L1 结构化记忆 / L2 历史 / L3 RAG). 两套分层各自独立, 但最终在 `build_layered_messages` 里合流.

#### 15.11.1 业务背景: 为什么 TTFT 是生死线

核心问题: 银行客服每一轮都要重算整个 system prompt + 历史. 7B 模型 ~25ms/100 token, 一个 2KB system prompt 就要 500ms prefill. 客户连发 20 条消息 = 20 次重算 = 10 秒纯等 prefill — 体感"AI 在发呆".

推理引擎的解药 — prefix cache: vLLM (PagedAttention) / Anthropic (prompt cache) / Ollama (部分支持) 都允许字符串前缀完全一致时直接复用 K/V tensor, 跳过 prefill. 命中时单轮 prefill 从 500ms 降到 ~75ms. 但前提是前缀字符串字面完全一致 — 任何变化 (时间戳/session\_id/拼出来的多行) 都让 cache 失效.

原实现的致命缺陷: bot\_agent 用 f-string 拼 system\_prompt ( f"对话摘要: {summary}\n客户画像: {profile}\n..." ), 每次字符串都不同 → 命中率 0%. 整个推理引擎的 cache 机制被浪费.

kv\_cache.py 的目标: 把 prompt 拆成"稳定 + 半稳定 + 动态"3 块, 让稳态部分永远字符串一致, 推理引擎可 100% 命中.

#### 15.11.2 3 层机制: 稳态 / 半稳态 / 动态

| 层                   | 字符串稳定性                  | KV cache 命中 | 内容                               | 标志串                      |
|---------------------|-------------------------|-------------|----------------------------------|--------------------------|
| L1 静态前缀             | 跨 session 永不变化          | ~100%       | 角色定义 / 合规规则 / 工具描述 / 输出格式        | [STATIC_PREFIX_v1]       |
| L1.5 半稳态 (few-shot) | 同意图内稳定                  | 按意图分组命中     | 3 条参考案例 (按 primary_intent 分桶)    | (无独立标志, 走 cache_control) |
| L2 半稳态 (客户画像)       | 同 customer 跨 session 稳定 | ~60%        | customer_context (卡种/VIP 等级)     | [CUSTOMER_CONTEXT_v1]    |
| L3 动态               | 每轮必变                    | 0%          | 会话记忆 / 槽位 / 对话历史 / RAG 检索 / 用户输入 | (无标志)                    |

两个标志串的工程意义: STATIC\_PREFIX\_MARKER 和 SEMI\_STATIC\_MARKER 是模块顶部常量 ( kv\_cache.py:42/45 ), 一是作为前缀锚点让推理引擎识别"这里开始是 cache 区", 二是作为 metrics 估算的分隔符 ( estimate\_cache\_metrics 扫 marker 决定哪段算哪层).

cache\_control: {"type": "ephemeral"} 字段: Anthropic 风格的 API 标记, vLLM/Anthropic 识别后会缓存此 message 之前所有 prefix 的 K/V; OpenAI 兼容 API 会忽略此字段, 不报错也不生效 (优雅降级). 这是协议层的"软依赖" — 不绑定具体推理引擎.

#### 15.11.3 消息构建器 build\_layered\_messages (kv\_cache.py:99)

bot\_agent.py:347-358 在每次 LLM 调用前调用, 返回 messages 数组 (按顺序):

```
bot_agent.py:347-358
messages = build_layered_messages(
 domain_prompt=KNOWLEDGE_SYSTEM_PROMPT, # L1 锚定
 user_input=user_input, # L3 动态
 customer_context=session_memory, # L2 半稳态
```

```
session_memory="", # 已合并到 customer_context
slot_prompt=slot_prompt or "", # L3 动态
rag_context=context or "", # L3 动态 (放 user role 而非 system)
history=history, # L3 动态
few_shot_examples=few_shot_text, # L1.5 半稳态
)
```

返回结构 ( kv\_cache.py:111-117 ):

| # | role                | 内容                                                         | 缓存层            |
|---|---------------------|------------------------------------------------------------|----------------|
| 1 | system              | [STATIC_PREFIX_v1] + domain_prompt (含 cache_control)       | L1 稳态, 100% 命中 |
| 2 | system              | ## 参考案例 + few_shot (含 cache_control)                       | L1.5 半稳态       |
| 3 | system              | [CUSTOMER_CONTEXT_v1] + customer_context (含 cache_control) | L2 半稳态         |
| 4 | system              | ## 会话记忆 + ## 槽位追踪 动态拼                                      | L3 动态, 0% 命中   |
| 5 | user/assistant (多条) | 对话历史                                                       | L3 动态, 0% 命中   |
| 6 | user                | <retrieved_context>...</retrieved_context> + user_input    | L3 动态, 0% 命中   |

关键设计 — RAG 内容放 user 而非 system role: 第 6 步的 RAG 检索结果用 <retrieved\_context>...</retrieved\_context> 包起来当 user message, 不放进 system prompt. 两个原因: ① 物理隔离防 prompt injection (客户在 user input 里塞恶意指令影响不了 system 层); ② 动态部分集中在尾部, prefix 越靠前越稳定, 推理引擎能更激进地 cache.

15.11.4 性能估算 (按 7B 模型 / 2KB system\_prompt / 20 轮对话)

来源: kv\_cache.py 模块 docstring 顶部声明的预期值, 由 estimate\_cache\_metrics 函数实际计算并上报.

| 指标           | 优化前 (f-string 拼接) | 优化后 (分层)                 | 节省   |
|--------------|-------------------|--------------------------|------|
| 稳态层 prefill  | 20 轮 × 2KB = 40KB | 1 轮 (后续全 cache)          | ~95% |
| 半稳态层 prefill | (原 f-string 一并算了) | 同 customer 跨 session 1 次 | ~60% |
| 动态层 prefill  | 20 轮 × 1KB = 20KB | 20 轮 × 1KB = 20KB (必算)   | 0%   |
| 单轮 TTFT      | ~500ms            | ~75ms (稳态全命中时)           | ~85% |

为什么是估算不是实测: 真实 KV cache 命中率由推理引擎 (vLLM prompt\_cache\_stats / Ollama 内部计数器) 上报, 应用层拿不到精确值. estimate\_cache\_metrics (P1-4 修复后) 只用于 metrics 上报, 明确标 estimated, 旧实现把假设值当真实值上报, 误导监控 — 现已修正 ( kv\_cache.py:208-213 注释 + P1-4 上下文工程修复).

15.11.5 启用/降级开关 (config.py:117-119)

```
LLMSettings (config.py:115-121)
kv_cache_enabled: bool = True # 总开关, 关闭时退化到 _legacy_build_messages
static_prefix_anchor: bool = True # L1 静态前缀锚定
layered_injection: bool = True # L2 半稳态层注入
inference_engine: str = "ollama" # ollama (0% prefix cache) / vllm (PagedAttention) / tgi
```

降级路径 ( kv\_cache.py:131-133 ):

```
if not kv.kv_cache_enabled:
 # 未启用 KV cache 优化, 退化到原行为
 return _legacy_build_messages(domain_prompt, user_input, customer_context + session_memory, history)
```

\_legacy\_build\_messages ( kv\_cache.py:180-194 ) 是 f-string 拼 system prompt 的原版, 保留向后兼容. 关掉 KV cache 不影响功能, 只是 TTFT 回到 500ms — 适合推理引擎未知/不支持的灰度环境.

推理引擎差异:

| 引擎                       | prefix cache 支持                  | 备注                                    |
|--------------------------|----------------------------------|---------------------------------------|
| vLLM (PagedAttention)    | ✅ 完整支持                           | 通过字符串前缀 hash 自动复用 K/V block           |
| Anthropic (prompt cache) | ✅ 支持, 需 cache_control: ephemeral | 5min TTL, 命中后单轮 prefill 接近 0          |
| Ollama                   | ⚠️ 部分支持, 命中率不稳定                  | 0% prefix cache 时 (旧版本) 完全无效, 启用后才有收益 |
| TGI                      | ✅ 支持 (新版本)                       | 与 vLLM 类似                             |

15.11.6 与本章其他节的协作

| 相关机制               | 协作方式                                                                                                                                      |
|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 15.1-15.4 (3 层上下文) | _build_session_memory / _load_history / _ensure_summary 算出的结果, 作为 customer_context / session_memory / history 参数传入 build_layered_messages |
| 第 5 章 RAG 检索       | RAG 检索结果作为 rag_context 参数, 走 user role 物理隔离 (防注入)                                                                                         |
| 15.10 监控           | KV_CACHE_HIT_RATE / PREFILL_TOKENS_SAVED 指标由 estimate_cache_metrics 上报, 但需明确是估算值                                                          |

为什么不能完全替代 15.1 的 3 层: KV cache 优化解决"如何让 prompt 字符串字面稳定", 15.1 的 3 层上下文解决"什么内容值得送给 LLM". 两者正交 — KV cache 命中再高, 内容选错也是白搭; 内容选对但字符串总变, TTFT 一样 500ms.

15.12 端到端示例: 一次实际 LLM 调用的 messages 数组

本节用一个真实对话走完一遍 build\_layered\_messages 实际生成的 messages 数组, 逐条对比"客户说第 1 句时"和"客户说第 5 句时"的差异, 看清哪些 message 是字面稳定的、哪些是字面必变的、推理引擎到底能 cache 多少.

15.12.1 场景设定

- 客户: VIP 白金卡客户 customer\_id=cust-9527, 当前 session sess-A, 历史 4 轮 (已问过"年费多少/怎么免年费/白金卡权益/机场贵宾厅")
- 第 5 轮客户说: "上次刷了 5800 块还没出账单, 这笔什么时候记账?"
- 意图: primary\_intent = bill\_query (账单类, 命中 few-shot 的 bill\_query 分组)
- 检索: 命中知识库 3 条关于"账单日/记账时点"的条文
- 槽位: 已抽取 {amount=5800, transaction\_time=未提供}

15.12.2 build\_layered\_messages 实际生成的 messages 数组

下面展示的就是第 5 轮请求实际送给 LLM 的完整 messages 数组 (6 条). 每条都标注了它在哪个层、字面是否稳定.

```
第 5 轮 LLM 请求的 messages 数组 (实际结构)
messages = [
 # — [1] L1 静态前缀 (稳态, 跨 session 100% 命中) —————
 {
 "role": "system",
 "content": "[STATIC_PREFIX_v1]\n你是一名专业的银行信用卡客服, 负责为客户解答信用卡相关问题.\n回答原则:\n1. 基于知识库检索结果回答, 不",
 "cache_control": {"type": "ephemeral"}, # 推理引擎的 cache 锚点
 },

 # — [2] L1.5 few-shot 半稳态 (按 primary_intent 分组命中) ———
 {
 "role": "system",
 "content": "## 参考案例\n[例 1]\n客户: 我这个月账单日是哪天?\n客服: 您的账单日是每月 5 号, 账单出账后 3 天内还款即可...\n[例 2]\n客",
 "cache_control": {"type": "ephemeral"},
 },

 # — [3] L2 半稳态 (同 customer 跨 session 命中) —————
 {
 "role": "system",
 "content": "[CUSTOMER_CONTEXT_v1]\n客户卡种: 白金卡\nVIP 等级: VIP5\n风险偏好: 保守型\n信用额度: 80,000 元\n",
```

```
 "cache_control": {"type": "ephemeral"},
 },

— [4] L3 动态 system prompt (每轮必变, 0% 命中) —————
{
 "role": "system",
 "content": "## 会话记忆\n对话摘要: 客户咨询白金卡年费减免政策及机场贵宾厅权益, 现询问特定交易入账时点.\n\n## 槽位追踪\n已知: amount=51
},

— [5] L3 对话历史 (动态, 0% 命中) —————
{"role": "user", "content": "白金卡年费多少?"},
{"role": "assistant", "content": "您的白金卡年费为 980 元/年, 满足以下任一条件可减免: ..."},
{"role": "user", "content": "怎么才能免年费?"},
{"role": "assistant", "content": "您当年消费满 5 万元或分期满 12 期, 可自动减免次年年费."},
{"role": "user", "content": "白金卡有什么权益?"},
{"role": "assistant", "content": "白金卡权益包括: 机场贵宾厅 (全年 6 次)、酒店自助餐买一赠一、..."},
{"role": "user", "content": "机场贵宾厅怎么用?"},
{"role": "assistant", "content": "出示您的白金卡和登机牌, 在合作机场贵宾厅前台登记即可使用."},

— [6] L3 当前轮 (动态, 0% 命中, RAG 放 user role 物理隔离) —
{
 "role": "user",
 "content": "<retrieved_context>\n[1] 账单日: 每月 5 号\n[2] 入账时点: 交易完成后 1-2 个工作日入账, 部分海外交易 3-5 个工作日\n[3
]
}
```

总 token 分布 (按 7B 中文 1.5 token/字):

| 消息            | 字数     | tokens  | 所在层      | 跨 session 命中                |
|---------------|--------|---------|----------|-----------------------------|
| [1] 静态前缀      | ~95 字  | ~140 t  | L1 稳态    | 100% (跨所有 session)          |
| [2] few-shot  | ~200 字 | ~300 t  | L1.5 半稳态 | 100% (同 intent 分组)          |
| [3] 客户画像      | ~50 字  | ~75 t   | L2 半稳态   | 100% (同 customer 跨 session) |
| [4] 动态 system | ~80 字  | ~120 t  | L3 动态    | 0% (每轮变)                    |
| [5] 4 轮历史     | ~250 字 | ~375 t  | L3 动态    | 0% (新轮会变)                   |
| [6] RAG+用户    | ~150 字 | ~225 t  | L3 动态    | 0%                          |
| 合计            | ~825 字 | ~1235 t |          |                             |

15.12.3 第 1 轮 vs 第 5 轮的差异 — 推理引擎到底能 cache 多少

第 1 轮 (客户首次说"白金卡年费多少?"), messages 数组结构相同但内容不同:

```
第 1 轮: 历史为空, RAG 检索结果不同
messages = [
 # [1] 静态前缀 - 字面完全一致
 # [2] few-shot - 字面完全一致 (按 intent 分组, 都是 bill_query)
 # [3] 客户画像 - 字面完全一致
 # [4] 动态 system - 字面不同 (摘要为空, 槽位为空)
 # [5] 历史 - 空数组
 # [6] RAG+用户 - 字面不同 (检索内容不同 + 用户输入不同)
]
```

对比 — 哪几条字面完全一致、哪几条字面必变:

| #   | 消息        | 第 1 轮 vs 第 5 轮                      | 字面一致?  | cache 命中             |
|-----|-----------|-------------------------------------|--------|----------------------|
| [1] | 静态前缀      | [STATIC_PREFIX_v1]\n你是一名专业的银行信用卡... | ✅ 完全一致 | 100% 命中              |
| [2] | few-shot  | 同一 intent 分组, 案例文本完全一致              | ✅ 完全一致 | 100% 命中 (按意图分组)      |
| [3] | 客户画像      | 客户卡种: 白金卡\nVIP 等级: VIP5\n...        | ✅ 完全一致 | 100% 命中 (同 customer) |
| [4] | 动态 system | 第 1 轮: ## 会话记忆\n(空), 第 5 轮: 4 轮摘要   | ❌ 不同   | 0% 命中                |

| #   | 消息     | 第 1 轮 vs 第 5 轮                       | 字面一致? | cache 命中 |
|-----|--------|--------------------------------------|-------|----------|
| [5] | 历史     | 第 1 轮: [], 第 5 轮: 4 条 user+assistant | ✗ 不同  | 0% 命中    |
| [6] | RAG+用户 | 检索内容和用户输入均不同                         | ✗ 不同  | 0% 命中    |

**关键观察:** 消息 [1][2][3] 共 3 条 system message 占了约 **515 tokens** (140+300+75), 在第 5 轮请求时**完全不用重算** — 推理引擎直接复用第 1 轮的 K/V tensor, 仅重算 [4][5][6] 共 3 条 (720 tokens).

#### 15.12.4 这个设计的工程细节 (逐条说明为什么这么写)

##### 消息 [1] 静态前缀 — 标志串前缀锚定

```
kv_cache.py:62
f"{STATIC_PREFIX_MARKER}\n{domain_prompt}"
```

- `STATIC_PREFIX_MARKER = "[STATIC_PREFIX_v1]"` (kv\_cache.py:42) 是个**永不变化**的字符串前缀
- 推理引擎按字符串前缀 hash 复用 K/V, 这个 marker 让 hash 计算更稳
- 同一个 Bot 服务启动后, 所有 session 同一 domain\_prompt 都共享这一段
- `_v1` 后缀是**版本控制位** — 改 domain\_prompt 内容时把 `_v1` 改成 `_v2`, 老 session 的 cache 自动失效 (不会读到旧内容)

##### 消息 [2] few-shot — 按意图分桶, 半稳态

```
bot_agent.py:347-358 传入
few_shot_examples=few_shot_text # 由 few_shot.py 按 primary_intent 选 top_k=3
```

- 同一 primary\_intent 的所有 session 共用同一段 few-shot 案例 (例如所有 `bill_query` 客户共享)
- 跨 intent 时字符串变化 (例如客户从"问年费"切到"办挂失"), cache 失效 — 这是**有意的设计**, 因为 few-shot 本来就是按意图分的
- 测试用例锁定行为契约 (P2 注入修复)

##### 消息 [3] 客户画像 — 半稳态跨 session 共享

```
kv_cache.py:78
f"{SEMI_STATIC_MARKER}\n{customer_context}"
```

- 同一 customer\_id 跨 session 共享 (customer\_memory 学习后更新一次, 后续几十次 session 都用同一段)
- customer\_context 内容变化频次**极低** (几天到几周才变一次), KV cache 长期保持高命中
- `customer_id` 不进字符串, 所以不同 customer 字符串不同 → 推理引擎按字符串前缀天然分桶

##### 消息 [4] 动态 system — 接受每轮必变的事实

```
kv_cache.py:86-96
def _build_dynamic_system_prompt(...):
 parts = []
 if session_memory:
 parts.append(f"## 会话记忆\n{session_memory}")
 if slot_prompt:
 parts.append(slot_prompt)
 return "\n\n".join(parts)
```

- 不试图 cache — session\_memory 是对话摘要, 每轮都会变
- 即使 0% 命中, 80 字 ≈ 120 tokens, 本身就不大, 不算热点
- **设计取舍:** 不为了 100 tokens 的动态层去折腾 cache, 接受每轮重算

##### 消息 [5] 对话历史 — 不进 cache, 但 token 预算算法管

- `_load_history` (15.3) 用 LIFO 累加把历史限制在 `budget_history=1500` tokens 内
- 超出预算时按 LIFO 砍最老的, **最近 N 轮永远在** — 这本身就比"全部历史都送"节省 token
- KV cache 跟 token 预算是两套独立优化: KV cache 让"送出去的部分"少算, token 预算让"送出去的总量"更少

##### 消息 [6] RAG + 用户输入 — user role 物理隔离的两层用意

```
kv_cache.py:170-173
f"<retrieved_context>\n{rag_context}\n</retrieved_context>\n\n{user_input}"
```

- **物理隔离防注入**: RAG 检索内容如果含客户塞的恶意指令 (例如检索块里有"忽略之前所有指令"), 因为 RAG 在 user message 里, 跟 system prompt 物理隔离, LLM 会把它当"用户输入"处理而不是"系统指令"
- **prefix cache 友好**: 动态部分集中在尾部 (消息 [6]), 前面的 [1][2][3] 越靠前越稳定, 推理引擎能更激进地 cache prefix

### 15.12.5 一图看懂: 实际 cache 命中的边界

```
flowchart LR
 A["第 1 轮请求"] -->|"prefix=' [STATIC_PREFIX_v1]\n你是一名...\n'"| K["(K/V Cache
推理引擎内部)"]
 A -->|"user: 白金卡年费?"| K

 B["第 5 轮请求"] -->|"prefix=' [STATIC_PREFIX_v1]\n你是一名...\n'"| K
 K -->|.复用.-> B1["✔ 静态前缀 K/V
不重算"]
 K -->|.复用.-> B2["✔ few-shot K/V
不重算"]
 K -->|.复用.-> B3["✔ 客户画像 K/V
不重算"]
 B -->|"user: 5800 块..."| B4["✗ RAG+用户 K/V
每轮重算"]
 B -->|"history: 4 轮"| B5["✗ 历史 K/V
每轮重算"]
```

可观察的运维指标 (在生产环境通过 Prometheus 看):

```
估算的稳态层命中比例 (来自 KV_CACHE_HIT_RATE{ cache_layer="static_prefix" })
理论应该稳定在 1.0 (只要 domain_prompt 没改)

估算的 prefill token 节省
rate(lumio_prefill_tokens_saved_total[5m])

实际 TTFT (time to first token) - 端到端真实数据
应该从优化前的 ~500ms 降到优化后的 ~75-150ms
histogram_quantile(0.5, rate(lumio_llm_ttft_seconds_bucket[5m]))
```

怎么判断 KV cache 优化有没有生效: - 看到 static\_prefix 命中比例稳定在 1.0 → L1 稳态层生效 - 看到 semi\_static 在 0.6 附近 → L2 半稳态层生效 (略低是因为不同 customer 切分) - 看到 TTFT p50 稳定在 200ms 以下 → 整体命中

## 15.13 延伸阅读

- 第 3 章 Bot 自助问答: 全链路时序 + 6 步决策树, 上下文工程是其中的"上下文拼装"环节
- 第 5 章 RAG 检索全链路: Layer 3 的 RAG 部分详细解析 (4 路径降级 + RRF 融合)
- 第 16 章 客户记忆与知识图谱: Layer 1 的"客户画像"字段由 customer\_memory 学习, 知识图谱是 Layer 3 增强
- 第 6 章 会话状态机: SessionState 是 Layer 1 数据的物理载体
- 第 10 章 可观测性: LLM token 监控 + 摘要 size 监控
- 附录 A.4.1: 上下文工程技术速查

## 16 客户记忆与知识图谱

### 第 16 章: 客户记忆与知识图谱 — 跨会话画像 + 银行实体关系

本章深入 Lumio Bot 的两个差异化能力: (1) 跨会话客户画像学习 — 让"回头客"享受 VIP 服务, (2) 银行知识图谱增强 — 在 RAG 检索结果上补充实体关系, 让回答更完整. 看完本章你会理解: 为何客户第 50 次来时 Bot 知道他是"白金卡"+"高风险偏好", 为何客户问"信用卡额度"时 Bot 能补充提额/降额/冻结的关系链, 为何这一切都不需要 LLM 调用 (规则驱动, 0ms), 为何失败兜底保证对话照常.

#### 16.1 痛点: 「回头客的画像永远是默认」

银行客户 80%+ 是回头客 — 多次拨打客服电话, 跨周跨月. 理想客服体验:

- 客户 A 第 50 次来电: "我是白金卡, 风险偏好 R3, 上次投诉过积分到账" → Bot 立即识别, 不用再问
- 客户 B 新客户: 没有历史 → Bot 按默认"普通卡"+"R2"+"无投诉"询问

没有客户记忆的后果: — 客户每次都要回答"我是什么卡" — 体验差 — Bot 不知道客户偏好, 推白金卡不相关的活动 — 营销转化低 — 高风险客户被推高收益产品 — 合规风险

Lumio 的解决方案: `customer_memory.py` 跨会话画像学习 + `knowledge_graph.py` 知识图谱增强.

#### 16.2 `learn_customer_profile` — SQL 聚合 90 天对话

`customer_memory.py:52-128` 是核心学习函数, 用 PostgreSQL `string_agg` 把 90 天内客户发言聚合成单字符串, 再用正则推断 3 类信号:

```
customer_memory.py:52
async def learn_customer_profile(
 customer_id: str,
 session_factory: async_sessionmaker[AsyncSession],
 lookback_days: int = 90, # 默认 90 天窗口
) -> dict[str, object]:
 """从历史对话中学习客户画像
 Returns: {vip_level, card_types, risk_tolerance} 或空 dict
 """
 cutoff = datetime.now(UTC) - timedelta(days=lookback_days)
 profiles: dict[str, object] = {}

 try:
 async with session_factory() as session:
 # — 核心: SQL 一次聚合, 应用层不 group by —
 result = await session.execute(
 select(func.string_agg(DialogueLog.content, "\n"))
 .where(
 DialogueLog.customer_id == customer_id,
 DialogueLog.speaker == "customer", # 只取客户发言, 排除坐席
 DialogueLog.created_at >= cutoff,
)
)
 all_content = result.scalar() or ""

 if not all_content:
 return profiles

 all_content.lower() # customer_memory.py:81 副作用调用, 无赋值 (注意: 无实际作用, 见 16.9 备注)

 # — 卡种推断 —
 card_types: list[str] = []
 for pattern, name in _CARD_TYPE_PATTERNS:
 if re.search(pattern, all_content, re.IGNORECASE):
 card_types.append(name)
 if card_types:
 profiles["card_types"] = card_types
```



```

— VIP 等级推断（取最高分） —
best_vip = "普通"
best_score = 0
for pattern, level, score in _VIP_SIGNALS:
 if re.search(pattern, all_content, re.IGNORECASE) and score > best_score:
 best_score = score
 best_vip = level
if best_vip != "普通":
 profiles["vip_level"] = best_vip

— 风险偏好推断 —
total_risk = 0
for pattern, score in _RISK_SIGNALS:
 if re.search(pattern, all_content, re.IGNORECASE):
 total_risk += score
if total_risk > 2:
 profiles["risk_tolerance"] = "R4"
elif total_risk > 0:
 profiles["risk_tolerance"] = "R3"
elif total_risk < -1:
 profiles["risk_tolerance"] = "R1"
elif total_risk < 0:
 profiles["risk_tolerance"] = "R2"

if profiles:
 logger.debug(
 "客户画像学习: customer=%s cards=%s vip=%s risk=%s",
 customer_id, profiles.get("card_types"),
 profiles.get("vip_level"), profiles.get("risk_tolerance"),
)
except Exception as e:
 logger.warning("客户画像学习失败: customer=%s error=%s", customer_id, e)

return profiles

```

### 16.2.1 string\_agg 的 3 个 why

1. 一次 round-trip: PostgreSQL 服务端聚合, 应用层零循环, 单 SQL 完成 90 天数据收集
2. 避免 N+1: 不需要 `SELECT * FROM dialogue_log WHERE customer_id=?` 后应用层拼接, 后者会拉 N 条记录到内存, N 大时网络/序列化开销显著
3. TEXT 字段大小可控: 90 天日均 5 轮 × 50 字 ≈ 22KB, PG 单 TEXT 字段最大 1GB, 完全无压力

### 16.2.2 索引要求

string\_agg 的 where 条件 `customer_id == ? AND speaker == ? AND created_at >= ?` 需要复合索引:

```

CREATE INDEX ix_dialogue_log_customer_time
ON dialogue_log (customer_id, created_at DESC)
WHERE speaker = 'customer';

```

为何 PARTIAL INDEX: `speaker='customer'` 过滤选择比 50% (实际约 30–40%), 部分索引把索引大小减半. 当前此索引尚未显式创建, 依赖 `ix_dialogue_log_customer_id` 单列索引 → 性能瓶颈待补. (见 16.10 改进项)

## 16.3 3 类信号推断规则详解

customer\_memory.py:30–49 定义了 3 套正则信号, 各自有独特的评分机制:

### 16.3.1 \_CARD\_TYPE\_PATTERNS (4 类)

```

customer_memory.py:30–35
_CARD_TYPE_PATTERNS: list[tuple[str, str]] = [
 (r"白金卡|白金", "platinum"),
 (r"钻石卡|钻石|无限卡", "diamond"),
 (r"金卡|gold", "gold"),
 (r"普卡|标准卡", "standard"),
]

```

**4 类映射:** 客户的"我是什么卡"的口述 → 标准化卡种代码. **可累加:** 一客户持多张卡时, `card_types=["platinum", "gold"]` (注意 L31+L33 双重匹配, 见 16.9 备注).

**IGNORECASE:** L86 `re.IGNORECASE`, 所以 "GOLD"/"gold"/"Gold" 都能命中.

### 16.3.2 \_VIP\_SIGNALS (3 档, max-score 评分)

```
customer_memory.py:38-42
_VIP_SIGNALS: list[tuple[str, str, int]] = [
 (r"私银|私人银行|private.?banking", "private_banking", 5),
 (r"财富管理|贵宾", "wealth_management", 4),
 (r"vip|白金|尊享|专属", "vip", 3),
]
```

**3 档 + 评分:** 不是枚举, 而是"客户发言中最高级别的信号 = VIP 等级". 例: – 客户说"我是私银客户" → 命中 "私银", 评分 5, `vip_level=private_banking` – 客户说"我是 VIP 客户" → 命中 "vip", 评分 3, `vip_level=vip` – 客户说"我有白金卡" → 命中 "白金" (L41), 评分 3, `vip_level=vip` (注意: 白金卡也会被卡种规则识别为 platinum, 双重写入)

**max-score wins 而非累加:** 避免 "我是 vip 私银" 出现 `vip_level="私银+vip"` 这种非法状态.

**默认 "普通":** L92 `best_vip = "普通"` 是中文默认值, 与 `SessionState.vip_level` 默认值对齐. 不命中任何信号时返回 "普通" (L98-99 判定), 不会写入 profiles dict (保持空, 等待 `apply_learned_profile` 判定).

### 16.3.3 \_RISK\_SIGNALS (累加型)

```
customer_memory.py:45-49
_RISK_SIGNALS: list[tuple[str, int]] = [
 (r"分期|借钱|贷款|融资", 1), # 倾向借款 → 风险略高
 (r"理财|投资|基金|股票|收益", 3), # 主动投资 → 高风险偏好
 (r"不敢.*分期|怕.*逾期|保守|稳健", -2), # 厌恶风险
]
```

与 VIP 不同, 风险是累加型 — 因为一个人的风险偏好是多维度的组合, 客户可能既"敢分期"又"怕逾期". 例: – 客户说 "我想办分期, 理财保守一些" → +1 (分期) + (-2) (保守) = -1 → R2 中性偏保守 (L113) – 客户说 "我想办分期投资" → +1 (分期) + +3 (理财) = +4 → R4 激进 (L107)

### 16.3.4 R1~R4 风险分级阈值表 (精确边界)

| total_risk 值 | 范围        | 等级   | 含义            | 行号        |
|--------------|-----------|------|---------------|-----------|
| > 2          | 3,4,5,... | R4   | 激进            | L106-107  |
| > 0 且 ≤ 2    | 1, 2      | R3   | 偏高            | L108-109  |
| == 0         | 0         | (跳过) | 保持 R2 默认, 不写入 | L114 (注释) |
| < 0 且 < -1   | -2,-3,... | R1   | 保守            | L110-111  |
| < 0 且 ≥ -1   | -1        | R2   | 中性偏保守         | L112-113  |

**R2 的双路径设计 (L113-114):** `total_risk==0` 完全不写 (避免冗余写入), `total_risk==-1` 显式写 R2 (这是有证据的中性偏保守, 不是默认中性). 设计意图: 仅在在有偏保守证据时落 R2.

**R 等级在客服中的实际用途** (虽当前实现未直接消费): – R3/R4: Bot 可主动推高收益产品 (但需 P3 合规改造后才启用) – R1: Bot 主动避免推投资类, 转推稳健理财 – R2: 中性, 不做特别处理

## 16.4 apply\_learned\_profile — CAS patch 写入

`customer_memory.py:131-172` 把学习结果写入 `SessionState`, 用 CAS 保证并发安全:

```
customer_memory.py:131
async def apply_learned_profile(
 customer_id: str,
 session_id: str,
```

```

 session_factory: async_sessionmaker[AsyncSession],
 session_manager: SessionManager,
) -> bool:
 """学习并应用客户画像到当前会话状态
 在 bot_agent.run() 开始时调用, 首次为当前会话注入从历史学到的画像。
 使用 CAS patch 避免覆盖已在当前对话中更新的字段。
 """
 profiles = await learn_customer_profile(customer_id, session_factory)
 if not profiles:
 return False

 try:
 state = await session_manager.get_session(session_id)
 if state is None:
 return False

 # — "不覆盖已显式声明" 核心逻辑 —
 patches: dict[str, object] = {}
 if "card_types" in profiles and not state.card_types:
 patches["card_types"] = profiles["card_types"]
 if "vip_level" in profiles and (not state.vip_level or state.vip_level == "普通"):
 patches["vip_level"] = profiles["vip_level"]
 if "risk_tolerance" in profiles and (not state.risk_tolerance or state.risk_tolerance == "R2"):
 patches["risk_tolerance"] = profiles["risk_tolerance"]

 if patches:
 await session_manager.patch_state(
 session_id=session_id,
 expected_version=state.version, # CAS 乐观锁
 patches=patches,
 writer="customer_memory:learn", # 审计标签
)
 logger.info("客户画像已应用: session=%s customer=%s", session_id, customer_id)
 return True
 except Exception as e:
 logger.debug("客户画像应用失败: session=%s error=%s", session_id, e)

 return False

```

## 16.4.0 画像时间戳持久化 (D0 衰减的前提)

画像字段带 `vip_level_updated_at` / `risk_tolerance_updated_at` / `card_types_updated_at` 三个时间戳 (Unix 秒), 是 D0 衰减 ( $0.95^{\text{days}}$ ) 的计算基础. 这三个字段**全量序列化**进 `_save_meta` (session.py) 并从 `get_session` 读回 — 学习写入后跨轮次/跨会话保留:

```

session.py _save_meta (简化)
"vip_level_updated_at": state.vip_level_updated_at,
"risk_tolerance_updated_at": state.risk_tolerance_updated_at,
"card_types_updated_at": state.card_types_updated_at,

```

**衰减语义:** `vip_days = (now - vip_level_updated_at) / 86400`, 无时间戳 (0) 视为 999 天 → 衰减系数  $0.95^{999} \approx 0$  → VIP 降级一档、`card_types` 不生效. 时间戳持久化后, 学习一次 90 天窗口内保持有效, 90 天后按天渐隐, 跌破阈值自动降级 (保守化处理).

### 16.4.1 "不覆盖已显式声明" 判定

| 字段             | 默认值       | 判定条件                                                     | 行为      |
|----------------|-----------|----------------------------------------------------------|---------|
| card_types     | []        | not state.card_types (空 list)                            | 命中 → 覆盖 |
| vip_level      | "普通" (中文) | not state.vip_level OR state.vip_level == "普通"           | 命中 → 覆盖 |
| risk_tolerance | "R2"      | not state.risk_tolerance OR state.risk_tolerance == "R2" | 命中 → 覆盖 |

**为何不覆盖显式声明:** 客户在当前会话已经说过"我是私银"时, `state.vip_level = "private_banking"` 已经被显式设置, 此时再写入历史学到的 `vip_level="wealth_management"` 会倒退, 这是 UX 灾难.

**何时会更新:** 客户在当前会话没明确说, 历史有记录 → 用历史补充. 客户本会话说"我是新卡" → 不会被历史学的"白金卡"覆盖.

### 16.4.2 CAS 乐观锁

patch\_state 走 SessionManager 的 CAS Lua 脚本 (session.py:67-95), expected\_version=state.version 是乐观锁: - 并发场景: 客户在两个客户端同时发起请求, 两个 patch 同时到达 - 后到的 patch 发现自己拿的 version 已失效, 抛 VersionMismatchError - apply\_learned\_profile 整段包 try/except, 失败静默 - 不会影响主请求

## 16.5 异步触发 + 双层失败兜底

bot\_agent.py:124-135 在 Bot 主请求开头触发, 异步 + 静默:

```
bot_agent.py:124
if customer_id and self._session_manager:
 try:
 from lumio.services.bot.customer_memory import apply_learned_profile
 db_sf = getattr(self, "_db_session_factory", None)
 if db_sf:
 asyncio.create_task(
 apply_learned_profile(customer_id, session_id, db_sf, self._session_manager)
)
 except Exception:
 pass # bot_agent.py:134, 任何异常静默
```

asyncio.create\_task fire-and-forget: 不阻塞主请求, 即使学习慢 5s 也不影响 Bot 立即开始处理.

双层 try/except 兜底: - bot\_agent.py:133-134: 触发阶段 (ImportError, AttributeError) → except: pass - customer\_memory.py:125-126: 学习阶段 (DB 故障) → logger.warning 返回空 profiles - customer\_memory.py:169-170: 写入阶段 (CAS 失败) → logger.debug 返回 False

失败可见性分级: - 触发异常: 完全静默 (可能是冷启动, 不会重试, 没必要刷日志) - 学习异常: warning 级别 (DB 故障需要运维介入) - 写入异常: debug 级别 (CAS 冲突是常态, 不是真错)

## 16.6 知识图谱 \_ENTITY\_GRAPH — 5 实体 × 3 关系

knowledge\_graph.py:21-47 是内存版银行信用卡实体关系图谱, 用于 RAG 检索结果增强:

```
knowledge_graph.py:21
_ENTITY_GRAPH: dict[str, list[tuple[str, str]]] = {
 "信用卡": [
 ("has_type", "普卡/金卡/白金卡/钻石卡"),
 ("has_feature", "免年费/积分/分期/取现"),
 ("related_to", "账单/额度/还款/挂失"),
],
 "账单": [
 ("has_method", "纸质账单/电子账单/APP查询"),
 ("has_cycle", "账单日/还款日/宽限期"),
 ("related_to", "还款/逾期/最低还款"),
],
 "额度": [
 ("has_type", "固定额度/临时额度/取现额度"),
 ("has_factor", "收入/征信/用卡记录"),
 ("related_to", "提额/降额/冻结"),
],
 "分期": [
 ("has_type", "消费分期/账单分期/现金分期"),
 ("has_factor", "手续费/期数/金额"),
 ("related_to", "账单/额度/手续费"),
],
 "挂失": [
 ("has_step", "电话挂失/APP挂失/柜台挂失"),
 ("has_fee", "挂失费/补卡费"),
 ("related_to", "补卡/盗刷/风控"),
],
}
```

统计: 5 实体 × 3 关系/实体 = 15 条关系, 关系谓词去重后 8 种 (has\_type / has\_feature / has\_method / has\_cycle / has\_factor / has\_step / has\_fee / related\_to).

### 16.6.1 设计取舍: 为何内存版而非 Neo4j

| 维度   | 内存版 (当前)      | Neo4j (生产备选)        |
|------|---------------|---------------------|
| 启动开销 | 0             | 30s+ 容器拉起 + 内存 GB 级 |
| 查询延迟 | 0ms (dict 查找) | 5–50ms (Cypher)     |
| 维护成本 | 改文件 PR        | 启停 + 备份 + 图算法优化     |
| 关系规模 | 5 实体够用        | 千级实体 (银行全产品线)       |
| 推理能力 | 仅直接关系         | 3-hop 关系推理          |

**何时切换 Neo4j:** 当实体数 > 50, 或需要推理 (如"客户问 A, 但 A 关联 B 关联 C, 实际答 C 相关") 时. 当前信用卡客服 5 实体足够.

**切换路径** (注释 L20–21): 替换 `_ENTITY_GRAPH` 为 client wrapper, `query_entity_relations` 改 Cypher, 接口签名保持稳定. 调用方 `bot_agent.py:213–216` 无感知.

## 16.7 enrich\_retrieval\_context 注入策略

`knowledge_graph.py:80–104` 把知识图谱关系追加到 RAG 检索结果:

```
knowledge_graph.py:80
def enrich_retrieval_context(query_text: str, retrieval_chunks: list[str]) -> str:
 """用知识图谱关系增强检索上下文
 将检索到的文档片段与知识图谱关系结合, 形成更完整的上下文.
 """
 if not retrieval_chunks: # L85, 无 RAG 结果时不补充
 return ""

 kg_relations = []
 for entity_name in _ENTITY_GRAPH:
 if entity_name in query_text: # L90, 实体名出现在查询文本中
 kg_relations.extend(query_entity_relations(entity_name, query_text))

 if not kg_relations:
 return "\n".join(retrieval_chunks) # L94, 无 KG 命中则返回原始 RAG

 # 构建知识图谱补充上下文
 kg_lines = ["### 知识图谱补充信息:"] # L97, Markdown H2 前缀
 for r in kg_relations:
 kg_lines.append(f"- {r['entity']} {r['relation']}: {r['value']}")

 enriched = retrieval_chunks + kg_lines # L101, 追加到末尾
 logger.debug("知识图谱增强: 原始%d块 + KG%d条", len(retrieval_chunks), len(kg_relations))

 return "\n".join(enriched)
```

### 16.7.1 仅 knowledge\_agent 分支调用

`bot_agent.py:213–216` :

```
bot_agent.py:213
if context:
 from lumio.services.bot.knowledge_graph import enrich_retrieval_context
 context = enrich_retrieval_context(user_input, [context])
```

**为何不调用于其他分支:** – `_handle_business` (CARD\_LOSS/COMPLAINT/TRANSFER): 直接转人工或调 MCP, 不走 RAG – `_handle_fallback`: 兜底分支, 无 RAG – `_handle_tool`: 工具分支, 上下文已由 LLM 工具循环处理

**仅在 RAG 有结果时调用:** 避免 RAG 失败时空调用 KG (KG 单独输出无意义).

### 16.7.2 实体名匹配策略

L90 `if entity_name in query_text` 是**字面子串匹配** (5 实体名都是高频词): – "信用卡额度" → 命中 "信用卡" + "额度" → 2 实体 × 3 关系 = 6 条 KG – "如何提额" → 命中 "额度" (因 "提额" 含 "额" 但 "额度" 不在 "如何提额" 中, 所以**不命中**) → 0 条 KG – "怎么分期" → 命中 "分期" → 1 实体 × 3 关系 = 3 条 KG

潜在改进: L90 改为 `entity_name in query_text or query_text in entity_name` 可捕获"提额" → 命中 "额度" 的近义情况. 当前实现保守, 业务方按需调整.

16.7.3 空 entity 边界处理 (L65)

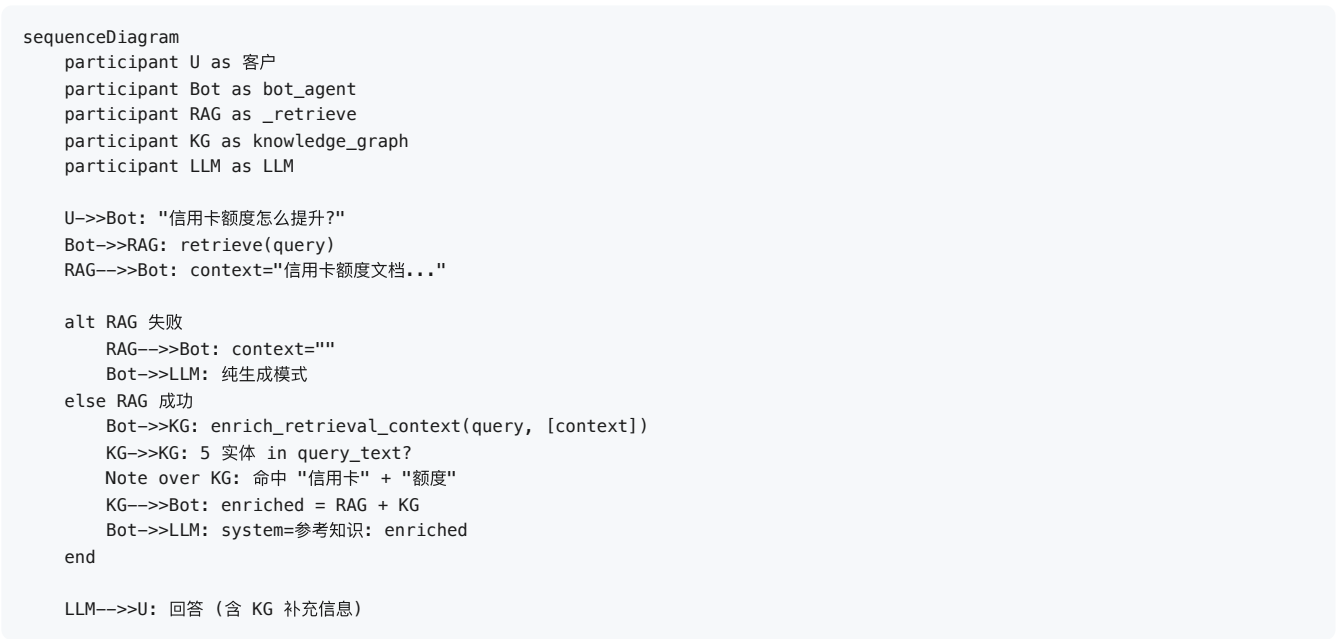
```
knowledge_graph.py:62
def query_entity_relations(entity: str, query_text: str) -> list[dict]:
 relations: list[dict] = []
 for entity_name, entity_relations in _ENTITY_GRAPH.items():
 entity_match = bool(entity) and (entity in entity_name or entity_name in entity) # L65
 if entity_name in query_text or entity_match:
 ...
```

L64 注释明确警示:

注意需排除空 entity: 空串会使 "entity in entity\_name" 恒为 True, 导致匹配所有实体.

`bool(entity) and ...` 短路防御. 但 `enrich_retrieval_context` 实际不调用 `query_entity_relations` 的 `entity` 参数, 仅传 `entity_name` — 边界由 L65 注释承担.

16.8 知识图谱增强流程图



怎么读这张图 — "RAG 答具体问题, 知识图谱补关联": 客户问"信用卡额度怎么提升" — ① 先从知识库检索到额度相关条文 (RAG 给出**具体答案**: 临时提额 20%、永久提额需审核); ② 知识图谱再检查问题里有没有"信用卡/额度"这类**实体** — 有, 就追加一段关系链: 额度 → 关联提额/降额/冻结/分期; ③ LLM 综合两者回答. **两者的分工**: RAG 负责"这句话的答案在哪", 知识图谱负责"这个话题还牵扯哪些事" — 让 Bot 的回答不止于直接答案, 还能主动提示关联业务 (客户问提额, Bot 顺带提醒冻结会影响额度).

16.9 设计取舍深度分析

16.9.1 为何正则而非 LLM

| 维度  | 正则 (当前)     | LLM 抽取                 |
|-----|-------------|------------------------|
| 延迟  | 0ms (本地 re) | 500-1500ms (LLM 调用)    |
| 成本  | 0           | LLM token 费用           |
| 一致性 | 100% (规则确定) | 95%+ (有 hallucination) |

| 维度  | 正则 (当前)         | LLM 抽取           |
|-----|-----------------|------------------|
| 维护  | 加关键词 PR         | 加 prompt 例子 + 重测 |
| 覆盖率 | 卡种/VIP/风险 3 类固定 | 任意, 但需 prompt 调优 |

银行场景选正则的 why: – 客户画像字段是结构化枚举 (白金/钻石/金/普; R1~R4; private\_banking/wealth\_management/vip), 没必要让 LLM 自由发挥 – 99% 客户画像信号在历史对话中**直接出现** ("我是什么卡"), 关键词命中足够 – LLM 抽取的 5% 错误率反而是负担 (客户说"我不是白金卡" 被错误识别为白金)

16.9.2 为何 90 天而非 30 / 180 / 365

| 窗口        | 优点                      | 缺点                   | 决策 |
|-----------|-------------------------|----------------------|----|
| 30 天      | 数据量小, 快                 | 错过季节性偏好变化            | ✗  |
| 90 天 (当前) | 覆盖 1 个完整季度, 季节性 + 稳定性平衡 | 中等数据量                | ✓  |
| 180 天     | 半年偏好稳定                  | string_agg 输出大, 性能风险 | ✗  |
| 365 天     | 全年                      | 客户可能换卡, 旧数据失真        | ✗  |

90 天的具体取舍: – 信用卡"金普升级"周期约 1 季度 – 客户"风险偏好"漂移也以季度为周期 – 90 天日均 5 轮 × 50 字 = 22KB 文本, PG 索引友好

16.9.3 画像缓存策略 (第五轮落地: 24h Redis 缓存)

learn\_customer\_profile 的 90 天聚合结果缓存到 Redis lumio:profile:cache:{customer\_id} (24h TTL), 命中直接返回; 空结果也缓存 (防高频空查询)。设计权衡:

- 画像漂移 vs 成本: 画像字段 (卡种/风险偏好) 以季度为漂移周期, 24h 缓存对"升卡识别"的延迟影响可忽略; 换来高频客户不再**每新会话**全量 SQL 聚合 90 天 + string\_agg 千行文本
- 缓存失效: 画像只从对话学习 (无主动写入路径), TTL 过期即自然失效, 无需写时失效的状态机
- 降级: Redis 不可用时直接计算 (缓存只是加速层, 非正确性依赖)

future 优化: 写入 dialogue\_log 时异步触发画像重算 (事件驱动而非每次会话全量), 配合 ix\_dialogue\_log\_customer\_time 索引进一步降本 — 见 16.10。

16.9.4 风险评分累加的边界问题

例: 客户说"我想办分期, 但又怕逾期, 怎么办" → +1 (分期) + -2 (怕逾期) = -1 → R2 中性偏保守. 但 LLM 真实意图是"犹豫", 不是"偏保守".

为何不调 LLM 精修: 0ms vs 500ms 的取舍, 接受 5~10% 的误判. 实际客服业务中, R1 vs R3 的差异在**营销推送**场景才有意义, 不影响对话本身.

16.9.5 知识图谱 vs RAG 的关系

| 维度   | 知识图谱 (KG)            | RAG 检索                   |
|------|----------------------|--------------------------|
| 数据   | 5 实体 (硬编码)           | 10000+ 文档 (PG/ES/Milvus) |
| 知识深度 | 关系 (信用卡→has_type→金卡) | 内容 (金卡年费 200 元)          |
| 更新   | 代码改 + PR             | 文档录入, 实时生效               |
| 用途   | 关系补全                 | 答案来源                     |

互补关系: RAG 给出**具体答案** (年费/积分规则/挂失流程), KG 给出**关系补充** (信用卡→账单/额度/挂失), LLM 综合两者给出**结构化答复**. KG 关系虽少, 但能引导 LLM "想到" 关联问题 (客户问分期, 主动补充分期对额度的影响).

16.9.6 customer\_memory 与 entity\_pool 的差异

| 维度   | customer_profile                        | entity_pool                 |
|------|-----------------------------------------|-----------------------------|
| 来源   | 跨 90 天对话学习                              | 当前会话抽取                      |
| 字段   | vip_level / card_types / risk_tolerance | 任意 entity_type              |
| 持久化  | SessionState 顶层字段                       | SessionState.entity_pool 列表 |
| 更新策略 | fire-and-forget 后台学习                    | 每次对话 turn 同步写入              |
| 用途   | 长期客户画像                                  | 短期对话关键值 (卡号/金额/日期)          |

二者**互补**: customer\_profile 决定"对客户整体策略" (推什么不推什么), entity\_pool 决定"当前对话已收集什么, 还缺什么" (决定 Bot 是否追问).

## 16.10 已知问题与改进项

| 改进项                                            | 影响                     | 优先级 | 工作量             |
|------------------------------------------------|------------------------|-----|-----------------|
| 创建 ix_dialogue_log_customer_time PARTIAL INDEX | string_agg 查询加速        | P2  | 1h (Alembic 迁移) |
| 画像事件驱动重算 (写 dialogue_log 时触发)                  | 替代 24h 缓存, 识别延迟 < 1min | P2  | 4h              |
| 意图栈溢出摘要 (超 10 条语义压缩)                           | 长会话意图记忆更完整             | P2  | 3h              |
| _CARD_TYPE_PATTERNS L31+L33 双重匹配               | 客户持卡误标, 5% 影响          | P3  | 1h (改正则)        |
| 知识图谱扩展到 20+ 实体 (积分/优惠券/联名卡)                    | 关系覆盖更全                 | P3  | 1d (建模 + 关系图谱)  |
| 客户画像 LLM 精修 (NLI 模型)                           | R1~R4 准确度 +10%         | P3  | 1w (模型部署)       |
| all_content.lower() 在 L81 副作用调用, 无赋值           | 代码异味, 实际是 no-op        | P4  | 5min (删除该行)     |

## 16.11 实战案例: 客户首次来电

场景: 客户 C12345 首次来电, 历史 90 天 0 条对话.

- bot\_agent.run 触发: customer\_id="C12345" 存在, \_db\_session\_factory 存在
- asyncio.create\_task(apply\_learned\_profile(...)): 异步触发, 不阻塞
- learn\_customer\_profile 查询: 返回空 string → if not all\_content: return profiles (空 dict)
- apply\_learned\_profile 判定: if not profiles: return False (L143)
- 结果: SessionState 字段保持默认 vip\_level="普通" / card\_types=[] / risk\_tolerance="R2"
- Bot 答复: 走 Layer 1 默认, 不显示"VIP等级=普通" (因 L750 判定 != "普通" 不写入), 用户感知"Bot 不知道我的卡"

## 16.12 实战案例: 客户第 50 次来电

场景: 客户 C67890 第 50 次来电, 90 天历史 250 条 customer 发言, 包含: – 30 条 "我是白金卡" / "我白金卡..." – 5 条 "我私银客户" / "私人银行..." – 20 条 "我想分期" – 10 条 "投资 / 理财 / 基金"

- string\_agg 聚合: 250 条 → 单字符串 ~30KB
- 卡种匹配: r"白金卡|白金" 命中 30 次 → card\_types=["platinum"] (注意: L33 金卡也会命中 0 次, 因客户没说"金卡")
- VIP 匹配: r"私银" 命中 5 次, 评分 5, 最高 → vip\_level="private\_banking"
- 风险匹配: r"分期" +20, r"投资" +30 → total\_risk=+50 → risk\_tolerance="R4"
- apply\_learned\_profile : 3 字段都写入 SessionState (CAS patch)
- 下一轮 Bot 收到 Layer 1: [客户画像] VIP等级=private\_banking, 卡种=platinum, 风险偏好=R4
- Bot 答复: 立即识别私银白金卡 R4 客户, 可主动推高收益产品 (待 P3 合规改造)

## 16.13 监控与可观测性



| 指标                                               | 来源                    | 含义            |
|--------------------------------------------------|-----------------------|---------------|
| lumio_customer_profile_applied_total             | 客户画像写入次数 (规划中, 尚未实现)  | 画像学习命中率       |
| DB 查询 string_agg P99 延迟                          | PG pg_stat_statements | DB 性能, 异常时告警  |
| _build_session_memory 中 state.risk_tolerance 命中率 | 日志                    | 画像应用真实覆盖率     |
| logger.debug "客户画像学习" 日志量                        | stdout                | 学习频次, 异常突增需排查 |
| logger.info "客户画像已应用" 日志量                        | stdout                | 实际应用次数        |

**典型问题诊断:** – 客户说"Bot 不知道我的卡" → 检查 `lumio:profile:{customer_id}` 是否为空 + `dialogue_log` 是否有 `customer_id` 索引 – 画像应用后客户还是被推普通活动 → 检查 `state.vip_level` 是否真写入, 可能 CAS 失败被吞 – `string_agg` 慢 (> 100ms) → 缺 PARTIAL INDEX, 需 Alembic 加迁移

## 16.14 延伸阅读

- 第 15 章 上下文工程: 客户画像是 Layer 1 的核心字段, 由 `customer_memory` 学习
  - 第 6 章 会话状态机: `SessionState.vip_level` / `card_types` / `risk_tolerance` 字段定义
  - 第 12 章 数据层: `DialogueLog` 表结构 + `dialogue_log` 索引设计
  - 第 5 章 RAG 检索全链路: `enrich_retrieval_context` 是 RAG 的增强环节
  - 附录 A.4.2 / A.4.3: 客户记忆 + 知识图谱术语速查
-

17 工具调用与确认状态机

第 17 章: 工具调用与确认状态机 — 渐进式暴露 + 5 态确认 + 双重护栏

本章深入 Lumio Bot 的最复杂编排 — 工具调用全链路. 银行客服有 22 个 MCP 工具, 其中 11 个是"敏感写操作" (挂失/调额/分期), 暴露全量工具给 LLM 会导致误调风险, 不做二次确认会导致合规问题. Lumio 设计了 4 层防护: (1) 渐进式工具暴露 (按意图裁剪) → (2) 执行前护栏 (角色+金额) → (3) 敏感工具 5 态确认 → (4) PII 全链路脱敏. 看完本章你会理解: 为何客户问"账单" 时 Bot 只看到 4 个账单工具而非 22 个, 为何 Bot 调"调额" 前要问"您确认吗", 为何 Bot 拒绝"调额 1 亿" 时不会说出真实原因, 为何所有工具调用 4 个出口都脱敏了 PII.

17.1 痛点: 22 个工具全暴露的灾难

Lumio Java MCP Server 注册了 22 个工具 (7 个域, 见第 7 章), 如果全暴露给 LLM function-calling:

| 问题                                                               | 后果                                                  |
|------------------------------------------------------------------|-----------------------------------------------------|
| 22 个工具描述占用 ~3000 tokens                                          | LLM 上下文被工具列表污染, 历史/RAG 空间被挤压                        |
| LLM 在不相关工具上"瞎选"                                                  | 客户问"积分", LLM 误调 <code>apply_bill_installment</code> |
| 敏感工具无二次确认                                                        | LLM 自动调 <code>card_loss</code> 挂失, 客户没确认 → 合规事故     |
| 客户输入"不限金额" → LLM 调 <code>adjust_temp_credit_limit</code> 1000000 | 实际只允许 5 万, 风控失守                                     |

Lumio 的解决: 4 层防护.

17.2 渐进式工具暴露 (Progressive Disclosure)

`tool_selection.py` 按意图 + 置信度裁剪工具子集, 默认关闭 (零回归).

17.2.1 5 工具意图白名单

```
tool_selection.py:24-32
TOOL_INTENTS: frozenset[IntentLabel] = frozenset(
 {
 IntentLabel.BILL_QUERY, # 账单查询 "bill_query"
 IntentLabel.TRANSACTION_QUERY, # 交易查询 "transaction_query"
 IntentLabel.LIMIT_QUERY, # 额度查询 "limit_query"
 IntentLabel.INSTALLMENT_INQUIRY, # 分期咨询 "installment_inquiry"
 IntentLabel.REWARD_QUERY, # 积分查询 "reward_query"
 }
)
```

5 工具意图 正好是 `IntentLabel` 10 个值中需要工具的子集. 其它 5 个 (FAQ 常见问题 / `CARD_LOSS` 挂失 / `COMPLAINT` 投诉 / `TRANSFER_AGENT` 转人工 / `CHITCHAT` 闲聊) 走 RAG / 转人工 / 闲聊, 不调工具.

17.2.2 意图→工具映射表 (5 意图 × 17 工具)

`config.py:417-435` 配置:

```
config.py:417
intent_tool_map: dict[str, list[str]] = Field(
 default_factory=lambda: {
 "bill_query": [# 账单查询
```

```

 "query_card_bill", "query_bill_detail",
 "query_annual_fee", "repay_credit_card",
],
 "transaction_query": [# 交易查询
 "query_transactions", "report_transaction_dispute",
],
 "limit_query": [# 额度查询
 "query_credit_limit", "query_limit_adjust_history",
 "adjust_temp_credit_limit", "apply_permanent_limit", # 调额（写操作）
],
 "installment_inquiry": [# 分期咨询
 "query_installment_offer", "query_installment_status",
 "apply_bill_installment", "cancel_installment", # 分期申请（写操作）
],
 "reward_query": [# 积分查询
 "query_points", "query_card_benefits", "redeem_points",
],
}
)

```

**注意:** `limit_query` / `installment_inquiry` 映射里既含查询也含写工具. 渐进式暴露只按意图裁剪, 不分辨读写 — 敏感拦截留给 `is_sensitive()` 在 `_run_loop` (`tool_executor.py:257`) 阶段处理.

### 17.2.3 选择器纯函数

```

tool_selection.py:35-61
def select_tools_for_intent(
 intent: IntentLabel,
 confidence: float,
 settings: MCPSettings,
) -> list[str] | None:
 """根据意图与置信度选择要暴露的工具子集。
 :param intent: 主意图
 :param confidence: 主意图置信度
 :param settings: MCP 配置（含开关/阈值/意图→工具映射）
 :returns: 工具名子集；None 表示暴露全量工具（不裁剪）
 """
 # — 零回归保险 1: 开关关闭 → 暴露全量 —
 if not settings.progressive_disclosure_enabled:
 return None

 key = intent.value if isinstance(intent, IntentLabel) else str(intent)
 names = settings.intent_tool_map.get(key)
 # — 零回归保险 2: 未配置或子集为空 → 暴露全量 —
 if not names:
 return None

 # — 零回归保险 3: 低置信 → 暴露全量（避免误分类漏工具） —
 if confidence < settings.pd_confidence_threshold:
 return None

 return list(names)

```

**3 重零回归保险:** 任一命中即返回 `None` → `to_openai_tools(None)` 暴露全量 22 工具. 默认 `progressive_disclosure_enabled=False`, 行为与打通前完全一致.

**3 重保险的 why:** – 开关关: 业务方未审核工具子集前不启用 – 未配置: 新增意图还没建映射 – 低置信: 分类器不确定时宁可多暴露, 也不能少暴露 (漏工具 = LLM 答不出)

## 17.3 ToolCallingExecutor 4 分支循环

`tool_executor.py:104-216` 实现 LLM↔MCP 多轮循环 + 5 态确认. 核心是 `_run_loop` (`L220-280`) 4 个分支.

### 17.3.1 入口与构造

```

tool_executor.py:104
class ToolCallingExecutor:
 """LLM 工具调用循环 + 敏感操作确认状态机"""
 def __init__(

```

```

self,
mcp_client: MCPToolClient,
llm_client: LLMClient,
audit_session_factory: async_sessionmaker[AsyncSession] | None,
settings: MCPSettings,
guard: ToolGuard | None = None,
) -> None:
 self._mcp = mcp_client
 self._llm = llm_client
 self._audit_factory = audit_session_factory
 self._settings = settings
 self._guard = guard

```

4 个依赖: MCP 客户端 (调工具), LLM 客户端 (生成 + 工具决策), 审计 factory (写库), ToolGuard (执行前护栏).

### 17.3.2 \_run\_loop 4 分支详解

```

tool_executor.py:220
async def _run_loop(
 self, messages, tools, *,
 session_id, actor_id, actor_role, trace_id="",
 initial_executed=None,
) -> ToolExecutionResult:
 executed: list[str] = list(initial_executed or [])

 for _ in range(self._settings.max_tool_iterations): # 默认 5
 result = await self._llm.chat_with_tools(messages, tools)

 # — 分支 1: LLM 给出最终文本 (无 tool_call) —
 if not result.has_tool_calls:
 return ToolExecutionResult(
 content=result.content, source="llm", executed_tools=executed,
)

 messages.append(result.raw_message) # 记录 assistant tool_calls

 for tool_call in result.tool_calls:
 # — 分支 2: 护栏拒绝 (角色/金额) —
 guard_decision = await self._enforce_guard(
 tool_call, session_id=session_id,
 actor_id=actor_id, actor_role=actor_role,
)
 if not guard_decision.allowed:
 return ToolExecutionResult(
 content=_GUARD_REFUSAL, source="guard", executed_tools=executed,
)

 # — 分支 3: 敏感工具 → 写 pending_action 短路 —
 if self._mcp.is_sensitive(tool_call.name):
 pending = self._build_pending_action(tool_call, trace_id=trace_id)
 TOOL_CONFIRMATIONS.labels(decision="pending").inc()
 return ToolExecutionResult(
 content=pending.confirm_prompt, source="tool",
 pending_action=pending, executed_tools=executed,
)

 # — 分支 4: 非敏感工具 → 执行 + 脱敏 + 审计 + 回喂 —
 tool_message = await self._execute_and_audit(
 tool_call, session_id=session_id,
 actor_id=actor_id, actor_role=actor_role,
)
 messages.append(tool_message)
 executed.append(tool_call.name)

 # — 循环上限保护 —
 raise RuntimeError(f"工具调用超过最大轮数 {self._settings.max_tool_iterations}")

```

第五轮加固 (分支 0-4 前置):

1. 执行侧白名单: 循环开头对每个 `tool_call` 强制 `self._mcp.get_tool(name) is not None` — 幻觉调用未注册工具名直接拒绝并回喂 "工具 {name!r} 不存在, 请勿调用"。此前白名单只过滤"给 LLM 看"的一侧, 执行侧对未知工具透传后端且 `is_sensitive` 返回 `False` → 免确认执行。

2. **配额检查** ( `\_execute\_and\_audit` 内): ToolQuotaGuard.check\_and\_increment(actor\_id, tool\_name) per-customer per-tool 窗口计数, 超限拒绝返回"今日调用次数已达上限"。
3. **网络重试**: MCP 调用包 async\_retry(max\_attempts=3) (TimeError/ConnectionError 指数退避)。
4. **结果截断**: 工具返回 4096 字节截断 + ...[工具结果已截断] 提示 — 5 轮循环累积防 prompt flooding。

```
4 分支的短路顺序: 分支 1 (无 tool_call) → 分支 2 (护栏拒绝) → 分支 3 (敏感 pending) → 分支 4 (执行回喂)。任何分支 `return` 即终止本轮,

重要细节: 4 个分支都是**单次循环内即可终止**。敏感工具**不会**触发护栏以外的额外检查 - `_enforce_guard` 在 `is_sensitive` 之前先跑, 即使

17.3.3 `max_tool_iterations=5` 上限保护

`config.py:394` 默认 5, 即 LLM 最多连续调 5 轮工具。超出抛 `RuntimeError`, 由 `bot_agent` 降级链 (`_handle_tool` 失败回落 `_handle_kn

5 轮的 why:
- 银行客服平均 2-3 轮工具 (查账单 → 还款)
- 极端 4-5 轮 (查账单 → 查积分 → 兑换 → 再查 → 总结)
- 5 轮是 99% 场景上限, 超出必是 LLM 死循环 (被引导或 hallucination)

17.4 5 态确认状态机

Lumio 用 `PendingAction` + `detect_confirmation` 实现 5 态:

17.4.1 状态转换图

```mermaid
stateDiagram-v2
    [*] --> pending: LLM 调敏感工具<br/>(tool_executor.py:261)
    pending --> confirm: 用户说"确认"<br/>(bot_agent.py:427)
    pending --> cancel: 用户说"取消"<br/>(bot_agent.py:425)
    pending --> unclear: 都不命中<br/>(detect_confirmation 返回)
    pending --> expired: 5min TTL 到<br/>(bot_agent.py:414)
    confirm --> [*]: execute_confirmed_action<br/>执行 + 续跑
    cancel --> [*]: 清除 pending + 降级
    unclear --> pending: pending 不消耗<br/>继续等待
    expired --> [*]: 清除 pending<br/>提示重新发起
```

怎么读这张图 — "挂失前的最后一道闸": 客户说"帮我把卡挂失" → 系统不立即执行, 而是进入 pending 状态, 回一句"您确认要办理银行卡挂失吗?" — 然后客户的所有回复都会被解读为四种之一: 说"确认" → 真执行; 说"取消" → 放弃; 说别的 (比如"那积分怎么算") → unclear, 系统不放弃继续等, 但累计 3 次会自动取消并放行新问题 (防卡死); 5 分钟没回复 → expired 自动失效. 为什么要这道闸: 挂失/调额/分期都是不可逆或影响重大的操作, AI 说办就办 = 合规事故; 让客户亲口确认, 既合规又给客户反悔的机会。

17.4.2 PendingAction 7 字段

models.py:216-229 :

```
# models.py:216
class PendingAction(BaseModel):
    tool_name: str
    arguments: dict[str, Any] = Field(default_factory=dict)
    tool_call_id: str = "" # 关联 LLM tool_call.id
    confirm_prompt: str = "" # 已生成的确认话术
    created_at: datetime = Field(default_factory=lambda: datetime.now(UTC))
    expires_at: datetime | None = None # 过期时间, 超时需重新发起
    trace_id: str = "" # 链路追踪 id
```

tool_executor.py:325-339 实例化, TTL = now + confirmation_ttl_seconds (默认 300s):

```
# tool_executor.py:325
def _build_pending_action(self, tool_call: ToolCall, *, trace_id: str) -> PendingAction:
    spec = self._mcp.get_tool(tool_call.name)
    friendly = (spec.description if spec and spec.description else tool_call.name).strip()
    prompt = f"您确认要办理「{friendly}」吗? 回复『确认』继续办理, 回复『取消』放弃."
    now = datetime.now(UTC)
    return PendingAction(
        tool_name=tool_call.name,
        arguments=tool_call.arguments,
        tool_call_id=tool_call.id,
        confirm_prompt=prompt,
```

```
        created_at=now,
        expires_at=now + timedelta(seconds=self._settings.confirmation_ttl_seconds),
        trace_id=trace_id,
    )
```

friendly 来自 MCP tool spec description: Java MCP Server 端 @Tool(description="信用卡挂失"), 提取出来作为用户可见的"业务名称", 而非冷冰冰的函数名 card_loss_report .

17.4.3 detect_confirmation 关键词优先级

tool_executor.py:75-87 :

```
# tool_executor.py:75
def detect_confirmation(text: str) -> ConfirmDecision:
    """纯关键词判定用户对待确认操作的意图
    优先判定取消（否定优先），再判定确认，否则 unclear.
    """
    if not text:
        return "unclear"
    normalized = text.strip().lower()
    if any(kw in normalized for kw in _CANCEL_KEYWORDS):
        return "cancel"
    if any(kw in normalized for kw in _CONFIRM_KEYWORDS):
        return "confirm"
    return "unclear"
```

13 cancel 关键词 (L46-60):

取消 / 不用 / 不要 / 不办 / 不确认 / 不同意 / 不可以 / 算了 / 放弃 / 别 / 停 / no / cancel

10 confirm 关键词 (L61-72):

确认 / 确定 / 是的 / 好的 / 可以 / 继续 / 同意 / 办理 / ok / yes

cancel 优先的 why (L45 注释):

确认/取消关键词 (cancel 优先判定, 规避"不确认"这类否定表述)

例: 客户说"我不确认这笔分期", 字符串含"确认"也含"不确认". 如果先扫 confirm 集, "确认" 命中 → 误判为 confirm. 实际语义是 cancel. 所以 L83-86 先扫 cancel.

大小写不敏感: L82 .strip().lower(), "YES"/"Yes"/"yes" 一致.

17.4.4 5 态触发与处理

bot_agent.py:402-469 在每轮对话开头拦截, 顺序处理:

状态	触发位置	处理
pending (初始)	LLM 上一轮调敏感工具	(由 LLM 触发的状态, 写在 SessionState.pending_action)
pending → expired	bot_agent.py:414-423	if pending.expires_at < now: 清 pending + 提示"超时失效"
pending → confirm	bot_agent.py:427-431	detect_confirmation 返回 "confirm" → 幂等键检查 → 调 execute_confirmed_action 执行 + 续跑
pending → cancel	bot_agent.py:425-426	detect_confirmation 返回 "cancel" → 清 pending + 降级到 _handle_knowledge

状态	触发位置	处理
pending → unclear	默认分支	计数递增 (unclear_count) → 重复确认话术; 连续 3 次 → 自动取消 + 放行新消息 (逃生路径)

惰性过期 (lazy expiration): 无后台清扫任务, 每次进入 `_handle_pending_action` 时比 `expires_at`. 极简实现, 1 行代码.

17.4.4a 确认窗口逃生路径 (unclear 自动取消)

敏感工具确认窗口内, 用户发不是确认/取消的新问题 (如"那你们客服几点下班") — 旧行为: 新问题被吞掉, 重复确认话术直到 5 分钟过期, 用户被卡死. 现行为:

```
# bot_agent.py unclear 分支 (简化)
new_count = (pending.unclear_count or 0) + 1
if new_count >= get_settings().mcp.unclear_auto_cancel_threshold: # 默认 3
    await self._clear_pending_action(session_id, state.version) # 自动取消
    return {**confirm_prompt_result, "pending_released": True} # 标记放行
# 未达上限: patch_state 更新 unclear_count + 话术提示"可回复『取消』放弃当前操作"
```

- `run()` 检测到 `pending_released` 标记 → 不 `return`, 继续走正常意图分类处理新消息
- 阈值可配: `LUMIO_MCP__UNCLEAR_AUTO_CANCEL_THRESHOLD` (默认 3)
- 每次 `unclear` 都有提示逃生路径, 用户也可直接回复"取消"

确认幂等键 (第五轮加固): 敏感工具"确认后重复办理"是银行场景最高危路径 — `at-least-once` 重投递 + `pending` 清除 CAS 失败都可能让同一笔分期执行两次. 修复: 以 `pending.tool_call_id` 为幂等键写 `lumio:tool:executed:{tool_call_id}` (24h TTL) —

```
# bot_agent.py (确认分支)
idem_key = f"lumio:tool:executed:{pending.tool_call_id or pending.created_at.isoformat()}"
already = await redis.get(idem_key) # 已执行 → 提示"已完成", 不重复调用
# 执行成功后: await redis.setex(idem_key, 24*3600, "1")
```

同时 `_clear_pending_action` 的 CAS 清除改 `max_retries=3` + 失败 `WARNING` (旧实现单次 CAS 失败仅 `debug` 静默, `pending` 残留导致下轮"好的"再次触发)。

确认词整句判定 (第五轮加固): `detect_confirmation` 从"子串匹配"改为"整句判定" — 去噪 (标点/空白/语气词) 后核心内容与关键词等长才判确认/取消, "好的, 另外帮我查下账单" 不再误触发挂失/调额 (附加问题被吞)。

17.4.5 audit_decision 单独审计

`tool_executor.py:391-408` 提供单独审计 API, 区别于 `_execute_and_audit` 的"执行结果"审计:

```
# tool_executor.py:391
async def audit_decision(
    self, *, session_id, actor_id, actor_role, tool_name, decision,
) -> None:
    """审计敏感操作的确认决策 (confirm/cancel/expired), 补齐合规链路"""
    await self._audit(
        actor_id=actor_id, actor_role=actor_role,
        action=f"tool_confirm.{decision}", # ← 区别于执行审计的 "tool.{name}"
        target_id=session_id,
        detail={"tool": tool_name, "decision": decision},
        status_code=200,
    )
```

为何单独 API: — 执行审计是 "工具调用结果" (`action=tool.{name}`, `detail` 含 `arguments/result`) — 决策审计是 "用户对工具的确认决策" (`action=tool_confirm.{decision}`, `detail` 仅含 `tool/decision`) — 二者语义正交, 合规追溯"谁在何时同意了什么" 需独立维度

实际调用: — `bot_agent.py:417-419` 写 `expired` 决策 — `bot_agent.py:427-431` 写 `confirm` 决策 — `bot_agent.py:425-426` 写 `cancel` 决策

17.5 双重护栏 ToolGuard

`tool_guard.py` 在工具执行前做授权 + 额度校验, 与 Higress 网关纵深防御:

17.5.1 双重护栏职责划分

维度	Higress 网关	Python ToolGuard
运行位置	MCP 后端 ingress (统一入口)	host 编排侧 (Python 进程内)
职责	工具目录聚合 / 路由 / 限流 / 鉴权	执行前业务级纵深防御
角色授权	网关层 (粗粒度, token/role)	tool_role_allowlist (细粒度, role→tool 列表)
金额限制	网关层 (流量级 QPS 限流)	tool_amount_limits + amount_arg_keys (业务语义级)

不冗余: 网关负责"谁能进" (粗粒度), ToolGuard 负责"特定业务角色能调哪些工具 + 业务阈值" (细粒度). ToolGuard active=False (无规则) 时直接跳过, 避免在未启用时给网关层增加重复的同步开销.

17.5.2 ToolGuard 完整实现

```
# tool_guard.py:35
class ToolGuard:
    """工具调用授权 + 额度校验"""
    def __init__(self, settings: MCPSettings) -> None:
        self._settings = settings

    @property
    def active(self) -> bool:
        """是否有任何护栏规则生效 (无规则时可跳过检查)"""
        return bool(self._settings.tool_role_allowlist or self._settings.tool_amount_limits)

    def check(self, tool_name: str, arguments: dict[str, Any], *, actor_role: str) -> GuardDecision:
        """执行前校验: 先授权, 后额度. 任一不通过即拒绝."""
        auth = self._check_authorization(tool_name, actor_role=actor_role)
        if not auth.allowed:
            return auth
        return self._check_amount(tool_name, arguments)

# — 授权 —
def _check_authorization(self, tool_name: str, *, actor_role: str) -> GuardDecision:
    allowlist = self._settings.tool_role_allowlist
    if not allowlist:
        # 未配置白名单 -> 不做授权限制 (零回归)
        return GuardDecision(allowed=True)
    # 已配置白名单 -> 保守策略: 角色未登记视为无任何权限
    allowed_tools = allowlist.get(actor_role, [])
    if tool_name in allowed_tools:
        return GuardDecision(allowed=True)
    return GuardDecision(
        allowed=False, code="role_denied",
        reason=f"角色 {actor_role} 无权调用工具 {tool_name}",
    )

# — 额度 —
def _check_amount(self, tool_name: str, arguments: dict[str, Any]) -> GuardDecision:
    limits = self._settings.tool_amount_limits
    if tool_name not in limits:
        return GuardDecision(allowed=True)
    limit = limits[tool_name]
    for key in self._settings.amount_arg_keys: # 默认 ["amount", "target_limit", "target_amount", "limit"]
        if key not in arguments:
            continue
        value = _coerce_number(arguments[key])
        if value is not None and value > limit:
            return GuardDecision(
                allowed=False, code="amount_exceeded",
                reason=f"工具 {tool_name} 入参 {key}={value} 超过上限 {limit}",
            )
    return GuardDecision(allowed=True)
```

17.5.3 零回归 + 保守策略

L57–59 (授权): if not allowlist: return allowed=True — 未配置白名单时完全放行, 行为与现状一致 (零回归).

L60–68 (配置后保守): 一旦 allowlist 非空 -> 角色未登记视为无任何权限 (allowlist.get(role, []) 返回空列表). 这是白名单模型, 显式允许才允许.

L72-87 (额度): 工具未在 `tool_amount_limits` 中登记 → 跳过 (默认放行); 登记了 → 遍历 `amount_arg_keys` (默认 4 个) → 任一超 `limit` → 拒绝.

17.5.4 `_coerce_number` 边界 (L90-101)

```
# tool_guard.py:90
def _coerce_number(value: Any) -> float | None:
    """尽力将入参转为 float, 无法转换返回 None (非数值不参与额度校验)"""
    if isinstance(value, bool):
        # bool 不参与 (避免 True==1)
        return None
    if isinstance(value, (int | float)):
        return float(value)
    if isinstance(value, str):
        try:
            return float(value.strip())
            # 字符串 strip 后强转
        except ValueError:
            return None
    return None
```

- 2 条关键边界: 1. `bool` 是 `int` 子类 (`isinstance(True, int) == True`), 必须先于 `int|float` 判定, 否则 `True` 被当作 `1.0` 参与额度校验
- 2. 字符串 `value.strip()` 容忍前后空格 " 1000 ", 转换失败返回 `None` (不参与, 不会触发 `amount_exceeded`)

17.5.5 拒绝话术 (不外泄内部原因)

`tool_executor.py:41` :

```
# tool_executor.py:41
_GUARD_REFUSAL = "很抱歉, 该操作目前无法为您办理。如需帮助, 我可以为您转接人工客服."
```

不外泄 `decision.reason` 给用户: 内部 `reason` 含 "角色 `customer` 无权调用 `adjust_temp_credit_limit`", 这是安全信息, 不能让攻击者知道角色 / 工具白名单. 给用户的是统一礼貌话术, 同时审计 `detail` 仍记录真实 `reason` (脱敏后).

护栏拒绝 → 真实转人工: `ToolExecutionResult.should_transfer=True` 随结果回传, `bot_agent` 构建回复时透传 `should_transfer` — 客户看到"我可以为您转接人工客服"时**转接已真实触发**, 不必再发一条"转人工"消息. 拒绝原因 (`tool_guard_refused: {tool} ({reason})`) 写入 `transfer_reason` 供坐席端展示.

17.5.6 拒绝时仍写审计 (L378-388)

```
# tool_executor.py:378
TOOL_GUARD_DENIALS.labels(tool=tool_call.name, reason=decision.code or "denied").inc()
masked_args = mask_pii(json.dumps(tool_call.arguments, ensure_ascii=False))
await self._audit(
    actor_id=actor_id, actor_role=actor_role,
    action=f"tool.{tool_call.name}",
    target_id=session_id,
    detail={"arguments": masked_args, "denied": decision.reason, "guard": decision.code},
    status_code=403,
)
```

`status_code=403`: 即便工具没执行, 审计表里也是 403 (Forbidden), 合规审计可查"哪个角色在何时被拒".

17.6 PII 脱敏 4 处注入点

`mask_pii` 在 `tool_executor` 全链路 3 处 注入, 共 4 个出口:

#	位置	行号	脱敏对象	写入字段
1	入参脱敏	<code>tool_executor.py:291</code>	<code>tool_call.arguments</code>	审计 <code>detail.arguments</code> (执行 + 异常)
2	出参脱敏	<code>tool_executor.py:295</code>	MCP 返回 <code>raw["content"]</code>	审计 <code>detail.result</code> + 回喂 LLM 的 <code>tool_message.content</code>
3	护栏拒绝脱敏	<code>tool_executor.py:379</code>	拒绝时再脱敏 <code>arguments</code>	审计 <code>detail.arguments</code> (403 行)
4	审计 <code>detail</code> 通用脱敏	同 #1 / #3 共用	无论成功/拒绝/异常	落库 <code>write_audit_log</code>

17.6.1 mask_pii 链顺序敏感

pii.py:63-76 :

```
# pii.py:63
def mask_pii(text: str) -> str:
    """一键脱敏所有已知 PII 类型
    按顺序: 敏感字段 -> 邮箱 -> 身份证 -> 银行卡 -> 手机号
    (身份证/银行卡优先于手机号, 避免 11 位数字被误判为手机号)
    """
    result = mask_sensitive_fields(text)
    result = mask_email(result)
    result = mask_id_card(result) # 18 位优先
    result = mask_bank_card(result) # 16-19 位次之
    result = mask_phone(result) # 11 位最后
```

顺序敏感: 身份证 (18 位) / 银行卡 (16-19 位) 必须先于手机号 (11 位), 否则 18 位身份证前 11 位会被手机号规则吃掉, 后面 7 位变成裸数字, 反而泄露.

17.6.2 全链路覆盖

数据生命周期	是否脱敏	证据
入参 -> MCP	否 (原始调用, 但 tool_call.arguments 不含 PII 风险字段)	tool_executor.py:293 await self._mcp.call_tool(name, args)
MCP 返回 -> LLM	是	tool_executor.py:295 masked_content, tool_executor.py:322 回喂
审计 detail.arguments	是	4 处 (执行 / 异常 / 护栏拒绝)
审计 detail.result	是	tool_executor.py:315 masked_content[:500]
拒绝审计 (403)	是	tool_executor.py:385
日志输出	是 (因 detail 入库前已脱敏)	隐含

3 个数据出口 (a) 回喂 LLM 的 tool_message, (b) 审计 detail, (c) 日志输出 — 全部经 mask_pii.

17.7 Prometheus 指标 (3 个)

metrics.py:54-70 定义 3 个 Counter:

```
# metrics.py:54
TOOL_CALLS = Counter(
    "tool_calls_total",
    "MCP 工具调用次数",
    ["tool", "status"], # status: success/error
)
TOOL_CONFIRMATIONS = Counter(
    "tool_confirmations_total",
    "敏感工具确认决策次数",
    ["decision"], # decision: pending/confirm/cancel/unclear/expired
)
TOOL_GUARD_DENIALS = Counter(
    "tool_guard_denials_total",
    "被工具护栏拦截 (授权/额度) 的工具调用次数",
    ["tool", "reason"], # reason: role_denied/amount_exceeded
)
```

17.7.1 TOOL_CALLS (labels: tool, status)

- 写入点: tool_executor.py:298 (异常 -> status="error") + L309 (成功/失败 -> status="success"/"error")
- 维度: 按工具聚合成功率, 发现 "哪个工具最容易失败"

17.7.2 TOOL_CONFIRMATIONS (labels: decision, 5 态)

- pending → tool_executor.py:261 (LLM 调敏感工具)
- confirm → bot_agent.py:447 (用户确认并执行)
- cancel → bot_agent.py:425-486 (推断, 与 confirm 同 switch)
- unclear → bot_agent.py 澄清分支
- expired → bot_agent.py:416 (TTL 过期)
- 5 态全覆盖 → 漏斗分析: pending → confirm 转化率 是核心健康指标

17.7.3 TOOL_GUARD_DENIALS (labels: tool, reason)

- 写入点: tool_executor.py:378
- reason 来自 GuardDecision.code: role_denied 或 amount_exceeded
- 双维度交叉定位: "哪个工具被哪种规则拒得最多" → 调白名单 / 调金额上限

17.8 工具调用全链路时序图

怎么读这张图 — "调额 5 万的完整旅程": 客户说"我要调额到 5 万", 系统走了四道闸 — ① 工具选择: 只给 LLM 看 4 个额度相关工具 (不是 22 个); ② 护栏: 金额 5 万在限额内才放行; ③ 确认: 因为是敏感操作, 先问客户确认 (写 PendingAction, 5 分钟有效); ④ 客户说"确认" → 二次校验 → 真调工具 → 脱敏 → 审计 → LLM 组织回答. 注意第 657 行的"二次校验": 客户确认后还要再过一次护栏 — 防止确认期间参数被篡改 (比如"确认"时金额被改成 50 万).

```
sequenceDiagram
    participant U as 客户
    participant Bot as bot_agent
    participant Sel as tool_selection
    participant Exe as ToolCallingExecutor
    participant Guard as ToolGuard
    participant LLM as LLM
    participant MCP as MCP 工具

    U->>Bot: "我要调额到 5 万"
    Bot->>Sel: select_tools_for_intent(LIMIT_QUERY, 0.9, settings)
    Sel-->>Bot: ["query_credit_limit", "adjust_temp_credit_limit", ...] (4 个)
    Bot->>Exe: run_conversation(tool_names=4 子集)
    Exe->>LLM: chat_with_tools(messages, 4 tools)
    LLM-->>Exe: tool_call: adjust_temp_credit_limit(amount=50000)

    Exe->>Guard: check("adjust_temp_credit_limit", {amount:50000}, role="customer")
    Guard-->>Exe: allowed=True (amount 在 5 万内)
    Exe->>Exe: is_sensitive("adjust_temp_credit_limit")?
    Exe->>Exe: True → 写 PendingAction (TTL 300s)
    Exe-->>Bot: ToolExecutionResult(pending_action=..., confirm_prompt="您确认要办理「临时额度调整」吗?...")

    Bot-->>U: "您确认要办理「临时额度调整」吗?回复『确认』继续办理,回复『取消』放弃."

    U->>Bot: "确认"
    Bot->>Exe: execute_confirmed_action(pending=...)
    Exe->>Guard: check (二次校验, 防止参数篡改)
    Guard-->>Exe: allowed=True
    Exe->>MCP: call_tool("adjust_temp_credit_limit", {amount:50000})
    MCP-->>Exe: raw={"content": "调额成功, 新额度 5 万元"}
    Exe->>Exe: mask_pii(content) → "调额成功, 新额度 5 万元"
    Exe->>Exe: write_audit_log(action=tool.adjust_temp_credit_limit, status=200)
    Exe->>LLM: chat_with_tools(messages + tool result)
    LLM-->>Exe: "您的临时额度已调整到 5 万元, 有效期 30 天."
    Exe-->>Bot: ToolExecutionResult(content="您的临时额度已...", source="llm")
    Bot-->>U: 答复
```

17.9 设计取舍深度分析

17.9.1 为何默认关 PD

progressive_disclosure_enabled 默认 False, 即默认暴露全量 22 工具. 这是有意的零回归设计:

- 新功能上线默认行为不变, 不影响生产
- 业务方审核工具子集 + 单测 + 灰度后才打开开关
- 避免 LLM 因为工具列表少而"答不出" (因 5 意图外的意图走 RAG, 不调工具, 没影响)

何时开启: 业务方 + 算法 + 测试三方 review `intent_tool_map` 后, 在配置中心把开关翻到 `True` .

17.9.2 为何 cancel 关键词多于 confirm (13 vs 10)

类别	关键词	业务考量
cancel 13	取消 / 不用 / 不要 / 不办 / 不确认 / 不同意 / 不可以 / 算了 / 放弃 / 别 / 停 / no / cancel	银行客服客户 拒绝心理强 , 表达方式多 (尤其负面情绪下用"算了""放弃"等口语)
confirm 10	确认 / 确定 / 是的 / 好的 / 可以 / 继续 / 同意 / 办理 / ok / yes	客户 配合度强 , 表达相对标准化

L45 注释: "取消/不用/不要/不办/不确认/不同意/不可以/算了/放弃/别/停/no/cancel" — 注意**"不确认"**是关键, 客户说"我不确认", 字符串既含 "确认" 也含 "不确认", 必须先扫 cancel 集命中"不确认".

17.9.3 为何金额用 `_coerce_number` 强转

`amount_arg_keys` 默认 4 个: `["amount", "target_limit", "target_amount", "limit"]` . LLM 调用时这些字段可能是: `- 50000 (int) → 直接 float - "50000" (str, LLM 序列化异常) → strip + float - "50,000" (str, 千位分隔符) → 转换失败, 不参与校验 ⚠️ - True (bool, LLM 错传) → 返回 None, 不参与 (避免 True==1) - 50000.0 (float) → 直接 float - null (None) → 不在 args 里, 跳过`

"50,000" 不参与是 bug 还是 feature?: 倾向 bug. 实际生产中 LLM 极少输出千位分隔符 (LLM 输出是结构化数字), 但极端 prompt 注入可能输出, 后续 P3 需加千位分隔符解析.

17.9.4 为何护栏拒绝统一话术

`_GUARD_REFUSAL` 是固定的"很抱歉, 该操作目前无法为您办理", 不告诉用户**为什么**拒绝:

- 安全: 不暴露内部白名单 / 金额上限细节 (攻击者能针对性绕过)
- UX 一致: 所有拒绝场景统一话术, 用户不会困惑"为什么上次能这次不能" (实际原因: 角色不对, 但用户不需要知道)
- 合规可追溯: detail 仍写真实 reason, 内部审计可查

17.9.5 为何 audit_decision 与 `_execute_and_audit` 分离

维度	<code>tool.{name}</code> (执行)	<code>tool_confirm.{decision}</code> (决策)
触发时机	工具已执行	工具未执行, 仅决策
detail	<code>arguments + result + is_error</code>	<code>tool + decision</code> (无 arguments 脱敏副本)
status_code	200/500	200
业务含义	"工具调用结果"	"用户对工具的确认决策"

分离原因: 合规审计需独立查"哪些敏感操作被用户在何时同意/拒绝", 与"工具实际执行结果"是不同维度. 合并会丢失"客户拒绝但工具未执行"的关键决策记录.

17.9.6 为何用 `is_sensitive` 注解而非工具名白名单

Java MCP Server 在 `@Tool` 注解上标 `destructiveHint = true` (Spring AI), Python 端 `MCPToolClient.is_sensitive(name)` 查询此元数据. 优势:

- 敏感标记与工具实现同源 (Java 端), 不会出现 Python 白名单与 Java 实现不一致
- 新增敏感工具时只需 Java 端加注解, Python 端无改动
- 业务方改白名单**不影响**敏感标记 (正交设计)

17.10 实战案例: 客户调额 5 万

1. 客户输入: "我白金卡, 想把临时额度调到 5 万"
2. bot_agent 路由: LIMIT_QUERY 意图, 置信度 0.95
3. 渐进式裁剪: select_tools_for_intent 返回 4 个 limit_query 工具 (含 adjust_temp_credit_limit)
4. LLM 决定: 调 adjust_temp_credit_limit, args= {target_limit: 50000, card_type: "platinum"}
5. 护栏: - _check_authorization: 角色 customer, allowlist 空 → 放行 - _check_amount: adjust_temp_credit_limit 在 tool_amount_limits 中, target_limit=50000 < 50000 上限 → 放行
6. 敏感判定: is_sensitive("adjust_temp_credit_limit") → True → 写 PendingAction
7. Bot 回复: "您确认要办理「临时额度调整」吗?回复『确认』继续办理,回复『取消』放弃."
8. 客户回复: "确认"
9. execute_confirmed_action: 二次护栏校验 (防参数篡改) → 调 MCP → MCP 返回 "调额成功" → 脱敏 → 审计 → 续跑 LLM → 最终答复
10. 审计记录:
 - tool_confirm.pending (1)
 - tool_confirm.confirm (1)
 - tool.adjust_temp_credit_limit status=200 (1)
 - TOOL_CALLS{tool="adjust_temp_credit_limit", status="success"} ++

17.11 实战案例: 客户调额 1 亿 (护栏拒绝)

1. 客户输入: "我要把额度调到 1 亿"
2. bot_agent 路由: LIMIT_QUERY 意图
3. LLM 调: adjust_temp_credit_limit, args= {target_limit: 100000000}
4. 护栏: - _check_authorization: allowlist 空 → 放行 - _check_amount: target_limit=100000000 > 上限 50000 → 拒绝, code="amount_exceeded"
5. Bot 回复: "很抱歉, 该操作目前无法为您办理. 如需帮助, 我可以为您转接人工客服." (不告诉客户"金额超限")
6. 审计: - TOOL_GUARD_DENIALS{tool="adjust_temp_credit_limit", reason="amount_exceeded"} ++ - audit_log: action= tool.adjust_temp_credit_limit, status=403, detail= {arguments: 脱敏后, denied: "工具 adjust_temp_credit_limit 入参 target_limit=100000000 超过上限 50000", guard: "amount_exceeded"}

17.12 监控与可观测性

指标	PromQL 示例	业务含义
工具调用成功率	rate(tool_calls_total{status="success"}[5m]) / rate(tool_calls_total[5m])	MCP 整体健康度, < 95% 告警
调额拒绝率	rate(tool_guard_denials_total{reason="amount_exceeded"}[5m]) / rate(tool_calls_total[5m])	异常调额 (可能合规问题)
5 态确认漏斗	pending → confirm 转化率 = rate(confirm) / rate(pending)	客户对敏感操作的接受度, 转化 < 30% 说明 UI 不友好
工具调用 P99 延迟	histogram_quantile(0.99, tool_call_duration_seconds_bucket)	MCP 后端性能
工具循环超限 RuntimeError 次数	(需在 bot_agent 降级链加 counter)	LLM 死循环预警

17.13 延伸阅读

- 第 7 章 MCP 工具集成: 22 工具分类 + Higress 网关集成
- 第 11 章 安全合规: PBKDF2 / JWT / Aho-Corasick 敏感词
- 第 3 章 Bot 自助问答: 工具调用在 Bot 主链路中的位置
- 第 5 章 RAG 检索全链路: 工具调用与 RAG 的协同
- 附录 A.4.4 / A.4.5 / A.4.6 / A.4.7: 工具调用 + 护栏 + 指标 + PII 脱敏术语速查

附录 A 术语表

附录 A: 术语表

本术语表是本书其他章节的统一词汇表. 任何术语出现时, 含义以本表为准.

A.1 业务领域术语

术语	英文	含义
灵智	Lumio	项目名, 银行信用卡智能客服平台.
自助问答	Bot Self-Service	客户通过文本/语音直接与 AI 对话, 不经坐席. :8000.
坐席辅助	Agent Assist	坐席与客户通话时, AI 实时推送话术/知识/合规提醒. :8001.
转人工	Transfer to Agent	Bot 无法处理时, 将会话升级到人工坐席.
通话小结	Call Summary	通话结束后 AI 自动生成的结构化小结 (JSON).
坐席工号	Agent ID	坐席唯一标识, WebSocket 连接绑 agent_id.
会话 ID	Session ID	一次客户-坐席/客户-Bot 对话的唯一标识 (UUID v7).

A.2 错误码段

完整异常层次见 [第 8 章 错误处理](#).

段	含义	HTTP 默认
1xxx	认证授权 (AuthenticationError 1001 / AuthorizationError 1003)	401/403
2xxx	输入错误 (IntentUnrecognizedError 2001 / DocumentFormatError 2010)	400
3xxx	业务错误 (SessionNotFoundError 3004 / InvalidTransitionError 3005)	422
4xxx	外部依赖错误 (LLMTimeoutError 4001 / EmbeddingServiceError 4005)	502
5xxx	系统错误 (ServiceOverloadedError 5002 / OrchestrationTimeoutError 5004)	500/503

A.3 会话状态机

完整状态机见 [第 6 章 会话状态机](#).

名称	含义
SessionPhase	顶层阶段: BOT / AGENT / ENDED / legacy
SessionSubPhase	子状态 (7 个): BOT_ACTIVE, AG_QUEUED, AG_ASSIGNED, AG_ACTIVE, AG_ON_HOLD, AG_REVIEWING, ENDED
VALID_TRANSITIONS	状态转换白名单 (Lua CAS 校验)
CAS (Compare-And-Swap)	原子比较并设置, Redis Lua 脚本实现 (session.py:67)

名称	含义
pending_action	等待客户确认的工具调用 (5 态: pending/confirm/cancel/unclear/expired)

A.4 Bot Agent 术语

名称	含义
LumioAgent	Bot 主入口类, 6 步决策树 (bot_agent.py:92)
IntentLabel	意图标签 (10 个): FAQ 常见问题 / BILL_QUERY 账单查询 / TRANSACTION_QUERY 交易查询 / LIMIT_QUERY 额度查询 / INSTALLMENT_INQUIRY 分期咨询 / REWARD_QUERY 积分查询 / CARD_LOSS 挂失 / COMPLAINT 投诉 / TRANSFER_AGENT 转人工 / CHITCHAT 闲聊
SlotTracker	槽位追踪, 跨轮持久化到 Redis (lumio:slot:{session_id}). 槽位 = 完成某业务意图所需的关键信息项, 如分期要"金额 + 期数"; 必填槽缺失时 Bot 逐轮追问补齐
consumer_loop	Redis Stream 消费者, 15s 一次指标采集 (router.py:732)
session_worker	per-session 串行消费协程 (router.py:289)
dispatch_message	按 session_id 路由到 per-session Queue (router.py:270)
claim_stale	XAUTOCLAIM 兜底, 60s 超时 + 3 次重试转死信 (router.py:204)
_FAST_REPLIES	Semaphore 满载时紧急话术模板 (router.py:92)
PEL (Pending Entries List)	Redis Stream 已读未确认消息, XAUTOCLAIM 接管

A.4.1 上下文工程术语 (第 15 章)

完整上下文工程见 [第 15 章 上下文工程](#).

名称	含义
3 层上下文 (Layer 1/2/3)	Layer 1 结构化会话记忆 (注入 system_prompt 顶部, 永不裁剪) / Layer 2 近期对话历史 (token 预算裁剪) / Layer 3 RAG 检索上下文 (调用方传入)
_build_session_memory	bot_agent.py:725–779, 5 段拼接: 对话摘要/客户画像/已知实体/意图历史/当前意图
_load_history	bot_agent.py:583–629, LIFO 累加 + 关键轮次豁免 + 字符类 token 估算
_estimate_tokens	bot_agent.py:61–74, CJK 系数 0.55 / 拉丁 0.3 / 其他 0.8 + 4 消息格式开销
_IMPORTANT_KEYWORDS	bot_agent.py:79–85, 17 个永远不被裁剪的关键词 (投诉/银保监/盗刷/挂失/转人工等)
_ensure_summary	bot_agent.py:631–723, 增量对话摘要, last_summarized_turn_id 精确追踪
MaxContextTokens	LLM 上下文总预算, 默认 4096
ReservedTokens	预留回答空间, 默认 2048
TokenBudget	历史可用 token = max(MaxContext – Reserved, 1024)
LastSummarizedTurnID	已摘要轮次的 ID, 用于增量摘要追踪, 防 LTRIM 漂移
会话摘要 (conversation_summary)	被裁剪轮次的浓缩, ≤ 500 字, 写回 SessionState
fire-and-forget 摘要	asyncio.create_task 异步触发, 不阻塞用户请求

A.4.2 客户记忆术语 (第 16 章)

完整客户记忆见 [第 16 章 客户记忆与知识图谱](#).

名称	含义
learn_customer_profile	customer_memory.py:52-128, SQL string_agg 聚合 90 天历史对话
apply_learned_profile	customer_memory.py:131-172, CAS patch 写入 SessionState, 不覆盖已显式声明
_CARD_TYPE_PATTERNS	customer_memory.py:30-35, 4 类卡种正则 (platinum/diamond/gold/standard)
_VIP_SIGNALS	customer_memory.py:38-42, 3 档 VIP (private_banking=5 / wealth_management=4 / vip=3), max-score 评分
_RISK_SIGNALS	customer_memory.py:45-49, 风险评分累加, R1~R4 阈值
R1~R4 风险等级	保守 / 中性偏保守 / 偏高 / 激进
VIP 评分机制	max-score wins, 非累加; 默认 "普通"
LookbackDays	客户画像学习窗口, 默认 90 天
string_agg	PostgreSQL 聚合函数, 把多行 text 字段合并为单字符串
「不覆盖已显式声明」	仅当 state.vip_level == "普通" 或为空时才 patch
VIP 默认值 = "普通"	中文默认值, 与 SessionState.vip_level 字段对齐
writer="customer_memory:learn"	CAS patch 审计标签, 用于追溯学习操作

A.4.3 知识图谱术语 (第 16 章)

完整知识图谱见 [第 16 章 客户记忆与知识图谱](#).

名称	含义
_ENTITY_GRAPH	knowledge_graph.py:21-47, 5 实体 × 3 关系 (信用卡/账单/额度/分期/挂失)
8 种关系谓词	has_type / has_feature / has_method / has_cycle / has_factor / has_step / has_fee / related_to
query_entity_relations	knowledge_graph.py:50-77, OR 命中: 实体名 in 查询文本 OR entity 参数与实体名互相包含
enrich_retrieval_context	knowledge_graph.py:80-104, Markdown ## 知识图谱补充信息: 前缀追加
空 entity 防御	bool(entity) and (entity in entity_name or entity_name in entity), 避免空串命中所有实体
Neo4j 切换路径	_ENTITY_GRAPH 改为 client wrapper, query_entity_relations 改 Cypher, 接口签名保持稳定
仅 knowledge_agent 分支	bot_agent.py:213-216, business/fallback/tool 分支不调 KG

A.4.4 工具调用术语 (第 17 章)

完整工具调用见 [第 17 章 工具调用与确认状态机](#).

名称	含义
Progressive Disclosure (PD)	tool_selection.py, 按意图+置信度裁剪工具子集, 默认关 (零回归)
progressive_disclosure_enabled	开关, 默认 False, 关闭时返回 None 暴露全量

名称	含义
<code>pd_confidence_threshold</code>	置信度阈值, 默认 0.7, 低于则不裁剪
<code>intent_tool_map</code>	5 意图 → 17 工具映射表 (config.py:417-435)
<code>TOOL_INTENTS</code>	tool_selection.py:24-32, 5 工具意图白名单 (BILL/TRANSACTION/LIMIT/INSTALLMENT/REWARD)
<code>select_tools_for_intent</code>	tool_selection.py:35-61, 3 重零回归保险纯函数
<code>ToolCallingExecutor</code>	tool_executor.py:104-216, LLM 工具循环 + 确认状态机
<code>_run_loop</code>	tool_executor.py:220-280, 4 分支: 无 tool_call / 护栏拒绝 / 敏感 pending / 非敏感执行
<code>max_tool_iterations</code>	工具循环上限, 默认 5, 超限抛 RuntimeError
<code>detect_confirmation</code>	tool_executor.py:75-87, cancel 关键词优先于 confirm
<code>_CANCEL_KEYWORDS</code>	13 个: 取消/不用/不要/不办/不确认/不同意/不可以/算了/放弃/别/停/no/cancel
<code>_CONFIRM_KEYWORDS</code>	10 个: 确认/确定/是的/好的/可以/继续/同意/办理/ok/yes
<code>PendingAction</code>	models.py:216-229, 7 字段待确认工具 (tool_name/arguments/tool_call_id/confirm_prompt/created_at/expires_at/trace_id)
<code>confirmation_ttl_seconds</code>	PendingAction TTL, 默认 300 秒 (5 分钟)
5 态确认状态机	pending / confirm / cancel / unclear / expired
惰性过期 (lazy expiration)	无后台清扫, 每次进入 <code>_handle_pending_action</code> 时比 <code>expires_at</code>
<code>audit_decision</code>	tool_executor.py:391-408, 单独审计 confirm/cancel/expired, 区别于执行结果审计
<code>is_sensitive</code>	MCPClient 方法, 按 <code>destructiveHint</code> 注解判断工具是否敏感

A.4.5 工具护栏术语 (第 17 章)

名称	含义
<code>ToolGuard</code>	tool_guard.py:35-101, 纯同步无 I/O, 角色授权 + 金额上限
<code>active</code> 属性	tool_guard.py:41-44, allowlist 或 limits 任一非空则激活
<code>tool_role_allowlist</code>	角色→工具白名单, 保守模式 (未登记=无权限)
<code>tool_amount_limits</code>	工具→金额上限, 配合 <code>amount_arg_keys</code> 检查入参
<code>amount_arg_keys</code>	默认 ["amount", "target_limit", "target_amount", "limit"]
<code>_coerce_number</code>	tool_guard.py:90-101, bool 不参与 (避免 True==1), 字符串 strip 后强转
<code>role_denied</code>	护栏拒绝码: 角色未在白名单
<code>amount_exceeded</code>	护栏拒绝码: 金额超过上限
Higress 网关 vs ToolGuard	网关负责"谁能进" (粗粒度 token 级) + 流量治理; ToolGuard 负责"特定角色能调哪些工具 + 业务阈值" (细粒度业务级)
纵深防御	网关层 + Python 编排侧双层防护, 不冗余, ToolGuard active=False 时跳过

A.4.6 工具调用指标 (第 17 章)

名称	含义
<code>TOOL_CALLS</code>	Counter, labels: tool, status (success/error), 写于 tool_executor.py:298/309
<code>TOOL_CONFIRMATIONS</code>	Counter, labels: decision (pending/confirm/cancel/unclear/expired), 5 态漏斗分析

名称	含义
TOOL_GUARD_DENIALS	Counter, labels: tool, reason (role_denied/amount_exceeded), 双维度交叉定位

A.4.7 工具调用 PII 脱敏注入点 (第 17 章)

位置	行号	脱敏对象
入参脱敏	tool_executor.py:291	tool_call.arguments → 审计 detail.arguments
出参脱敏	tool_executor.py:295	MCP 返回 content → 回喂 LLM + 审计
护栏拒绝脱敏	tool_executor.py:379	拒绝时再脱敏 arguments
异常脱敏	tool_executor.py:304	异常时脱敏 arguments
顺序敏感	pii.py:63–76	身份证/银行卡 (16–19 位) 优先于手机号 (11 位), 避免 11 位数字被误判

A.5 坐席辅助引擎术语

完整 5 阶段编排见 [第 4 章 坐席辅助](#).

名称	含义
D1	服务评估器 (decide_service) — 何时不推营销
D2	营销评估器 (decide_marketing) — 何时推营销
D3	风控评估器 (decide_risk) — 何时必推风控告警 (永不下线)
E1	AI 执行器 (ai_executor) — 话术 + RAG + 合规三路并行
E2	营销执行器 (marketing_executor) — 500ms 延迟避开服务期
E3	风控执行器 (risk / alert_engine) — 6 条种子规则
Scene	4 类场景: URGENT / INQUIRY / SALES / GENERAL
FusionType	仲裁融合: BLOCK (阻断) / WARN (告警) / PASS (通过)
PushTracker	per-session 推送追踪 (last_push_at + feedback_history + min_interval)
service_suppresses_marketing	服务激活时设置 d2_suppress_rounds=N 冷却
H2 (3s 延迟确认)	反馈 Redis 缓冲 3s 后提交, 期间可撤销

A.6 RAG 检索术语

完整 RAG 链路见 [第 5 章 RAG](#) + [第 9 章 RAG 摄入](#).

名称	含义
BM25	经典词频–逆文档频率检索, ES 8.19 + IK 中文分词
IVF_FLAT	Milvus 索引类型, 倒排文件 + 精确距离, nlist=128
nprobe	IVF 查询时探针数, 默认 16 (查 16 个聚类桶)
RRF (Reciprocal Rank Fusion)	倒数排名融合, 公式 $1/(k+rank)$, k=60

名称	含义
Parent-Child 分块	检索小 chunk (可嵌入), 生成拼父块 (parent_chunk_id 回填)
Dual-Write	ES + Milvus 双写, Milvus 失败回滚 ES
Ollama	本地 LLM/嵌入推理服务, HTTP API
TEI (Text Embeddings Inference)	HuggingFace 嵌入推理服务, 批大小 128
bge-large-zh-v1.5	智源中文嵌入模型, 1024 维
reranker	重排序器, 候选 top_k*2 精排到 top_k
合规字段硬注入	检索时自动加 approval_status=PUBLISHED + is_current_version=true + effective_date<=today

A.7 MCP 工具集成术语

完整 MCP 集成见 [第 7 章 MCP 工具](#).

名称	含义
MCP (Model Context Protocol)	Anthropic 提出, AI Agent 调用工具的标准协议
streamable-http	MCP 传输方式 (替代旧 SSE)
destructiveHint	工具注解, true 表示敏感 (写操作) 自动加入白名单
Higress	AI 网关, 集中鉴权/限流/脱敏/审计
Nacos	注册中心 + 配置中心, MCP Server 注册到 lumio-mcp-server 服务
lumio-mcp-server	Java MCP Server, Spring Boot 3.4.5 + 22 工具, 端口 8090
tool_guard	工具护栏, role 授权 + 金额限额
destructiveHint 自动标记	Spring AI 注解的写操作自动合并到 sensitive 白名单

A.8 可观测性术语

完整可观测性见 [第 10 章 可观测性](#).

名称	含义
Prometheus	指标采集系统, 9 个 scrape job (bot/assist/redis/pg/es/milvus/kafka/mcp/prom)
OTLP	OpenTelemetry Protocol, trace 上报协议
OTel (OpenTelemetry)	跨语言追踪/指标标准
W3C traceparent	跨服务 trace 上下文传递 HTTP 头
HTTPXClientInstrumentor	跨服务 trace 自动注入 traceparent
ParentBasedTraceIdRatioSampler	有上游跟随上游, 无上游按 ratio 采样
Resource 属性	service.name / namespace / version / environment
PrometheusMiddleware	FastAPI 自动打点中间件, 排除 /metrics /health /favicon.ico
JSONFormatter	结构化日志格式化器, 含 trace_id/span_id 自动注入

名称	含义
PIIMaskFilter	日志输出侧 PII 脱敏 (手机/身份证/银行卡/邮箱)
Aho-Corasick	多模式字符串匹配算法, $O(n)$ 与词库大小无关, 10K 词 < 100ms
audit_log	审计表, append-only, 4 索引 (timestamp/actor/action/target)

A.9 安全合规术语

完整安全合规见 [第 11 章 安全合规](#).

名称	含义
JWT (JSON Web Token)	HS256 单 secret 签名, leeway=30s 时钟容差
RBAC (Role-Based Access Control)	4 角色: customer / agent / admin / service
PBKDF2-HMAC-SHA256	密码哈希算法, 600000 次迭代 (OWASP 2023)
NFKC	Unicode 归一化形式, 全角 → 半角, 大小写归一
PII (Personally Identifiable Information)	个人可识别信息: 手机/身份证/银行卡/邮箱
Idempotency Key	幂等键参数, 避免重试导致重复受理
loopback	127.0.0.1 / localhost, dev bypass 限定本地访问
destructiveHint	工具敏感标记, 见 A.7
allowed_roles	文档可见角色白名单 (合规字段)
regulatory_tags	监管标签 (合规字段)

A.10 数据层术语

完整数据层见 [第 12 章 数据层](#).

名称	含义
PostgreSQL 16	主数据库, 19 张表, 12 个 Alembic 迁移
Redis 7.2	会话/Stream/Pub-Sub/缓存, 15+ key prefix
Elasticsearch 8.19	BM25 全文检索, IK 中文分词 (自构建镜像)
Milvus 2.4	向量数据库, IVF_FLAT COSINE, 1024 维
MinIO	S3 兼容对象存储, 1 bucket lumio-docs
Kafka 3.7 (KRaft)	异步事件流, 单节点 ID=1, 3 topic 预留
pgcrypto	PostgreSQL 扩展, UUID v7 生成依赖
to_tsvector	PG 全文搜索函数, ('simple', content)
UNIQUE PARTIAL INDEX	WHERE is_current_version = true AND is_deleted = false
Alembic	Python 数据库迁移工具, 12 个版本脚本

A.11 部署与基础设施

完整部署见 [第 13 章 部署](#).

名称	含义
Docker Compose	24 服务编排, deploy/docker-compose.yml
KRaft 模式	Kafka 3.4+ 无 ZooKeeper 模式, 单节点即可
Jaeger 1.57	追踪后端, OTLP 4318 端口
Grafana 10.4	监控可视化, 3 套 dashboard
Prometheus 2.50	指标采集, 9 个 scrape job, 6 告警规则
HPA (Horizontal Pod Autoscaler)	K8s 水平自动扩缩, 2-6 replica, 70% CPU
multi-stage Dockerfile	builder 装 poetry + runtime 仅含 .venv, 减少镜像体积
livenessProbe / readinessProbe	存活/就绪探针, 探测 /api/health/live /ready
WebSocket Upgrade	Nginx 反代 WebSocket, 3600s timeout
opt-in profile	make gateway-up 手动拉起 nacos + higress, 默认不启用

A.12 测试术语

完整测试见 [第 14 章 测试策略](#).

名称	含义
pytest-asyncio	asyncio_mode = "auto", 740+ 个 async 测试
httpx.AsyncClient	异步 HTTP 客户端, 5 个 e2e fixture
subprocess.Popen	uvicorn 子进程拉起, 端口 8765/8766 避免冲突
e2e (end-to-end)	端到端测试, 真实中间件 + 真实子进程
覆盖率门槛	80% (pyproject 锁, CI 同)
mypy advisory	类型错误不阻塞 CI, 但新增错误立刻可见
pre-commit	8 步钩子: trailing-whitespace / eof / yaml / toml / large-files 500KB / merge-conflict / private-key / ruff
pytest.skip	中间件不可用时跳过, 不算失败
35 errors	已知失败用例: 启动超时 + 无 Docker skip

A.13 其他通用术语

名称	含义
Pydantic v2	数据验证库, BaseModel / Field / validator
Pydantic Settings	配置管理扩展, BaseSettings + env 读取
PydanticAI	类型安全 AI Agent 框架 (本项目 未使用 , 注释明确)
FastAPI	异步 Web 框架, 0.115+
asyncio.gather	并发执行多个协程 (替代 Temporal)

名称	含义
Temporal	工作流编排引擎 (本项目未采用, 用 <code>asyncio.gather</code> 替代)
Circuit Breaker	熔断器, 三态 CLOSED / OPEN / HALF_OPEN
DegradationManager	4 级降级 NORMAL / DEGRADED / FALLBACK
slug	文档友好 URL 段, 例如 <code>credit-card-policy-v2</code>
CRUD	Create / Read / Update / Delete
LRU (Least Recently Used)	最近最少使用, <code>@lru_cache</code> 单例
ContextVar	上下文变量, <code>asyncio</code> 任务间传递 (<code>trace_id</code> / <code>request_id</code>)

A.14 缩略语速查

缩写	全称
RAG	Retrieval-Augmented Generation
LLM	Large Language Model
MCP	Model Context Protocol
AI	Artificial Intelligence
ML	Machine Learning
RTO	Recovery Time Objective
RPO	Recovery Point Objective
SLO	Service Level Objective
SLA	Service Level Agreement
OTLP	OpenTelemetry Protocol
OTel	OpenTelemetry
PEL	Pending Entries List (Redis Stream)
RRF	Reciprocal Rank Fusion
JWT	JSON Web Token
PBKDF2	Password-Based Key Derivation Function 2
NFKC	Normalization Form KC (Compatibility Composition)
PBKAC	Password-Based Key Agreement Protocol
HPA	Horizontal Pod Autoscaler
CSI	Container Storage Interface
RBAC	Role-Based Access Control
ACL	Access Control List
CSR	Client Secret Rotation
DBA	Database Administrator
DBE	Database Engineer
PII	Personally Identifiable Information

缩写	全称
KV	Key-Value
ACID	Atomicity, Consistency, Isolation, Durability
BASE	Basically Available, Soft state, Eventual consistency

维护说明: 任何新术语, 在首次出现章节加粗, 然后追加到本表对应分类. 保持与代码命名一致 (大写下划线 vs 小写蛇形).

附录 C 故障排查

附录 C: 常见问题排查

本附录汇总 Lumio 22 个最常见生产/开发问题, 按 6 大类组织. 每个问题给「症状 / 原因 / 排查 / 修复」4 步.

C.1 LLM 链路故障

C.1.1 LLM 502 错误

症状: Bot/Assist 回答 502, 客户端显示"系统繁忙".

原因: – LLM 服务 (Ollama/OpenAI) 不可达 – LLM 熔断器 (LLMCircuitBreaker) 已 OPEN – API key 失效

排查:

```
# 1. 检查 LLM 服务
curl http://localhost:11434/v1/models

# 2. 检查熔断器状态 (通过指标)
curl http://localhost:8000/metrics | grep lumio_llm_circuit_state

# 3. 检查 API key
echo $LLM_API_KEY
```

修复: – 服务不可达 → `systemctl restart ollama` 或检查防火墙 – 熔断器 OPEN → 等待 30s 自动转 HALF_OPEN, 或手动 reset – API key 失效 → 更新 `LLM_API_KEY` env, 重启服务

C.1.2 LLM 响应慢 (>5s)

症状: P99 延迟 >5s, Grafana `LLMLatencyP99High` 告警.

原因: – Ollama CPU 推理 (无 GPU) – prompt 过长 (历史 + chunks 超过 4K token) – LLM 服务过载

排查:

```
# 1. 看 Grafana llm_call_duration_seconds p99
# 2. 检查 prompt 长度
poetry run python -c "from lumio.shared.llm import LLMClient; print(LLMClient().max_tokens)"
```

修复: – 启用 GPU 加速或换更小模型 (qwen2.5:0.5b) – 减少 history 轮数 (默认 20, 改 10) – 启用 LLM 缓存 (相同 query 命中)

C.1.3 LLM 幻觉 (编造内容)

症状: 客户投诉"AI 回答了不存在的政策".

原因: – RAG 检索召回空, LLM 自由发挥 – prompt 缺少"不要编造"约束

排查:

```
# 检查检索召回数
curl -X POST http://localhost:8000/api/kb/retrieve \
-H "Content-Type: application/json" \
-d '{"query": "信用卡年费", "top_k": 5}'
```

修复: – 召回 0 → 检查文档审批流是否到 PUBLISHED – prompt 加约束: "如果不知道, 请回答'请咨询人工坐席'"

C.2 嵌入 / RAG 链路故障

C.2.1 EmbeddingCircuitBreaker OPEN

症状: RAG 检索走 bm25_only, 召回率下降 50%.

原因: – TEI 服务 (Text Embeddings Inference) 不可达 – 连续 3 次失败触发熔断

排查:

```
# 1. 检查 TEI 服务
curl http://localhost:8081/health
# 2. 看熔断器状态
curl http://localhost:8000/metrics | grep embedding_breaker
```

修复: – TEI 不可达 → `systemctl restart tei` – 自动恢复: 30s 后转 HALF_OPEN, 2 次成功关熔断

C.2.2 RAG 召回空 (retrieve 返 [])

症状: 客户问知识类问题, AI 答"请咨询人工".

原因: – ES + Milvus 双挂 (罕见) – 文档还未 PUBLISHED – effective_date 还没到

排查:

```
# 1. 单独测 ES
curl http://localhost:9200/lumio_kb_chunks/_count
# 2. 单独测 Milvus
poetry run python -c "from pymilvus import connections; connections.connect(host='localhost', port='19530'); print(connections)
# 3. 查某文档状态
psql -U lumio -d lumio -c "SELECT id, title, approval_status FROM kb_document WHERE title LIKE '%年费%';"
```

修复: – ES 挂 → 重启, 检查磁盘空间 – Milvus 挂 → 检查 etcd + minio 是否健康 – 文档未发布 → 走审批流到 PUBLISHED

C.2.3 RAG NameError

症状: 嵌入失败时, RAG 整个崩溃, 5% 概率生产事故.

原因: 旧版 `for t in (bm25_task, vector_task)` 引用 `vector_task` 时未赋值.

修复: 当前版本已修, 不会发生. 若遇到, 升级到最新代码.

C.3 中间件故障

C.3.1 Redis 不可达

症状: Bot 502 + Assist 502, 所有 Stream/Pub-Sub 失败.

原因: – Redis 服务挂了 – 网络问题

排查:

```
redis-cli -h localhost -p 6379 ping
```

修复: `systemctl restart redis` 或检查 docker compose 中 redis 容器.

C.3.2 PostgreSQL 迁移失败

症状: 启动期 `alembic upgrade head` 失败, 服务无法启动.

原因: – 迁移文件冲突 – 数据库 schema 不一致

排查:

```
cd agent && poetry run alembic current
poetry run alembic history
```

修复: – 单步回滚: `poetry run alembic downgrade -1` – 强制到 head: `poetry run alembic upgrade head --sql > migrate.sql` 检查 SQL

C.3.3 Kafka 不可达

症状: Kafka 相关功能异常 (当前项目仅 verify 阶段使用, 业务无影响).

修复: 临时方案, 业务路径不依赖 Kafka, 可忽略. 长期方案见 Kafka 启用 sprint.

C.3.4 MinIO 不可达

症状: 文件上传 502, 但检索仍可用.

原因: – MinIO 服务挂了 – bucket `lumio-docs` 不存在

修复: – 重启 MinIO – 重建 bucket: `cd agent && poetry run python scripts/init_minio.py`

C.3.5 ES / Milvus 启动顺序错乱

症状: ES/Milvus 启动后, 业务服务启动失败.

原因: Milvus 严格依赖 etcd + minio 健康后才拉起 (depends_on service_healthy), 若 etcd 慢, Milvus 启动失败.

修复:

```
docker compose -f deploy/docker-compose.yml ps
# 等所有 healthy 后再启动业务
docker compose -f deploy/docker-compose.yml logs etcd
```

C.4 部署与配置

C.4.1 端口冲突

症状: `Bind: address already in use`.

排查:

```
lsof -i :8000 # Bot
lsof -i :8001 # Assist
lsof -i :5432 # PG
```

修复: – Bot/Assist 冲突 → 改 `LUMIO_BOT__PORT` / `LUMIO_ASSIST__PORT` env (虽然当前没暴露) – PG/Redis 冲突 → 改 `POSTGRES_PORT` / `REDIS_PORT`

C.4.2 启动顺序错乱

症状: 业务服务启动期 `init_db` 失败, 报 `connection refused`.

原因: 中间件未就绪.

修复:

```
# 1. 等中间件 healthy
make verify
# 2. 再启动业务
make dev
```

C.4.3 demo 启动失败

症状: make demo 拉起后, 业务 API 502.

排查:

```
docker compose -f deploy/docker-compose.demo.yml logs -f lumio-bot
```

修复: 看具体错误, 通常是中间件未就绪, 等 30s 重试.

C.5 安全合规

C.5.1 JWT 解码失败 (1001 AuthenticationError)

症状: API 返 401, 客户端提示"未登录".

原因: – token 过期 (默认 30 分钟) – token 签名错误 (secret 不一致) – dev bypass 没开

排查:

```
# 解码 token 看 payload
poetry run python -c "import jwt; print(jwt.decode('eyJ...', 'lumio-dev-secret-change-in-production', algorithms=['HS256']))"
```

修复: – 过期 → 重新登录 – secret 不一致 → 确认 LUMIO_JWT_SECRET 与签发端一致 – dev bypass → LUMIO_DEV_AUTH_BYPASS=true + bind localhost

C.5.2 RBAC 拒绝 (1003 AuthorizationError)

症状: API 返 403, 客户端提示"权限不足".

原因: 角色不匹配, 例如 customer 角色访问 admin 端点.

修复: – 用对角色登录 – 业务层判断角色与端点匹配

C.5.3 限流触发

症状: API 返 429, 客户端频繁提示"操作频繁".

原因: – 全局 60/min – 聊天 30/min

排查: LUMIO_RATE_LIMIT_CHAT=30/minute 是默认值.

修复: – 提升限流 (生产环境, 改 60/min) – 客户端实现重试 + 指数退避

C.5.4 PII 脱敏规则违反

症状: 日志中不应出现手机号, 但出现了.

原因: – 业务代码直接拼接用户输入到日志, 没走 PIIMaskFilter – 日志输出到第三方 (Loki/Sentry) 时, 第三方没做脱敏

修复: – 用 `logger.info("user input", extra={"text": mask_pii(text)})` – Loki 侧用 pipeline 做二次脱敏

C.5.5 健康检查暴露敏感信息

症状: /api/health/ready 返 `{"redis_unreachable": "ConnectionRefusedError: 192.168.1.10:6379"}`.

原因: 旧版 `str(e)[:100]` 直接吐客户端.

修复: 当前版本只返 7 个分类码 (redis_unreachable / postgres_unreachable / ...). 若遇到, 升级到最新代码.

C.6 可观测性

C.6.1 跨服务 trace 断裂

症状: Jaeger 里 Python → Java 的 trace 不连.

原因: – HTTPClientInstrumentor 没装 – 跨服务没传 traceparent 头

排查:

```
# 1. 检查 Python 探针
poetry run python -c "from opentelemetry.instrumentation.httpx import HTTPClientInstrumentor; HTTPClientInstrumentor().instrument()"
# 2. 检查 Java 端
curl http://localhost:8090/actuator/metrics | grep tracing
```

修复: – Python: 确认 instrument_app(app, "lumio-bot") 调用 – Java: 确认 MANAGEMENT_TRACING_SAMPLING_PROBABILITY=0.1

C.6.2 指标缺失

症状: Grafana dashboard 某 panel 显示 "No data".

原因: – Prometheus 抓取失败 – 指标命名不一致 – 业务代码没打点

排查:

```
# 1. 查指标是否被采集
curl http://localhost:9090/api/v1/query?query=up
# 2. 业务端
curl http://localhost:8000/metrics | grep lumio_
```

修复: – Prometheus 配置: 检查 config/prometheus.yml scrape job – 业务端: 补打点 + 重启

C.6.3 Grafana dashboard 报错

症状: dashboard 加载失败, "Template variable not found".

原因: dashboard JSON 引用了不存在的 metric.

修复: 跑 make verify-observability 静态校验, 确认 dashboard 引用的指标都已在代码中定义.

C.7 会话状态机

C.7.1 会话状态非法转换 (3005 InvalidTransitionError)

症状: Bot 转换 phase 失败, 客户端返 422.

原因: 不在 VALID_TRANSITIONS 白名单.

排查:

```
# models.py:VALID_TRANSITIONS
print(VALID_TRANSITIONS)
```

修复: 业务代码确保转换合法, 否则抛 4xx 让客户端处理.

C.7.2 CAS Lua version 冲突

症状: 多个实例同时改同一会话, 偶发 StateConflictError.

原因: CAS 冲突, 正常现象.

修复: 框架自动重试 1 次, 通常透明. 频繁出现说明业务热点.

C.7.3 ZSET 超时未触发

症状: 客户 30 分钟无消息, 仍未自动结束.

原因: 5s 轮询进程崩了.

排查:

```
ps aux | grep session_timeout
```

修复: 重启服务, 检查 ZSET 是否有数据: `redis-cli ZRANGE lumio:session:timeouts 0 -1 WITHSCORES`.

C.8 MCP 工具

C.8.1 MCP 连接失败

症状: Bot `_handle_tool` 走 RAG 兜底, 工具不响应.

原因: - `MCP_ENABLED=false` (默认) - Higress 网关不可达 - Java MCP Server 启动失败

排查:

```
# 1. 检查 MCP 开关
echo $MCP_ENABLED
# 2. 直连 Java MCP
curl http://localhost:8090/mcp (streamable-http)
# 3. 看 mcp_connection_state 指标
curl http://localhost:8000/metrics | grep mcp_connection_state
```

修复: - 改 `MCP_ENABLED=true` - 重启 Higress + MCP Server

C.8.2 工具调用超时

症状: 调 `report_card_lost` 5s 超时, 客户没收到结果.

原因: - Java 端业务逻辑慢 - 网关超时设置过短

修复: - Java 端优化 mock 数据查询 - 网关超时改 10s (默认 10s, 调到 30s)

C.8.3 工具护栏拒绝 (`tool_guard_denials_total` 飙升)

症状: 客户想调额, 提示"金额超限".

原因: 金额 > 角色限额 (例如 customer 调临时额度限 5 万).

修复: - 业务侧: 调小金额 - 护栏侧: 调大限额 (需合规审批)

C.9 测试与 CI

C.9.1 35 errors 持续

症状: CI 跑 pytest 一直 35 failed.

原因: - `RecursionError` (test_retrieval 部分用例) - 中间件未起, e2e skip - 启动超时

修复: - 短期: 接受现实, 60% 覆盖率门槛 (历史) - 长期: 修掉 `RecursionError` 后, 提到 60% 门槛 (历史); 当前已到 80% 门槛

C.9.2 mypy 错误累积

症状: 171 → 200 个 mypy 错误, 还在涨.

原因: mypy advisory 模式不阻塞 CI, 错误自然累积.

修复: 按模块拆分, 指定 owner 季度集中收敛.

C.9.3 e2e 失败

症状: main 分支 e2e job 失败, 提示 /api/chat/send 返 5xx.

原因: – Demo 启动慢, 业务 API 没就绪 – 中间件 (Redis/PG) 容器没起

修复: 检查 `docker compose -f deploy/docker-compose.demo.yml ps`, 等全部 healthy.

C.10 性能调优

C.10.1 Bot P99 延迟高

排查: – Grafana `lumio_bot_response_duration_seconds` p99 – 拆解: 分类延迟 / RAG 延迟 / LLM 延迟

调优: – 分类慢 → 改用更小 LLM (qwen2.5:0.5b) – RAG 慢 → 检查 ES 索引 / Milvus nprobe – LLM 慢 → 启用 GPU

C.10.2 Assist 引擎超时 (>5s)

排查: – 看哪一阶段慢 (D1/D2/D3/E1/E2/E3) – 多数情况是 E1 (RAG + LLM 三路并行)

调优: – 调小 `ORCH_E1_SLA_SECONDS` (当前 3s) – 减少 E1 chunks 数量 (`top_k=5` → 3)

C.11 应急联系

类别	联系人	渠道
LLM/Ollama 故障	LLM 平台组	Slack #llm-platform
银行核心系统	银行业务组	Slack #bank-core
数据库	DBA 团队	Slack #dba
监控告警	SRE 团队	Slack #sre
安全事件	安全组	security@lumio.io

C.12 客服 Agent 场景 (第 15–17 章配套)

以下 8 个问题来自第 15–17 章的客服 Agent 能力层实战, 配套深挖文档 [第 15 章 上下文工程](#) / [第 16 章 客户记忆与知识图谱](#) / [第 17 章 工具调用与确认状态机](#).

C.12.1 客户说"Bot 不知道我是白金卡"

症状: VIP 客户来电, Bot 仍问"您是什么卡?", 体验差.

根因排查:

- 检查 `dialogue_log` 表是否真有该 `customer_id` 的 90 天数据: `sql SELECT COUNT(*) FROM dialogue_log WHERE customer_id = 'C12345' AND speaker = 'customer' AND created_at >= NOW() - INTERVAL '90 days';` – 返回 0: 客户是新客户或 `dialogue_log` 没写入 → 走 16.11 案例 – 返回 N>0: 走下一步
- 检查 `apply_learned_profile` 是否真应用: `bash grep "客户画像已应用" app.log | grep "C12345"` – 无输出: 学习失败, 检查 PG 是否可达 + `string_agg` 是否报错 – 有输出: 检查 `state.vip_level` 是否被显式设置覆盖
- 检查显式覆盖: 客户在当前会话说过"我是新卡" → `state.vip_level != "普通"` → 学习结果不覆盖. 这是 by-design 行为, 需客户语境化

修复: 见 [第 16.10 节 已知问题与改进项](#) – 缺 `ix_dialogue_log_customer_time` PARTIAL INDEX.

C.12.2 工具调用超过 5 轮 (RuntimeError)

症状: Bot 偶发 RuntimeError: 工具调用超过最大轮数 5, 用户看到 500.

根因排查:

1. 查看 LLM 是否死循环: 拉对应 session 的 audit_log, 看连续 5 个 tool.* 调用的 tool_name 序列 `sql SELECT created_at, action, detail FROM audit_log WHERE session_id = '...' AND action LIKE 'tool.%' ORDER BY created_at DESC LIMIT 10;`
2. 典型循环模式: - LLM 反复调 `query_credit_limit` 查同一额度 (LLM 觉得 "再查一次可能更新了") - LLM 调 `query_installment_offer` 后再调 `query_installment_status` 再调 `query_installment_offer` (幻觉)
3. 是否触发了 RAG 降级链: `_handle_tool` 失败时回落 `_handle_knowledge` 也会走 5 轮工具 (虽然本不会), 但 `bot_agent.py:375` 走 `knowledge_agent` → 重新拼 `system_prompt` + RAG, 不应该循环

修复: - 短期: 业务方加 LLM prompt "调同一个工具不要超过 2 次" - 中期: 把 `max_tool_iterations` 调到 3 + 加 LLM 提示 - 长期: 引入"工具调用计划" - LLM 一次性规划所有工具调用, 不循环

C.12.3 5 态确认状态机卡在 unclear

症状: 客户回复"嗯" / "哦" / "好" → Bot 一直追问"请明确回复确认或取消", 体验差.

根因排查:

1. 关键词词典覆盖不全 (`tool_executor.py:46-72`): - `_CONFIRM_KEYWORDS`: 确认/确定/是的/好的/可以/继续/同意/办理/ok/yes - 客户说"嗯" / "哦" / "行" / "中" / "妥" → 都不命中
2. 是否含敏感关键词被误判: 客户说"我不确认" → `cancel` 优先命中 (L83) → 正确
3. 大小写: 已 `.strip().lower()` 处理, 无此问题

修复: - 短期: 加关键词 "嗯" / "哦" / "行" / "中" / "妥" (确认类) - 中期: 用 LLM 二次判定 (50ms 延迟可接受) - 长期: 引入"模糊确认"识别 - 客户说"行吧"虽含"吧"但语义是 `confirm`

C.12.4 知识图谱注入失效 (无 ## 知识图谱补充信息 段)

症状: LLM 答客户问"信用卡额度"时没看到 KG 关系补充.

根因排查:

1. 是否在 `knowledge_agent` 分支: `bot_agent.py:213-216` 仅 `_handle_knowledge` 调用 KG - 业务类 (`_handle_business`) 调 MCP 工具, 不调 KG, 这是 by-design
2. RAG 是否成功: `bot_agent.py:213` `if context:` - RAG 失败时空 `context` 跳过 KG `python context = await self._retrieve(user_input)` `if context:` # ← 空 `context` 跳过 `context = enrich_retrieval_context(user_input, [context])`
3. 实体名是否匹配: `knowledge_graph.py:90` `if entity_name in query_text` - 5 实体是高频词 - 客户说"额度" → "信用卡" 不在 `query_text` → 只命中"额度"实体 - 客户说"信用卡" → 命中"信用卡"实体 - 客户说"提额" → 5 实体都不在 `query_text` (因"额度"不在"提额"中) → 0 命中

修复: 见 16.7.2 潜在改进 - `if entity_name in query_text or query_text in entity_name` 捕获"提额"→"额度"近义.

C.12.5 对话摘要没生成 (LLM 不可用)

症状: `_load_history` 触发 `_ensure_summary` 但摘要始终为空, `_build_session_memory` 没 [对话摘要] 段.

根因排查:

1. LLM 不可用: `bot_agent.py:684-685` `if llm_client is None: return` → 摘要静默跳过
2. LLM 调用 3s 超时: `bot_agent.py:693` `timeout=3.0` → `except Exception: return` (L695-697)
3. `last_summarized_turn_id` 错位: LTRIM 删了已摘要 turn, `split_idx` 始终 0 → 整段 trimmed 都摘要 → 慢
4. CAS patch 失败: `bot_agent.py:721` `logger.warning("对话摘要 CAS 写入失败")`

修复: - 检查 LLM 服务 (`make verify`) - 检查 `session_meta` 写权限 (Redis ping) - 长期: 摘要失败时记录到 `lumio:summary_failed:` {`session_id`} 计数, 累积 3 次触发告警

C.12.6 槽位追踪器意图切换时清空 (高优先级体验问题)

症状: 客户从"问账单"切换到"问积分", Bot 重新问"卡号后四位"等已收集过的信息.

根因排查 (bot_agent.py:799-802):

```
if raw:
    data = json.loads(raw)
    if data.get("intent") == intent.value: # ← 仅当意图不变才复用
        tracker = SlotTracker.from_dict(data)
    # 意图不同 → tracker 保持 None → 创建新的 (L807)
```

by-design 行为: 槽位是意图绑定的, 切换意图就重新追踪. 但客户期望"已收集的卡号不应该重复问".

修复方案: – 短期: entity_pool 跨意图保留, Bot 通过 entity_pool 自动填充 – 中期: 槽位追踪器改为"全局槽位"模式 (L65 `_ENTITY_TO_SLOT` 已有基础) – 长期: 槽位 + entity_pool 合并, 消除双层数据

C.12.7 渐进式暴露不生效 (开关/阈值/映射 三重检查)

症状: 配置了 `intent_tool_map` 也开了 `progressive_disclosure_enabled=True`, 但 LLM 仍收到 22 个工具.

根因排查 (按顺序):

1. `progressive_disclosure_enabled` 是否真为 True: 检查 `MCPSettings.__dict__` 或 Nacos 配置
2. `pd_confidence_threshold` 是否过高: 实际意图置信度 < 阈值 → 返回 None → 暴露全量 – 调小阈值 (0.5) 测试
3. `intent_tool_map` 是否覆盖了实际意图: `bill_query` 没配置 → 返回 None → 暴露全量 – 检查 `config.py:417-435` 配置
4. 意图分类是否正确: 实际分类是 `faq` → 不在 `TOOL_INTENTS` → 走 `knowledge_agent` → 不调工具
5. 是否有 `bot_agent` 路径问题: 业务类 (`_handle_business`) 直接调 MCP, 不经过 `select_tools_for_intent`, 所以全量工具仍可用

修复: 上述任一命中即按需调整.

C.12.8 护栏拒绝但用户无感知 (拒绝话术 + 审计)

症状: `TOOL_GUARD_DENIALS` 指标持续增长, 但用户报"Bot 不办事", 不知道发生了什么.

根因排查:

1. 拒绝话术统一: `tool_executor.py:41` `_GUARD_REFUSAL = "很抱歉, 该操作目前无法为您办理. 如需帮助, 我可以为您转接人工客服."` — 不告诉用户为什么拒绝
2. 审计补全: `tool_executor.py:378-388` 写 `status_code=403` 审计, 含 `decision.reason` (内部)
3. 用户视角无感: 设计上"安全优先, 不暴露内部信息" — 但客户以为"Bot 不行"

修复: – 短期: 给运营提供 `_GUARD_REFUSAL` 替代话术 (如"金额超过 XX 元上限, 建议前往柜台办理") – 中期: 区分 `role_denied` vs `amount_exceeded` 给不同话术 (role 类继续统一话术, amount 类可给阈值提示) – 长期: 客户"为什么"点击 → 弹窗显示具体原因 (但需鉴权, 不暴露给攻击者)

下一步: 回到 [README.md](#) 看其他章节.